

Arquitetura de Computadores

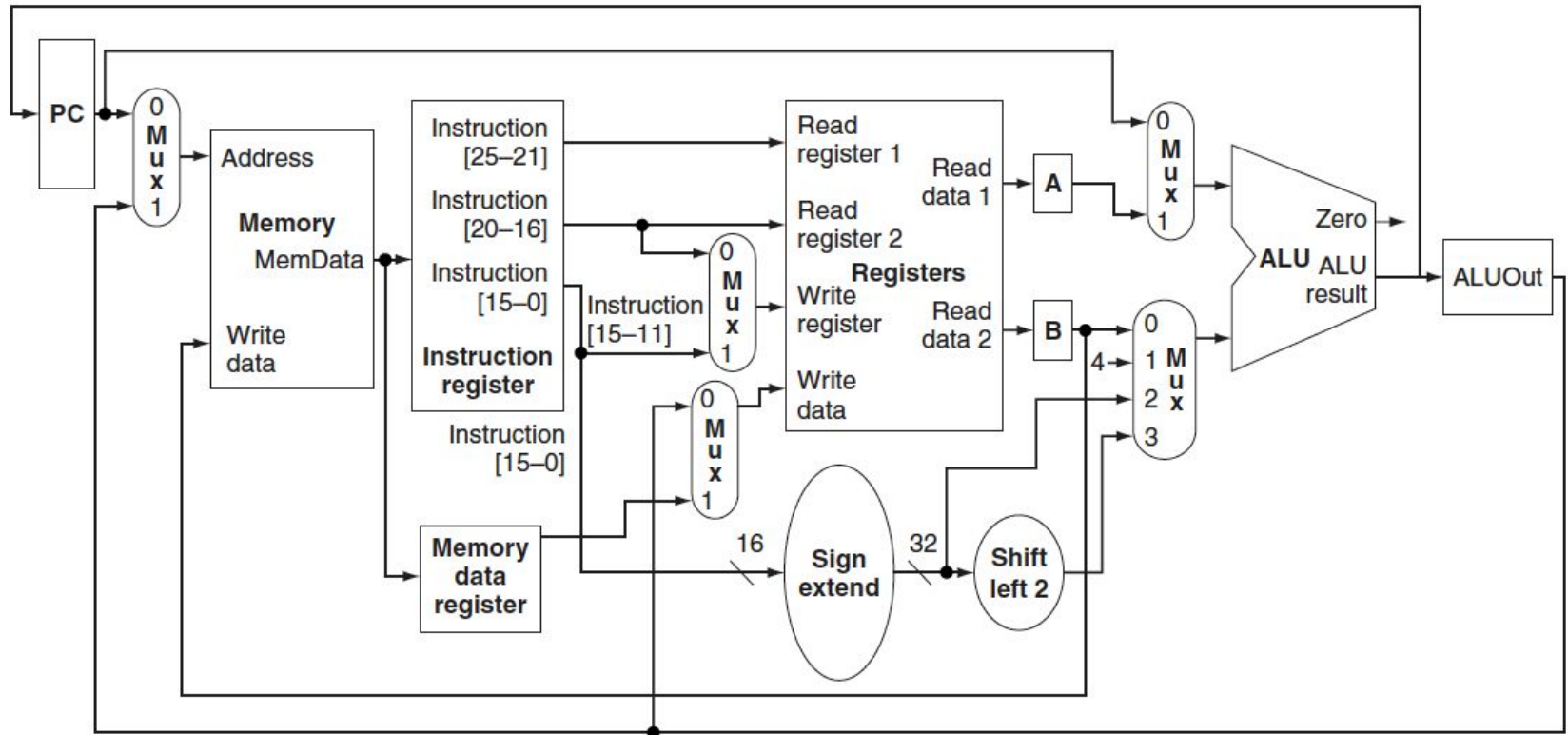
Tópico 4- Melhorando o Desempenho Através do Paralelismo

Em relação a desempenho...

- O que é vantajoso no processador Monociclo?
 - $CPI = 1$
- O que é desvantajoso no processador Monociclo?
 - Frequência baixa (Regrada pela instrução mais lenta)
- O que se ganhou na implementação Multiciclo?
 - Aumento na Frequência (Regrada pelo passo mais lento)
- O que se perdeu na implementação Multiciclo?
 - $CPI > 1$

Como agregar as vantagens das duas implementações?
CPI baixo e Frequência alta

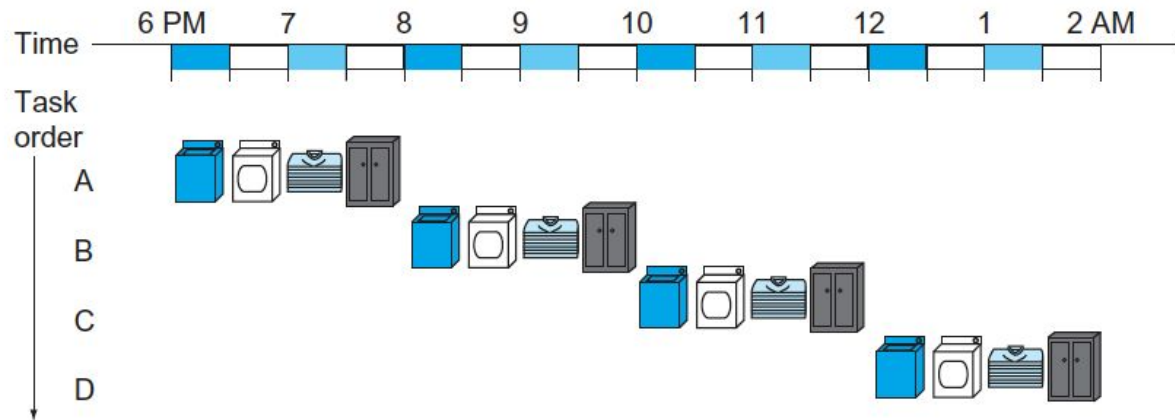
No multiciclo..o que se pode melhorar?



- Hardware dividido para executar os 5 passos
- Somente **uma/duas** parte do hardware está propagando a execução da instrução
- Solução?

Multiciclo

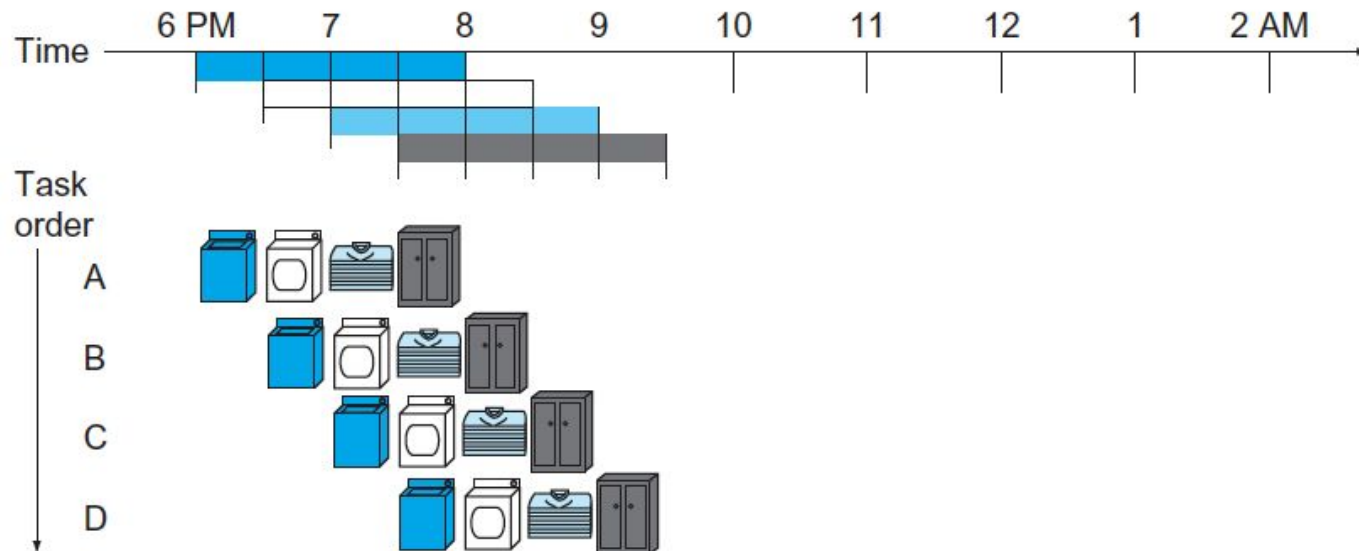
- Analogia com o processo de lavagem de roupas
 - Dividido em 4 passos:
 - Lavar
 - Secar
 - Dobrar
 - Guardar
 - Assumindo que cada passo leve 30 minutos



- Cada processo leva 2 horas
- Três processos levam 6 horas
- Recursos ociosos na maior parte do tempo!

Pipeline

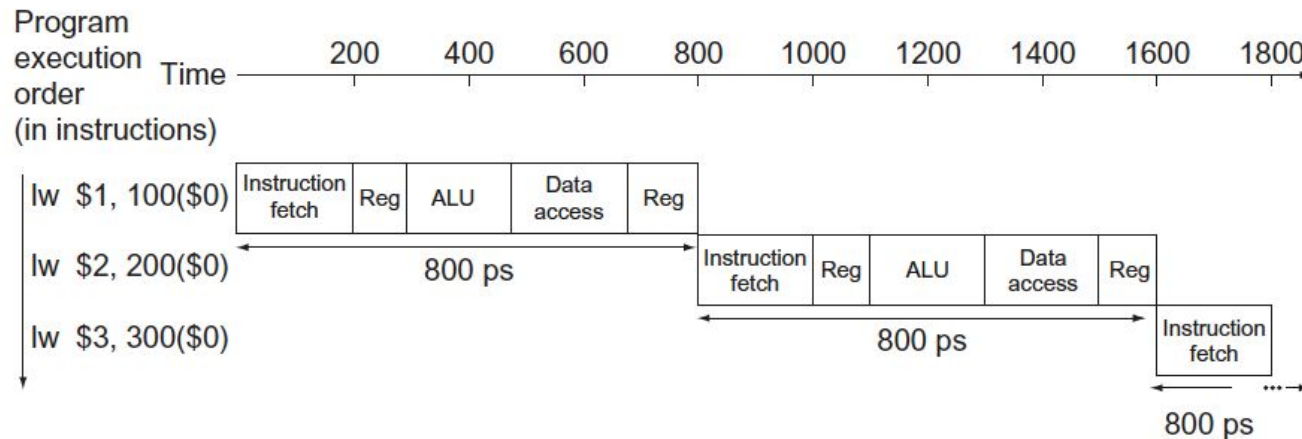
- Eliminação da ociosidade dos recursos
- Inicia-se um processo a cada passo
- Um processo continua levando 2 horas
- Três processos levam 3 horas e 30 minutos



Monociclo

Assuma:

- 200 ps para acessos à memória e operação na ULA.
- 100 ps para acesso ao banco de registradores.

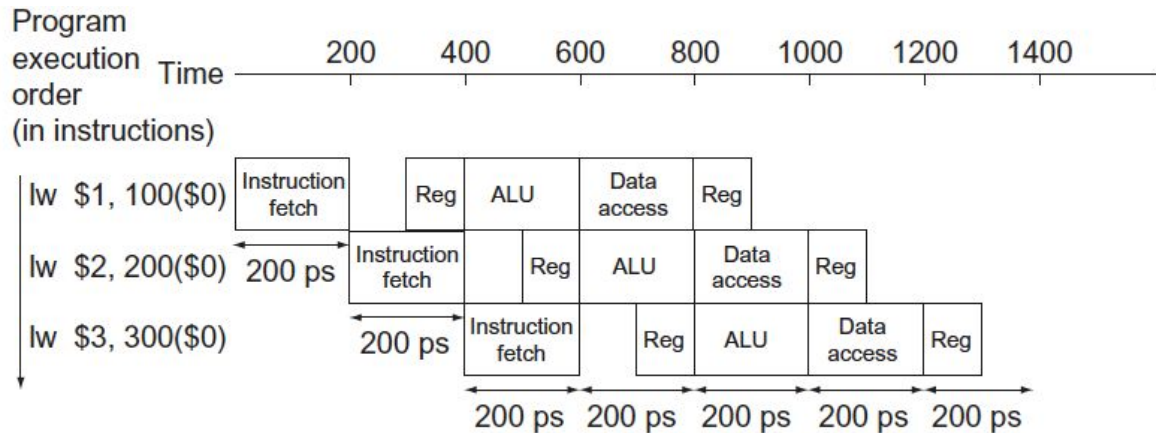


- Período do monociclo é 800 ps
- CPI = 1
- Tempo de execução de um load = 800 ps
- Tempo de execução de três loads = 2400 ps

Pipeline

Assuma:

- 200 ps para acessos à memória e operação na ULA.
- 100 ps para acesso ao banco de registradores.



- Período do pipeline é 200 ps
- CPI ≈ 1
- Tempo de execução de um load = 1000 ps
- Tempo de execução de três loads = 1400 ps

Características do Pipeline

- Pipelining **não reduz o tempo de execução** de uma instrução, mas **diminui** o tempo de execução de muitas instruções
 - Questões:
 - O caminho crítico do pipeline é limitado pelo **estágio mais longo**
 - Quanto mais estágios melhor?
 - Períodos dos estágios devem ser balanceados
 - Múltiplas instruções estão no pipeline concomitantemente
 - Uma em cada estágio do pipeline
 - Mas somente uma instrução pode terminar (write back) por ciclo
 - Exemplo: Intel 486 (1991)

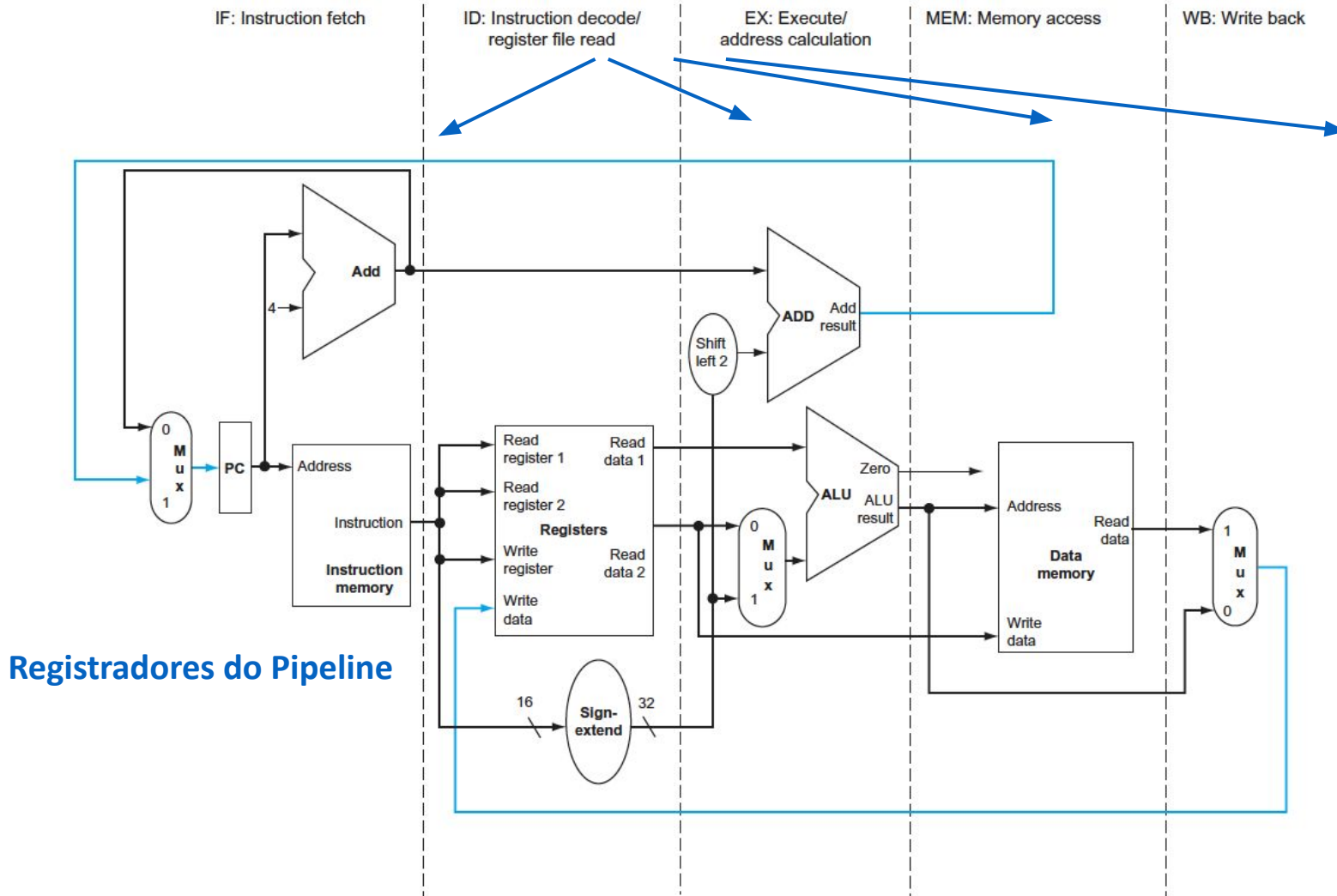
Caminho de dados

- Vamos construir o bloco operativo
- Lembrando: lw, sw, add, sub, beq
- Primeiro, lembre-se dos cinco passos
 - Busca de Instruções e incremento do PC (IF)
 - Decodificação da instrução e busca dos registradores (ID)
 - Execução ou cálculo do endereço de memória de dados/branch (EX)
 - Acesso à Memória (MEM)
 - Gravação do resultado no banco de registradores (WB)

Caminho de dados

- No monociclo
 - Todos os passos feitos em apenas um ciclo
 - Hardware dedicado para executar toda a instrução em um ciclo
- No multiciclo
 - Passos são realizados em ciclos diferentes
 - Alguns componentes de hardware podem compartilhados em diferentes ciclos da execução da instrução
 - Memórias, somadores, etc.
- No pipeline
 - Devemos implementá-lo a partir do caminho de dados do monociclo ou multiciclo?
 - Monociclo
 - Temos instruções diferentes em cada passo que necessitam de HW dedicado

Divisão em Estágios de Pipeline

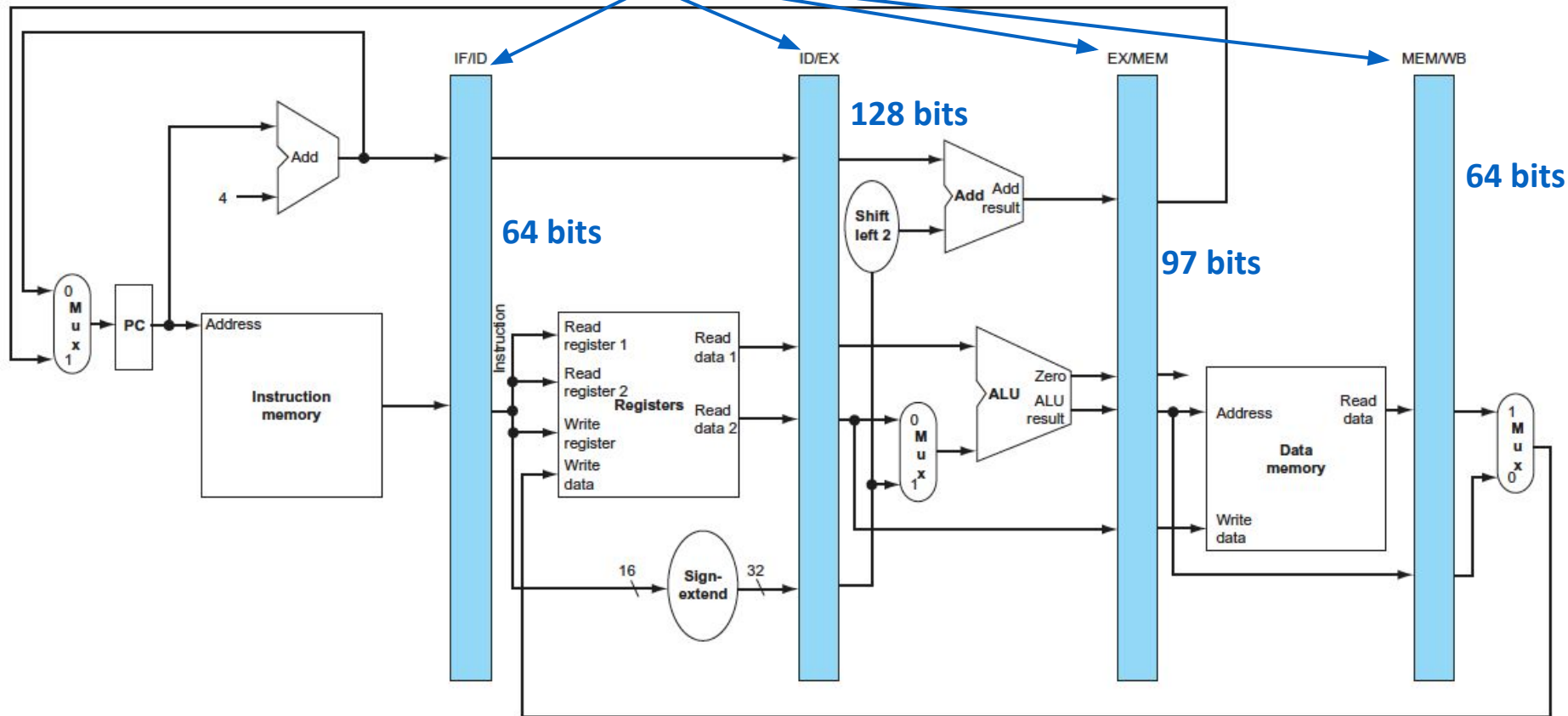


Pipeline

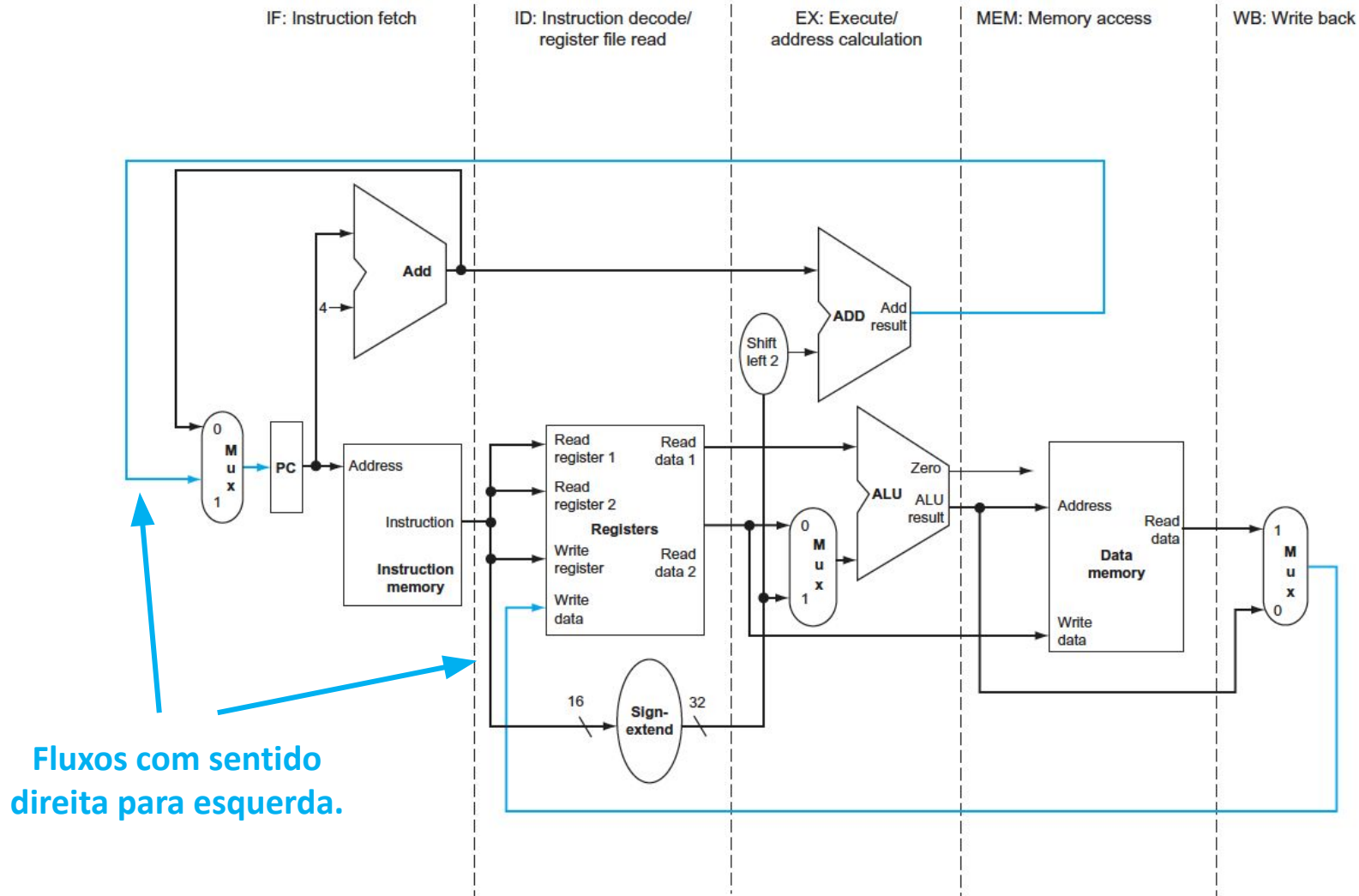
- **Como iremos isolar os resultados de cada instrução em cada estágio do pipeline?**
 - Precisamos de registradores ***extras*** para guardar os dados entre os estágios
 - Registradores do pipeline

Bloco operativo com pipeline

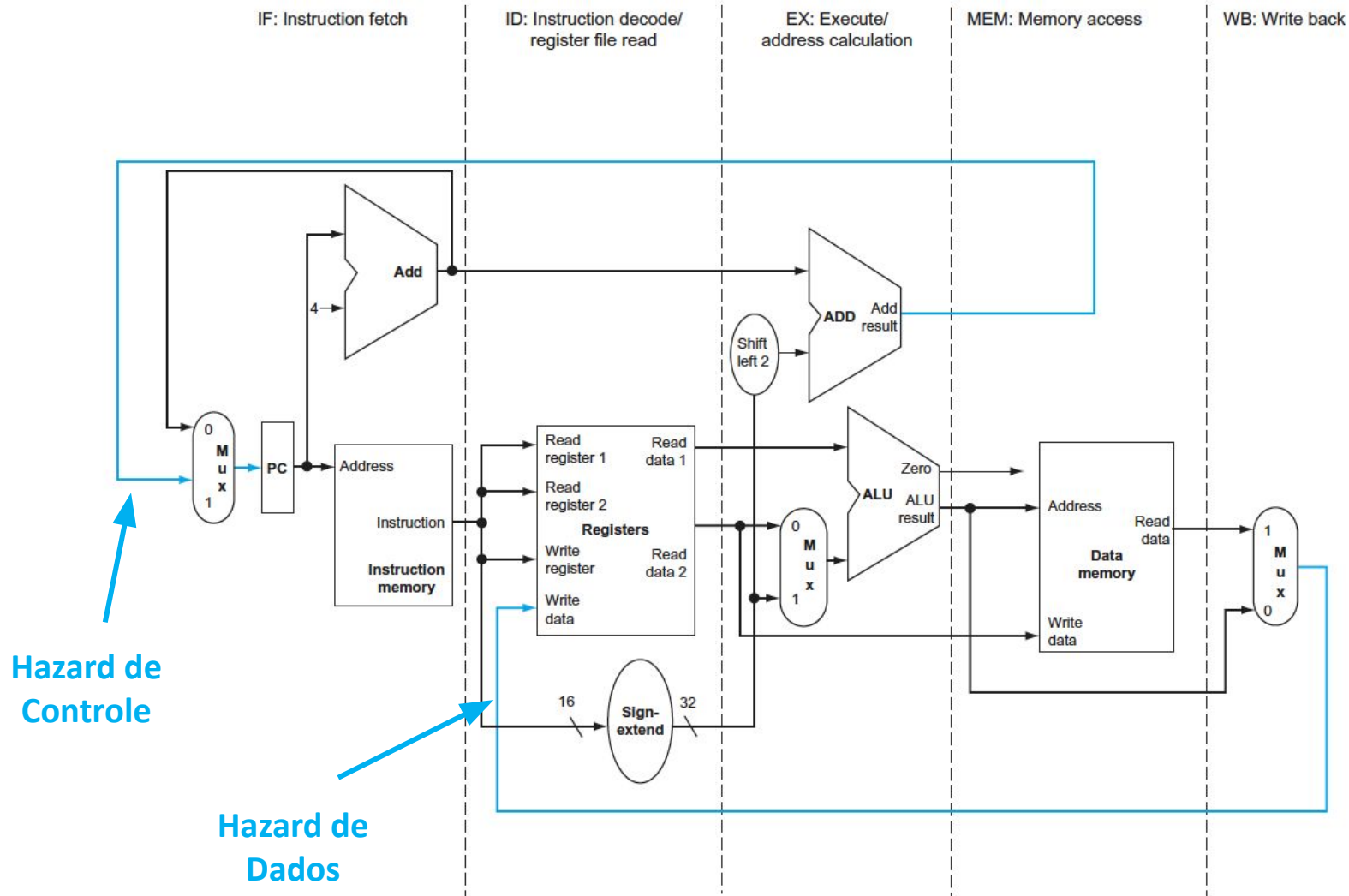
Registadores do Pipeline Largos o suficiente para manter os dados



Divisão em Estágios de Pipeline

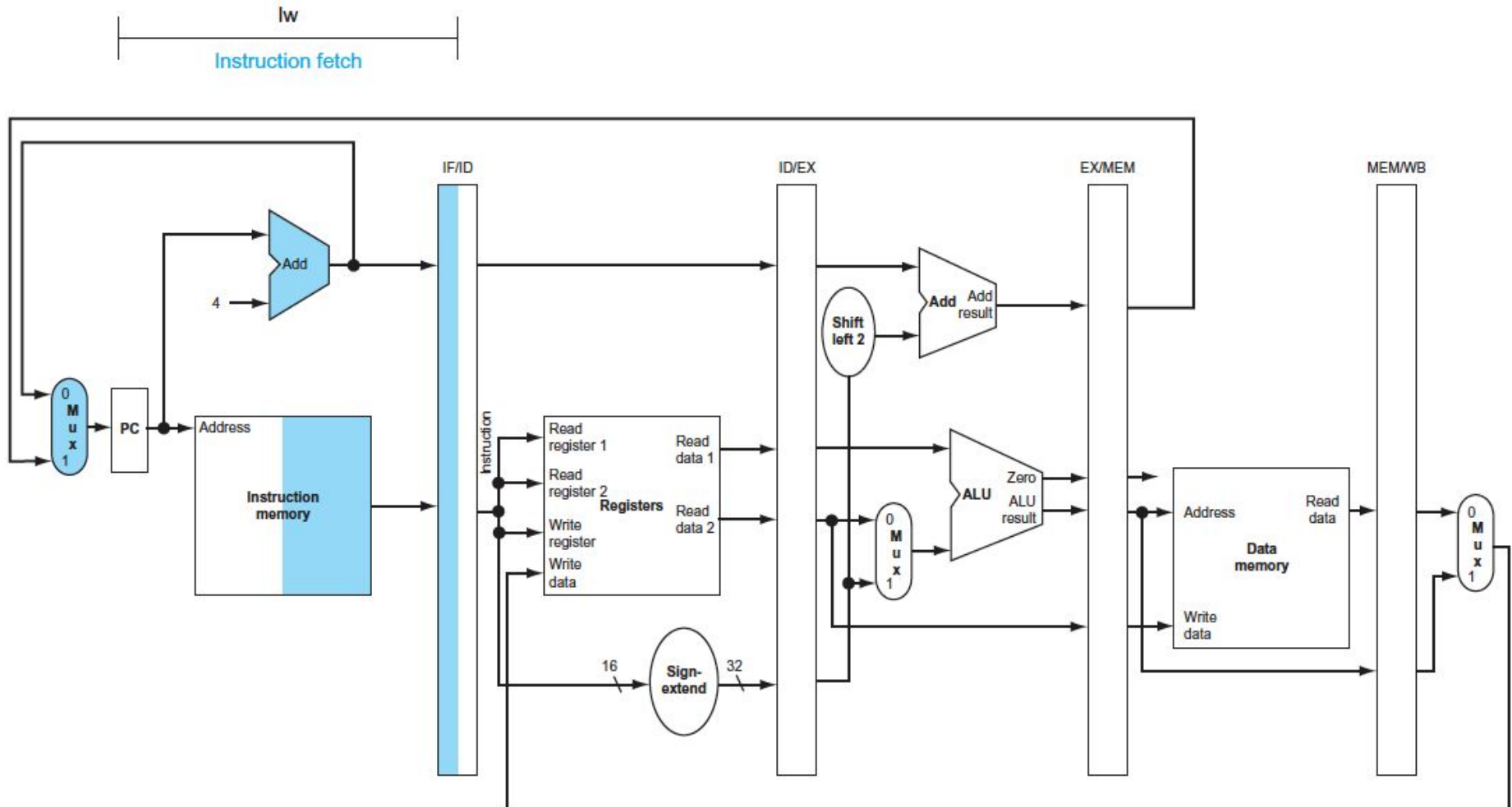


Divisão em Estágios de Pipeline



Exemplo no Pipeline

- Considere a execução de uma instrução lw



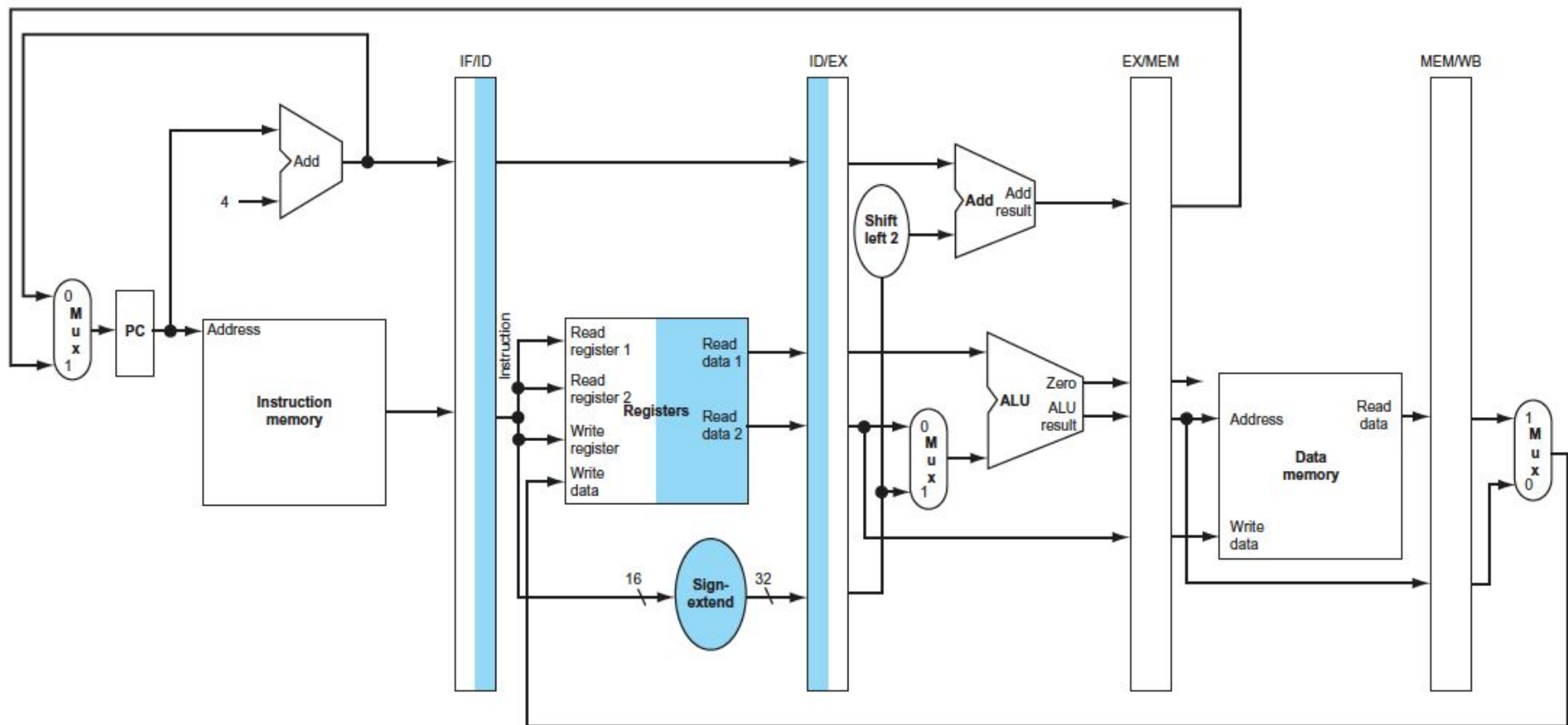
Exemplo no Pipeline

- Considere a execução de uma instrução lw

Instr + 1

lw

Instruction decode



Exemplo no Pipeline

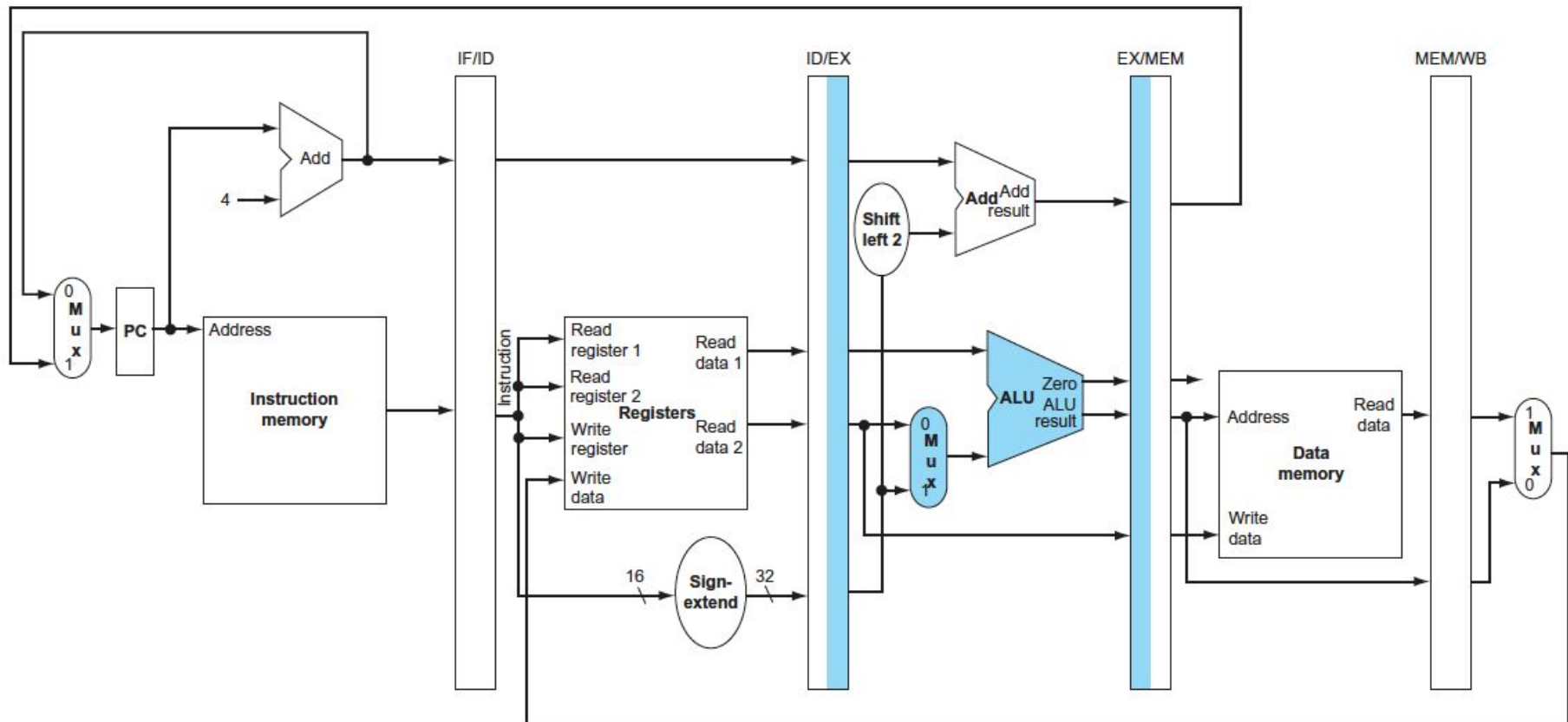
- Considere a execução de uma instrução lw

Instr + 2

Instr + 1

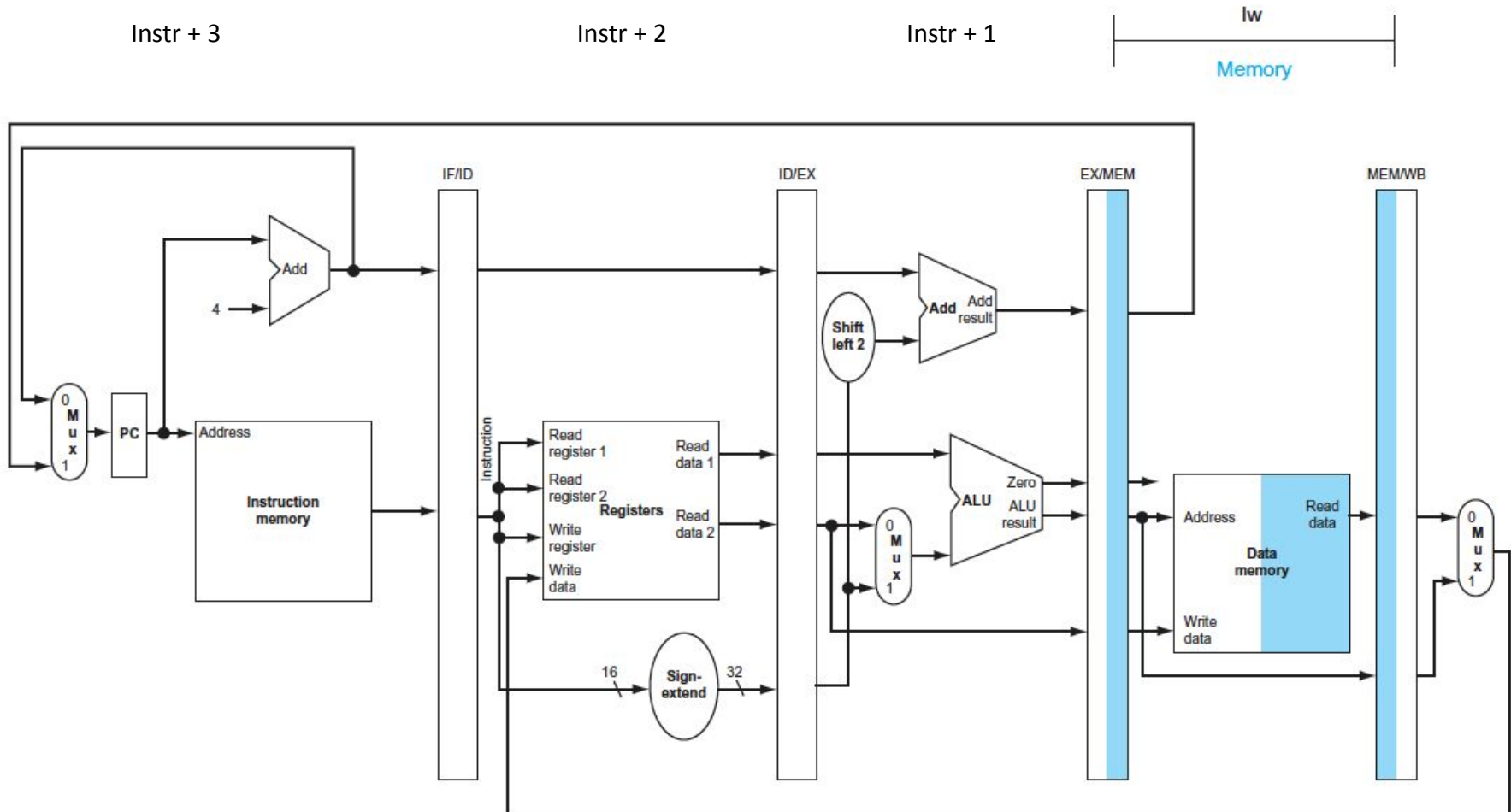
lw

Execution



Exemplo no Pipeline

- Considere a execução de uma instrução lw



Exemplo no Pipeline

- Considere a execução de uma instrução lw

Instr + 4

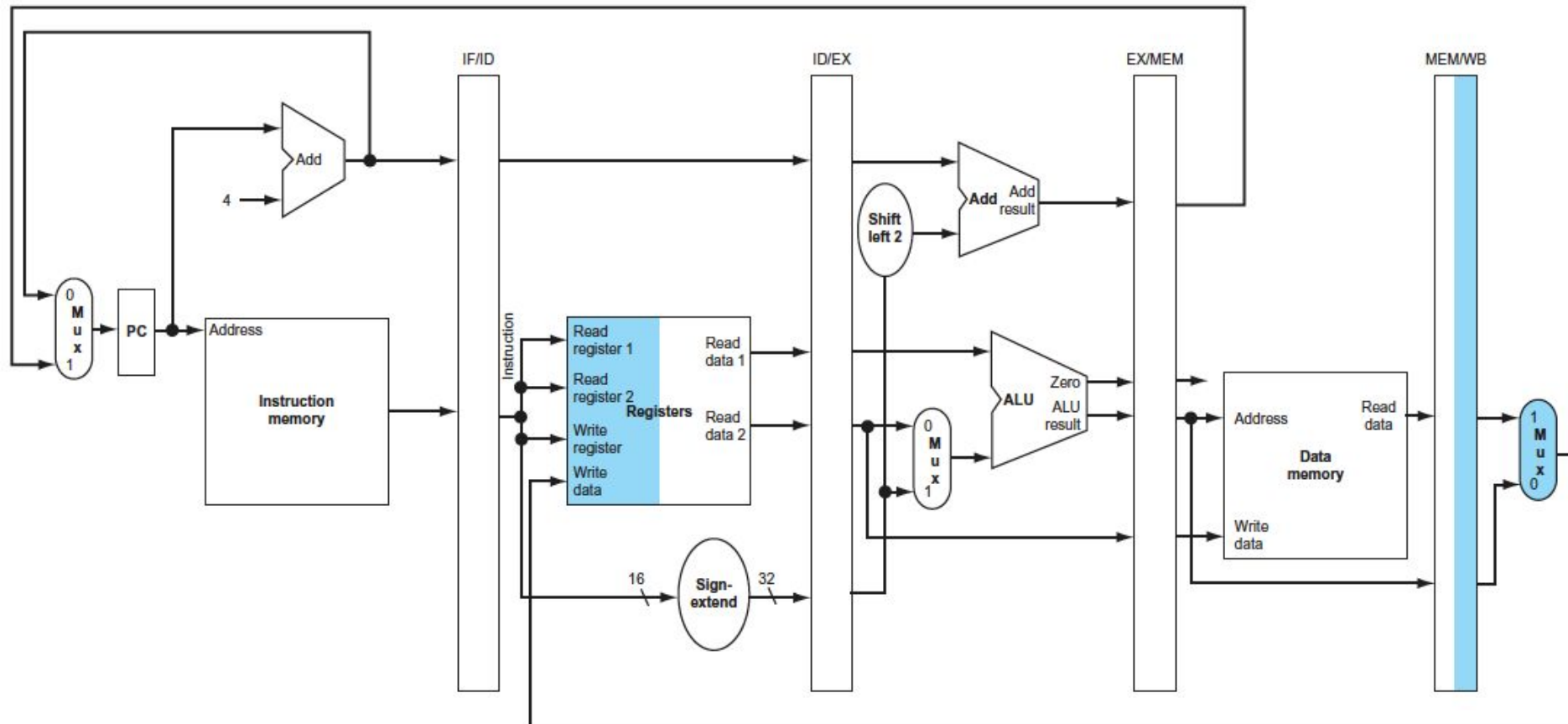
Instr + 3

Instr + 2

Instr + 1

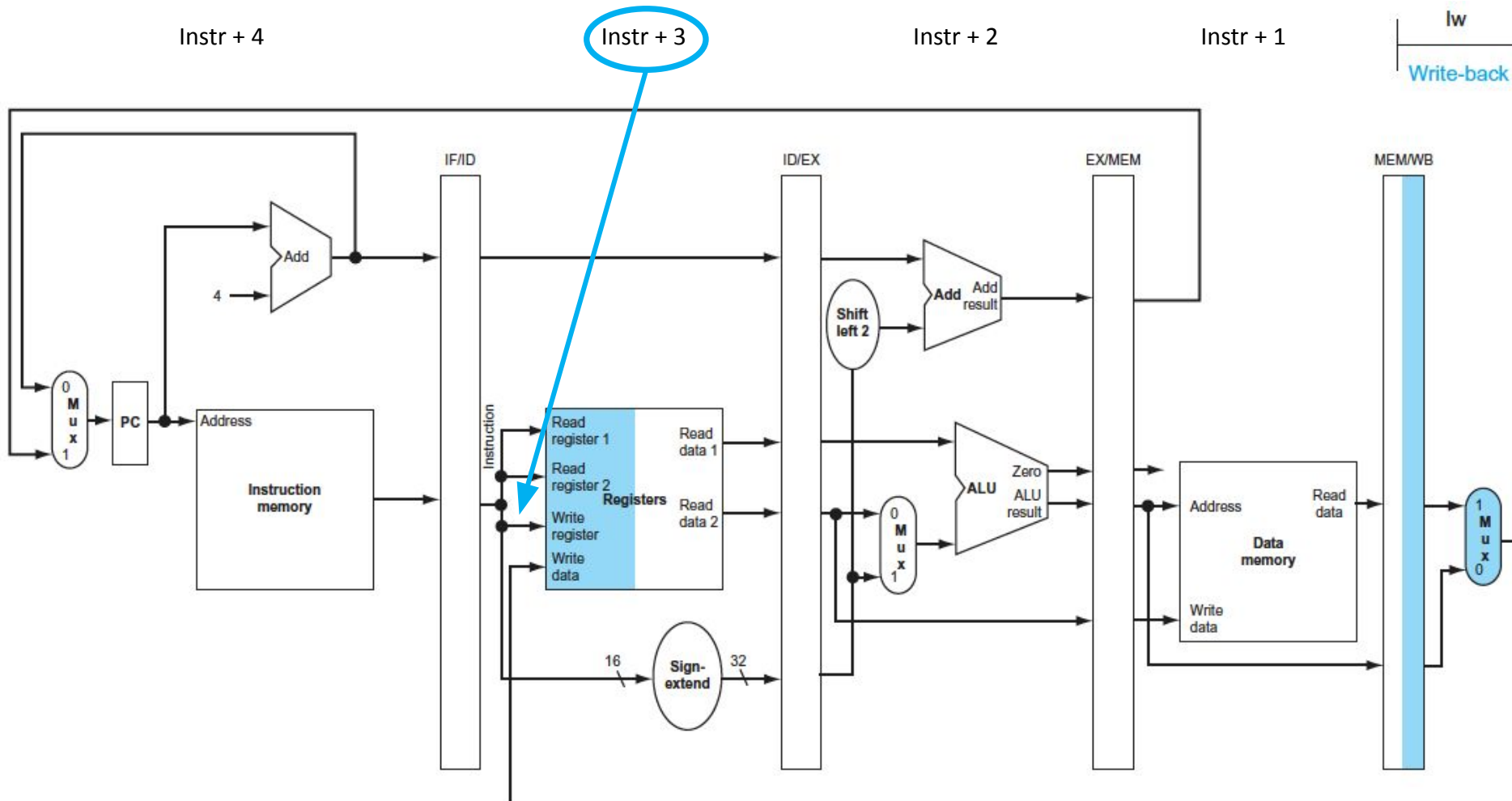
lw

Write-back



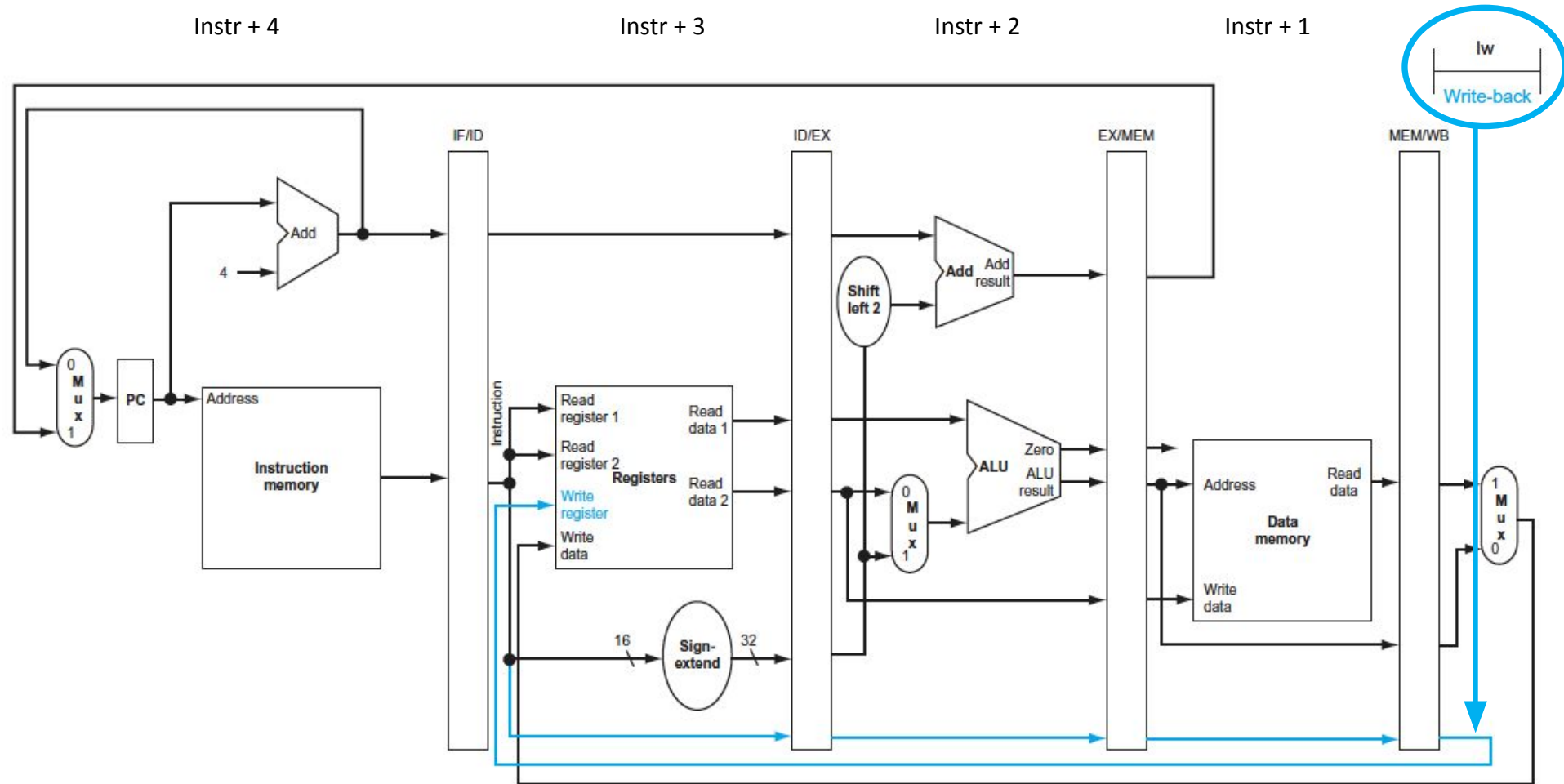
Exemplo no Pipeline

- Considere a execução de uma instrução lw



Exemplo no Pipeline

- Considere a execução de uma instrução lw



Exemplo no Pipeline

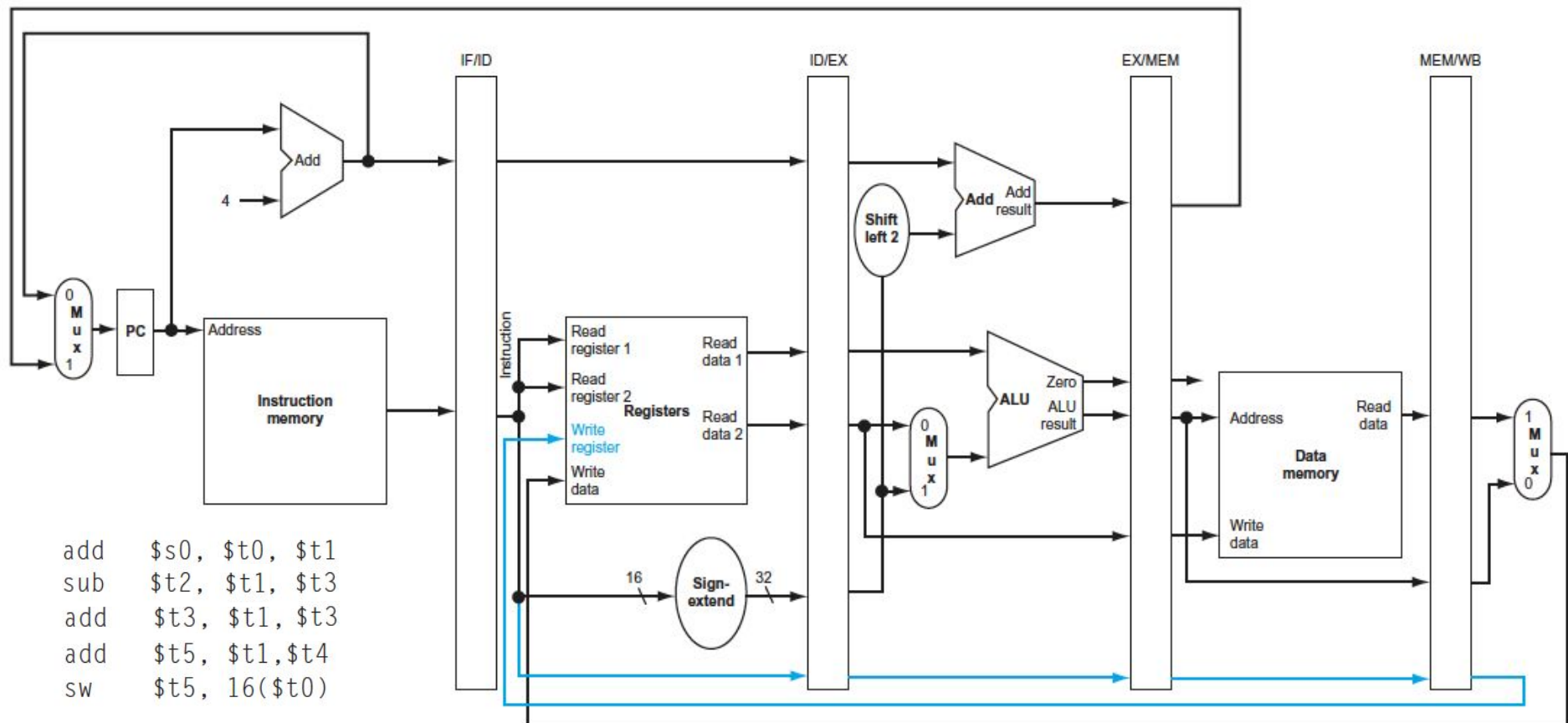
- Considere a execução do trecho MIPS abaixo:

```
add    $s0, $t0, $t1
sub     $t2, $t1, $t3
add     $t3, $t1, $t3
add     $t5, $t1, $t4
sw      $t5, 16($t0)
```

Exemplo no Pipeline

• Ciclo 1

add \$s0, \$t0, \$t1

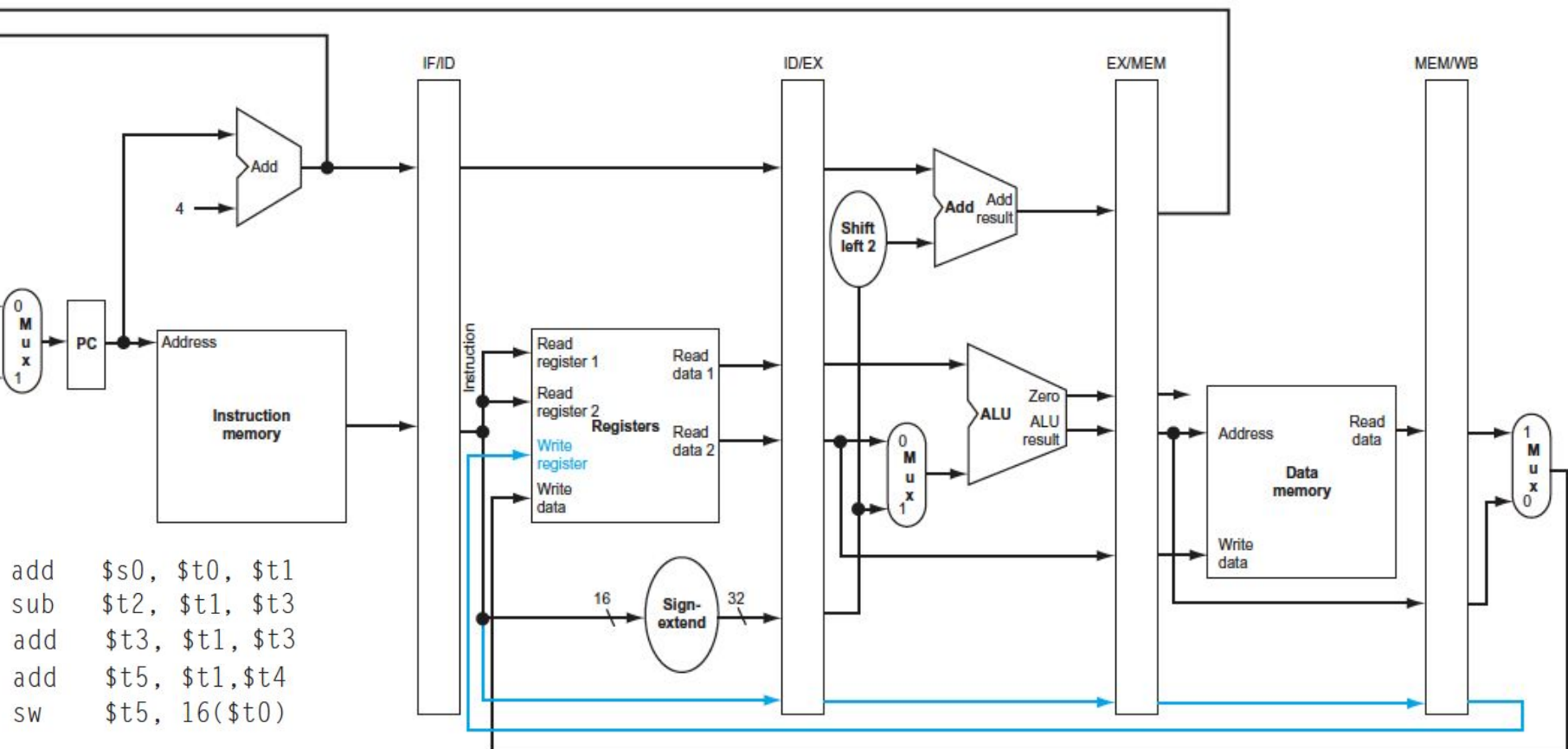


Exemplo no Pipeline

• Ciclo 2

sub \$t2, \$t1, \$t3

add \$s0, \$t0, \$t1



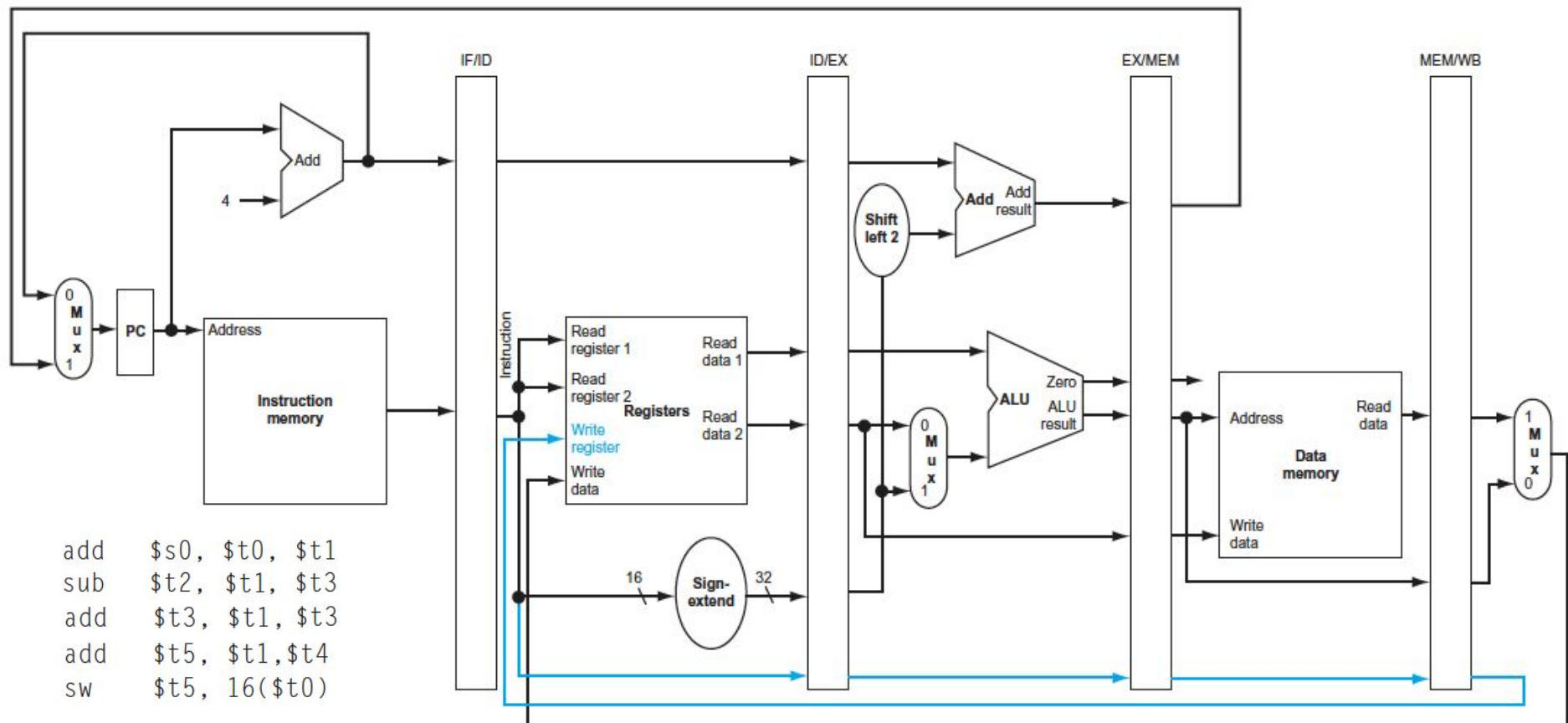
Exemplo no Pipeline

• Ciclo 3

add \$t3, \$t1, \$t3

sub \$t2, \$t1, \$t3

add \$s0, \$t0, \$t1



```
add $s0, $t0, $t1
sub $t2, $t1, $t3
add $t3, $t1, $t3
add $t5, $t1, $t4
sw $t5, 16($t0)
```

Exemplo no Pipeline

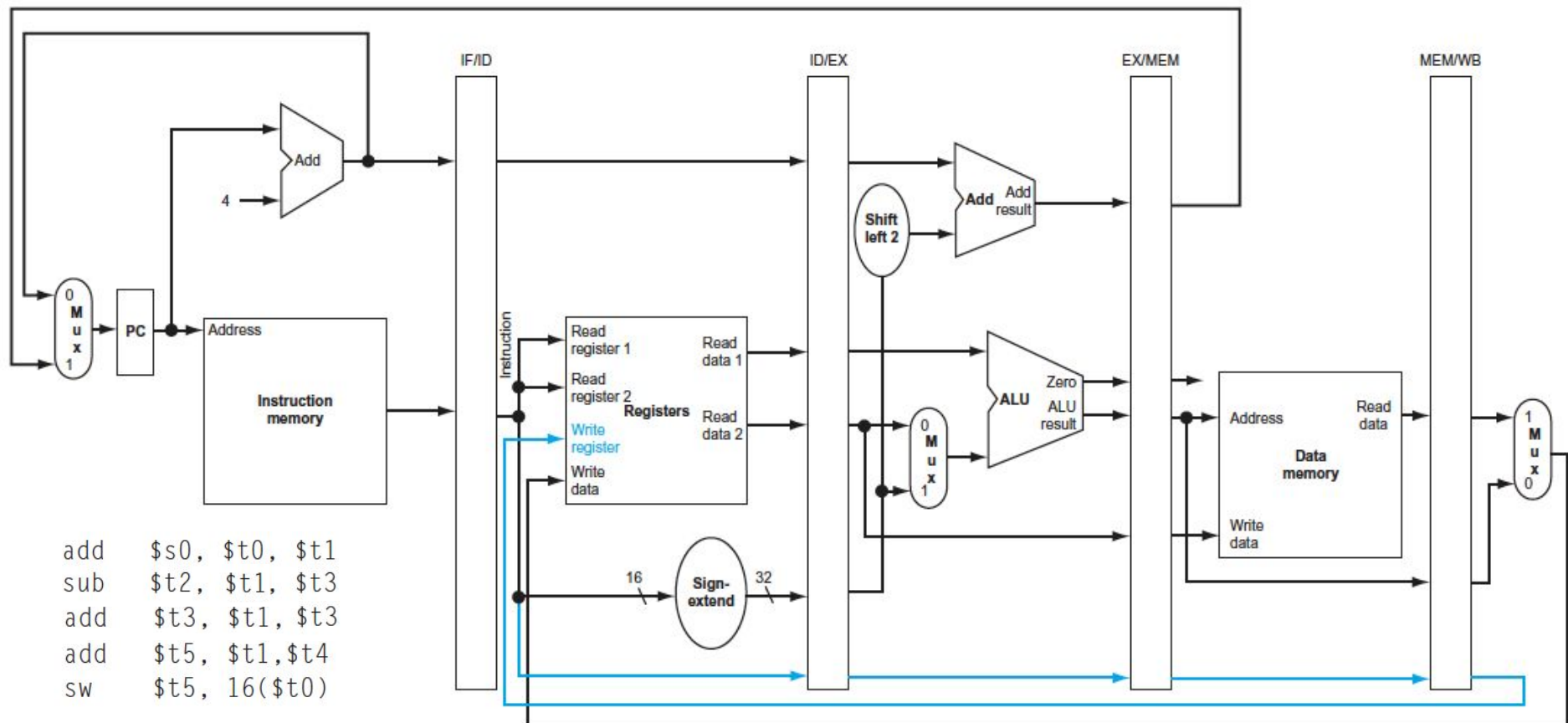
• Ciclo 4

add \$t5, \$t1, \$t4

add \$t3, \$t1, \$t3

sub \$t2, \$t1, \$t3

add \$s0, \$t0, \$t1



Exemplo no Pipeline

• Ciclo 5

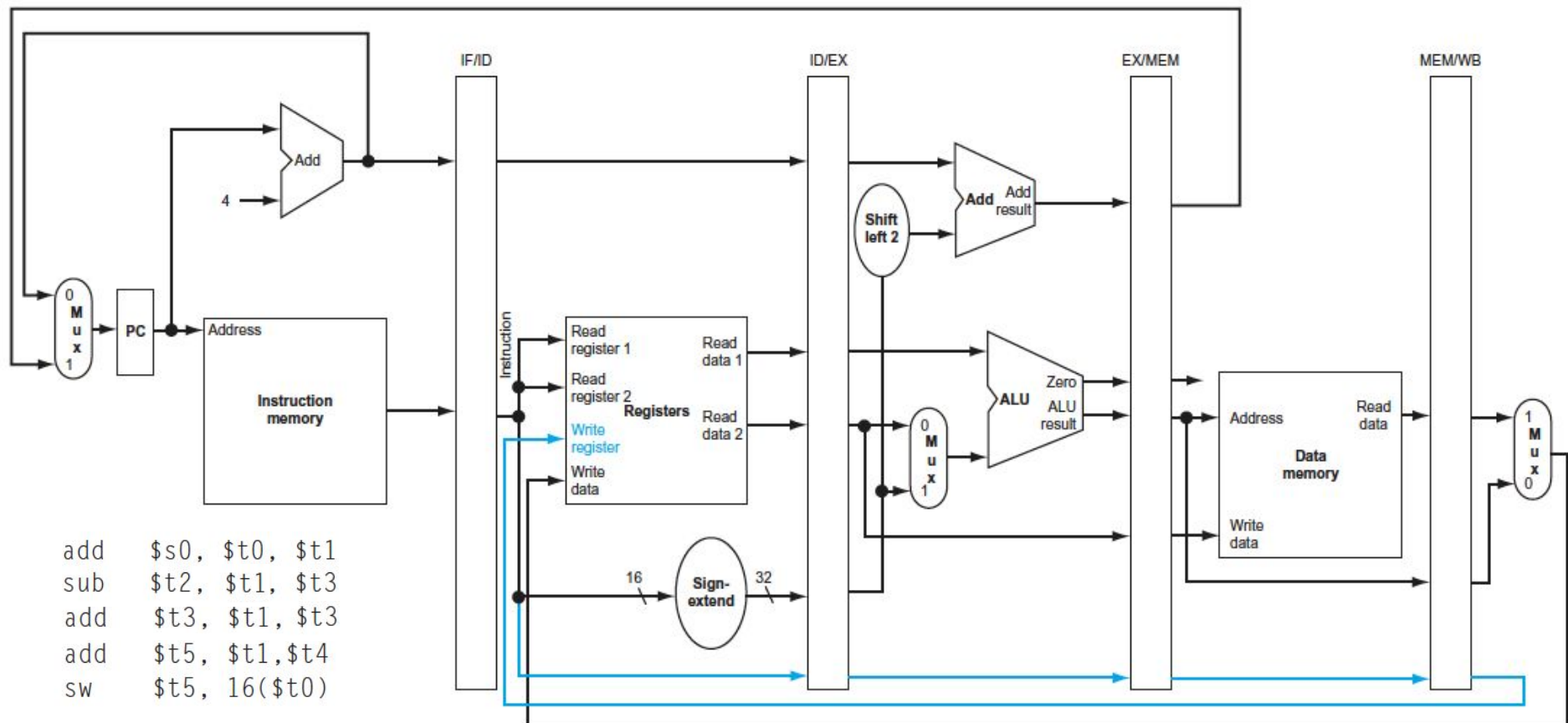
sw \$t5, 16(\$t0)

add \$t5, \$t1, \$t4

add \$t3, \$t1, \$t3

sub \$t2, \$t1, \$t3

add \$s0, \$t0, \$t1



Exemplo no Pipeline

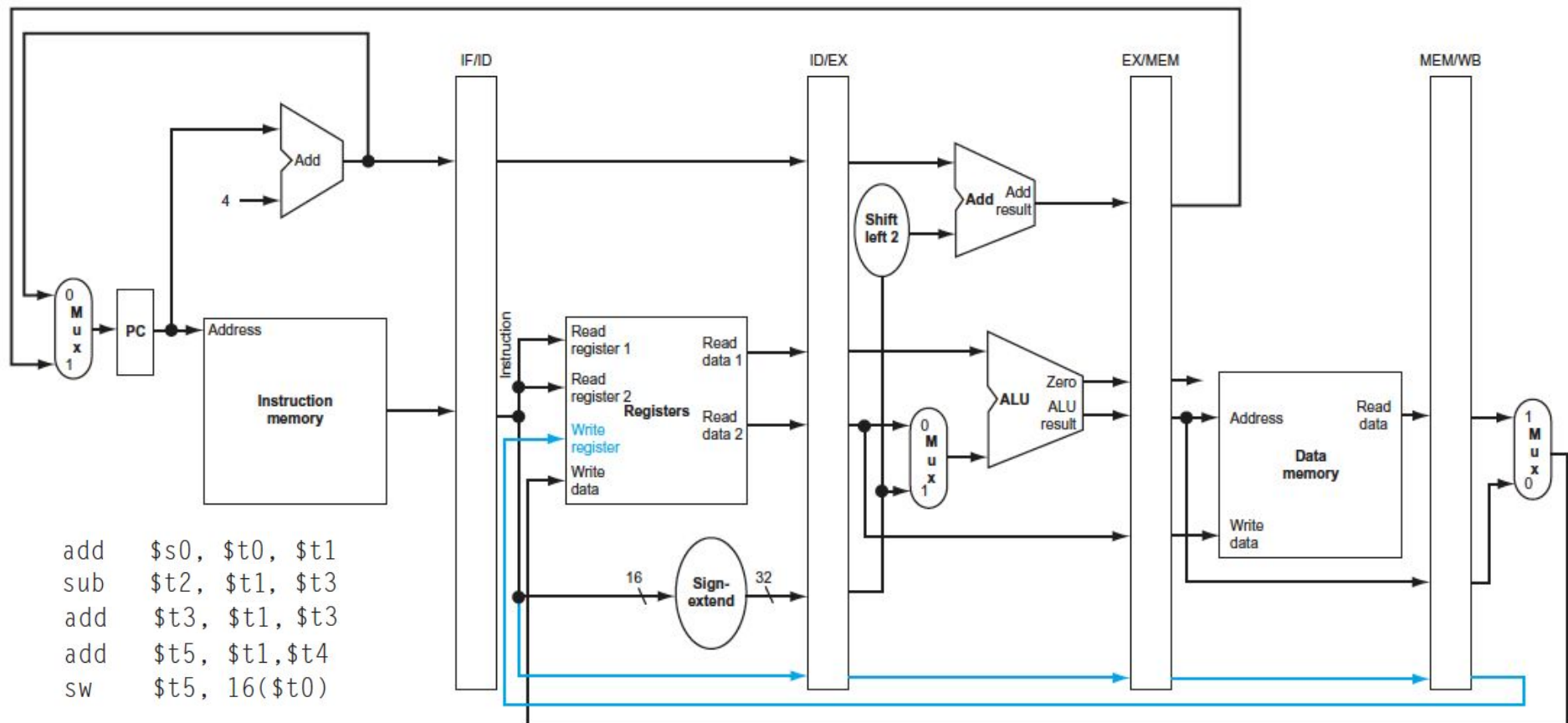
• Ciclo 6

sw \$t5, 16(\$t0)

add \$t5, \$t1, \$t4

add \$t3, \$t1, \$t3

sub \$t2, \$t1, \$t3



```
add $s0, $t0, $t1
sub $t2, $t1, $t3
add $t3, $t1, $t3
add $t5, $t1, $t4
sw $t5, 16($t0)
```

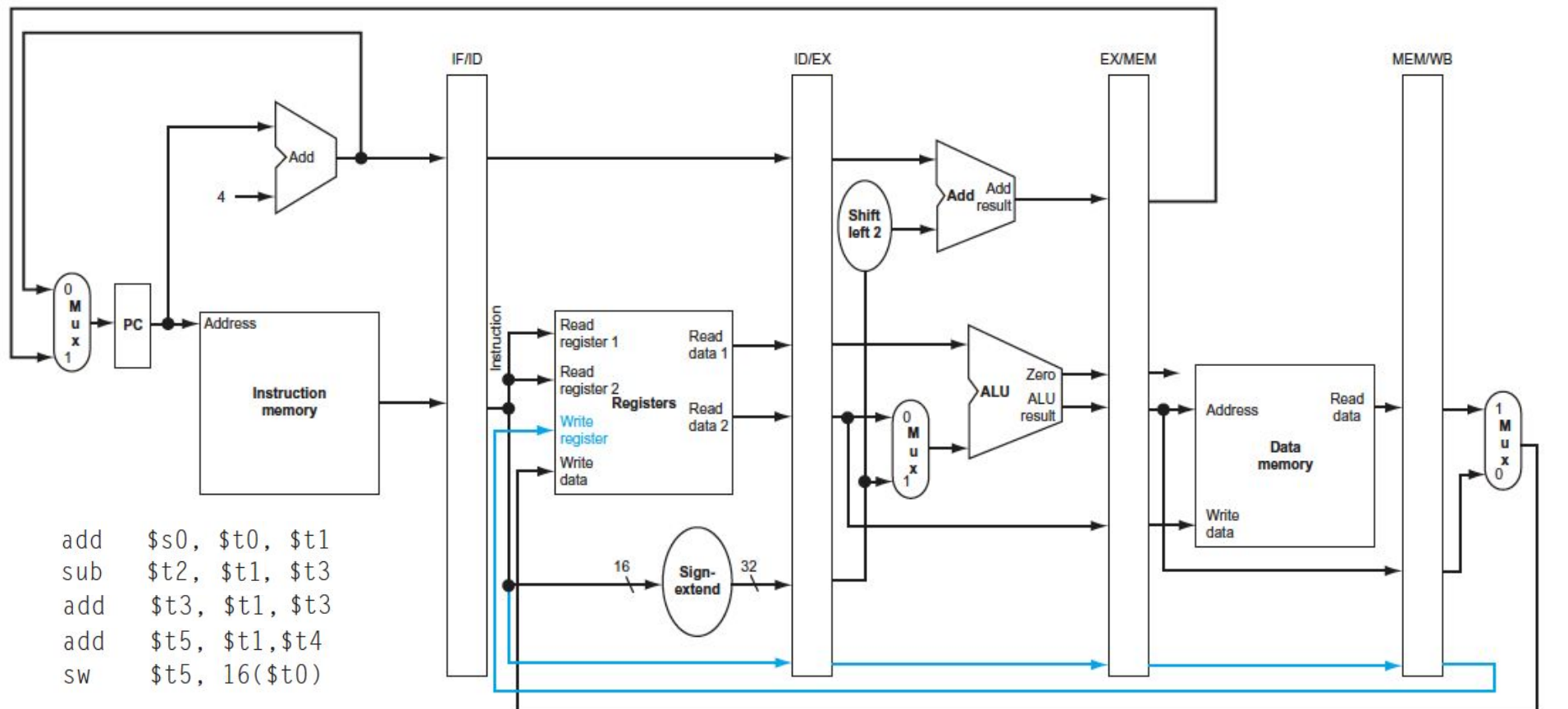
Exemplo no Pipeline

• Ciclo 7

sw \$t5, 16(\$t0)

add \$t5, \$t1, \$t4

add \$t3, \$t1, \$t3

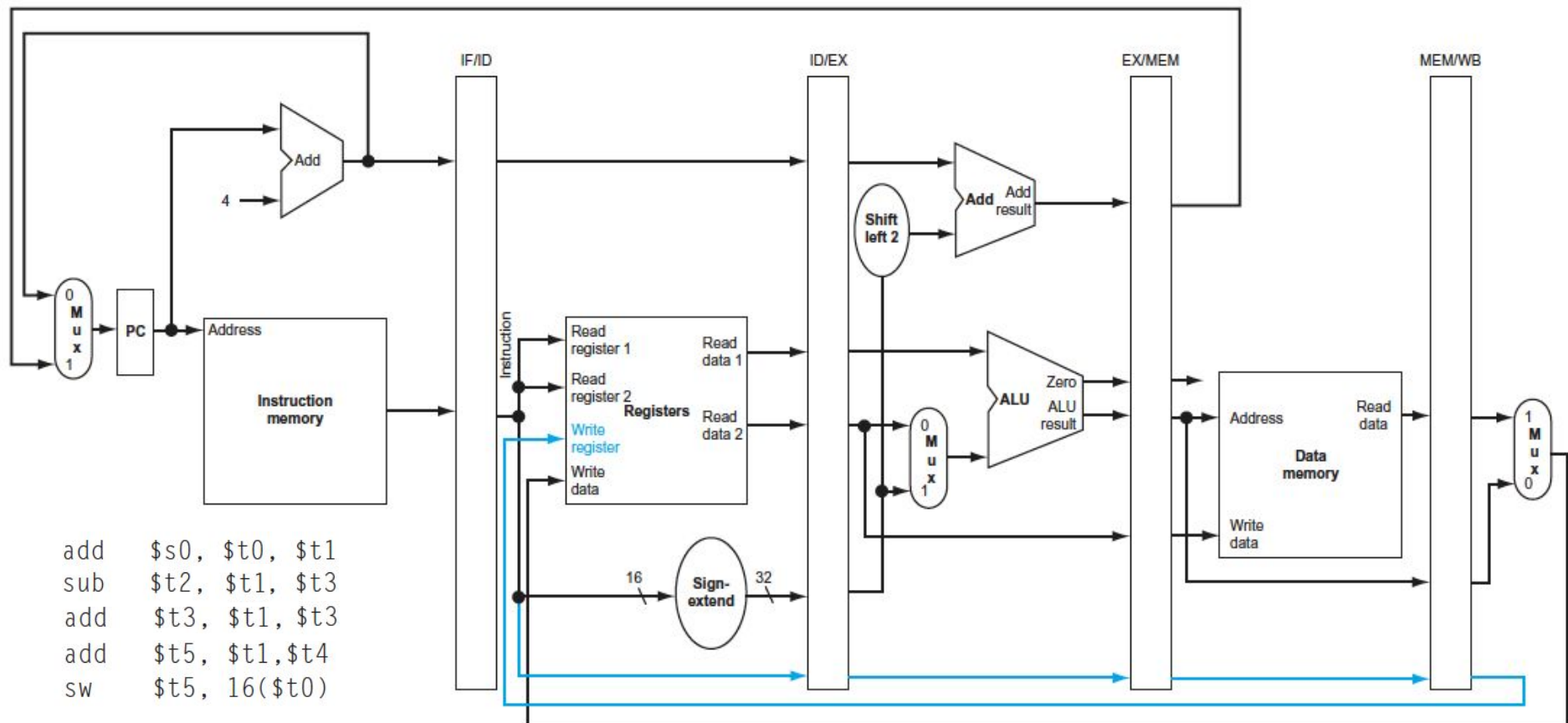


Exemplo no Pipeline

• Ciclo 8

sw \$t5, 16(\$t0)

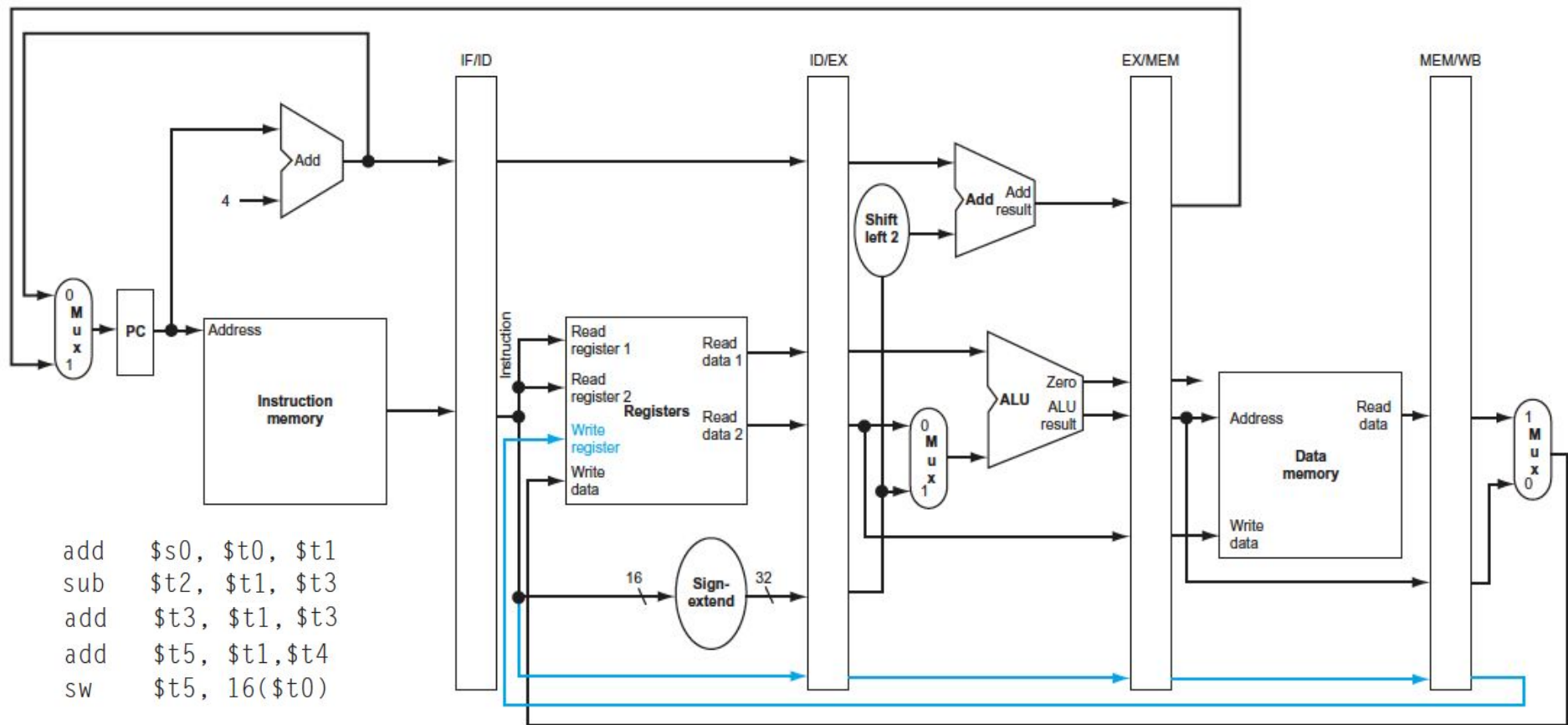
add \$t5, \$t1, \$t4



Exemplo no Pipeline

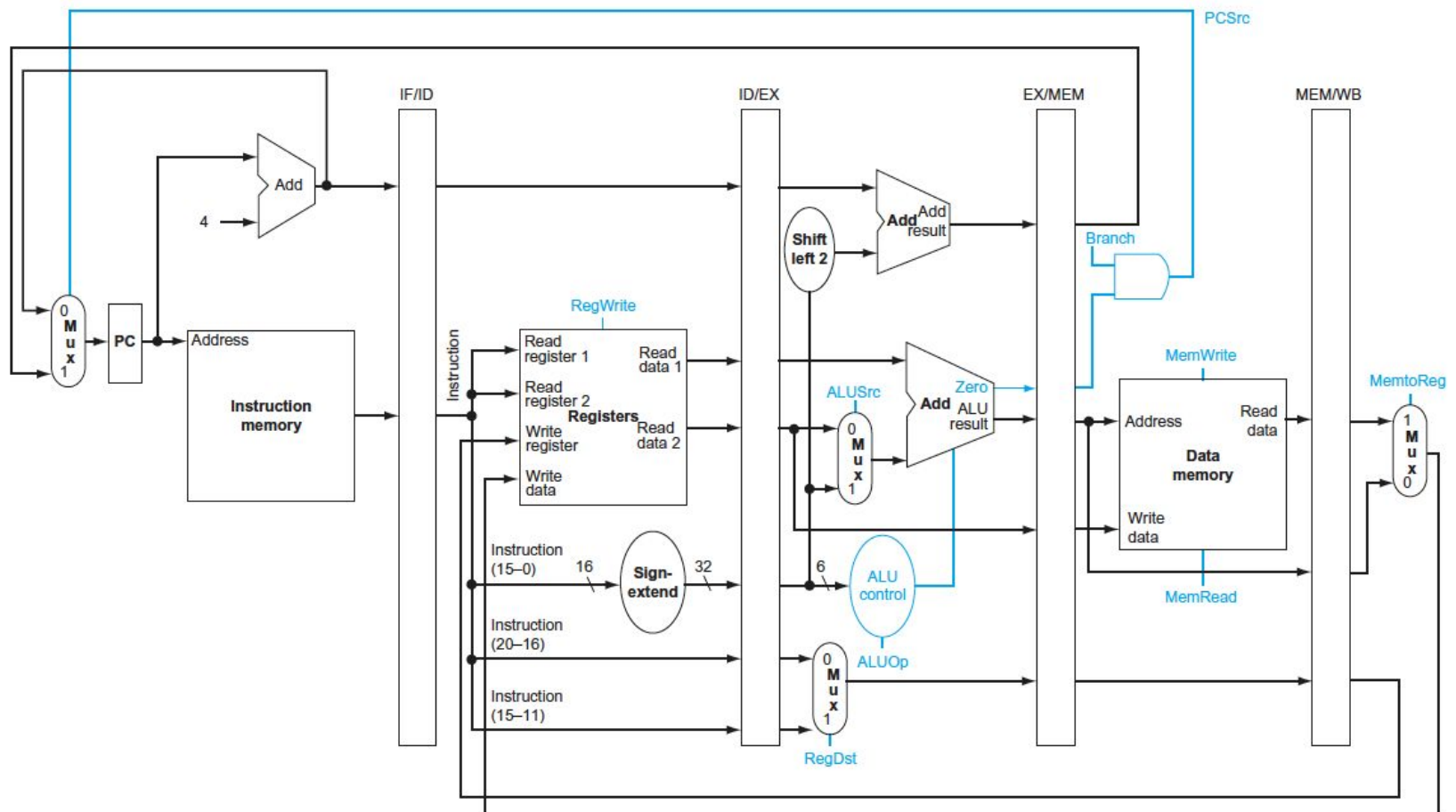
• Ciclo 9

sw \$t5, 16(\$t0)



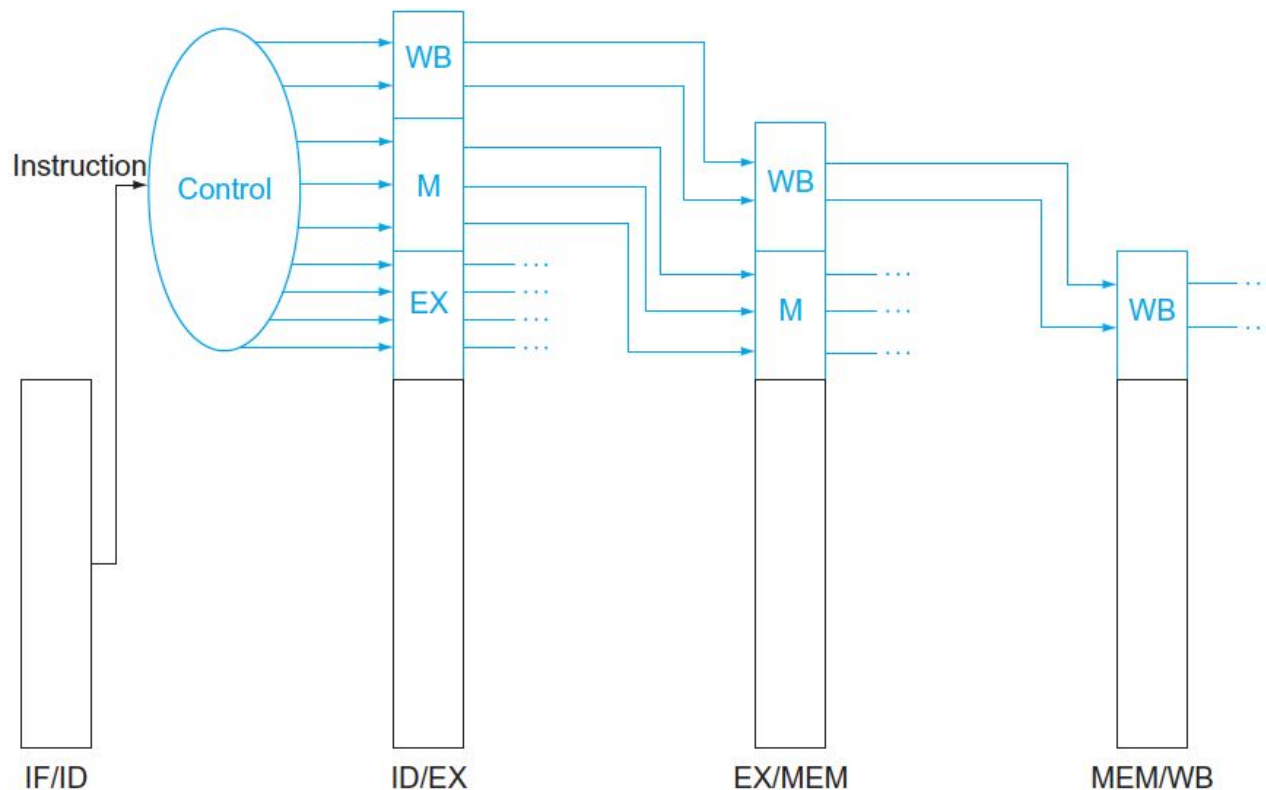
Controle do Pipeline

- Mesmos sinais de controle do MIPS Monociclo



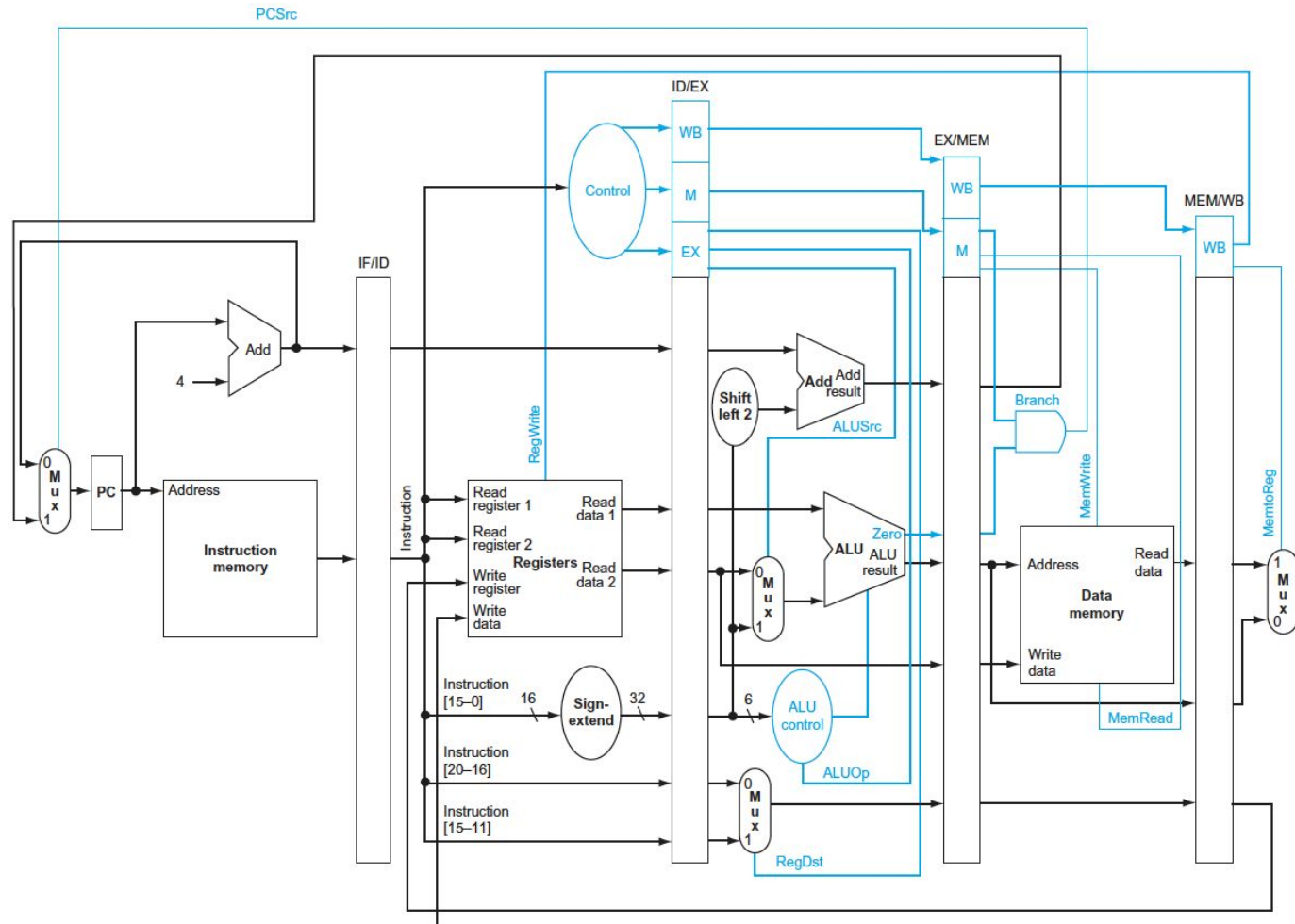
Controle do Pipeline

- Sinais de controle se propagam junto com as instruções
- Registradores de pipeline armazenam os bits de controle



Controle do Pipeline

- Sinais de Controle acoplados ao Caminho de dados



Pipeline Hazards

- **Hazards Estruturais** – duas instruções concorrem para executar no mesmo elemento de hardware no mesmo ciclo de clock
 - No MIPS Pipeline – Se somente existisse uma memória de dados e instruções
- **Hazards de Dados** – uma instrução que está em um estágio anterior do pipeline depende de um dado gerado por uma instrução que está em um estágio posterior do pipeline
 - No MIPS Pipeline – no exemplo abaixo, se não houver intervenção, a segunda instrução estará no segundo estágio e a primeira no terceiro. Assim, a segunda carregará do banco de registradores o valor anterior de \$s0.

```
add    $s0, $t0, $t1  
sub    $t2, $s0, $t3
```

- **Hazards de Controle** – uma instrução que modifica o fluxo da execução (beq, j) deve tomar uma decisão enquanto outras instruções já estão em execução no pipeline.

Hazard de Dados

- O que são?

- Hazards de dados entre registradores que dificultam a execução simultânea de instruções entre os estágios do pipeline
- Uma instrução em um estágio anterior do pipeline precisa de um dado que está sendo computado por uma instrução que está em um estágio posterior do pipeline

sub	\$2, \$1, \$3	# Register \$2 written by sub
and	\$12, \$2, \$5	# 1st operand(\$2) depends on sub
or	\$13, \$6, \$2	# 2nd operand(\$2) depends on sub
add	\$14, \$2, \$2	# 1st(\$2) & 2nd(\$2) depend on sub
sw	\$15, 100(\$2)	# Base (\$2) depends on sub

- RAW- Read After Write – Dependência Verdadeira

Hazard de Dados

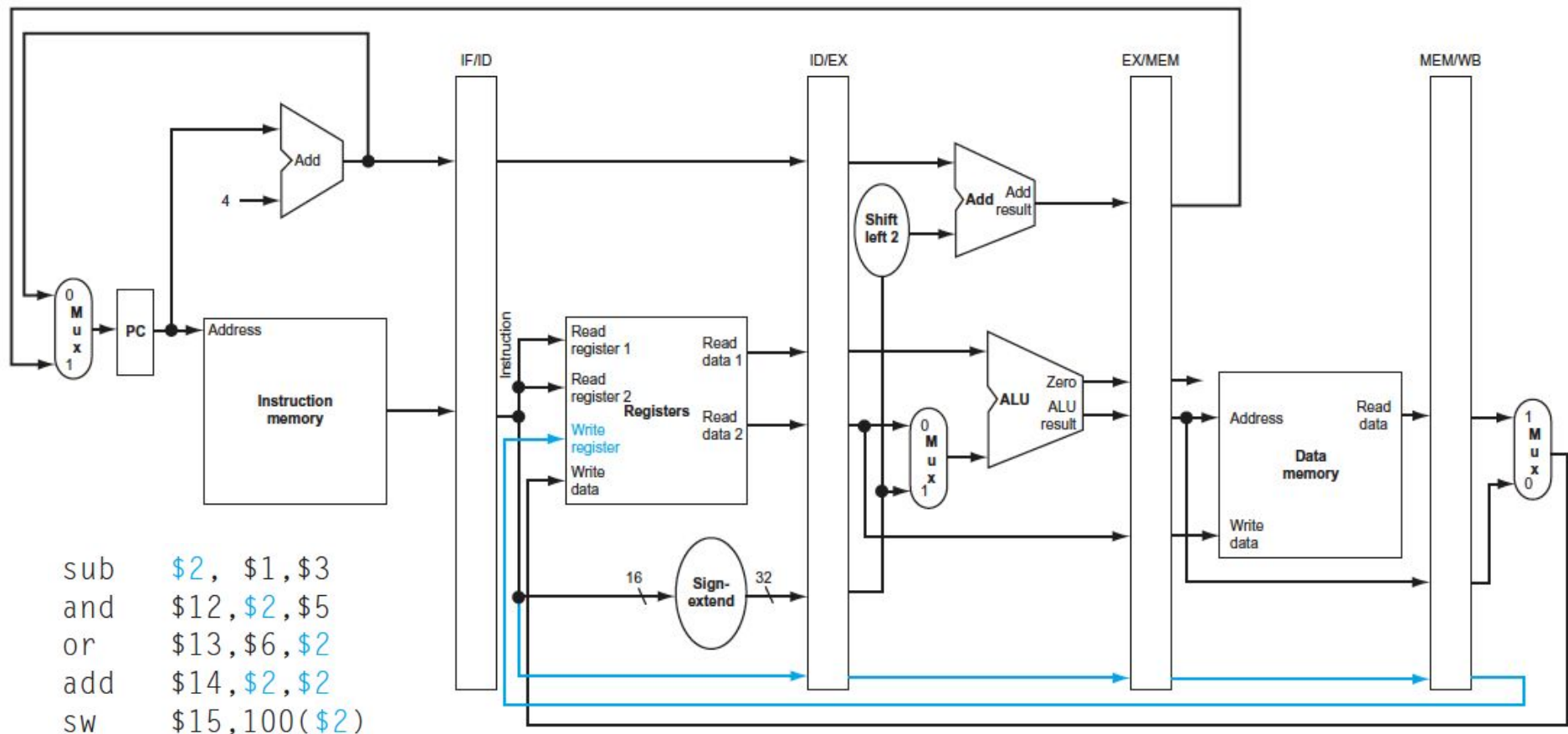
- Caso 1: Hazards entre instruções Aritméticas e Lógicas

sub	\$2, \$1, \$3	# Register \$2 written by sub
and	\$12, \$2, \$5	# 1st operand(\$2) depends on sub
or	\$13, \$6, \$2	# 2nd operand(\$2) depends on sub
add	\$14, \$2, \$2	# 1st(\$2) & 2nd(\$2) depend on sub
sw	\$15, 100(\$2)	# Base (\$2) depends on sub

Hazards de Dados

• Ciclo 1

sub \$2, \$1, \$3



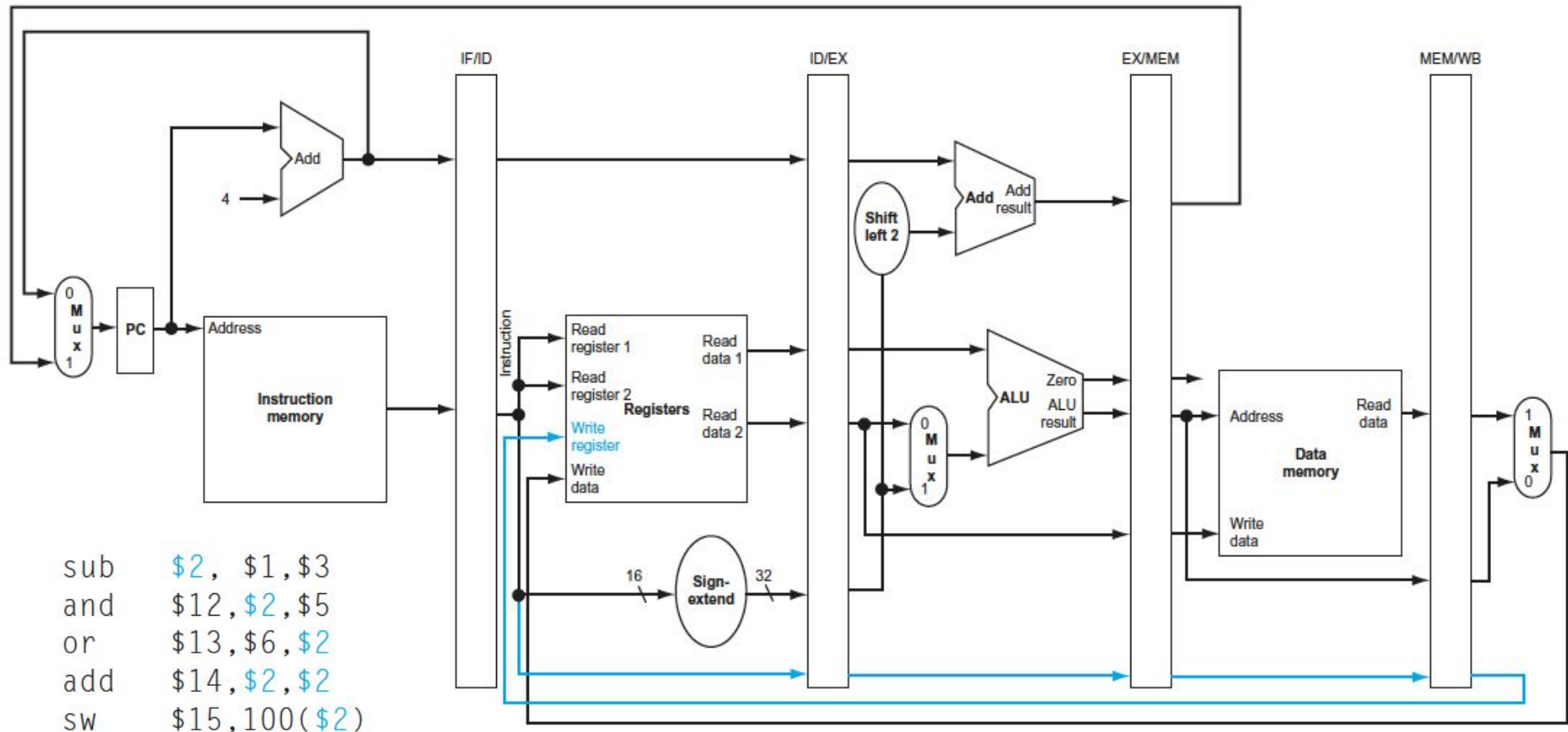
sub \$2, \$1, \$3
and \$12, \$2, \$5
or \$13, \$6, \$2
add \$14, \$2, \$2
sw \$15, 100(\$2)

Hazards de Dados

• Ciclo 2

and \$12, \$2, \$5

sub \$2, \$1, \$3



Hazards de Dados

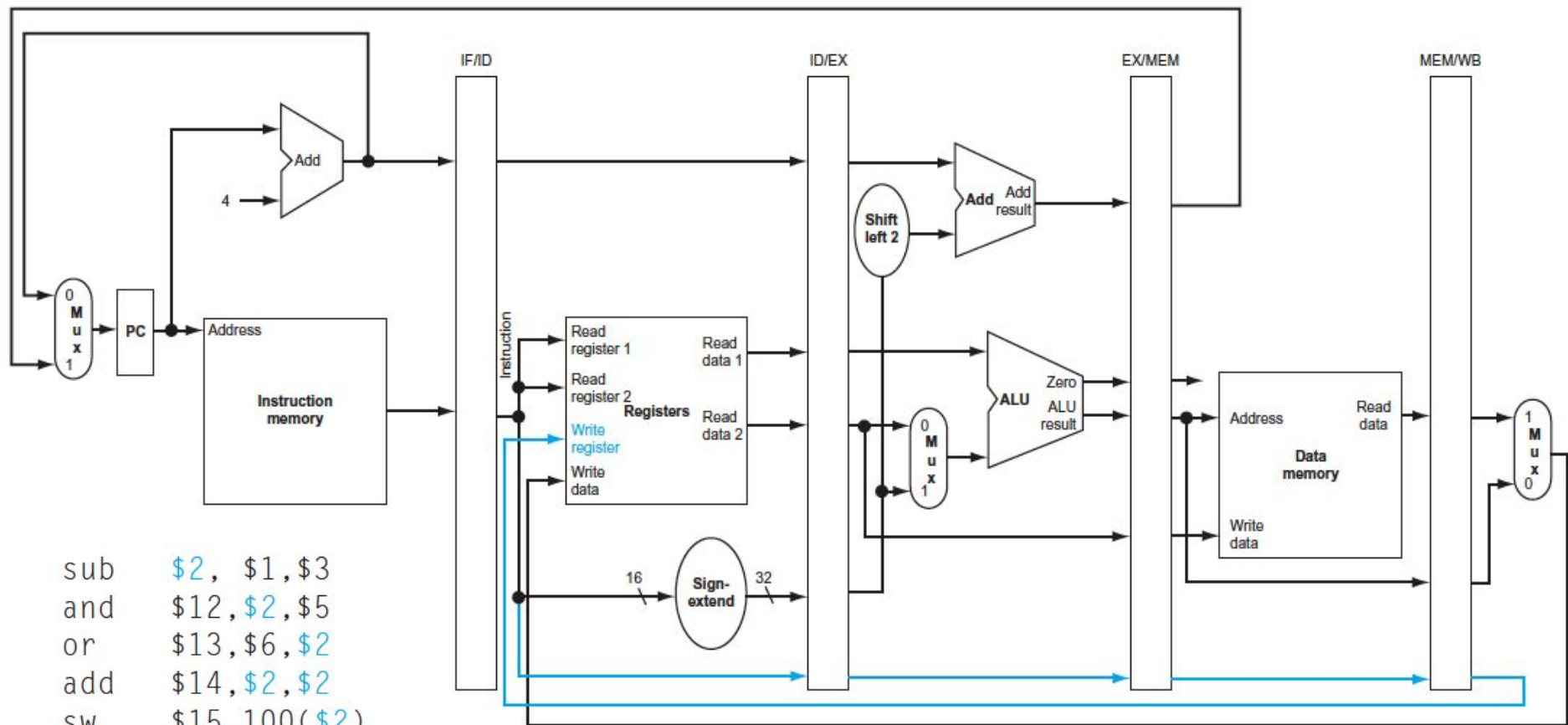
• Ciclo 3

or \$13, \$6, \$2

and \$12, \$2, \$5

sub \$2, \$1, \$3

sub \$2, \$1, \$3
and \$12, \$2, \$5
or \$13, \$6, \$2
add \$14, \$2, \$2
sw \$15, 100(\$2)



Hazards de Dados

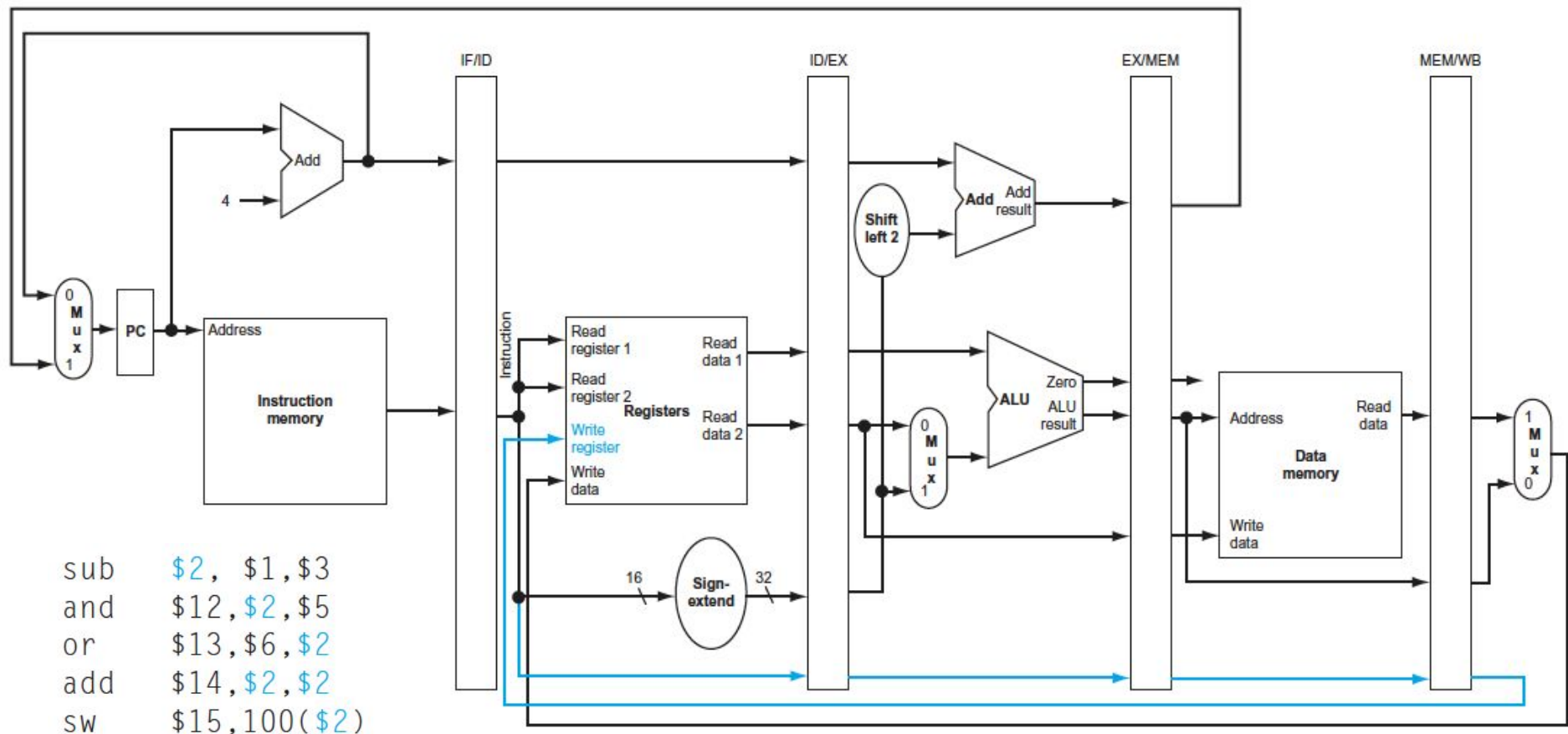
• Ciclo 4 – Pipeline Stall

or \$13, \$6, \$2

and \$12, \$2, \$5

Bolha

sub \$2, \$1, \$3



Hazards de Dados

• Ciclo 5 – Pipeline Stall

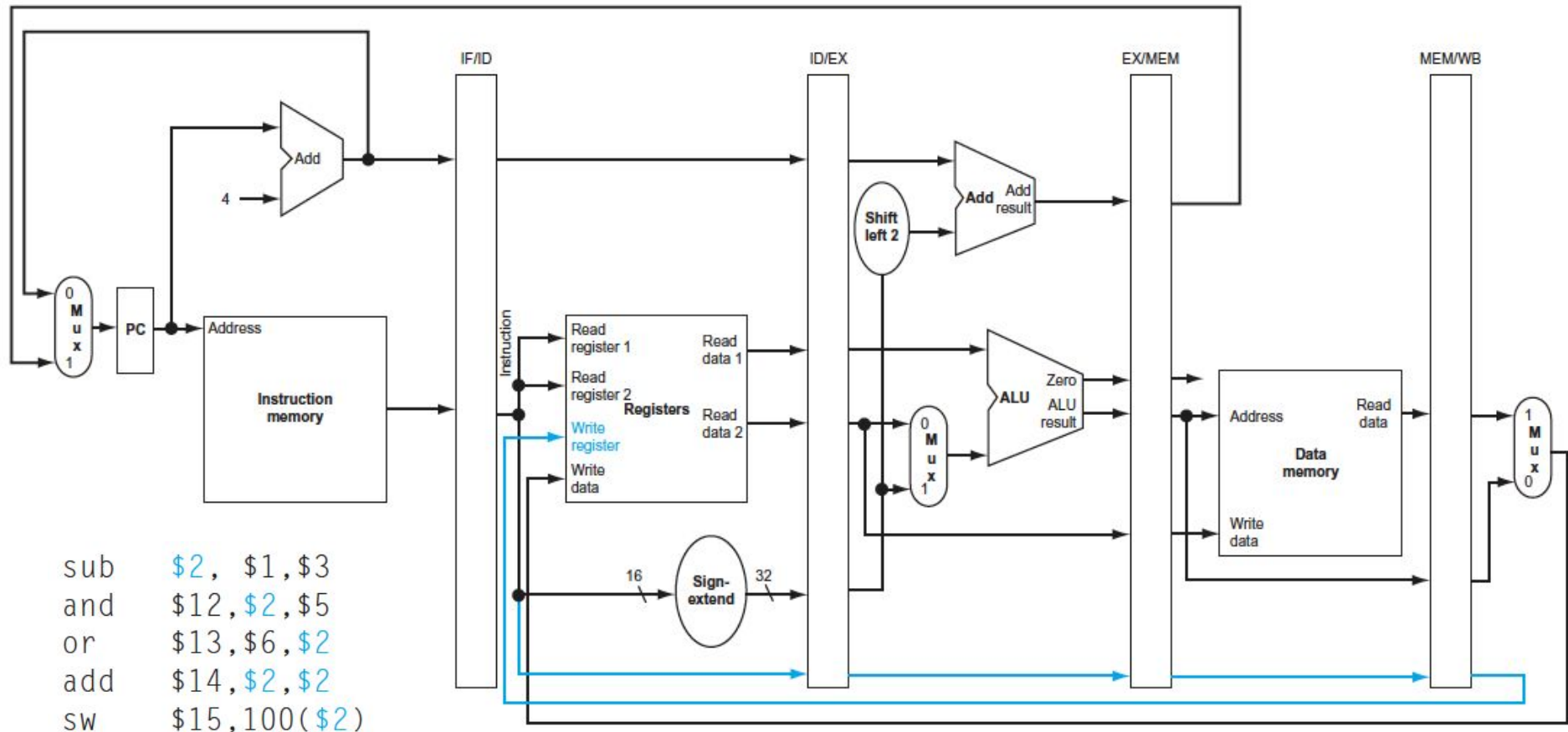
or \$13, \$6, \$2

and \$12, \$2, \$5

Bolha

Bolha

sub \$2, \$1, \$3



Hazards de Dados

• Ciclo 6

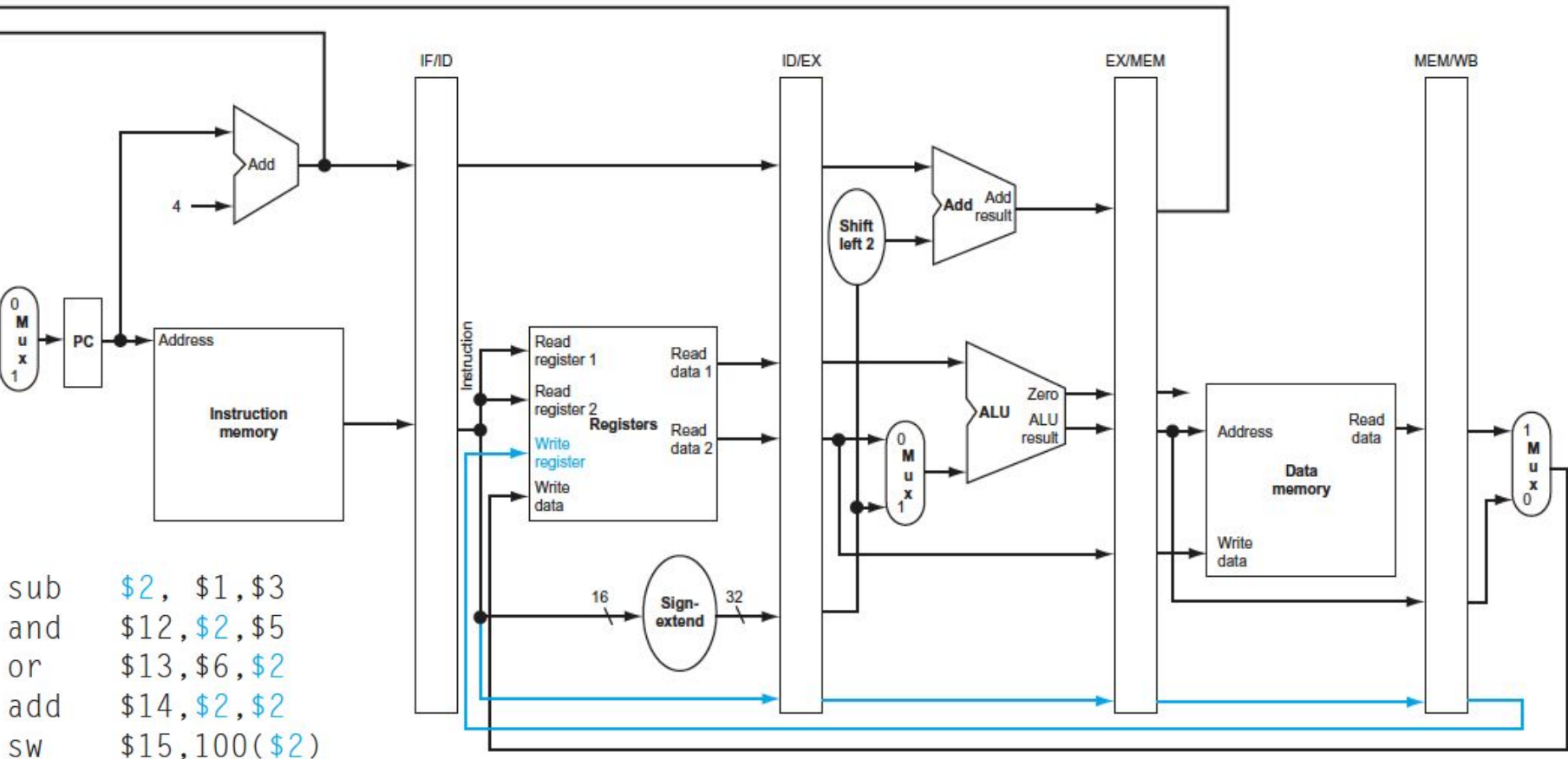
add \$14, \$2, \$2

or \$13, \$6, \$2

and \$12, \$2, \$5

Bolha

Bolha



Hazards de Dados

• Ciclo 7

sw \$15, 100(\$2)

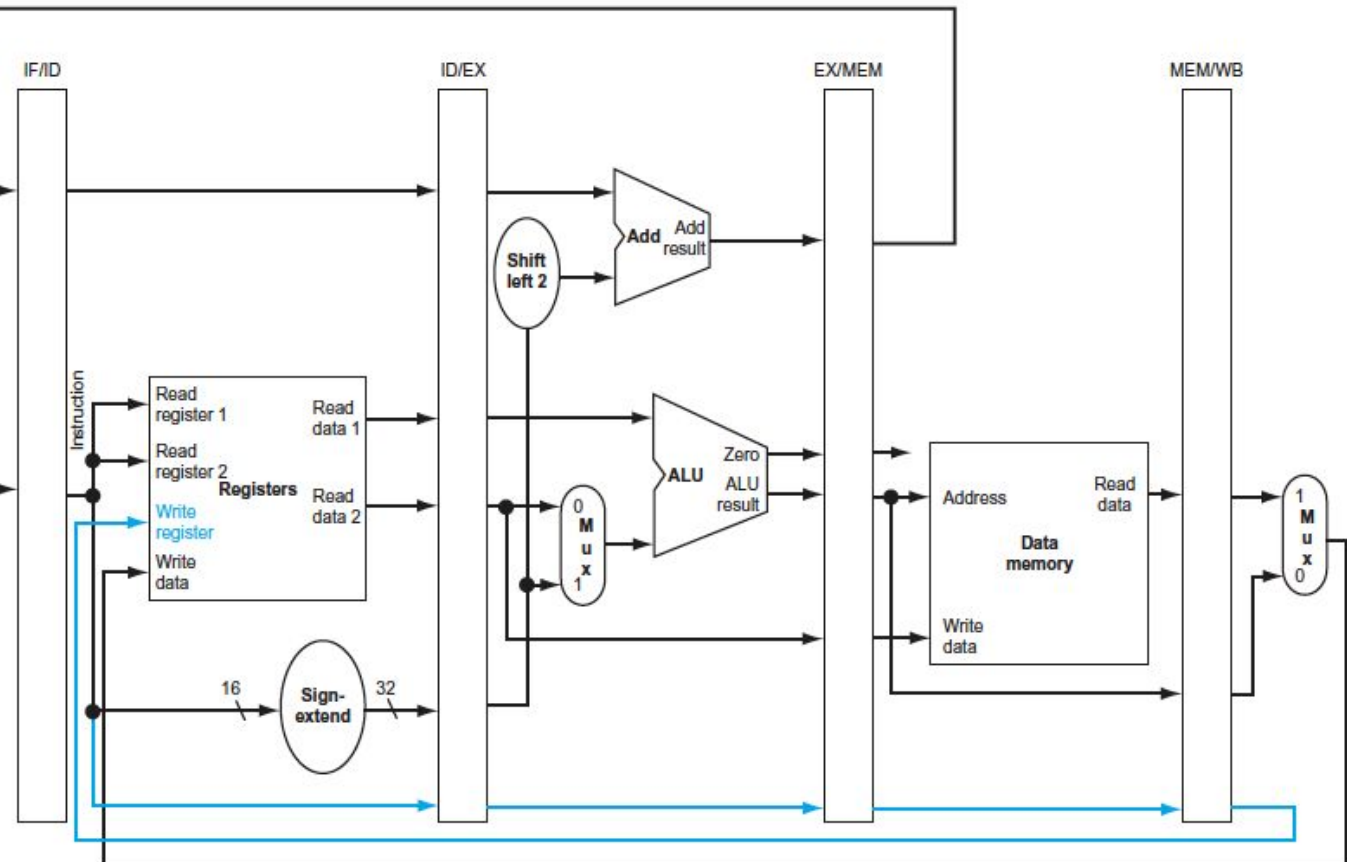
add \$14, \$2, \$2

or \$13, \$6, \$2

and \$12, \$2, \$5

Bolha

sub \$2, \$1, \$3
and \$12, \$2, \$5
or \$13, \$6, \$2
add \$14, \$2, \$2
sw \$15, 100(\$2)



Hazards de Dados

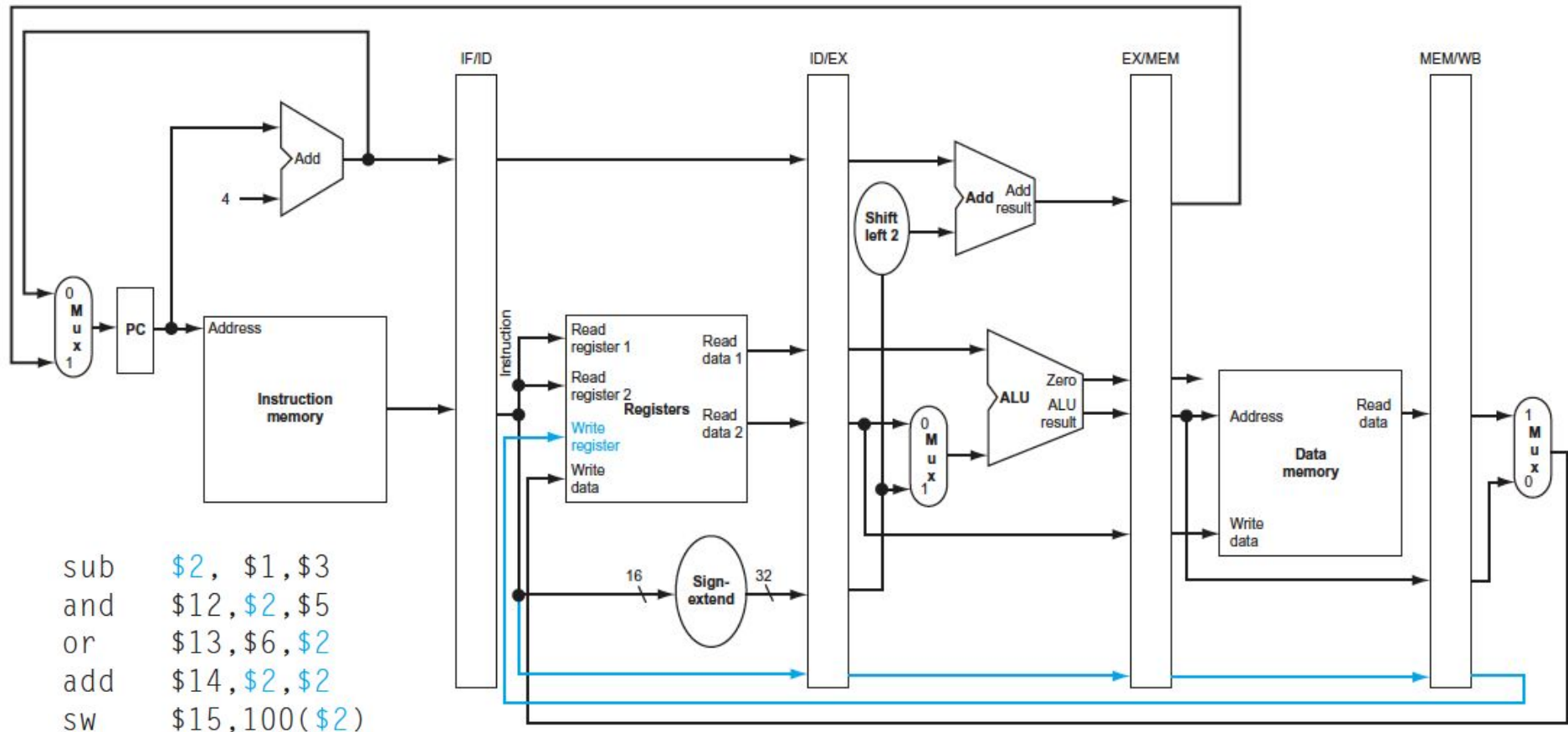
• Ciclo 8

sw \$15, 100(\$2)

add \$14, \$2, \$2

or \$13, \$6, \$2

and \$12, \$2, \$5



sub \$2, \$1, \$3
and \$12, \$2, \$5
or \$13, \$6, \$2
add \$14, \$2, \$2
sw \$15, 100(\$2)

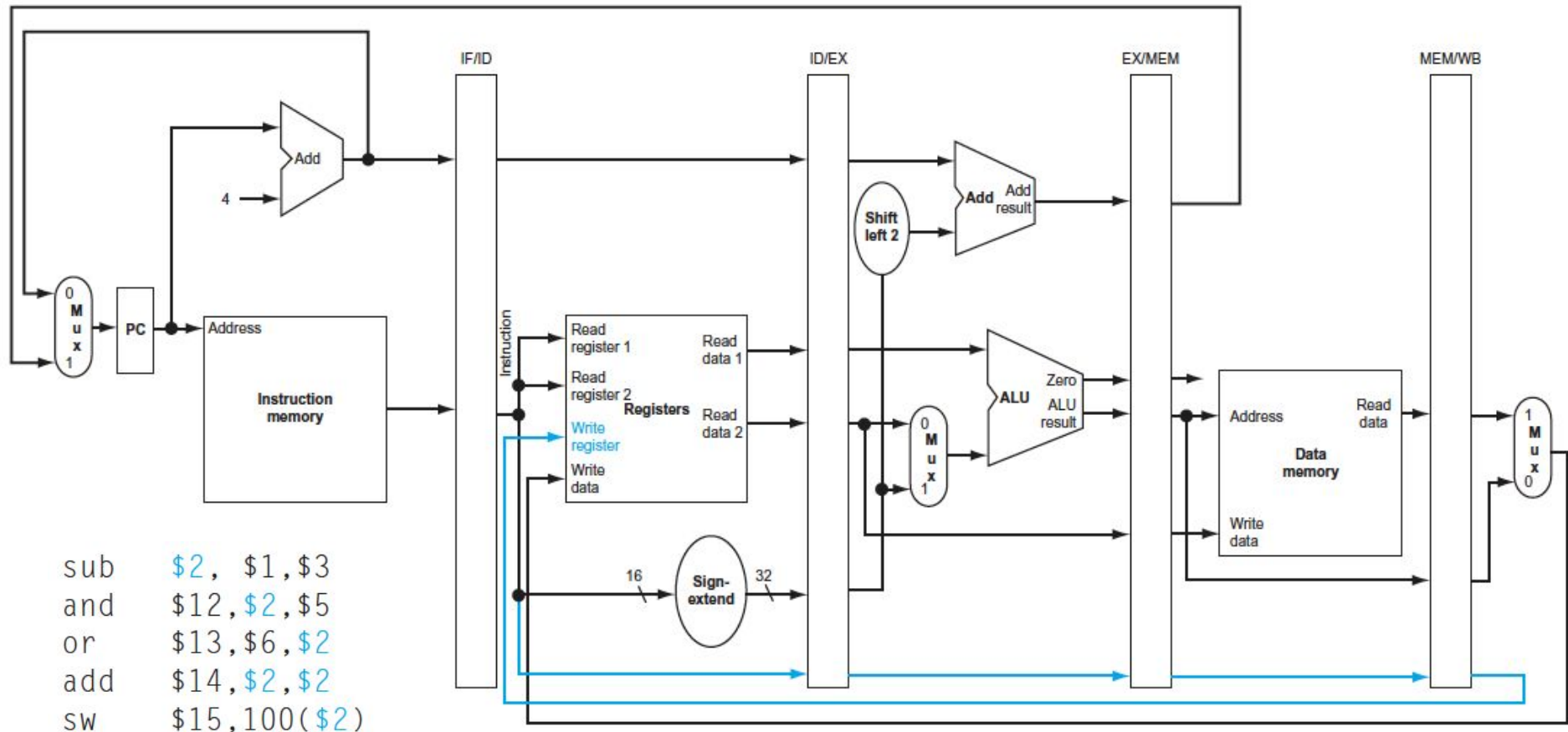
Hazards de Dados

• Ciclo 9

sw \$15, 100(\$2)

add \$14, \$2, \$2

or \$13, \$6, \$2



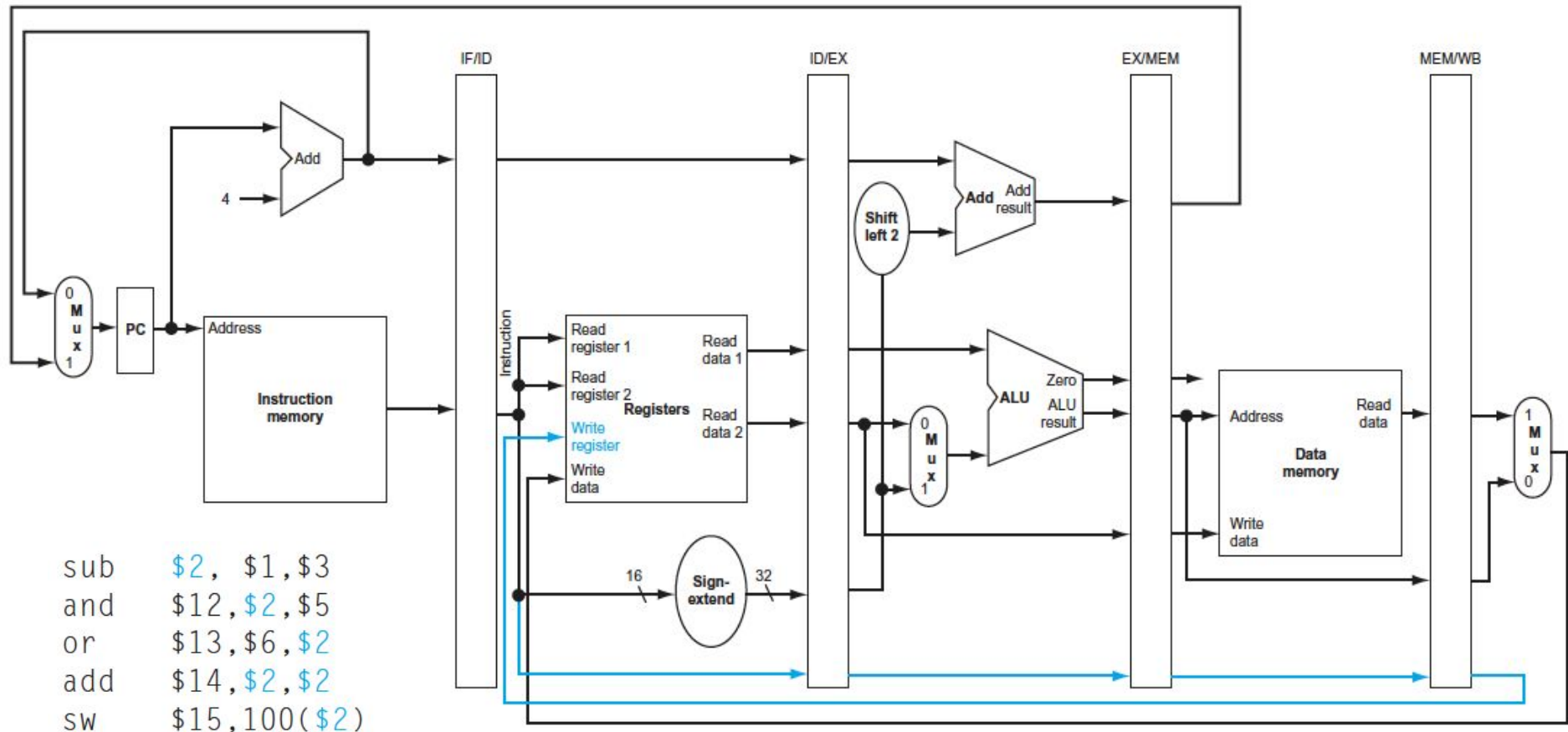
sub \$2, \$1, \$3
and \$12, \$2, \$5
or \$13, \$6, \$2
add \$14, \$2, \$2
sw \$15, 100(\$2)

Hazards de Dados

• Ciclo 10

sw \$15, 100(\$2)

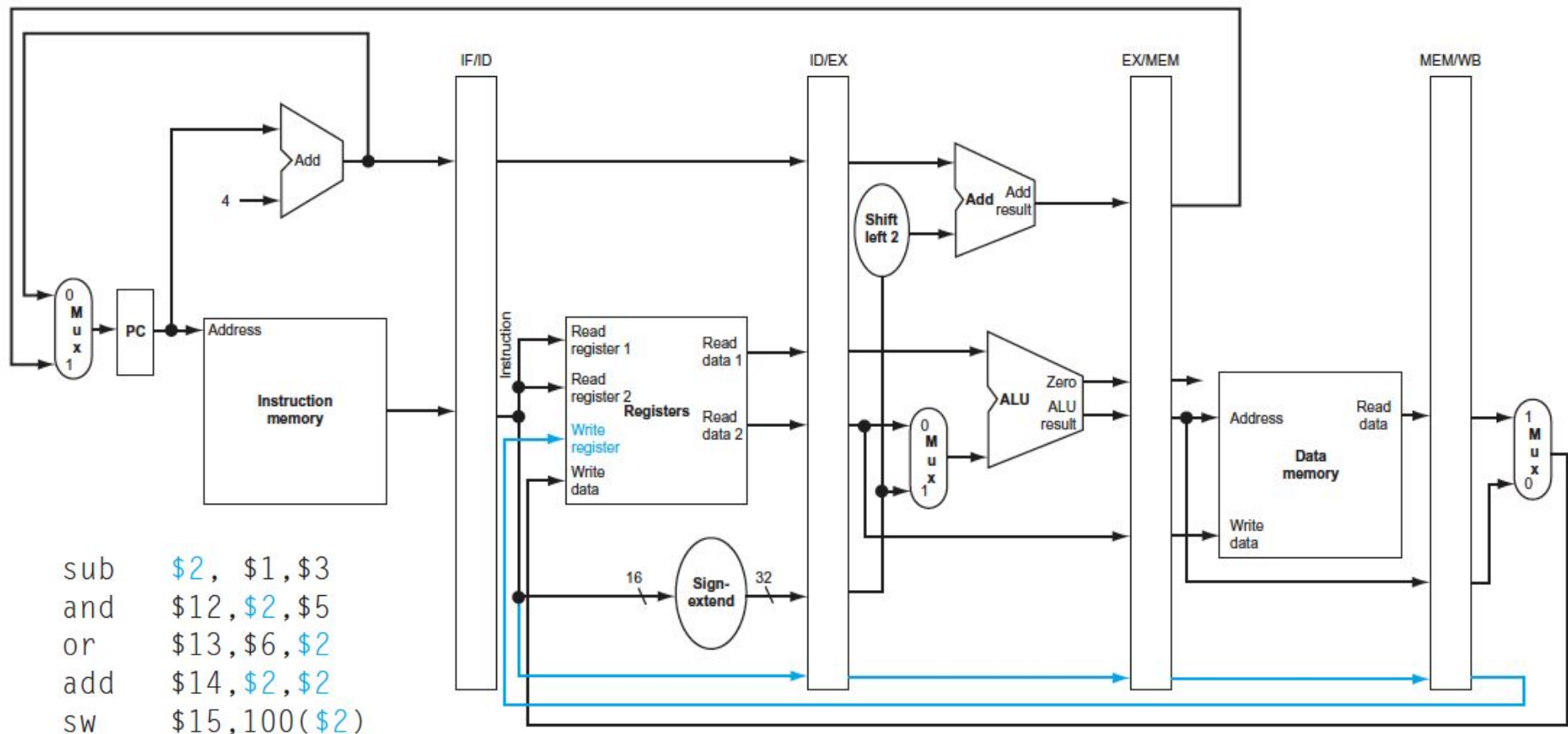
add \$14, \$2, \$2



Hazards de Dados

• Ciclo 11

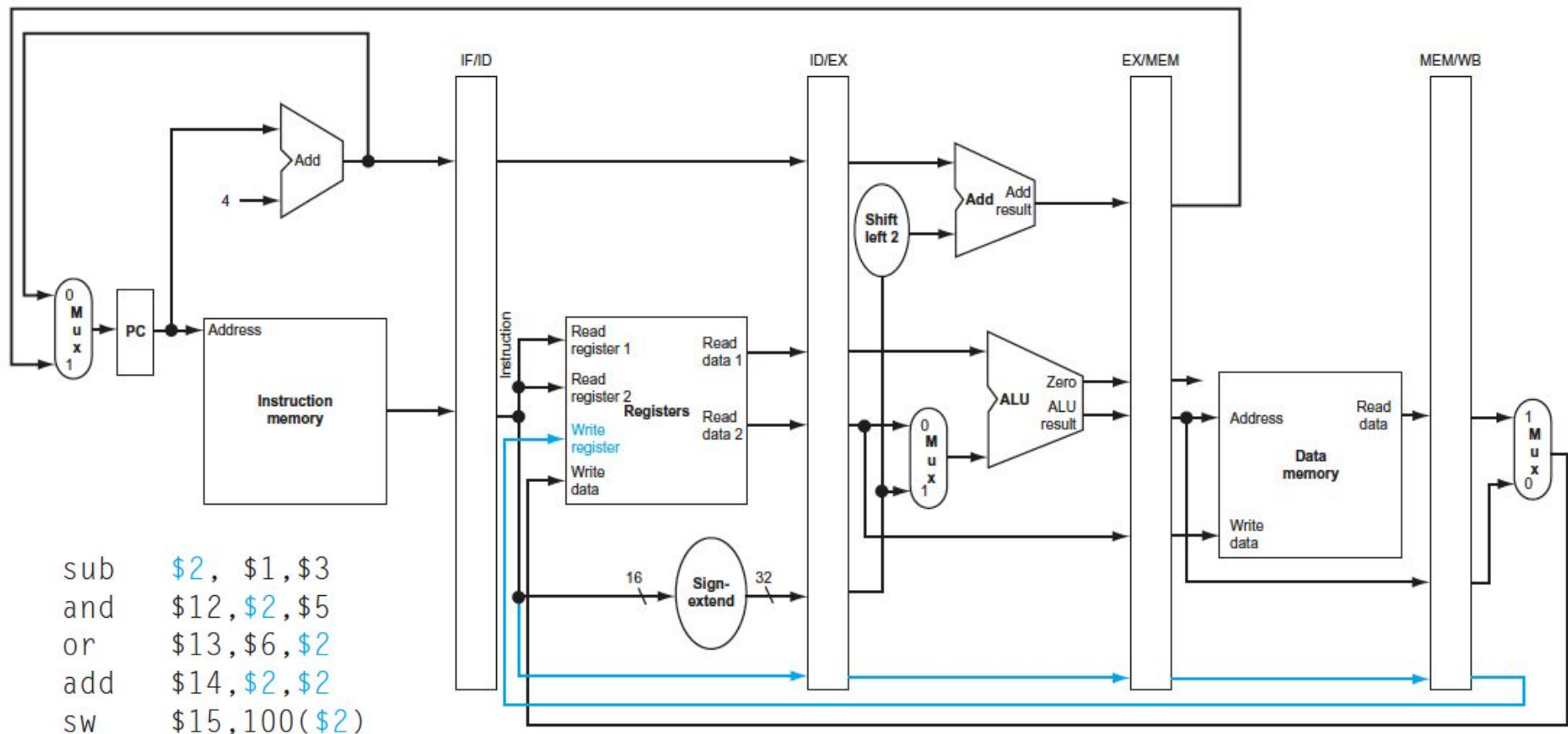
sw \$15, 100(\$2)



Hazards de Dados

- $CPI = \text{Ciclos/Instruções} = 11/5 = 2,2$

sw \$15, 100(\$2)



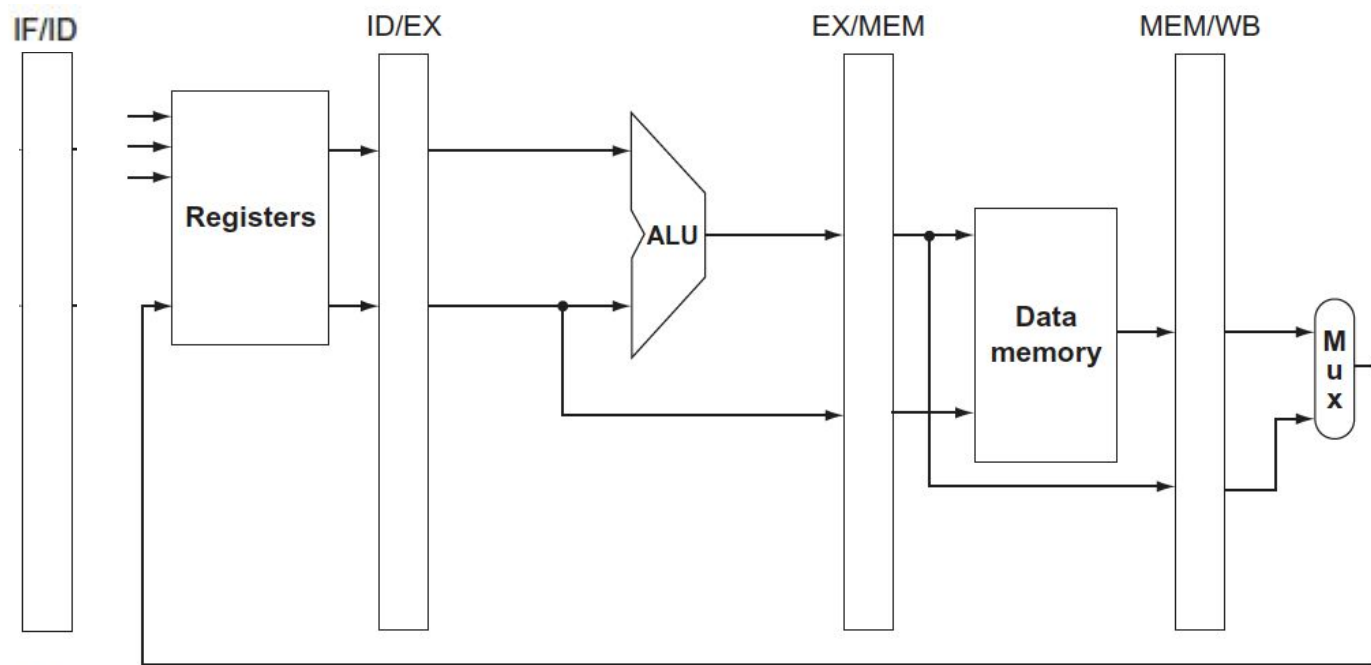
Hazard de Dados - Forwarding

- Adiantamento de resultados entre estágios do pipeline
- Hardware de controle que decide quando adiantar dados deve ser inserido
- Hardware para adiantar os dados no bloco operativo deve ser inserido

Hazards de Dados

• Ciclo 1

sub \$2, \$1, \$3



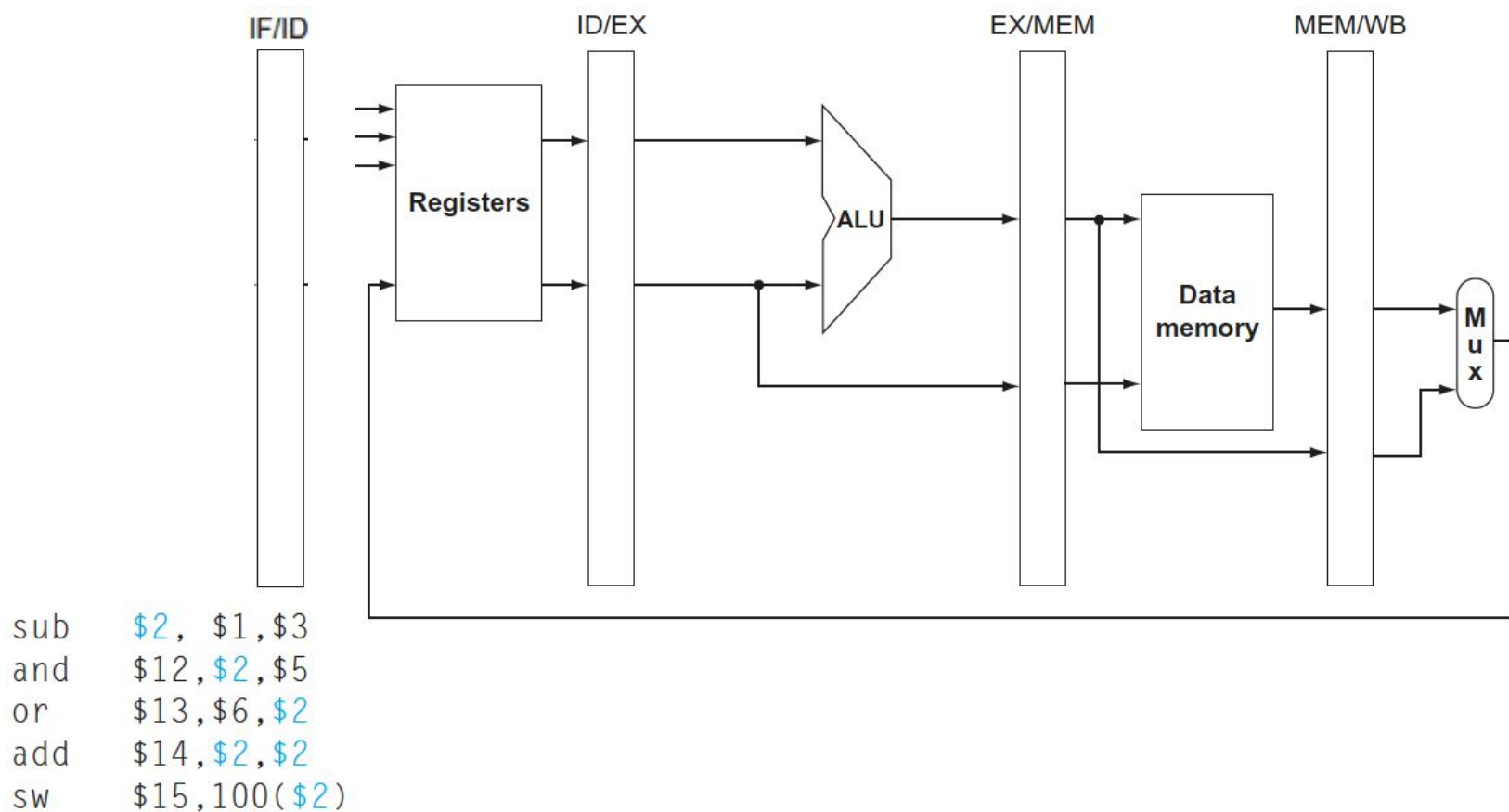
```
sub    $2, $1, $3
and    $12, $2, $5
or     $13, $6, $2
add    $14, $2, $2
sw     $15, 100($2)
```

Hazards de Datos

• Ciclo 2

and \$12, \$2, \$5

sub \$2, \$1, \$3



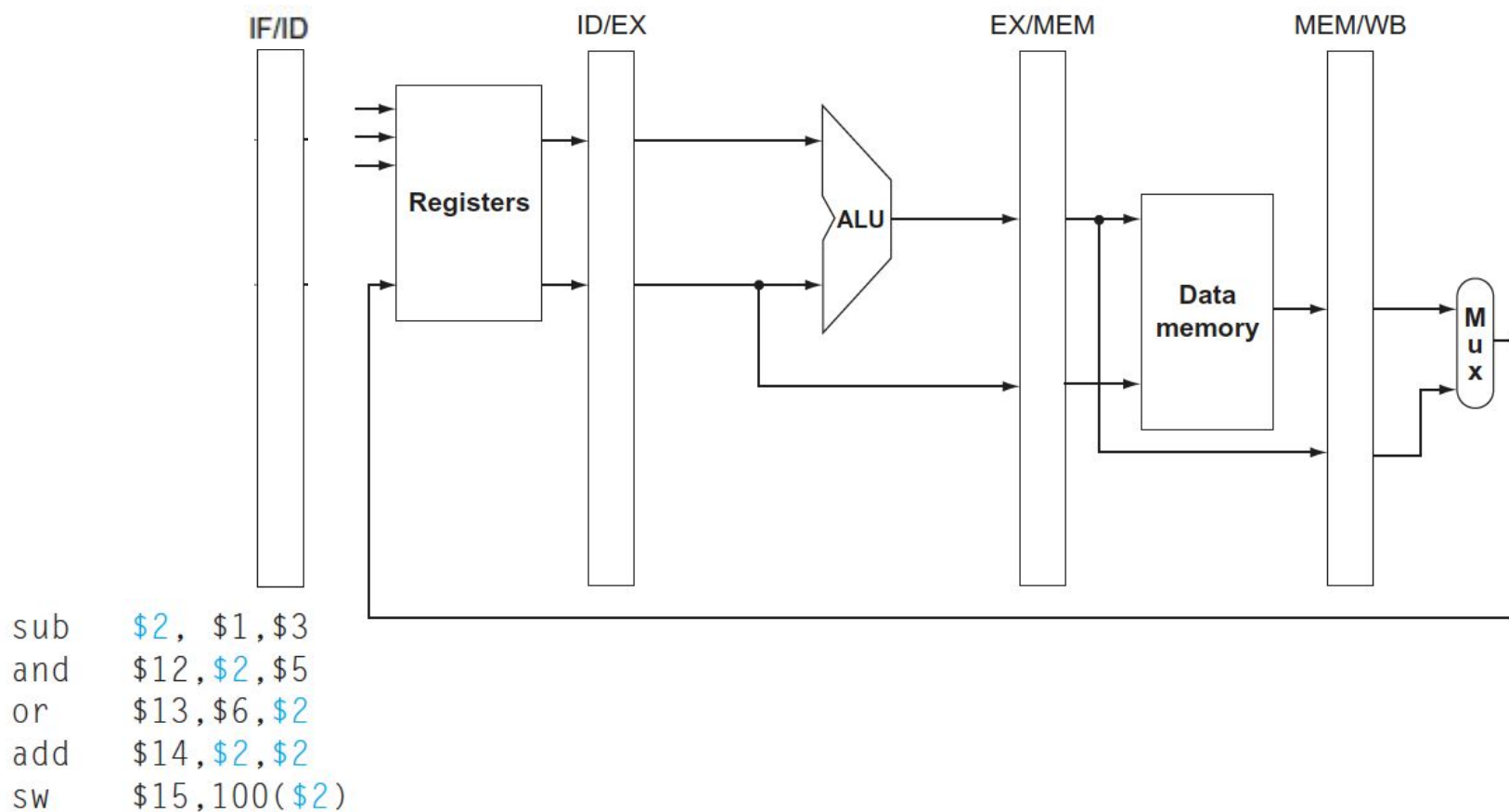
Hazards de Datos

• Ciclo 3

or \$13, \$6, \$2

and \$12, \$2, \$5

sub \$2, \$1, \$3



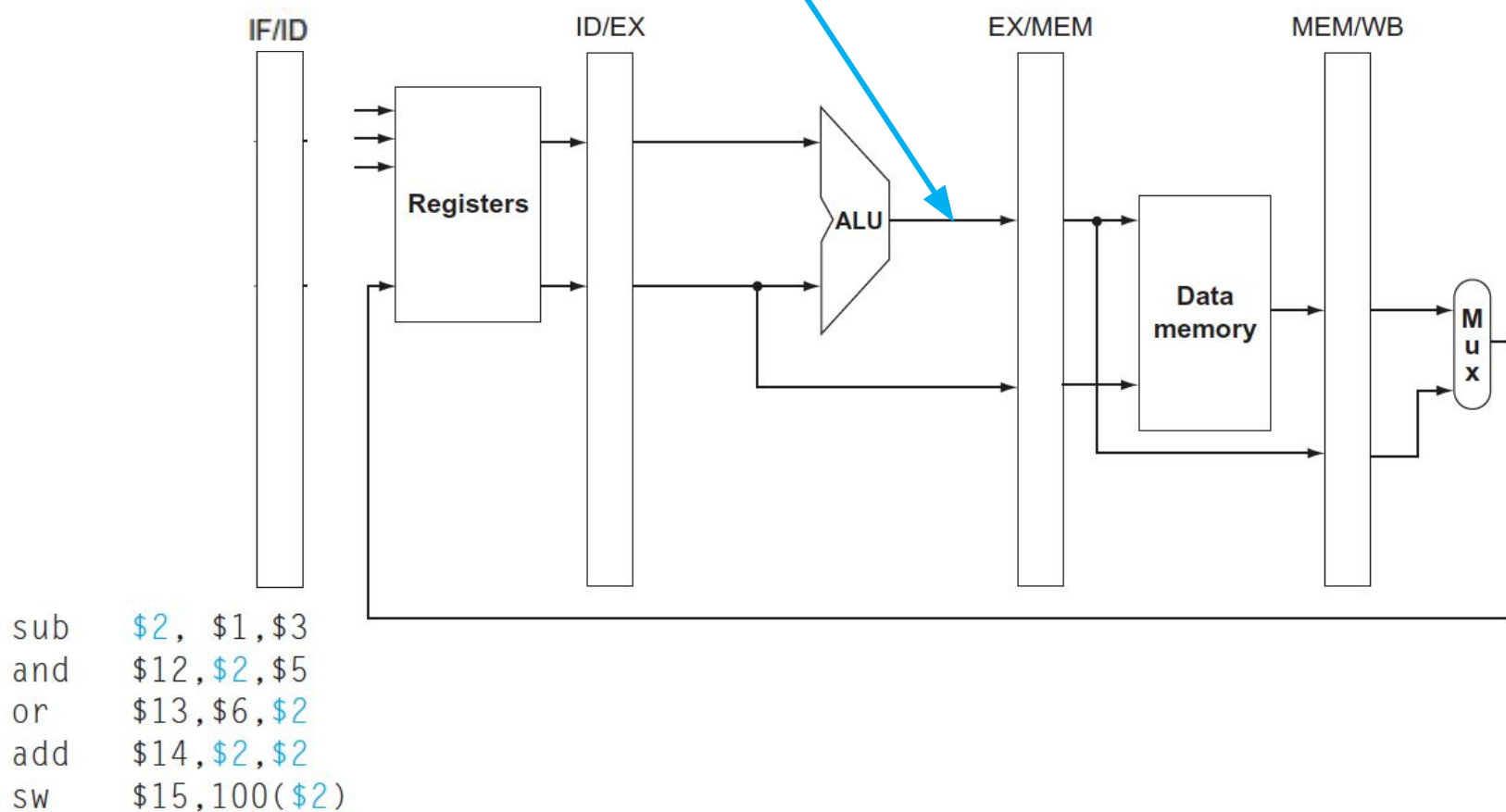
Hazards de Datos

• Ciclo 3

or \$13, \$6, \$2

and \$12, \$2, \$5

sub \$2, \$1, \$3



Hazards de Dados

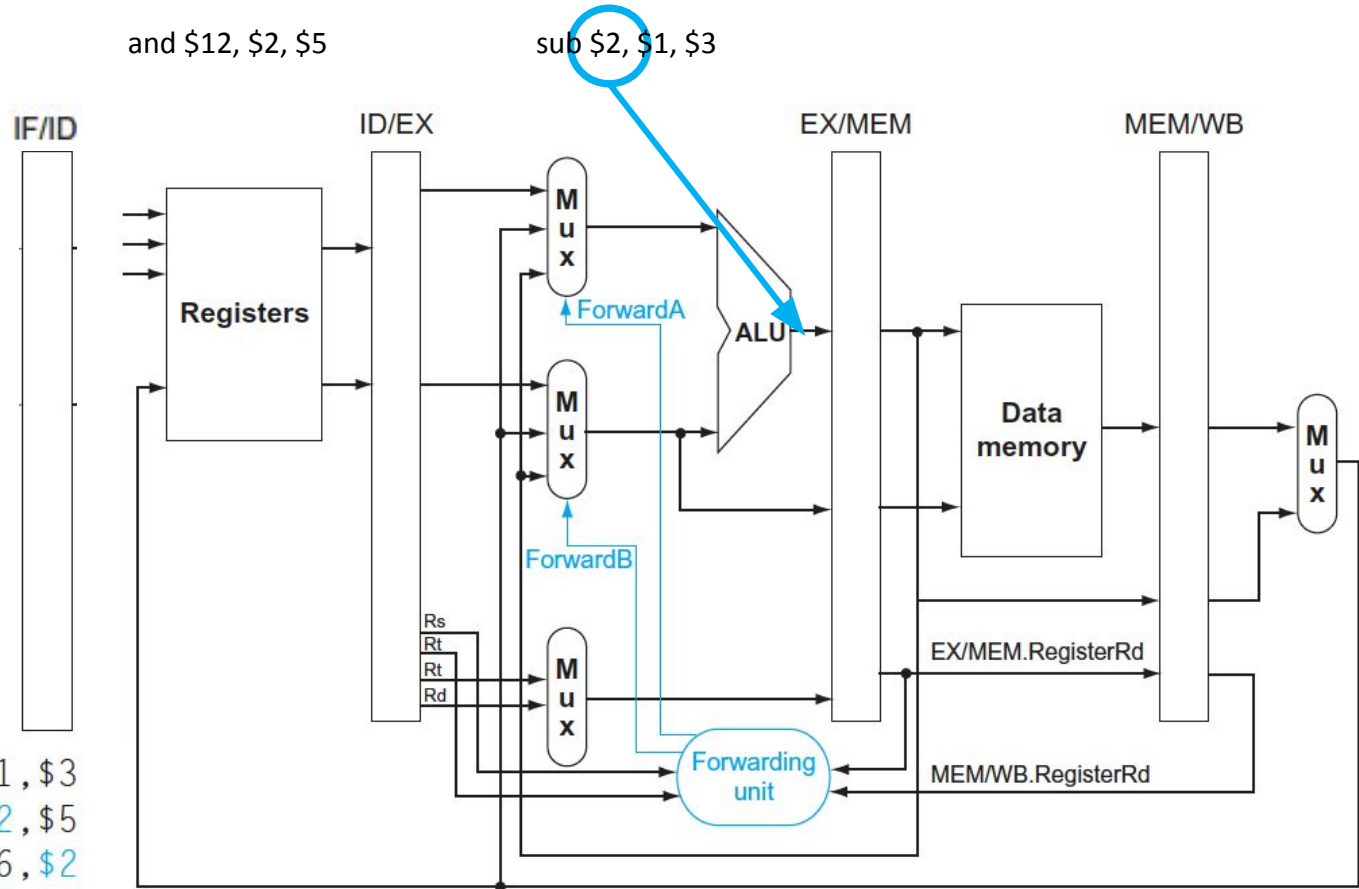
• Ciclo 3

or \$13, \$6, \$2

and \$12, \$2, \$5

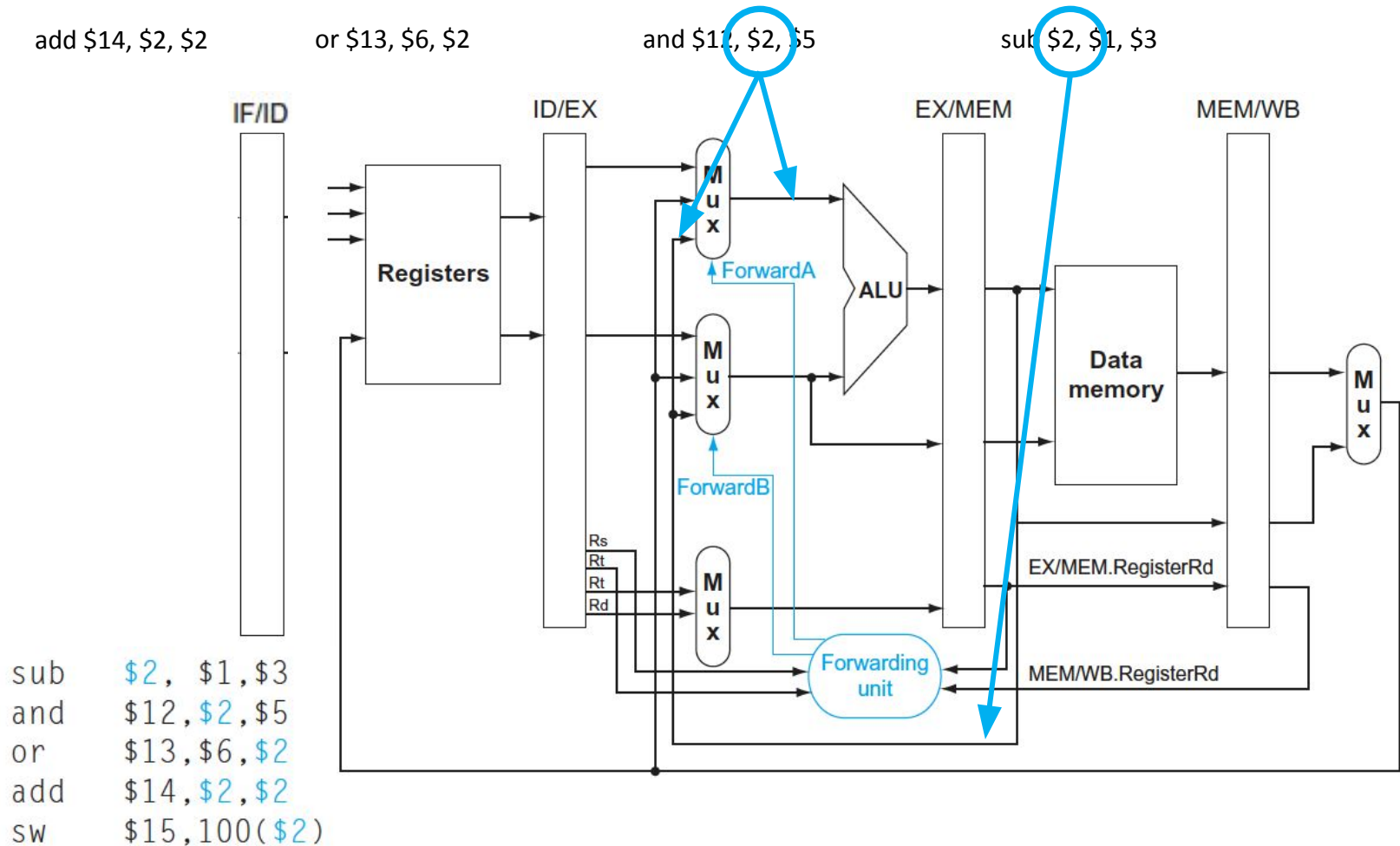
sub \$2, \$1, \$3

sub \$2, \$1, \$3
and \$12, \$2, \$5
or \$13, \$6, \$2
add \$14, \$2, \$2
sw \$15, 100(\$2)



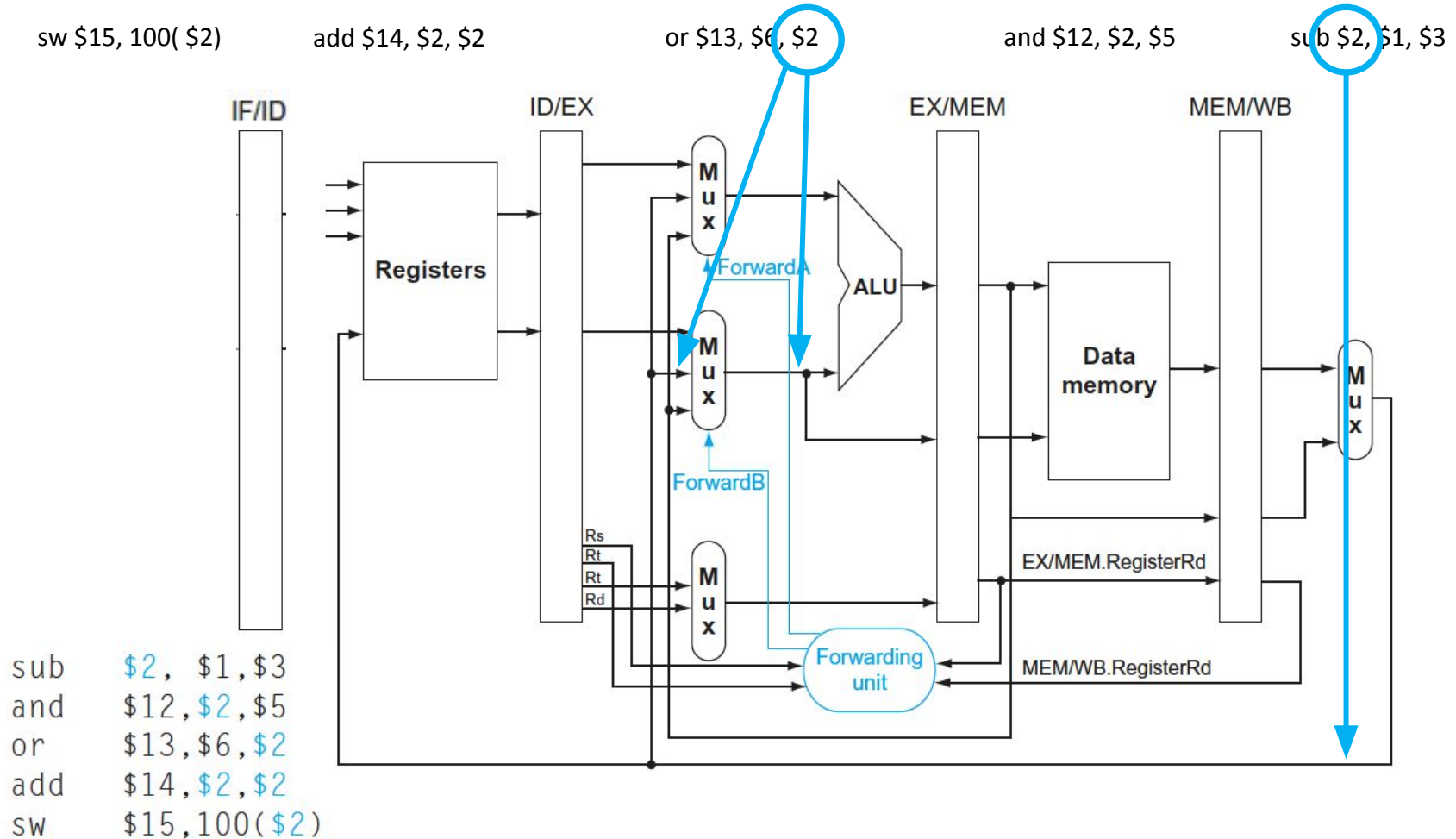
Hazards de Dados

• Ciclo 4 – Forwarding Ativo – MEM -> EX



Hazards de Dados

- Ciclo 5 – Forwarding Ativo – WB -> EX



Hazards de Dados

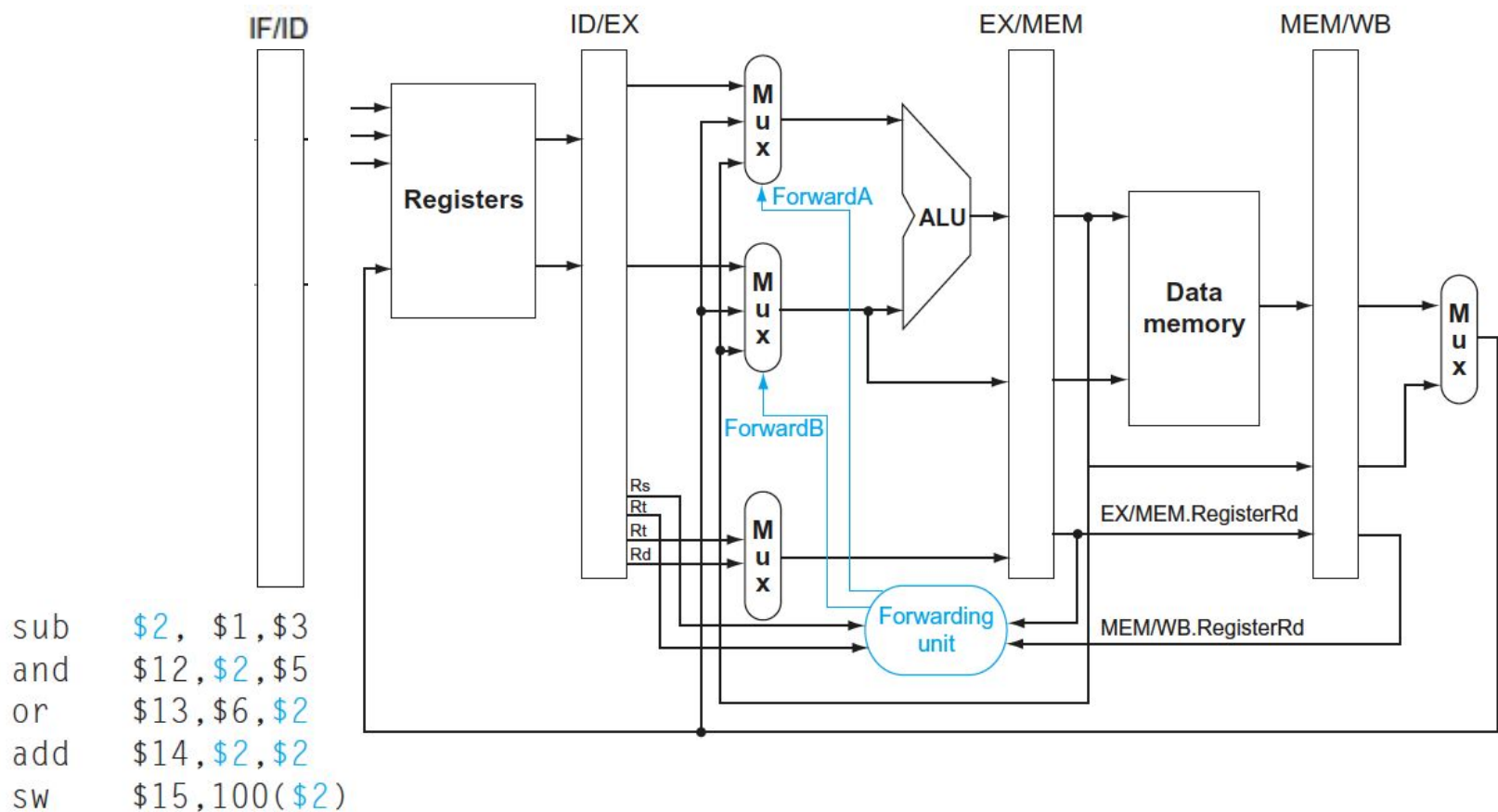
• Ciclo 6

sw \$15, 100(\$2)

add \$14, \$2, \$2

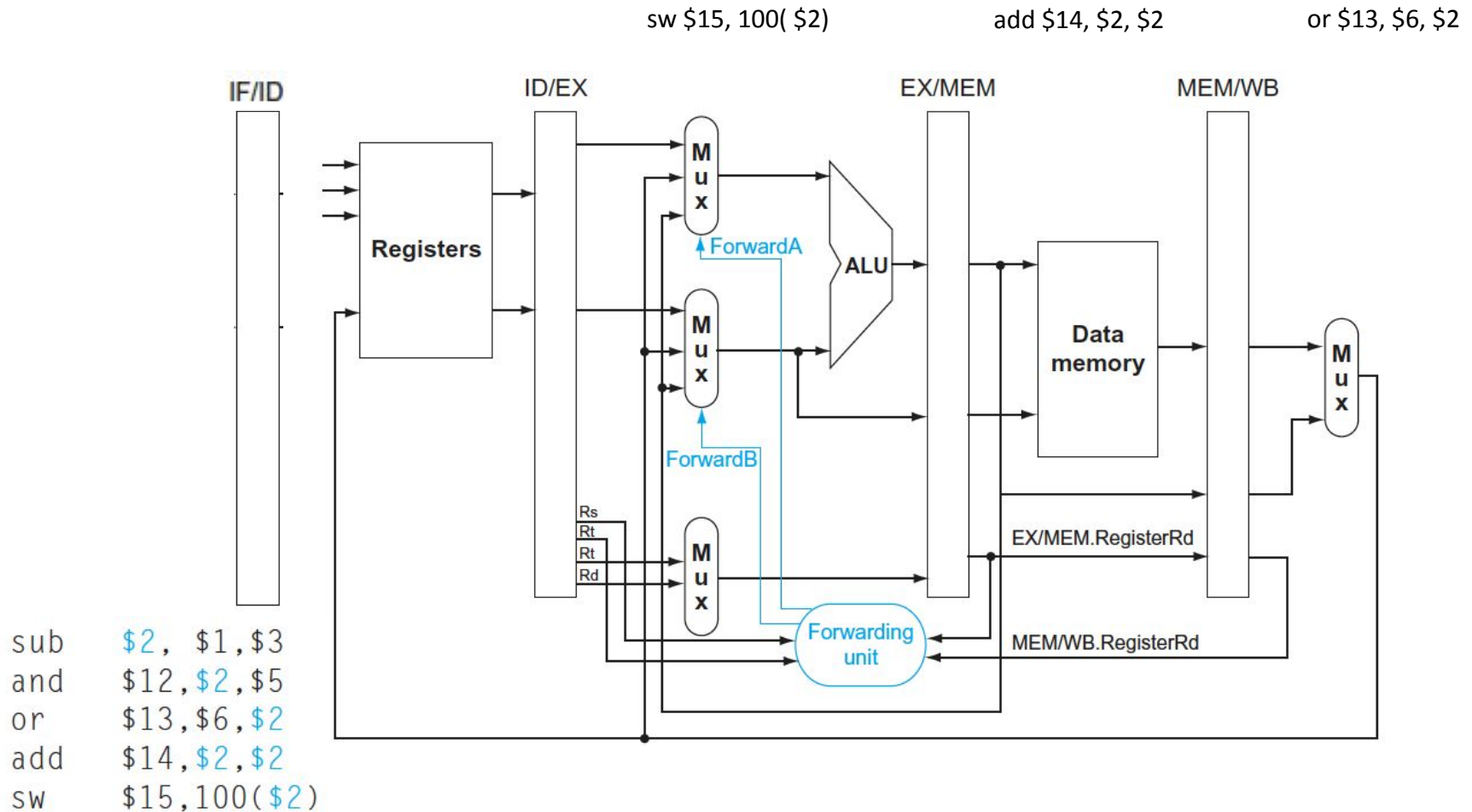
or \$13, \$6, \$2

and \$12, \$2, \$5



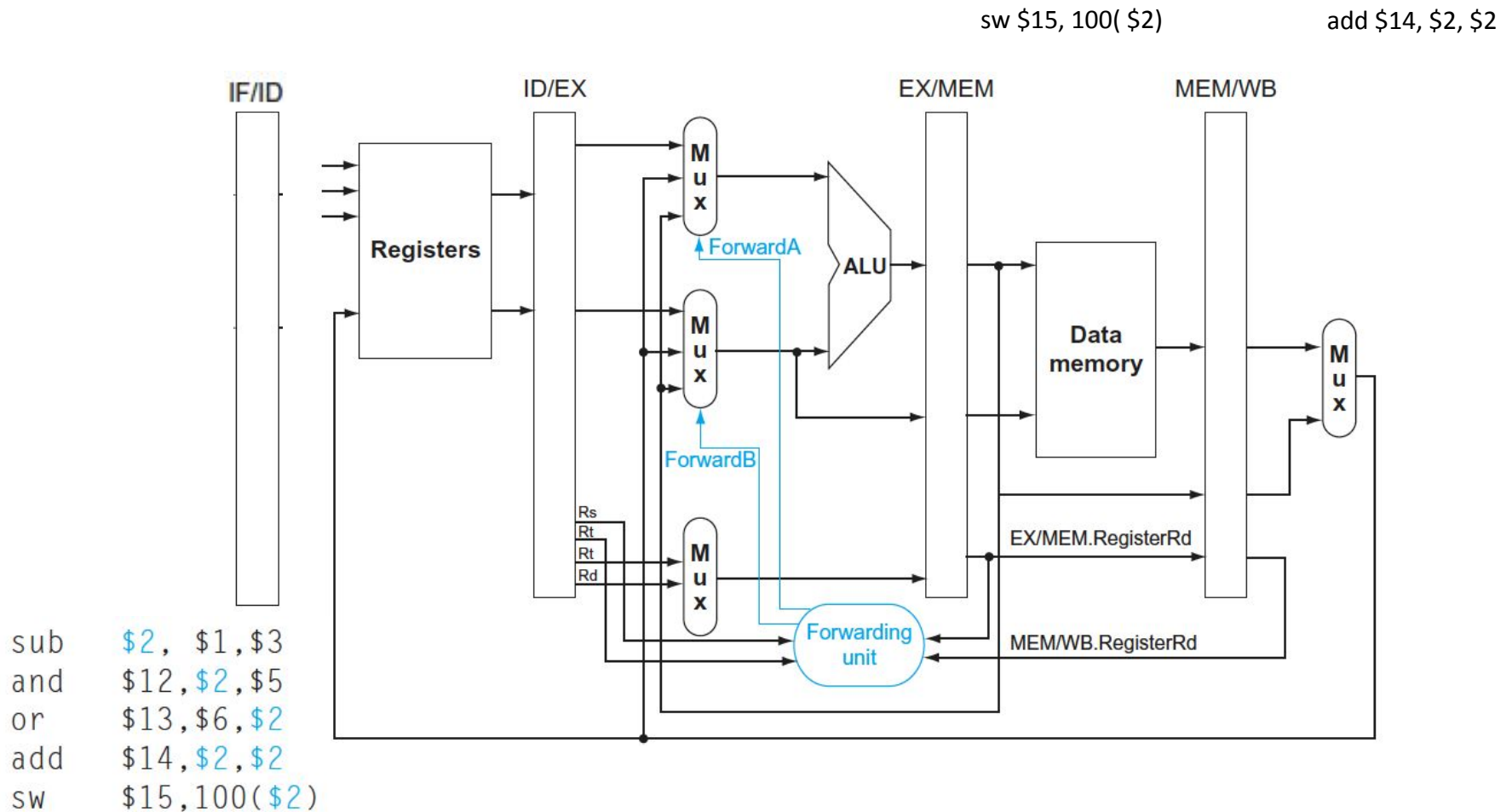
Hazards de Dados

• Ciclo 7



Hazards de Datos

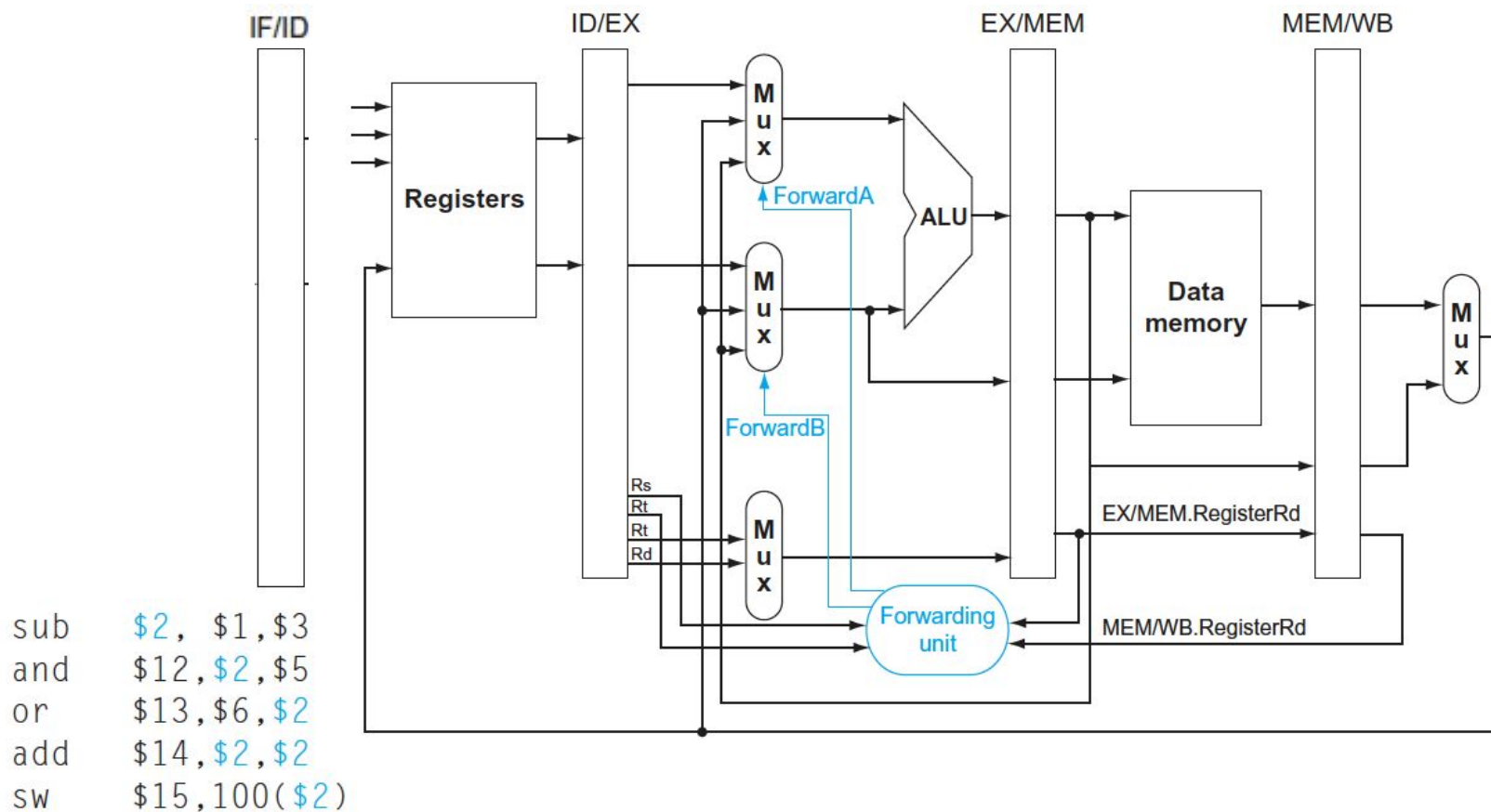
• Ciclo 8



Hazards de Dados

- Ciclo 9 - Com Forwarding

sw \$15, 100(\$2)



Hazards de Dados

- Forwarding Unit

- Recebe registradores de leitura e escrita das instruções
- Realiza comparações entre registradores de escrita de instruções que estão em estágios posteriores com registradores de leitura de instruções que estão em estágios anteriores
- Resultado das comparações geram os controles dos muxes

1a. EX/MEM.RegisterRd = ID/EX.RegisterRs

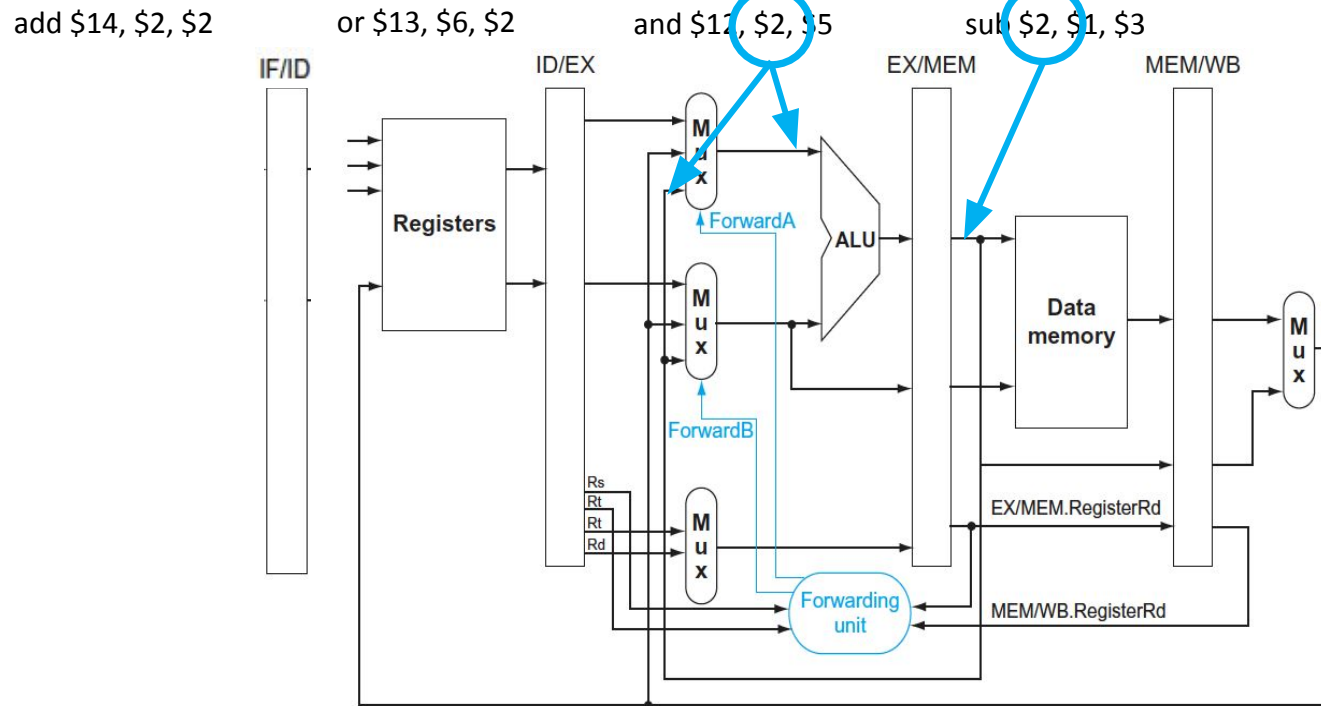
1b. EX/MEM.RegisterRd = ID/EX.RegisterRt

2a. MEM/WB.RegisterRd = ID/EX.RegisterRs

2b. MEM/WB.RegisterRd = ID/EX.RegisterRt

Hazards de Dados

- Adiantamento resultado da ALU que está em MEM→ALU



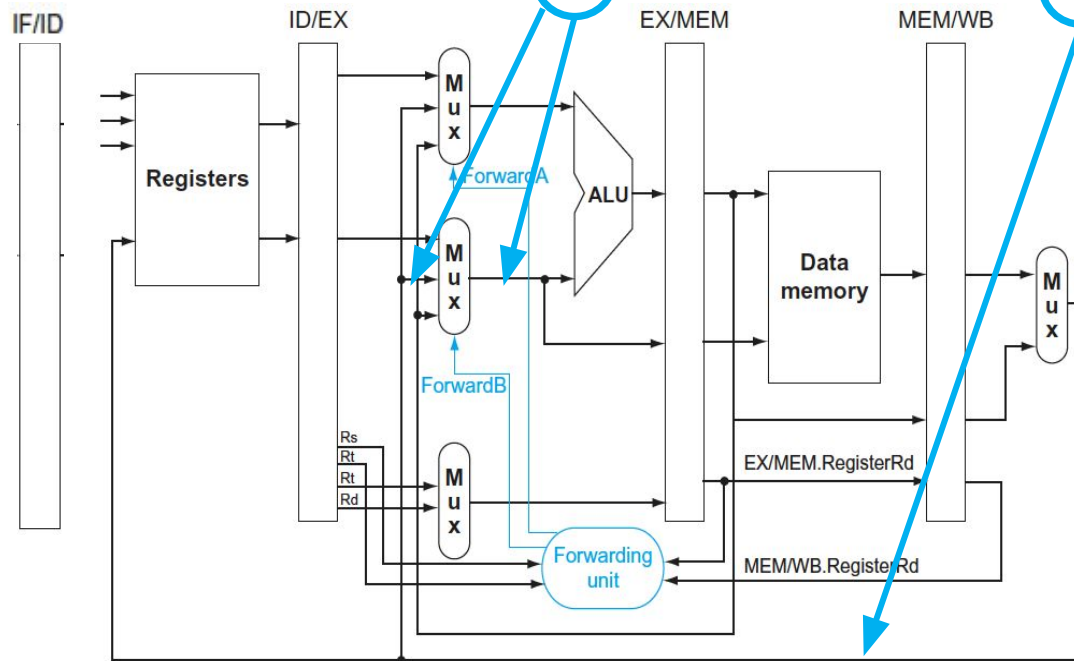
```
sub    $2, $1, $3
and    $12, $2, $5
or     $13, $6, $2
add    $14, $2, $2
sw     $15, 100($2)
```

```
if (EX/MEM.RegWrite
and (EX/MEM.RegisterRd ≠ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs)) ForwardA = 10
if (EX/MEM.RegWrite
and (EX/MEM.RegisterRd ≠ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRt)) ForwardB = 10
```

Hazards de Dados

- Adiantamento resultado da ALU que está em WB→ALU

sw \$15, 100(\$2) add \$14, \$2, \$2 or \$13, \$6, \$2 and \$12, \$2, \$5 sub \$2, \$1, \$3



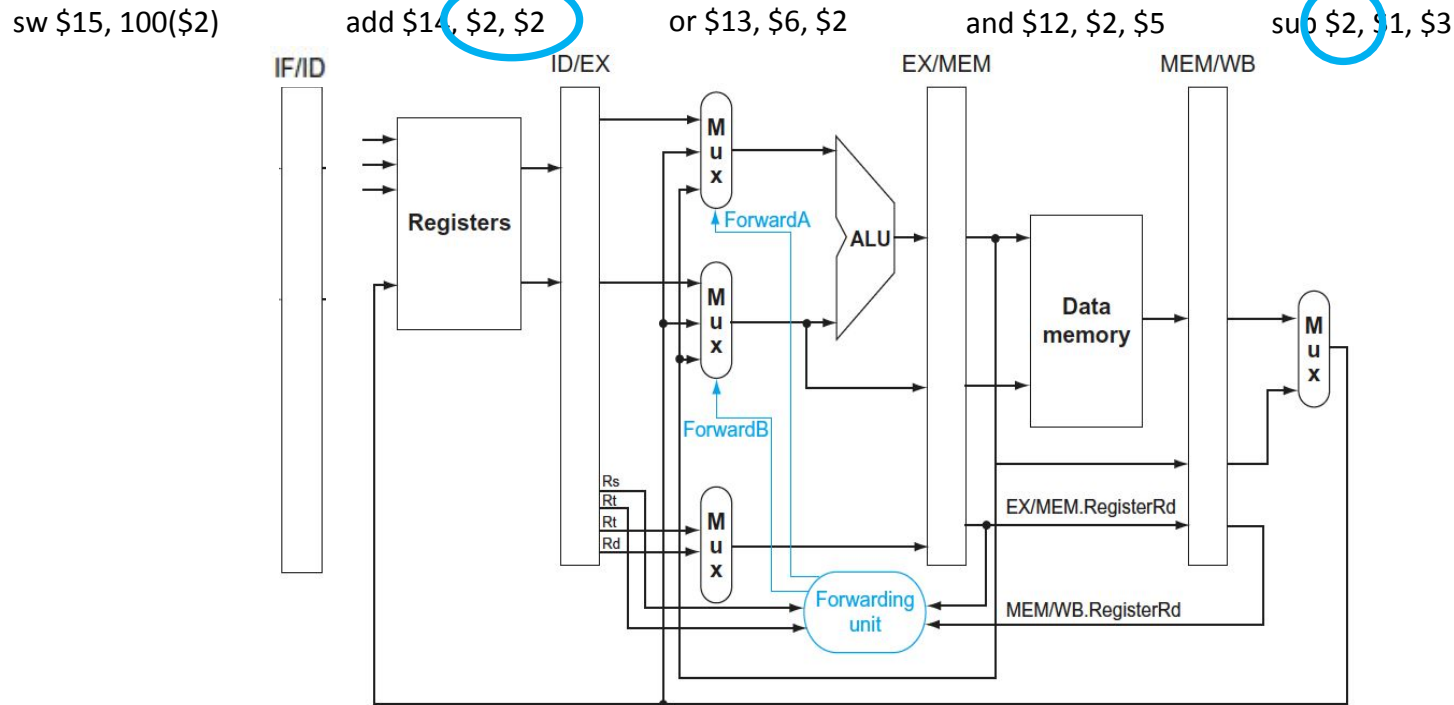
```
sub    $2, $1, $3
and    $12, $2, $5
or     $13, $6, $2
add    $14, $2, $2
sw     $15, 100($2)
```

```
if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd ≠ 0)
and (MEM/WB.RegisterRd = ID/EX.RegisterRs)) ForwardA = 01

if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd ≠ 0)
and (MEM/WB.RegisterRd = ID/EX.RegisterRt)) ForwardB = 01
```

Hazards de Dados

- Não existe Hazard – sub escreve e add lê no mesmo ciclo



sub	\$2, \$1, \$3
and	\$12, \$2, \$5
or	\$13, \$6, \$2
add	\$14, \$2, \$2
sw	\$15, 100(\$2)

Hazards de Dados

- A sequência abaixo é executada corretamente com detecção de forwarding atual?

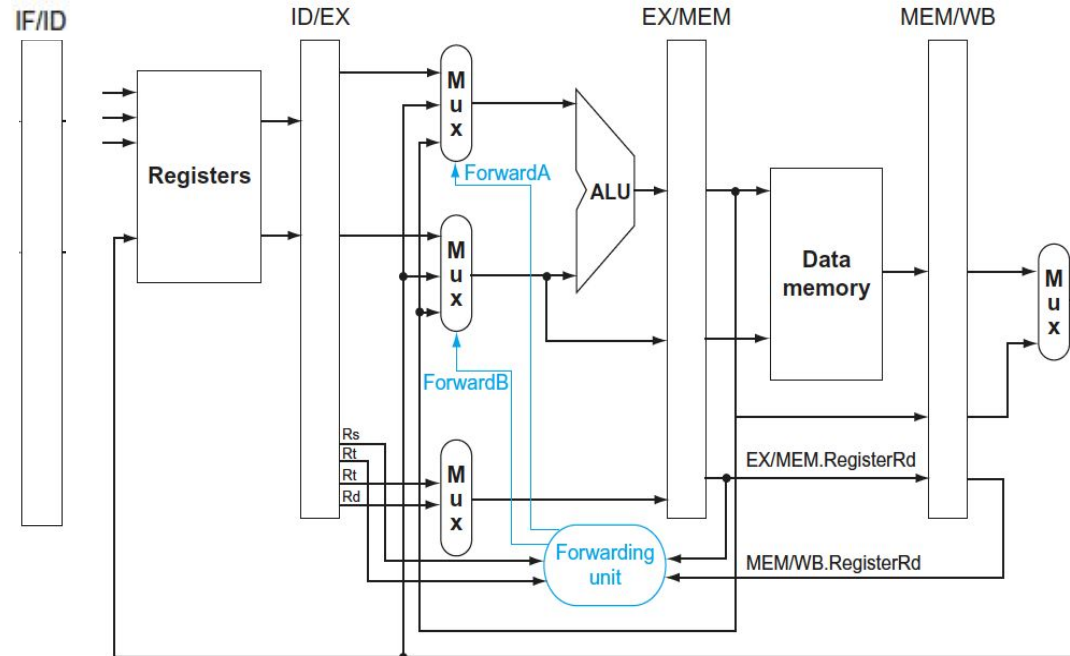
Correto

Errado

add \$1, \$1, \$4

add \$1, \$1, \$3

add \$1, \$1, \$2



```
add $1,$1,$2  
add $1,$1,$3  
add $1,$1,$4
```

Hazards de Dados

- Verifica se existe dependência entre MEM e EX e evita que o forwarding seja feito de WB para EX

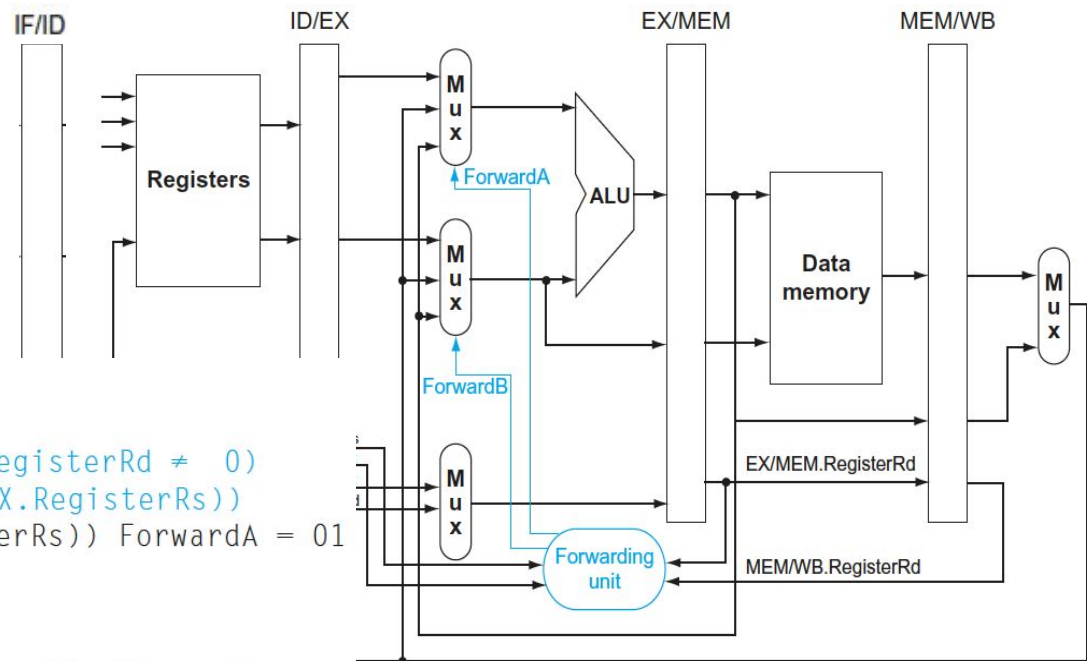
add \$1, \$1, \$4

Correto

add \$1, \$1, \$3

Errado

add \$1, \$1, \$2



```
if (MEM/WB.RegWrite  
and (MEM/WB.RegisterRd ≠ 0)  
and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)  
and (EX/MEM.RegisterRd ≠ ID/EX.RegisterRs))  
and (MEM/WB.RegisterRd = ID/EX.RegisterRs)) ForwardA = 01
```

```
if (MEM/WB.RegWrite  
and (MEM/WB.RegisterRd ≠ 0)  
and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)  
and (EX/MEM.RegisterRd ≠ ID/EX.RegisterRt))  
and (MEM/WB.RegisterRd = ID/EX.RegisterRt)) ForwardB = 01
```

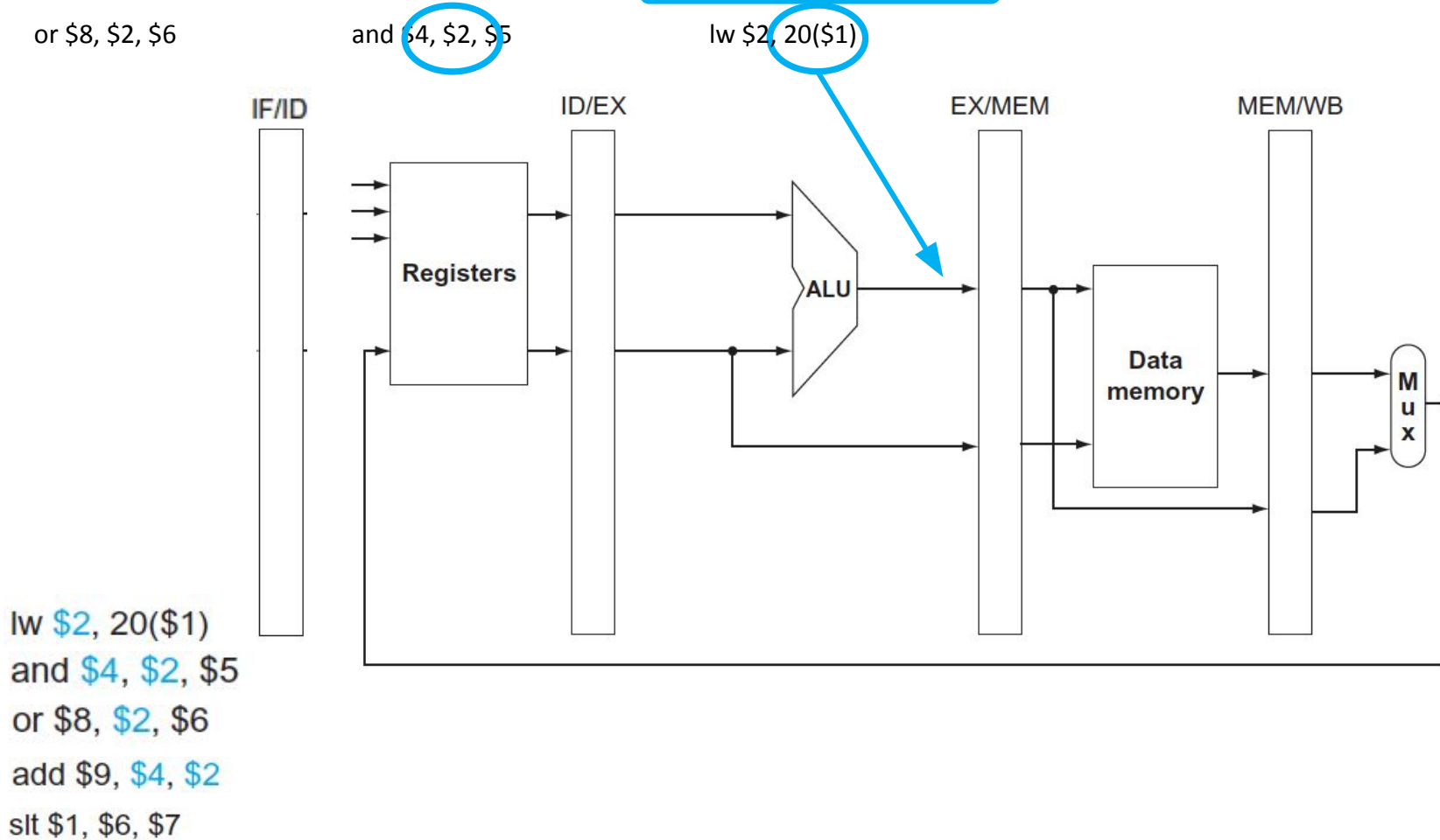
Hazard de Dados

- Caso 2: Hazards entre instruções LW e Aritméticas/Lógicas

```
lw $2, 20($1)
and $4, $2, $5
or $8, $2, $6
add $9, $4, $2
slt $1, $6, $7
```

Hazards de Dados

• Ciclo 3 – Sem forwarding



Hazards de Dados

• Ciclo 4 - Sem forwarding

or \$8, \$2, \$6

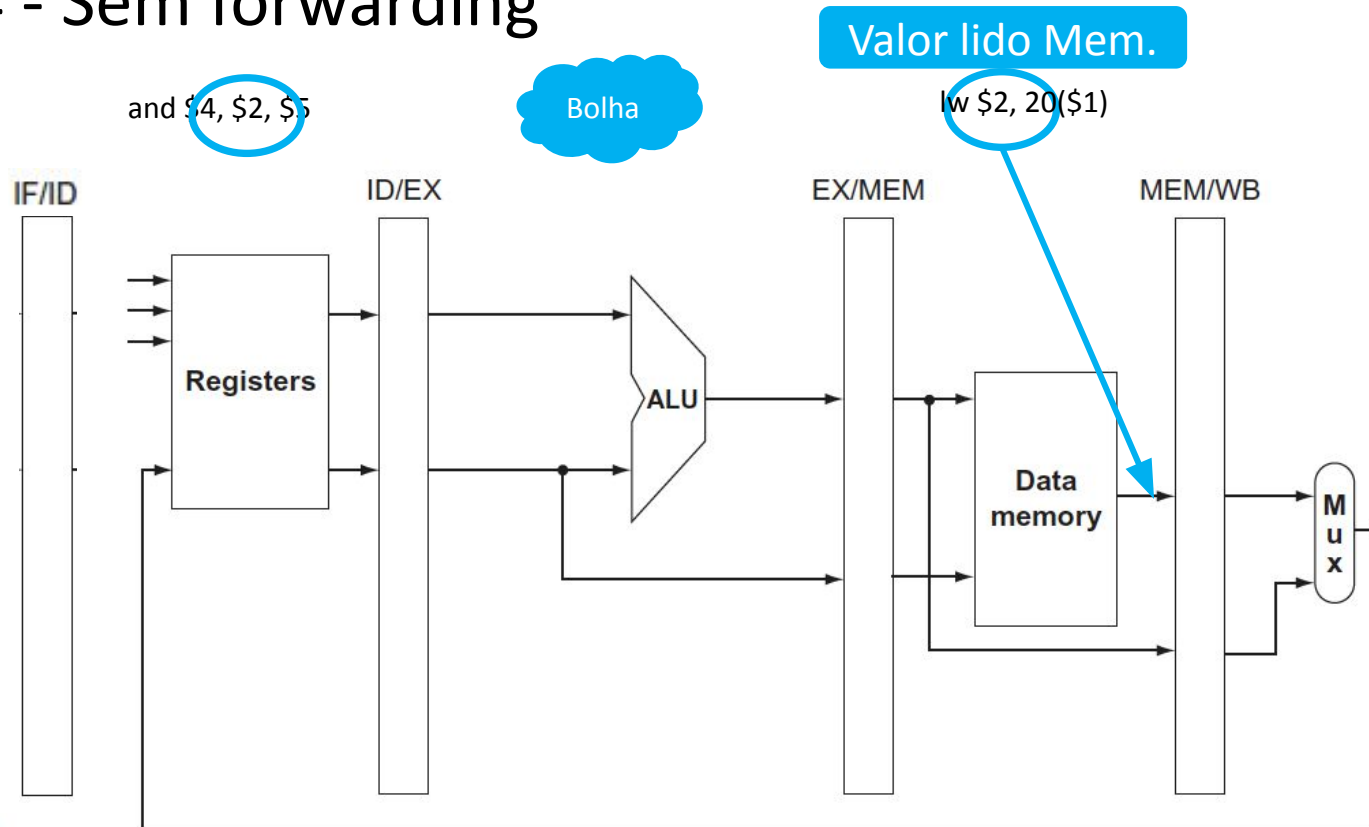
and \$4, \$2, \$5

Bolha

Valor lido Mem.

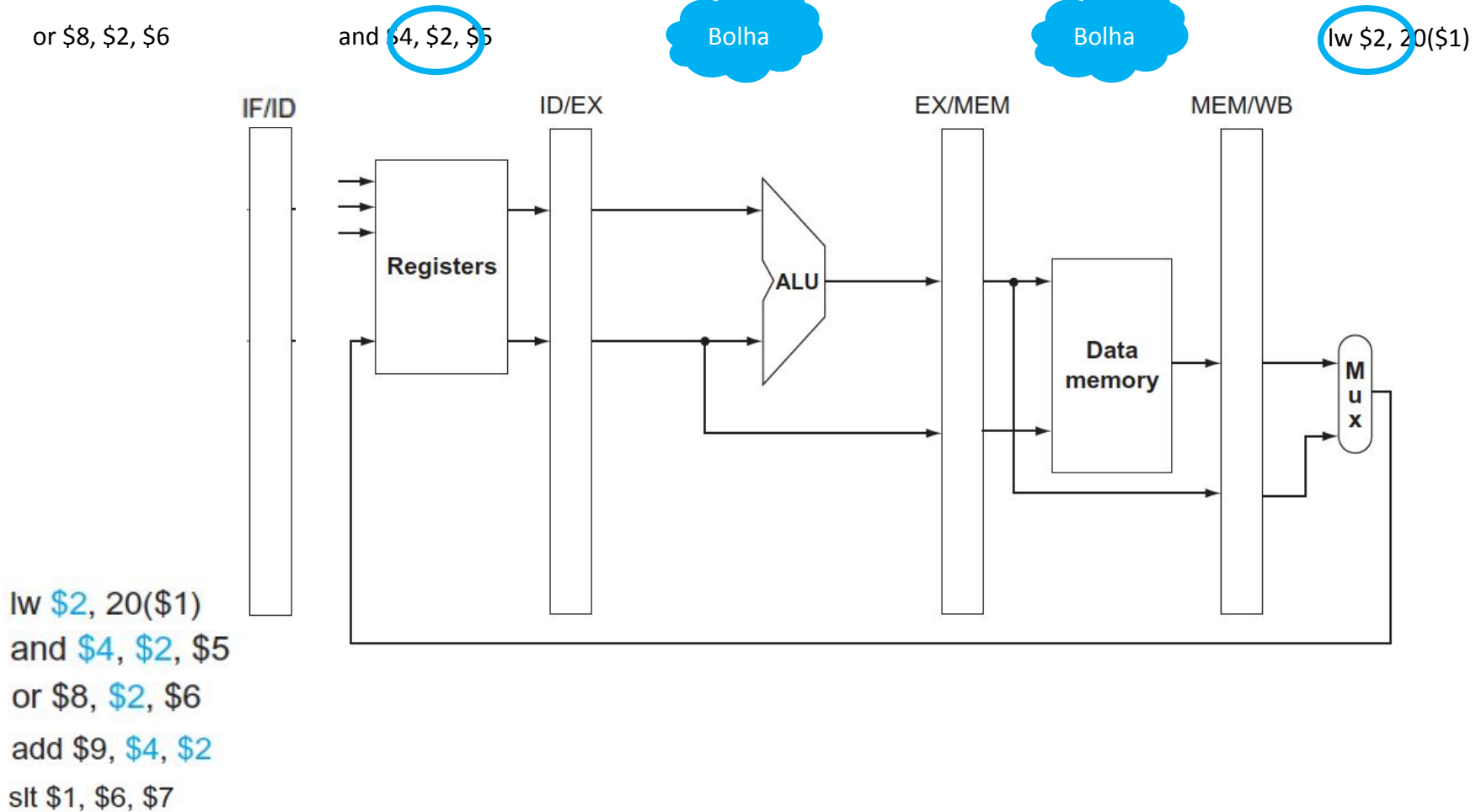
lw \$2, 20(\$1)

lw \$2, 20(\$1)
and \$4, \$2, \$5
or \$8, \$2, \$6
add \$9, \$4, \$2
slt \$1, \$6, \$7



Hazards de Dados

• Ciclo 5 - Sem forwarding



Hazards de Dados

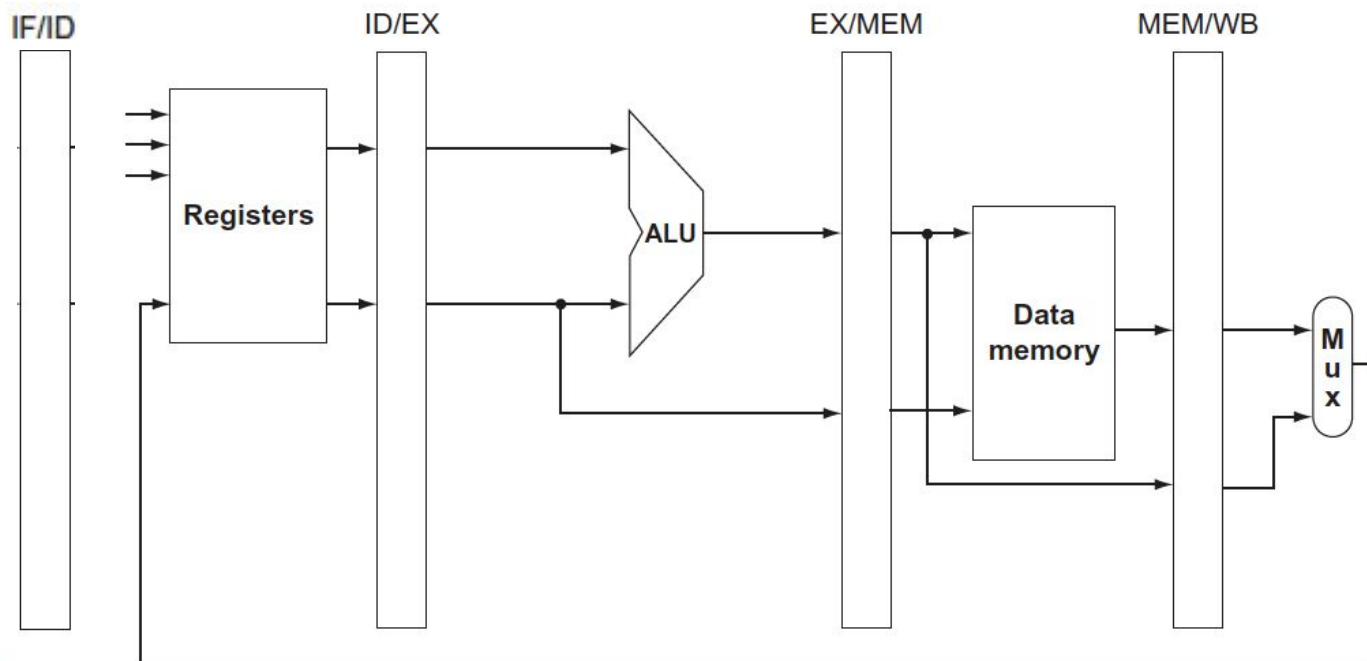
- Ciclo 6

or \$8, \$2, \$6

and \$4, \$2, \$5

Bolha

Bolha



lw \$2, 20(\$1)
and \$4, \$2, \$5
or \$8, \$2, \$6
add \$9, \$4, \$2
slt \$1, \$6, \$7

Hazards de Dados

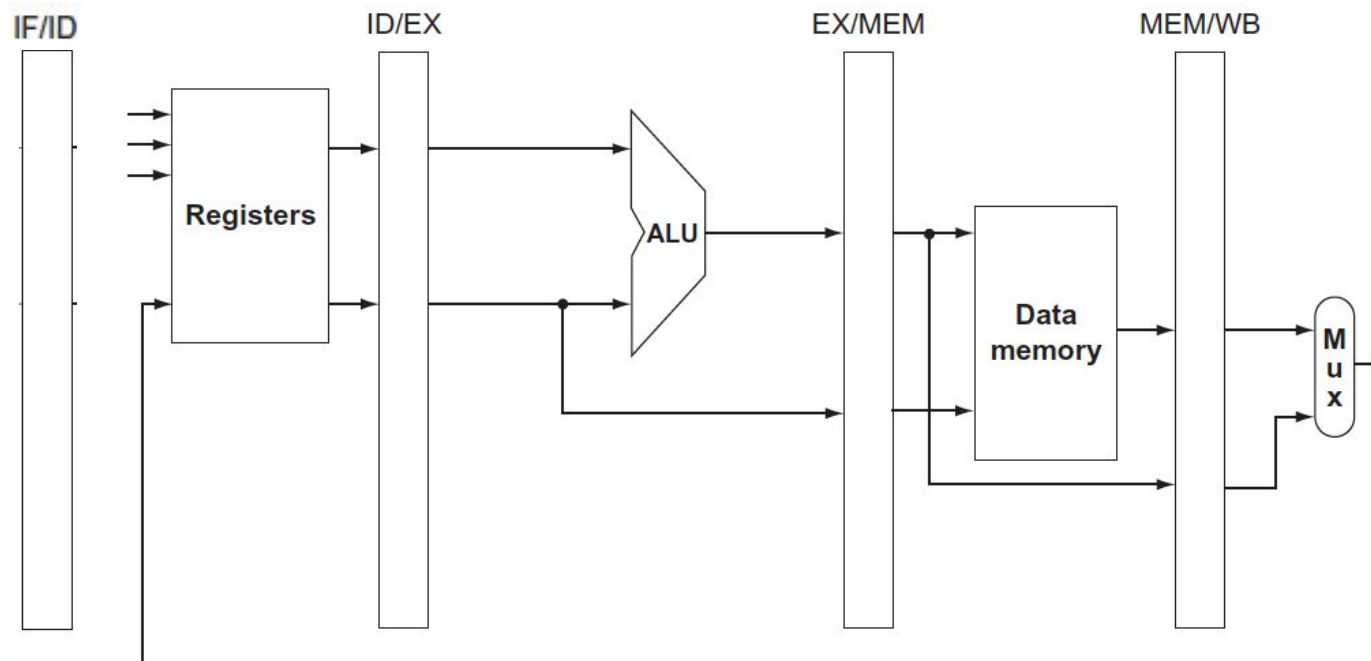
• Ciclo 7

or \$8, \$2, \$6

and \$4, \$2, \$5

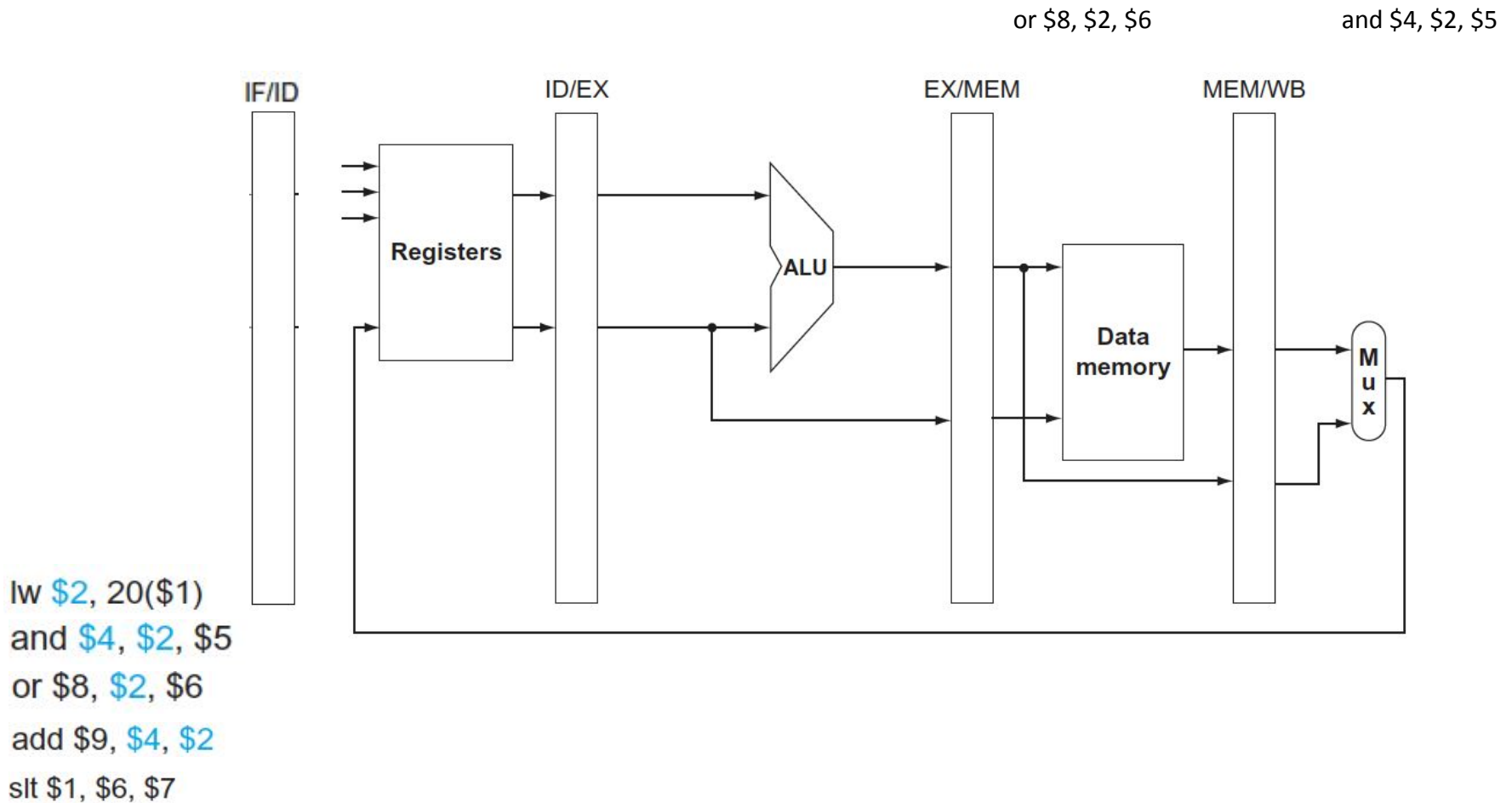
Bolha

lw \$2, 20(\$1)
and \$4, \$2, \$5
or \$8, \$2, \$6
add \$9, \$4, \$2
slt \$1, \$6, \$7



Hazards de Datos

- Ciclo 8

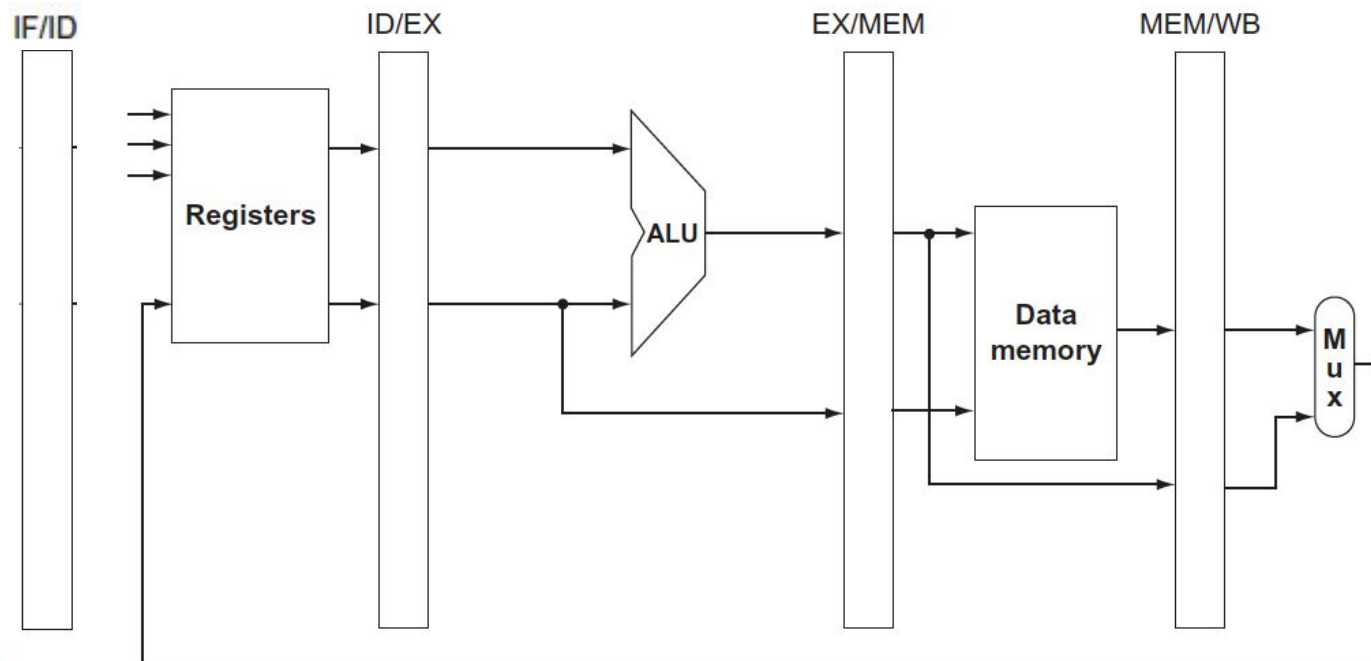


Hazards de Dados

- Ciclo 9

or \$8, \$2, \$6

lw \$2, 20(\$1)
and \$4, \$2, \$5
or \$8, \$2, \$6
add \$9, \$4, \$2
slt \$1, \$6, \$7



Hazards de Dados

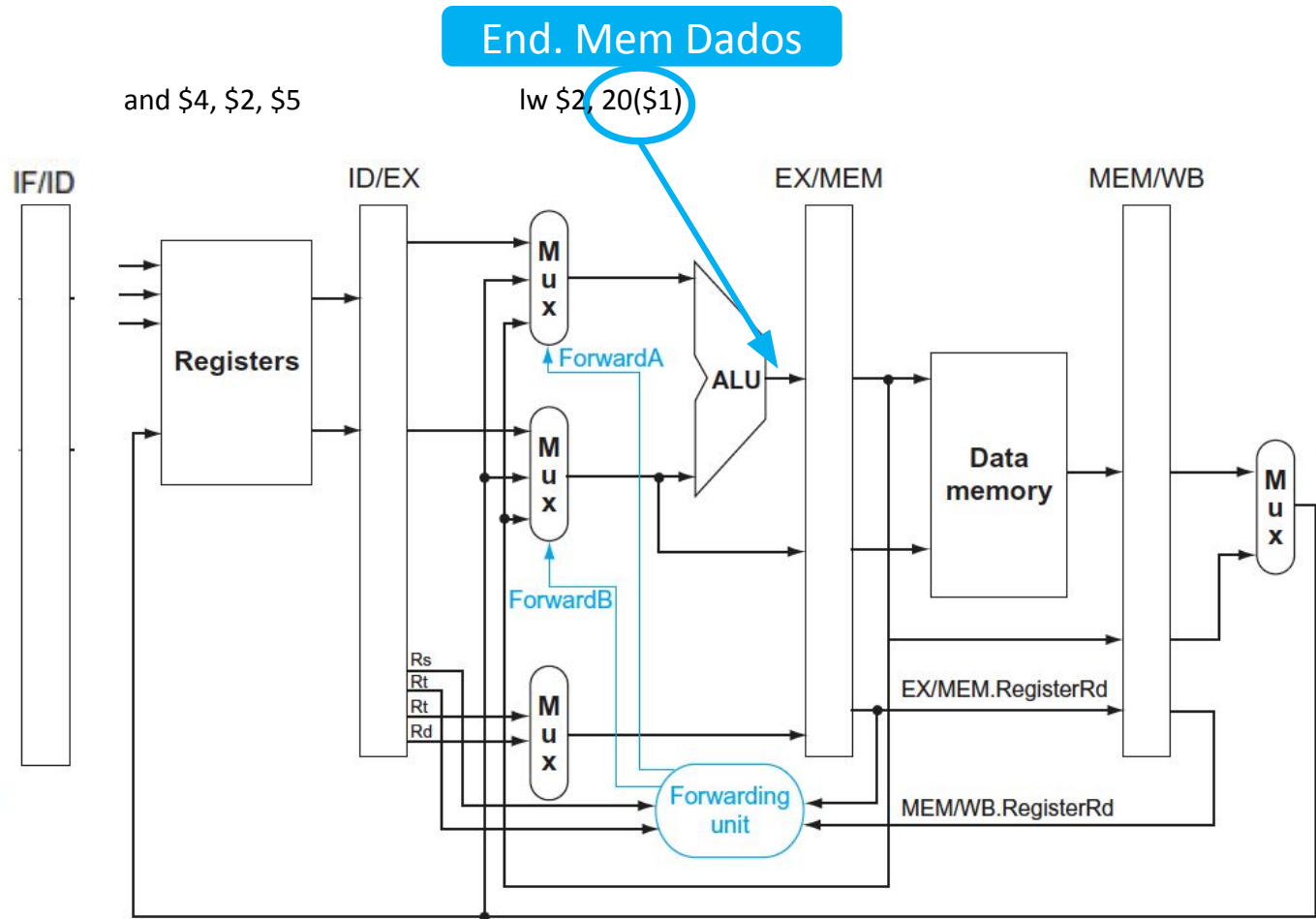
• Ciclo 3

or \$8, \$2, \$6

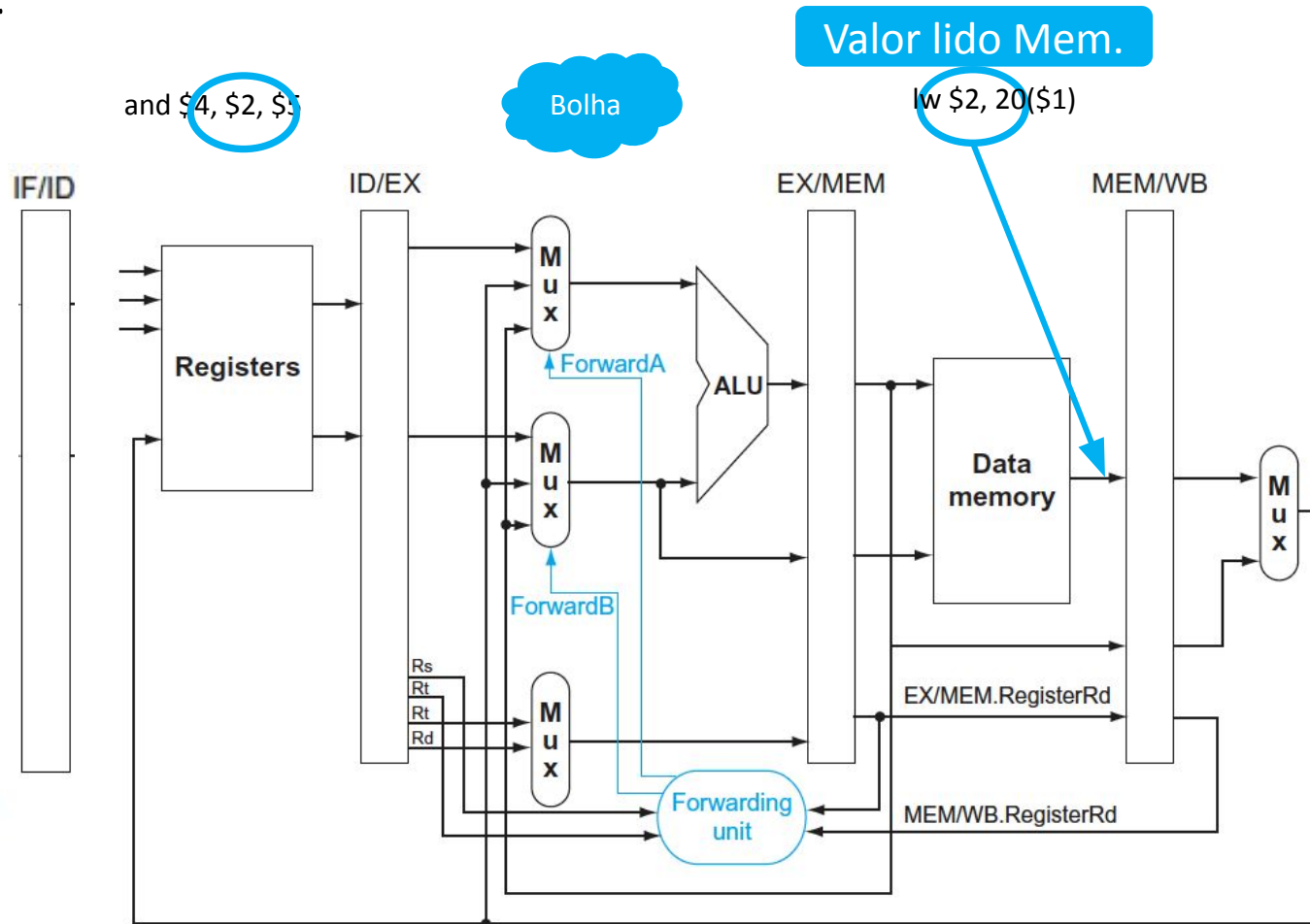
and \$4, \$2, \$5

lw \$2, 20(\$1)

lw \$2, 20(\$1)
and \$4, \$2, \$5
or \$8, \$2, \$6
add \$9, \$4, \$2
slt \$1, \$6, \$7



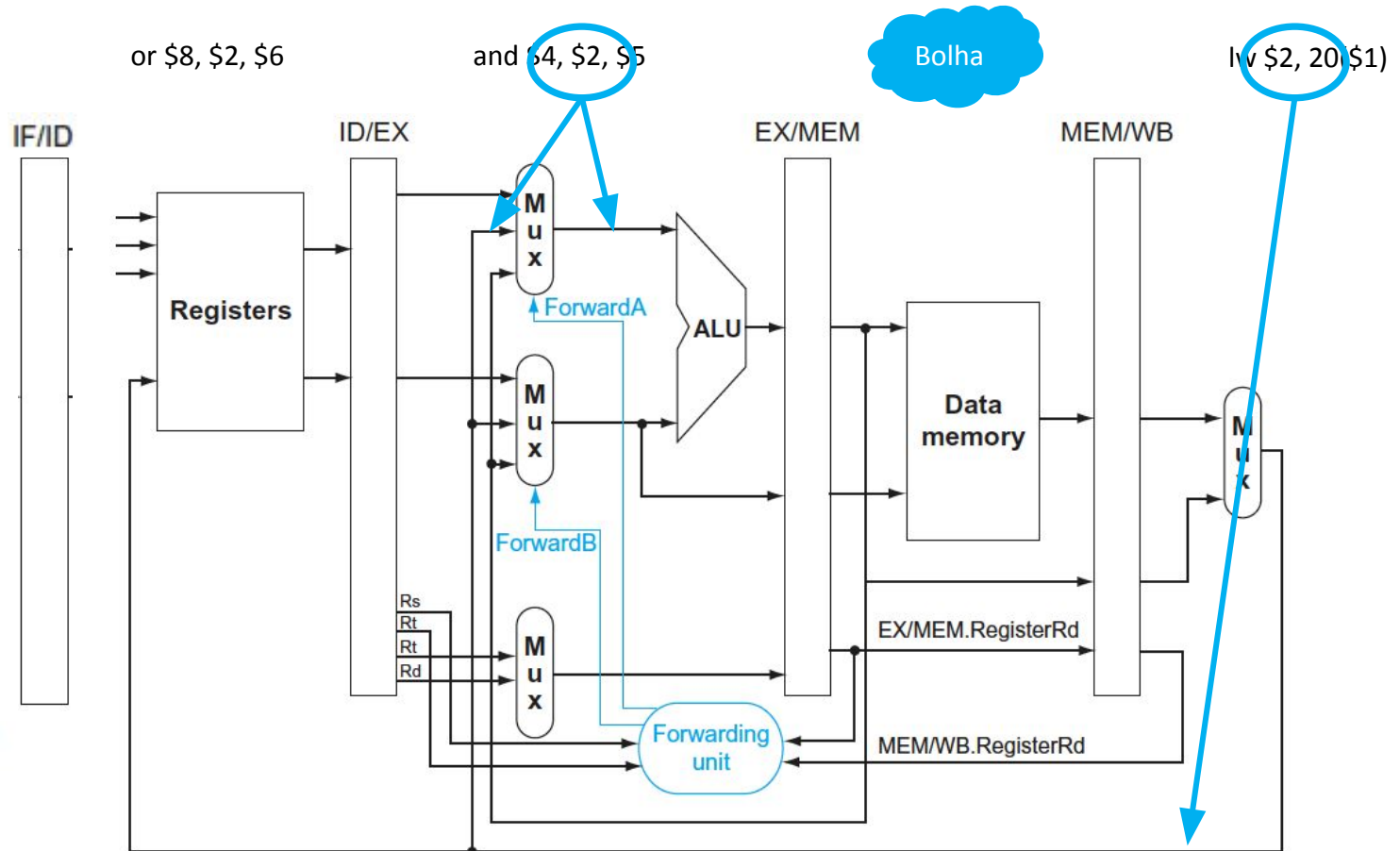
lw \$2, 20(\$1)



Hazards de Dados

• Ciclo 5 – Forwarding Ativo

lw \$2, 20(\$1)
and \$4, \$2, \$5
or \$8, \$2, \$6
add \$9, \$4, \$2
slt \$1, \$6, \$7



Hazards de Dados

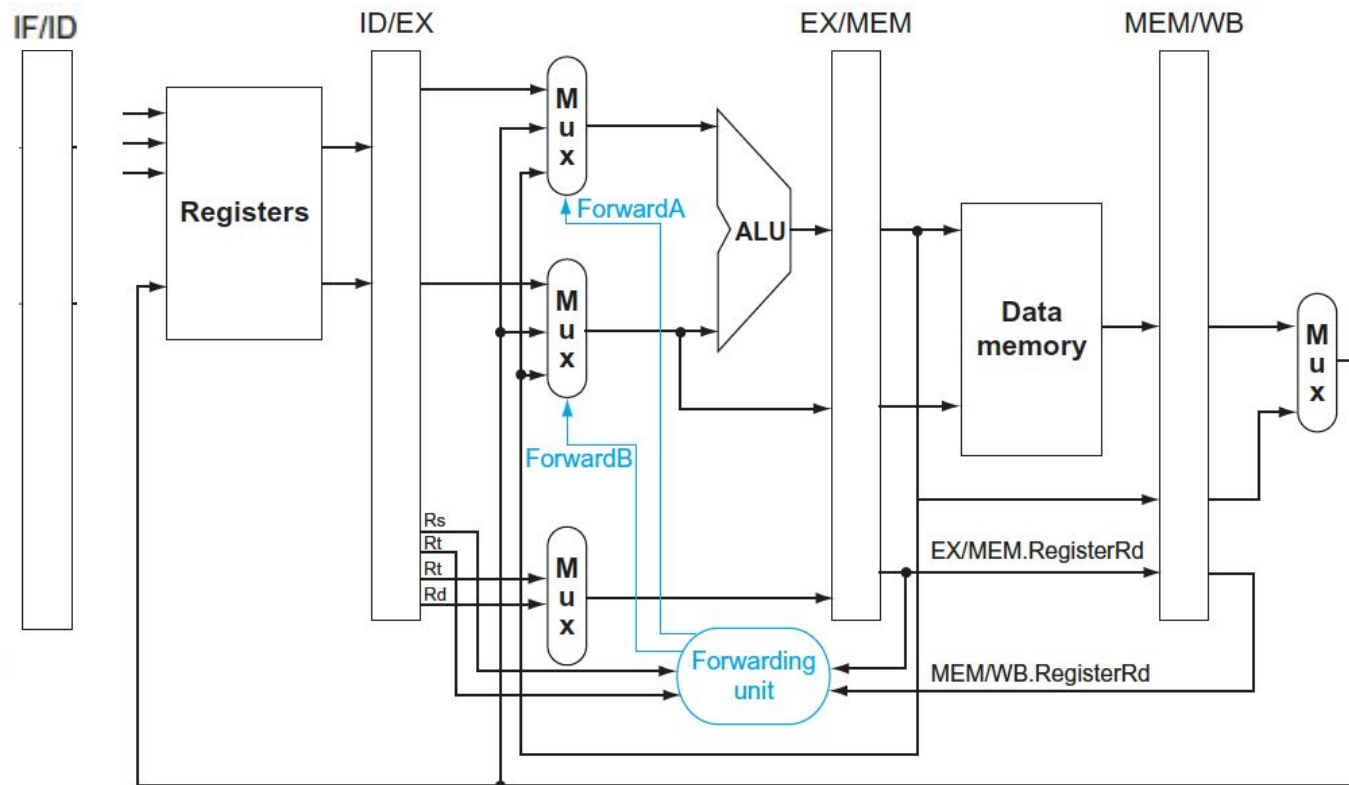
• Ciclo 6

lw \$2, 20(\$1)
and \$4, \$2, \$5
or \$8, \$2, \$6
add \$9, \$4, \$2
slt \$1, \$6, \$7

or \$8, \$2, \$6

and \$4, \$2, \$5

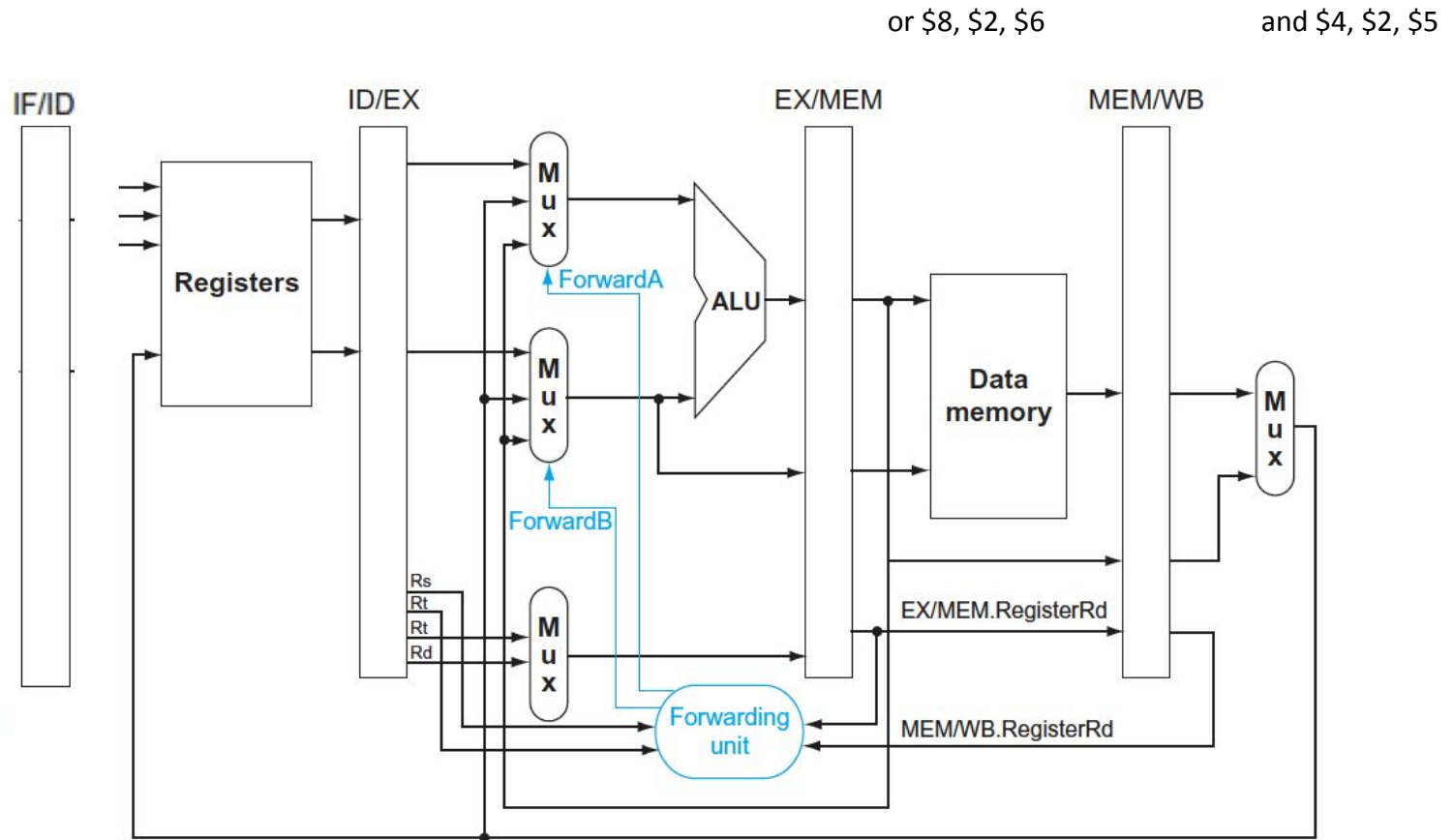
Bolha



Hazards de Dados

- Ciclo 7

lw \$2, 20(\$1)
and \$4, \$2, \$5
or \$8, \$2, \$6
add \$9, \$4, \$2
slt \$1, \$6, \$7

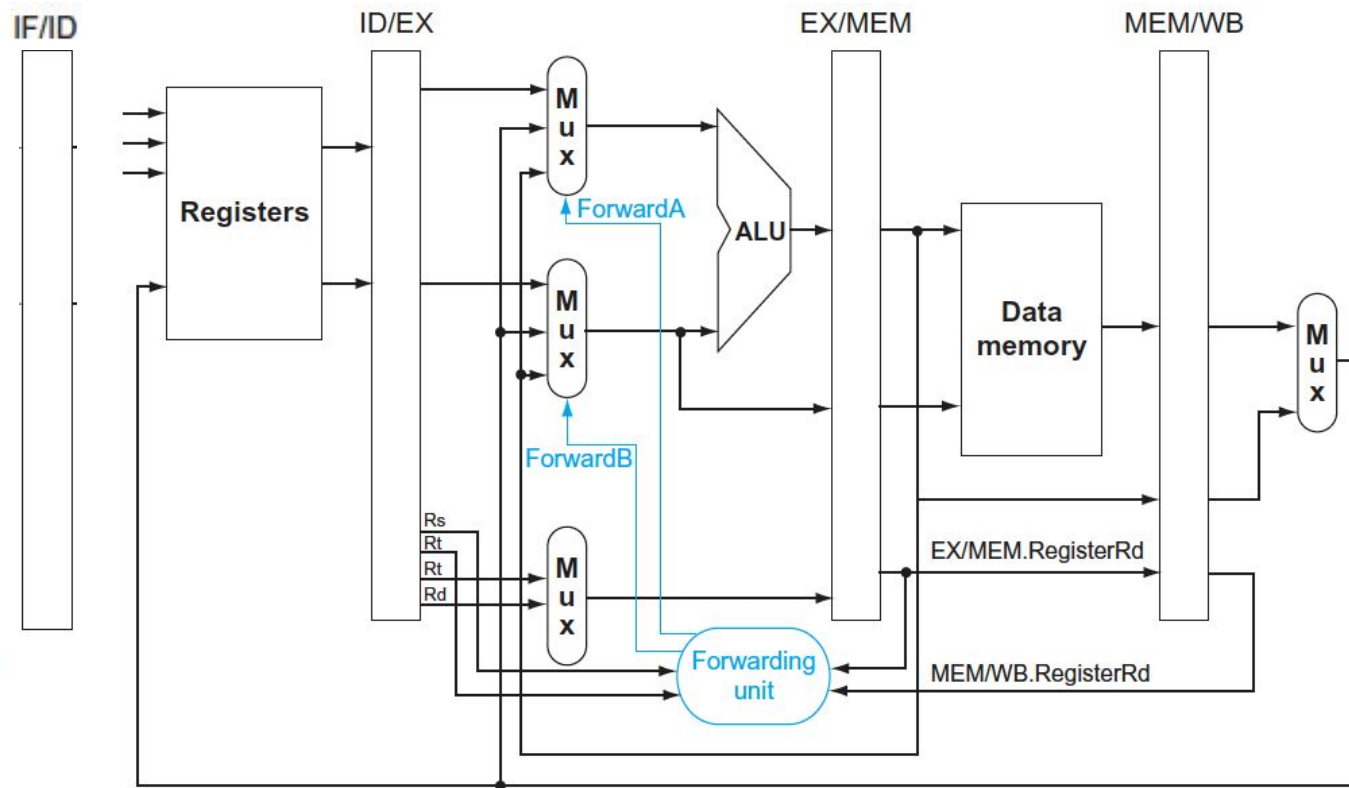


Hazards de Dados

- Ciclo 8

or \$8, \$2, \$6

lw \$2, 20(\$1)
and \$4, \$2, \$5
or \$8, \$2, \$6
add \$9, \$4, \$2
slt \$1, \$6, \$7

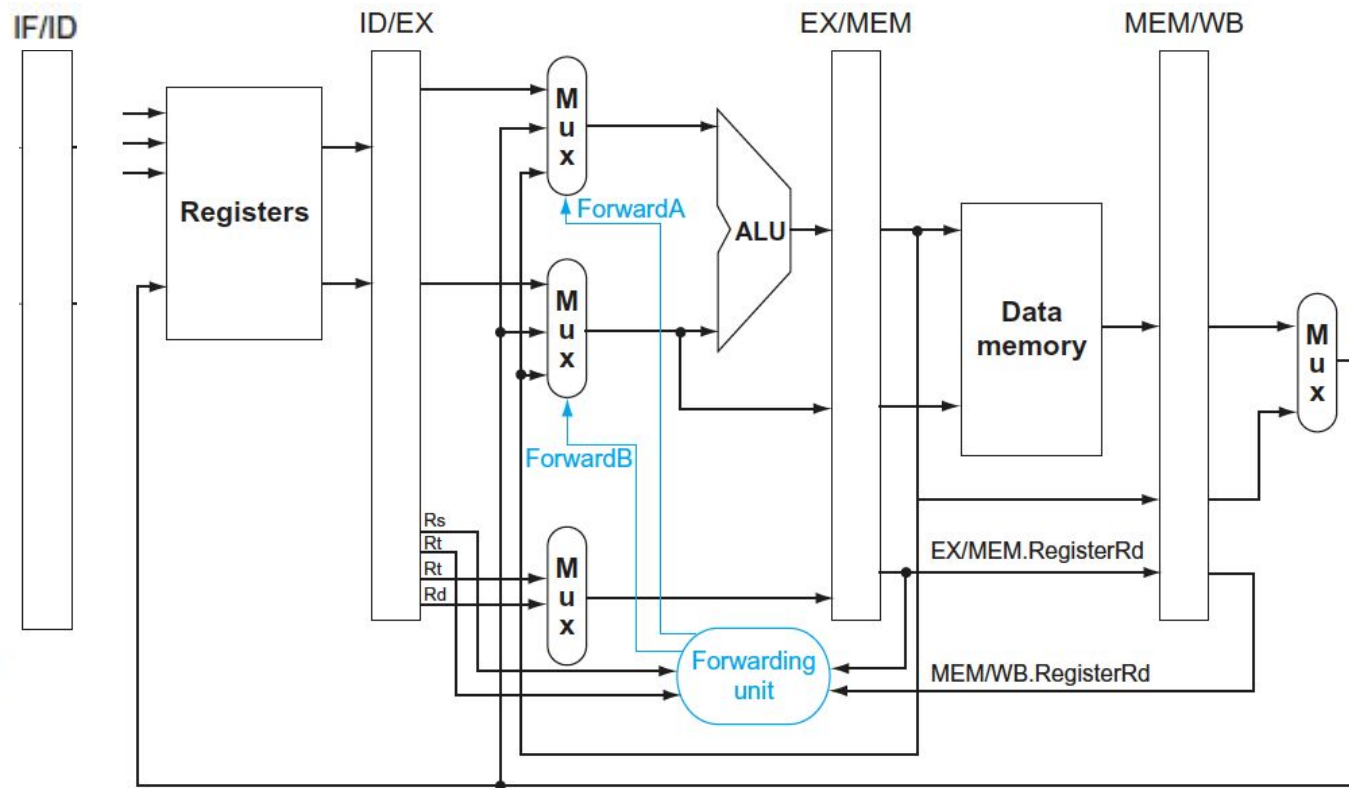


Hazards de Dados

- Ciclo 8

or \$8, \$2, \$6

lw \$2, 20(\$1)
and \$4, \$2, \$5
or \$8, \$2, \$6
add \$9, \$4, \$2
slt \$1, \$6, \$7



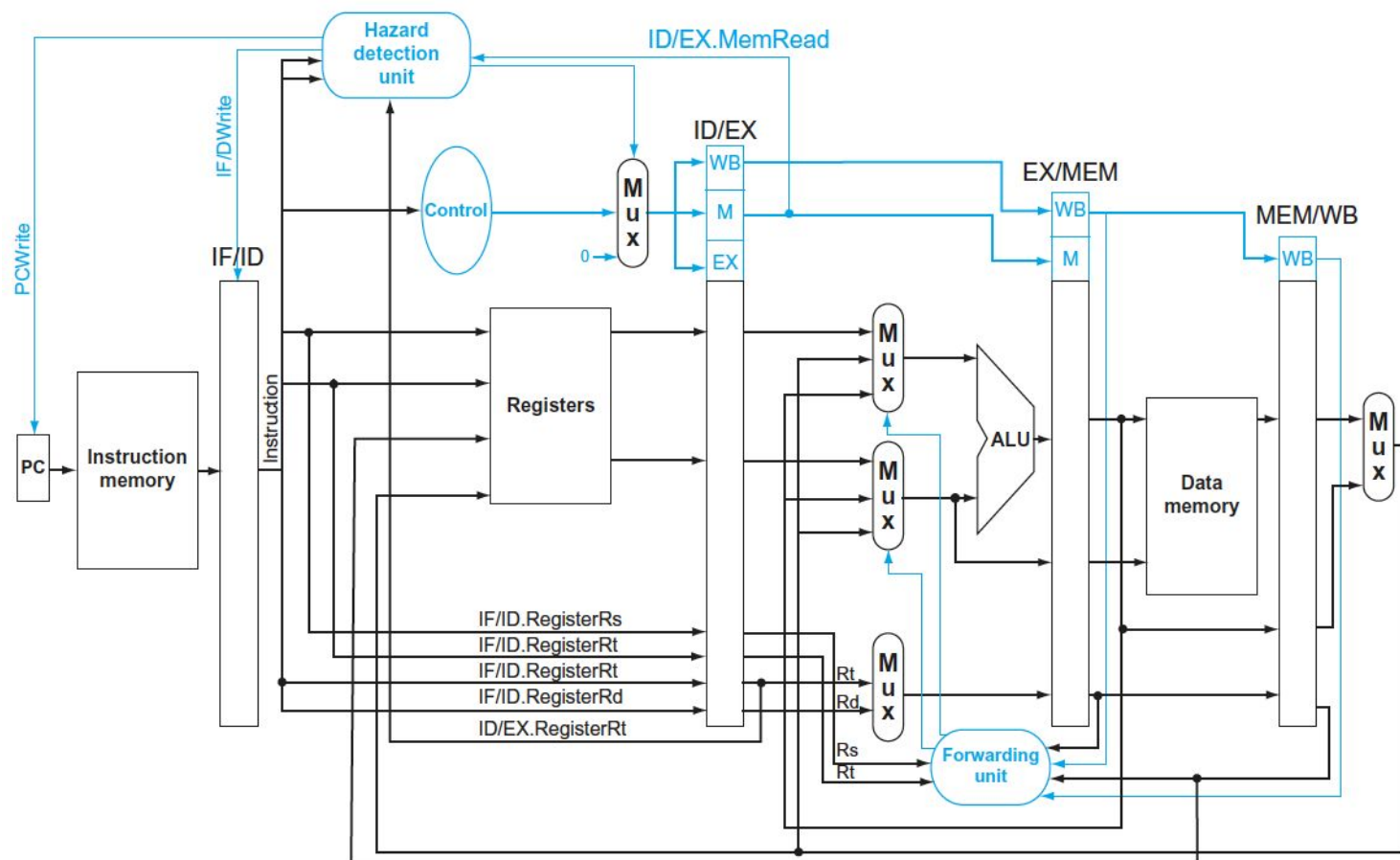
Hazard de Dados

- Caso 2: Hazards entre instruções LW e Aritméticas/Lógicas
 - Adicionar a condição na Forwarding Unit para inserir a Bolha

```
if (ID/EX.MemRead and  
    ((ID/EX.RegisterRt = IF/ID.RegisterRs) or  
     (ID/EX.RegisterRt = IF/ID.RegisterRt)))  
    stall the pipeline
```

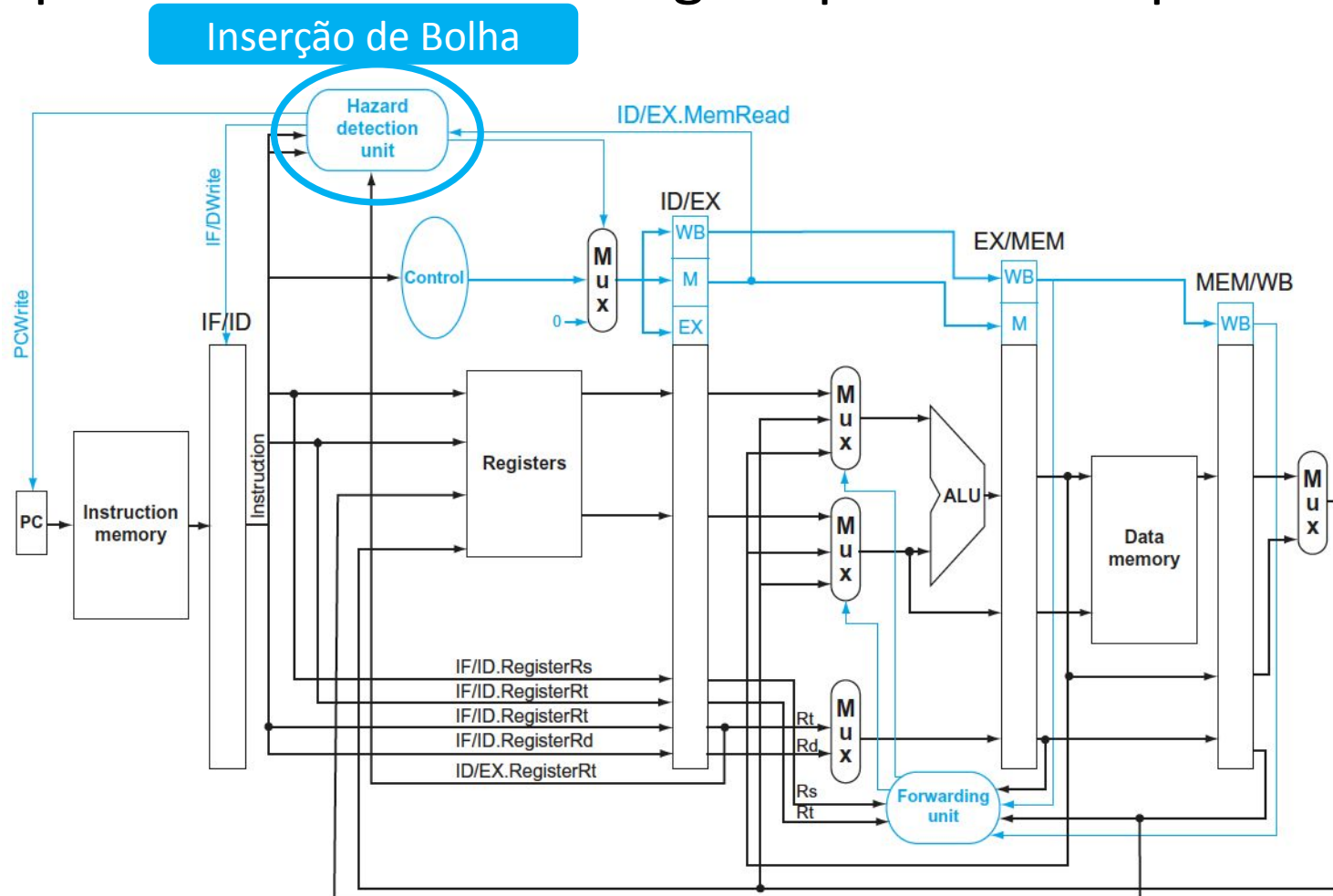
Hazard de Dados

- Componentes de Forwarding acoplados ao Pipeline



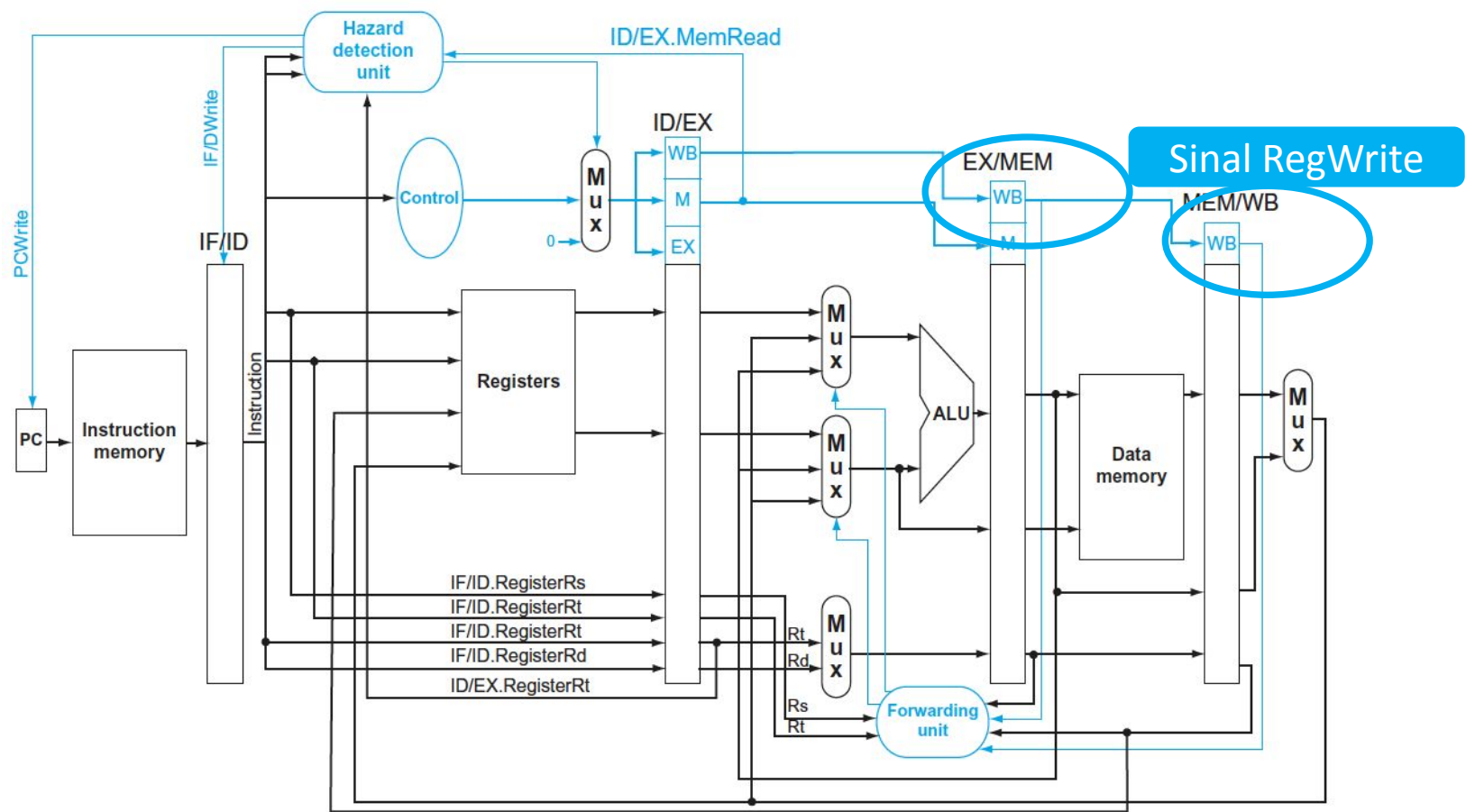
Hazard de Dados

- Componentes de Forwarding acoplados ao Pipeline



Hazard de Dados

- Componentes de Forwarding acoplados ao Pipeline



Exemplo no Pipeline

- Reordenação de código pelo compilador pode evitar stalls

Código Original

```
lw    $t1, 0($t0)
lw    $t2, 4($t0)
add   $t3, $t1, $t2
sw    $t3, 12($t0)
lw    $t4, 8($t0)
add   $t5, $t1, $t4
sw    $t5, 16($t0)
```

Código Reordenado

```
lw    $t1, 0($t0)
lw    $t2, 4($t0)
lw    $t4, 8($t0)
add   $t3, $t1, $t2
sw    $t3, 12($t0)
add   $t5, $t1, $t4
sw    $t5, 16($t0)
```

Hazards de Controle

- Uma instrução que modifica o fluxo da execução (beq) deve tomar uma decisão de controle de fluxo, entretanto já existem instruções de um fluxo em execução em estágios anteriores do pipeline.

```
36 sub $10, $4, $8
40 beq $1, $3, 7 # PC-relative branch to 40+4+7*4=72
44 and $12, $2, $5
48 or  $13, $2, $6
52 add $14, $4, $2
56 slt $15, $6, $7
. . .
72 lw  $4, 50($7)
```

- Branch pode resultar em:
 - Not Taken: próxima instrução será executada (44 beq \$1, \$3, 7)
 - Taken: fluxo irá saltar para o PC calculado (72 lw \$4, 50(\$7))

Hazards de Controle

- Previsão Estática - Assumir branch Not Taken
 - Sempre buscar as próximas instruções, caso:
 - Acerte a previsão: seguir com as instruções no pipeline
 - Erre a previsão: remover as instruções no pipeline (flush)

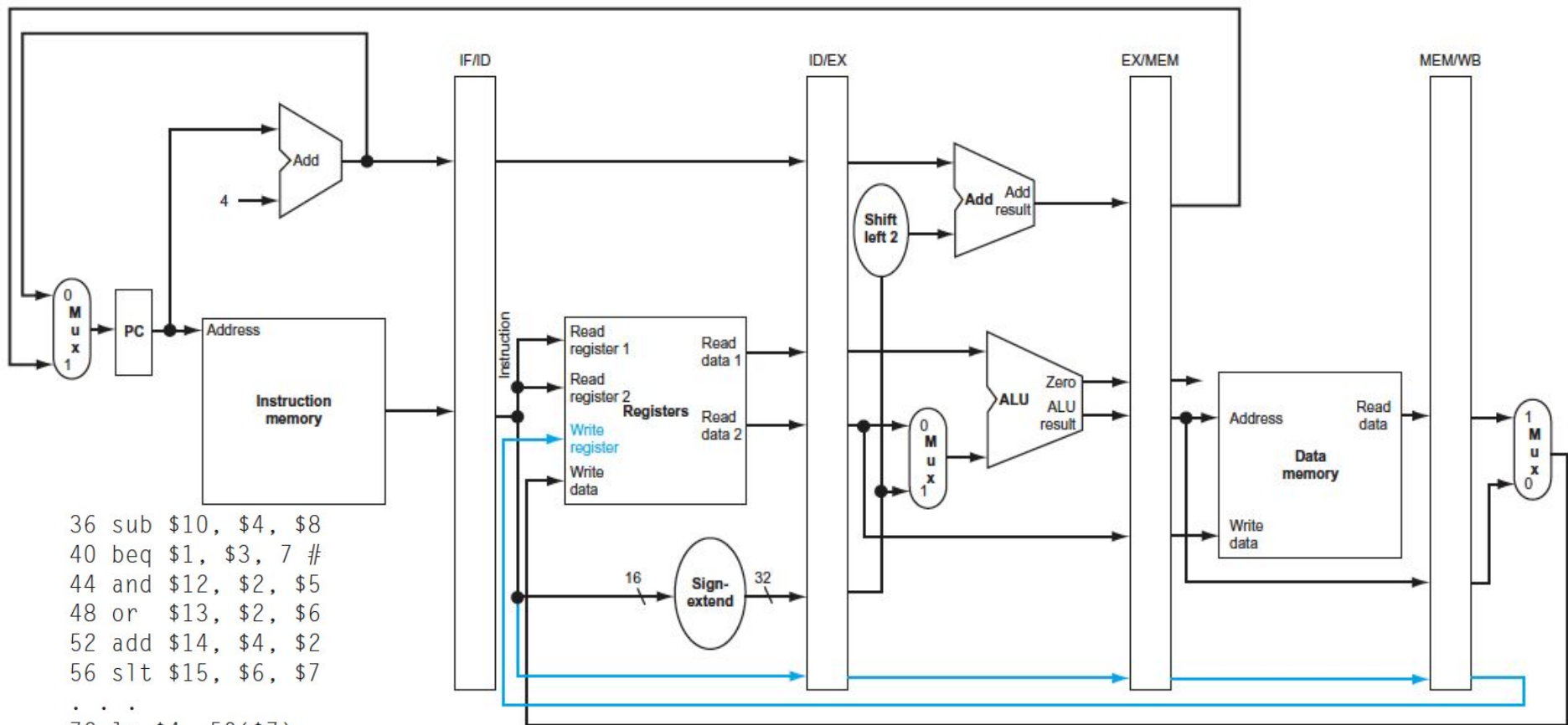
```
36 sub $10, $4, $8
40 beq $1, $3, 7 # PC-relative branch to 40+4+7*4=72
44 and $12, $2, $5
48 or  $13, $2, $6
52 add $14, $4, $2
56 slt $15, $6, $7
. . .
72 lw  $4, 50($7)
```

Hazards de Controle

• Ciclo 1

sub \$10, \$4, \$8

```
36 sub $10, $4, $8
40 beq $1, $3, 7 #
44 and $12, $2, $5
48 or $13, $2, $6
52 add $14, $4, $2
56 slt $15, $6, $7
. . .
72 lw $4, 50($7)
```



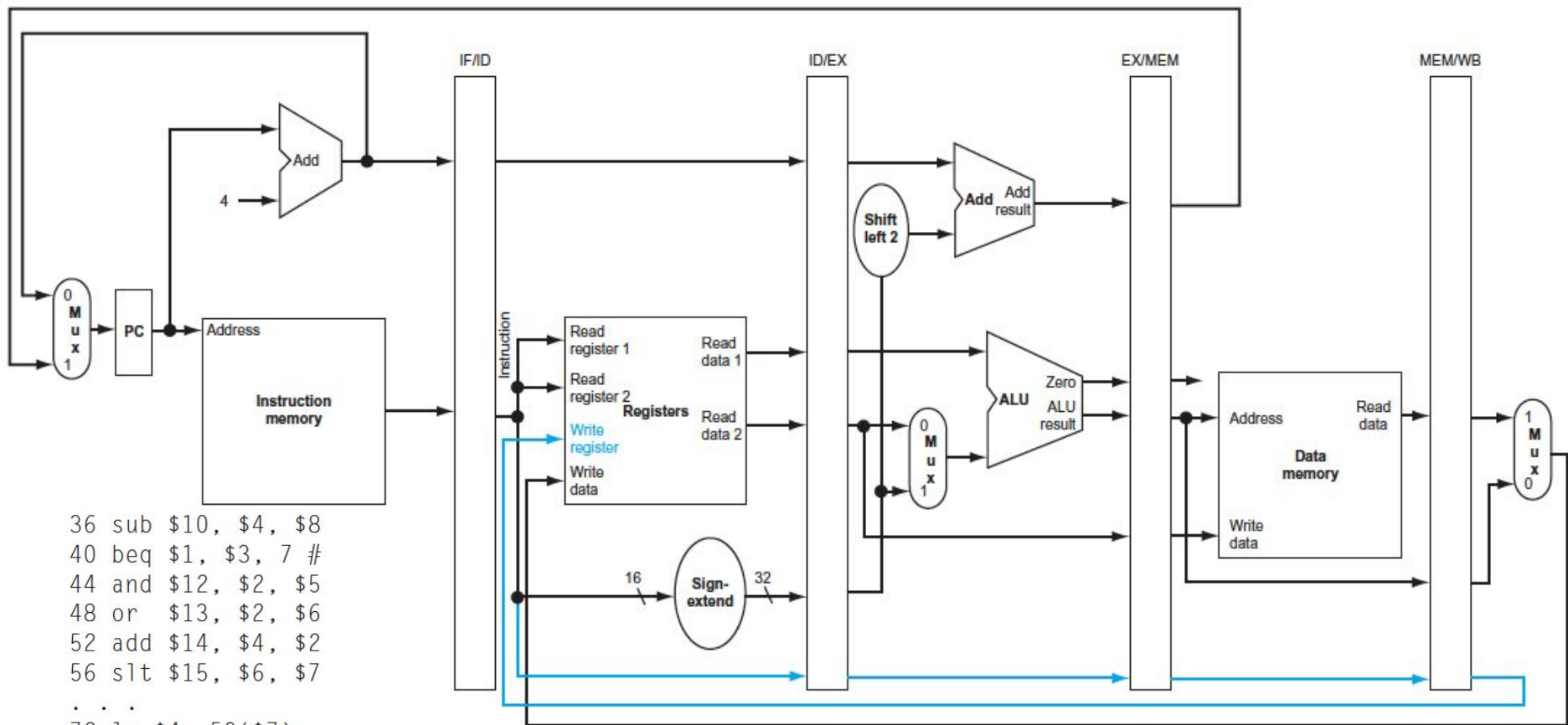
Hazards de Controle

• Ciclo 2

beq \$1, \$3, 7

sub \$10, \$4, \$8

```
36 sub $10, $4, $8
40 beq $1, $3, 7 #
44 and $12, $2, $5
48 or $13, $2, $6
52 add $14, $4, $2
56 slt $15, $6, $7
. . .
72 lw $4, 50($7)
```



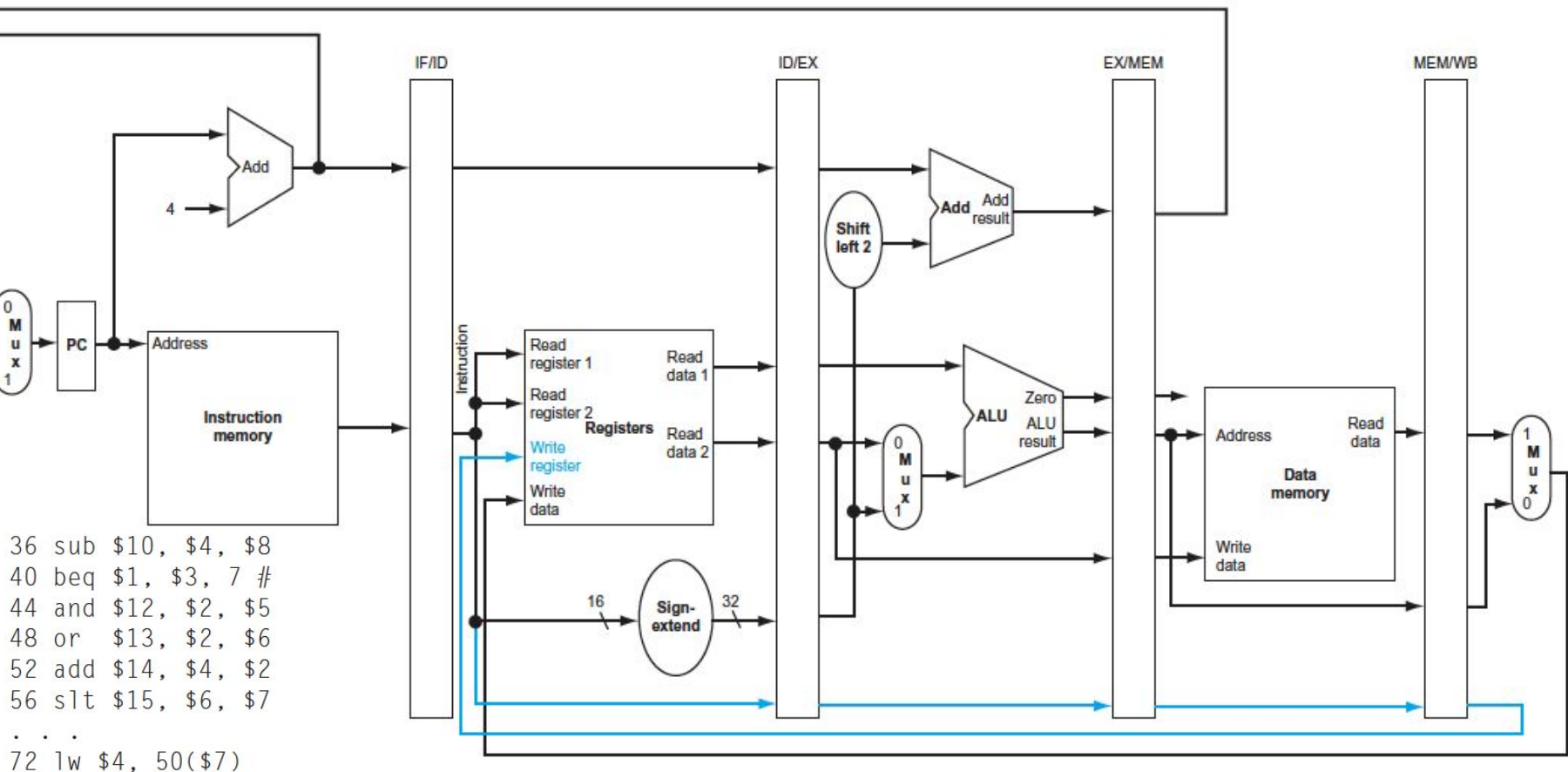
Hazards de Controle

- Ciclo 3 – única possibilidade é buscar o PC+4

and \$12, \$2, \$5

beq \$1, \$3, 7

sub \$10, \$4, \$8



```
36 sub $10, $4, $8
40 beq $1, $3, 7
44 and $12, $2, $5
48 or $13, $2, $6
52 add $14, $4, $2
56 slt $15, $6, $7
. . .
72 lw $4, 50($7)
```

Hazards de Controle

- Ciclo 4 – Decisão será tomada neste ciclo

or \$13, \$2, \$6

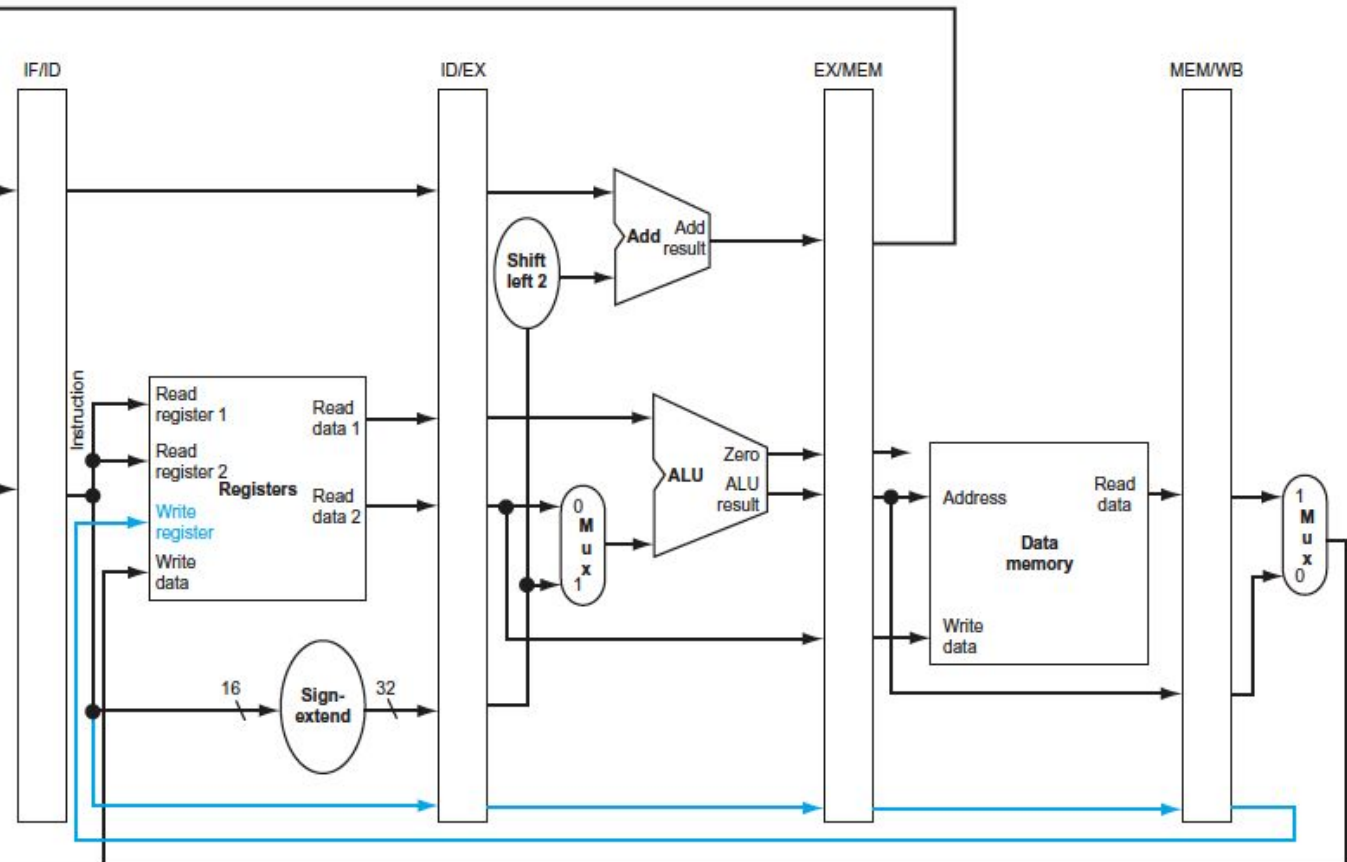
and \$12, \$2, \$5

beq \$1, \$3, 7

sub \$10, \$4, \$8

```

36 sub $10, $4, $8
40 beq $1, $3, 7 #
44 and $12, $2, $5
48 or $13, $2, $6
52 add $14, $4, $2
56 slt $15, $6, $7
. . .
72 lw $4, 50($7)
    
```



Hazards de Controle

• Ciclo 5 – Caso Not Taken – Fluxo segue normalmente

add \$14, \$4, \$2

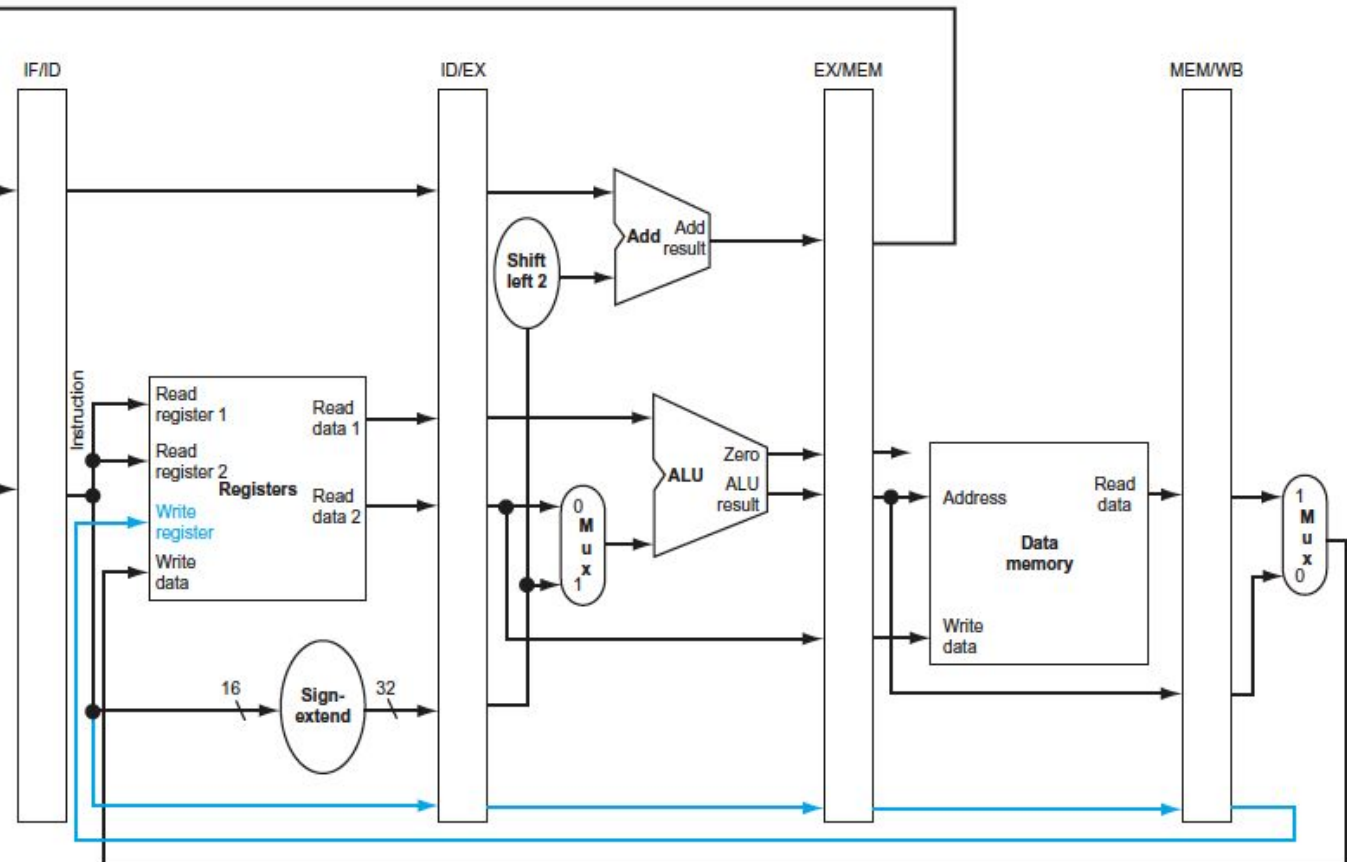
or \$13, \$2, \$6

and \$12, \$2, \$5

beq \$1, \$3, 7

sub \$10, \$4, \$8

```
36 sub $10, $4, $8
40 beq $1, $3, 7 #
44 and $12, $2, $5
48 or $13, $2, $6
52 add $14, $4, $2
56 slt $15, $6, $7
. . .
72 lw $4, 50($7)
```



Hazards de Controle

- Ciclo 4 – Decisão será tomada neste ciclo

or \$13, \$2, \$6

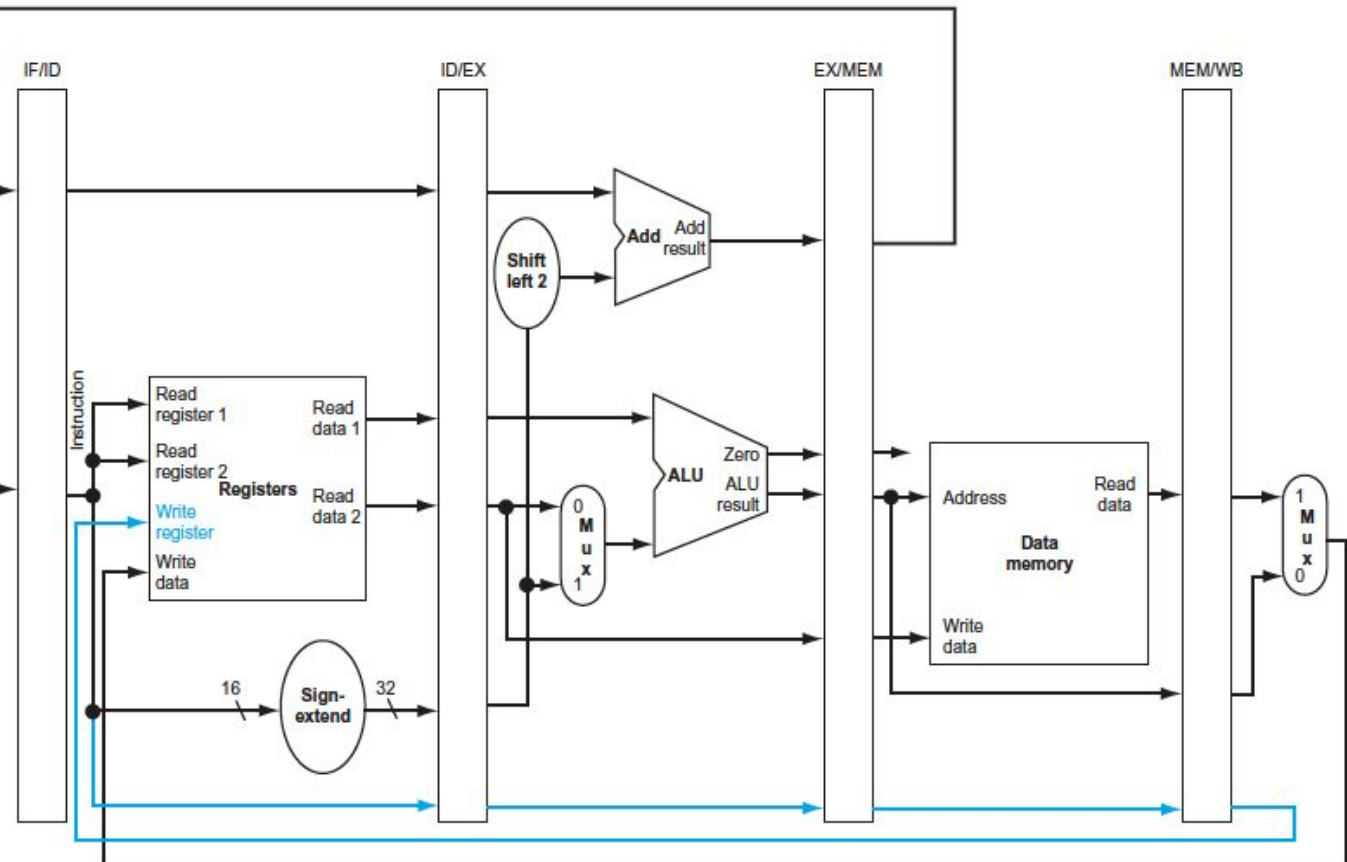
and \$12, \$2, \$5

beq \$1, \$3, 7

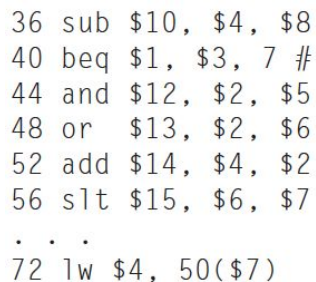
sub \$10, \$4, \$8

```

36 sub $10, $4, $8
40 beq $1, $3, 7 #
44 and $12, $2, $5
48 or $13, $2, $6
52 add $14, $4, $2
56 slt $15, $6, $7
. . .
72 lw $4, 50($7)
    
```



sub \$10, \$4, \$8



Hazards de Controle

• Ciclo 6 – Caso Taken – Flush no pipeline

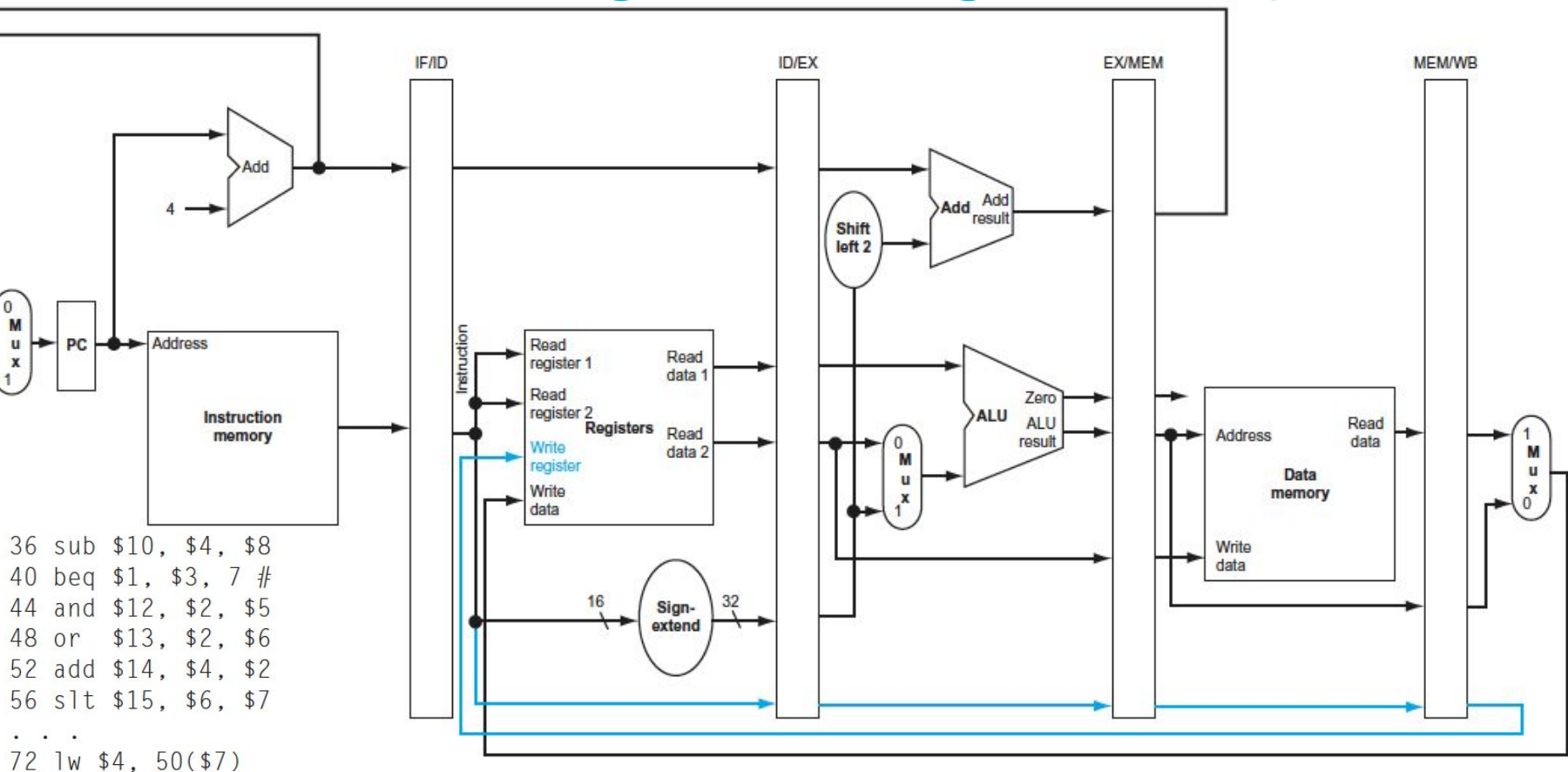
lw \$4, 50(\$7)

Bolha

Bolha

Bolha

beq \$1, \$3, 7



Hazards de Controle

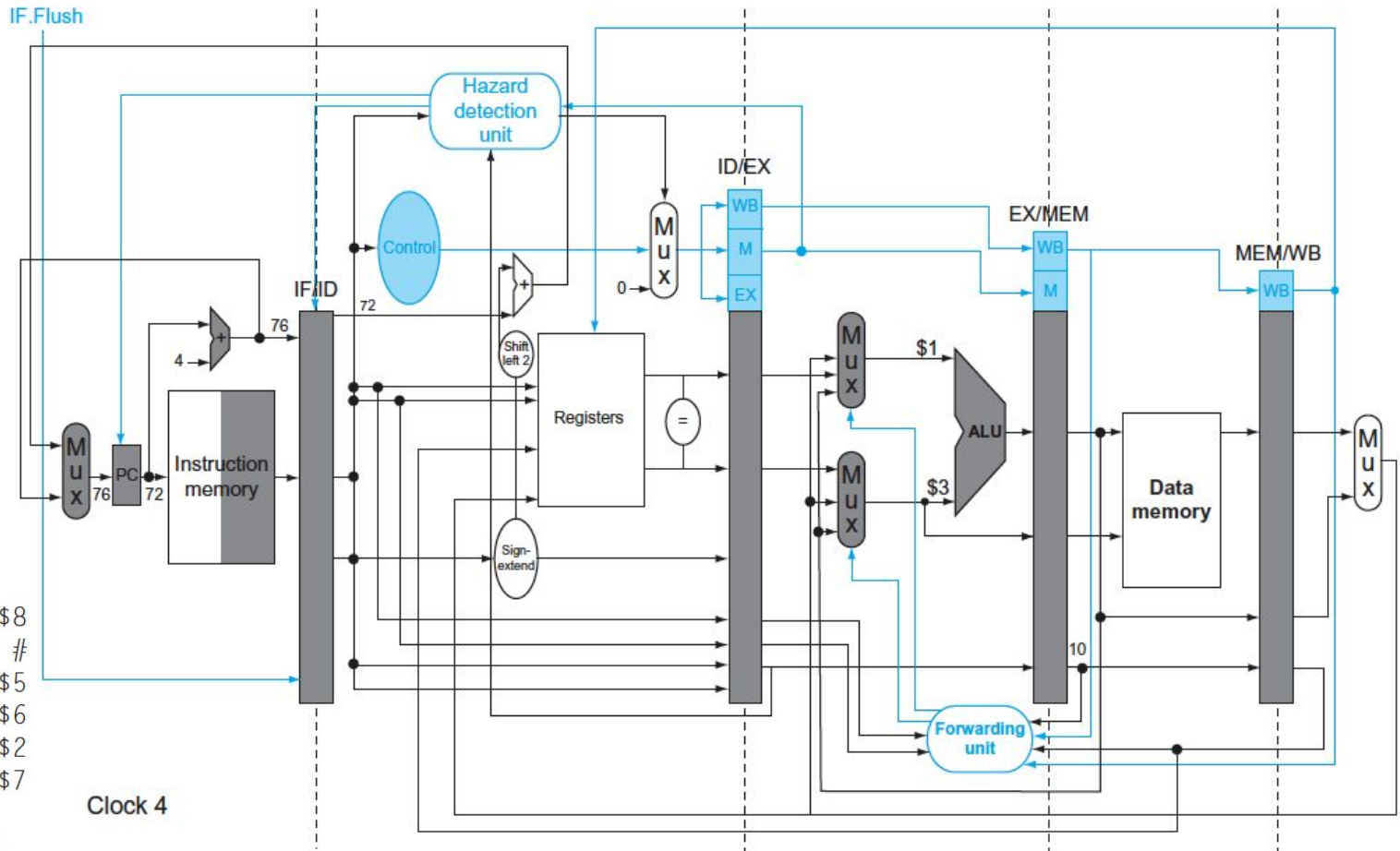


- Previsão Estática - Assumir branch Not Taken
 - Hardware de decisão é adiantado para o segundo estágio do pipeline
 - Reduzir bolhas quando Taken
 - caso em que a previsão resulte em erro

Hazards de Controle

• Ciclo 1

sub \$10, \$4, \$8

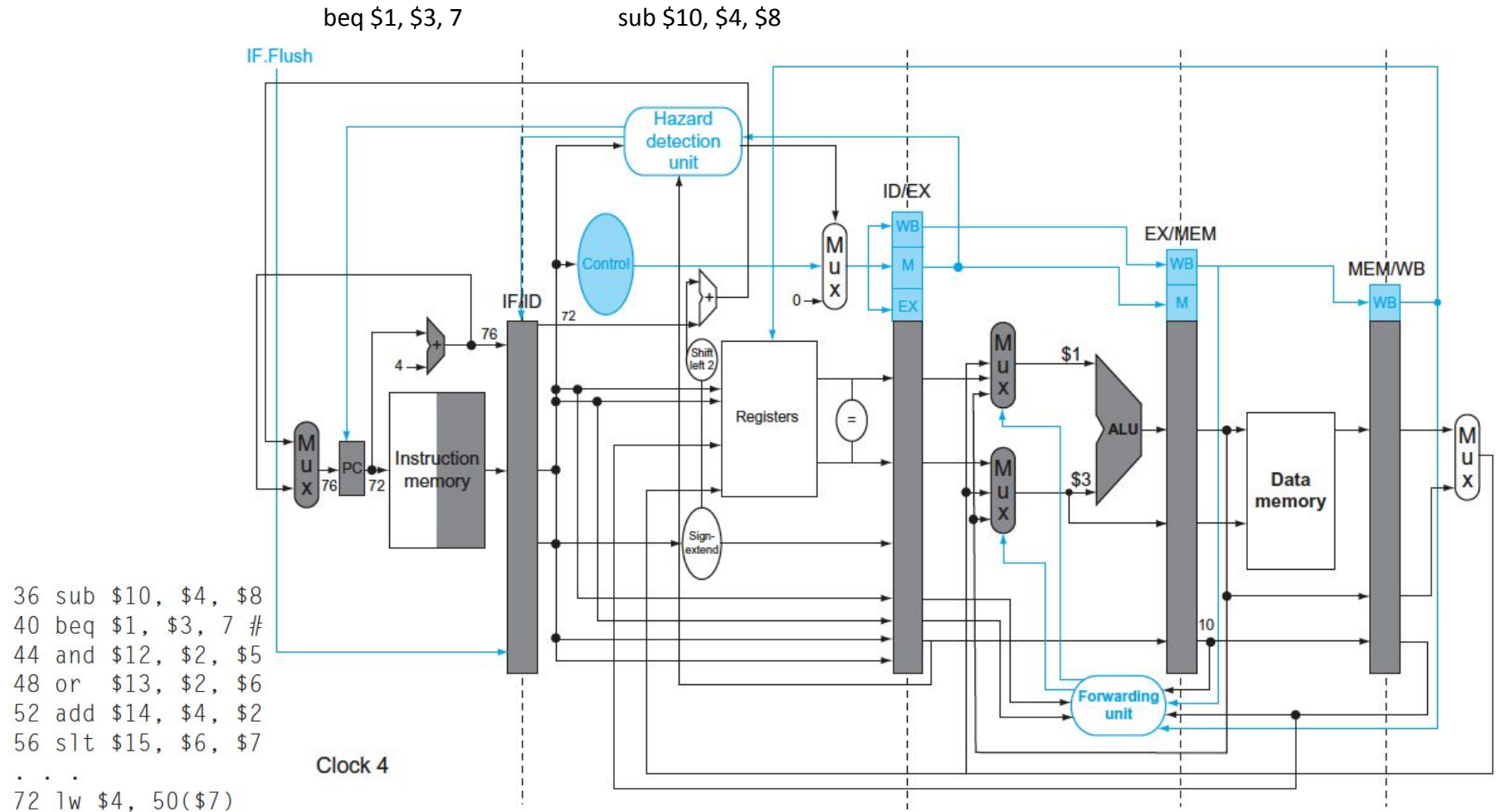


```

36 sub $10, $4, $8
40 beq $1, $3, 7 #
44 and $12, $2, $5
48 or $13, $2, $6
52 add $14, $4, $2
56 slt $15, $6, $7
. . .
72 lw $4, 50($7)
    
```

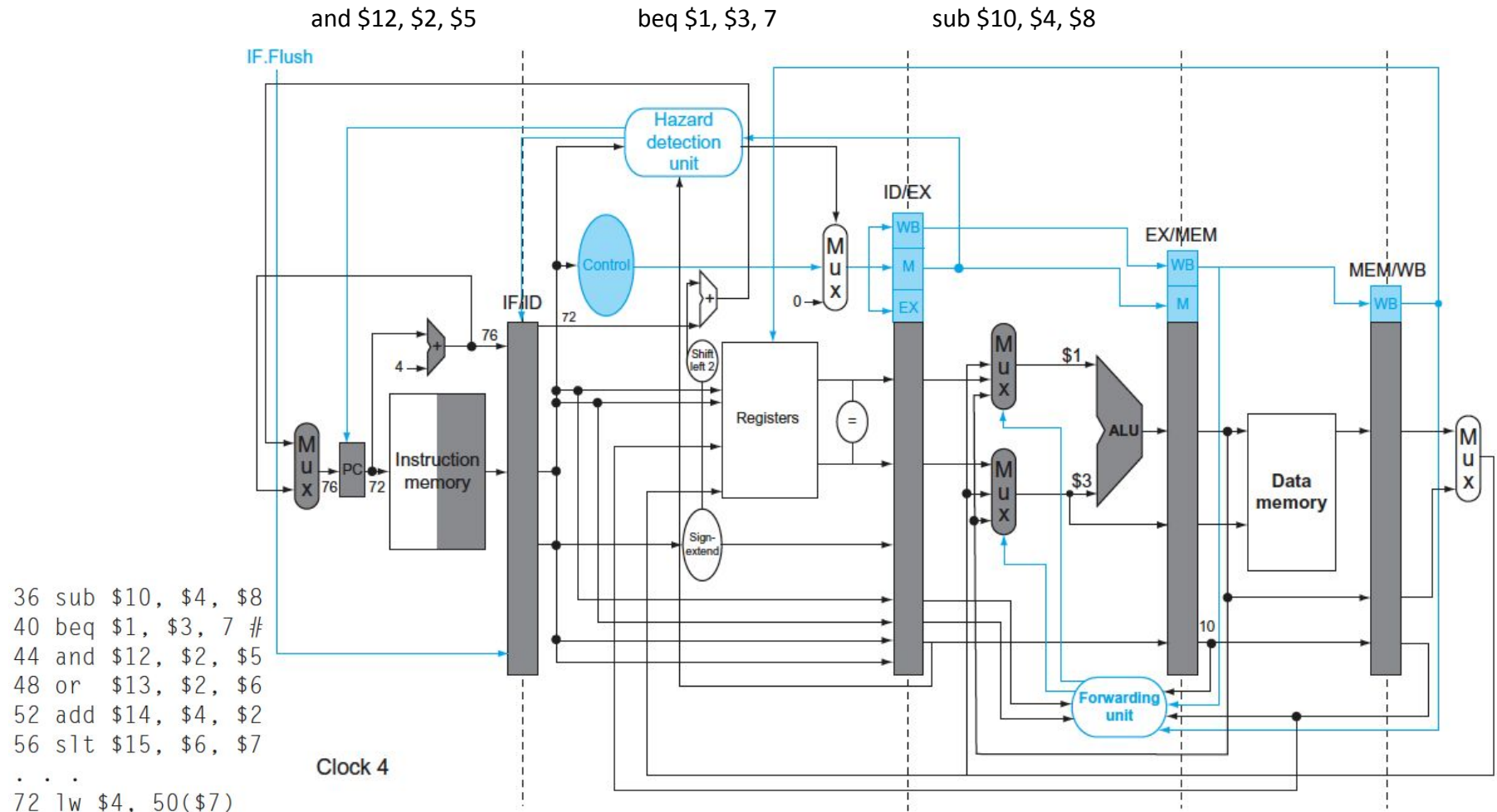
Hazards de Controle

• Ciclo 2



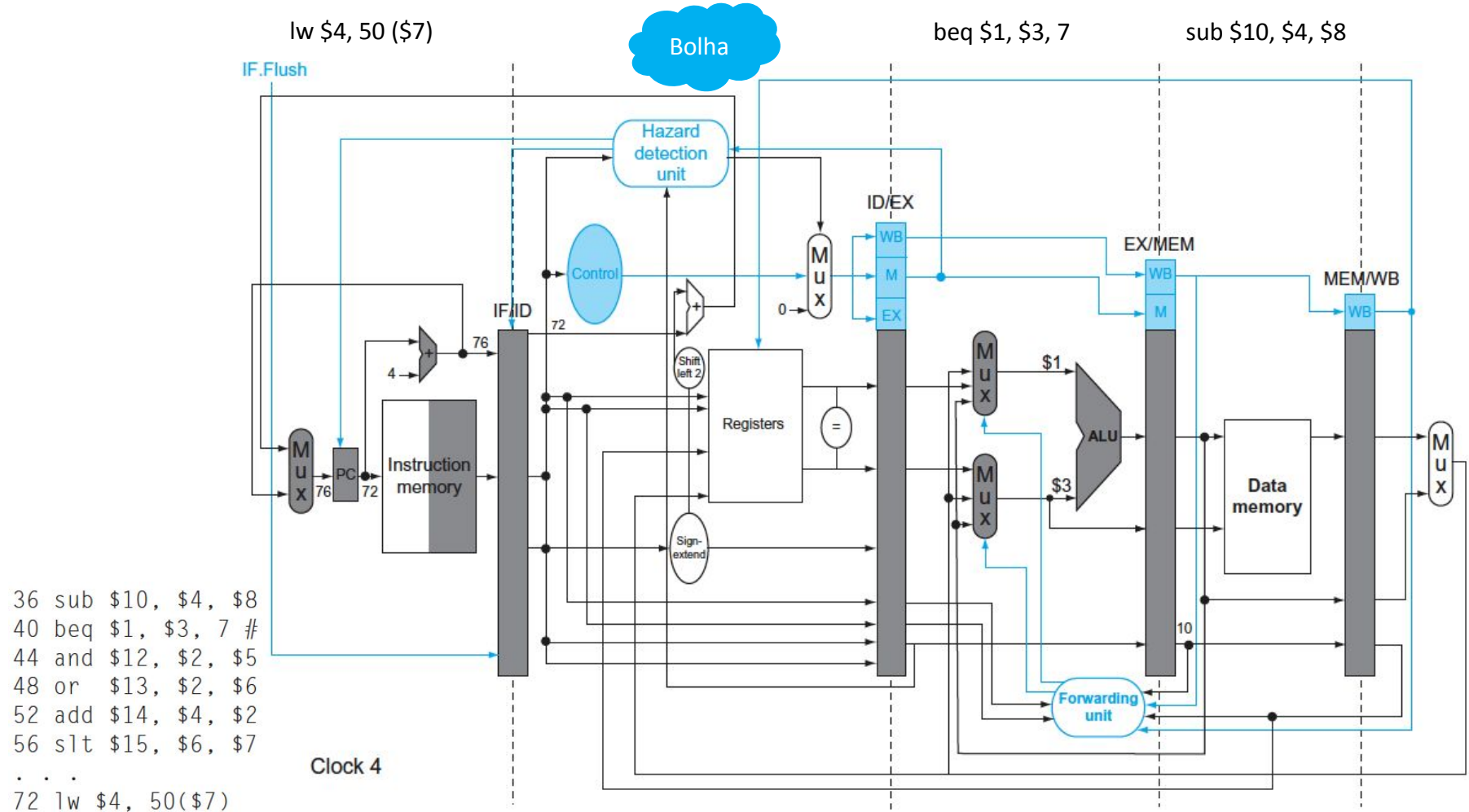
Hazards de Controle

- Ciclo 3 – Decisão será tomada neste ciclo



Hazards de Controle

- Ciclo 4 – Caso Taken – Flush no pipeline - Somente 1 bolha



Hazards de Controle

- Previsão Estática- Assumir branch Not Taken
 - Branch Delay Slot
 - Compilador encontra e insere uma instrução após a instrução
 - Esta instrução deve ser independente do branch
 - A instrução abaixo do branch sempre será executada, não pertencendo ao corpo condicional
 - Processador ciente que deverá executar a instrução após o branch

Código sem Delayed Branch

```
36 sub $10, $4, $8
40 beq $1, $3, 7 #
44 and $12, $2, $5
48 or  $13, $2, $6
52 add $14, $4, $2
56 slt $15, $6, $7
. . .
72 lw  $4, 50($7)
```

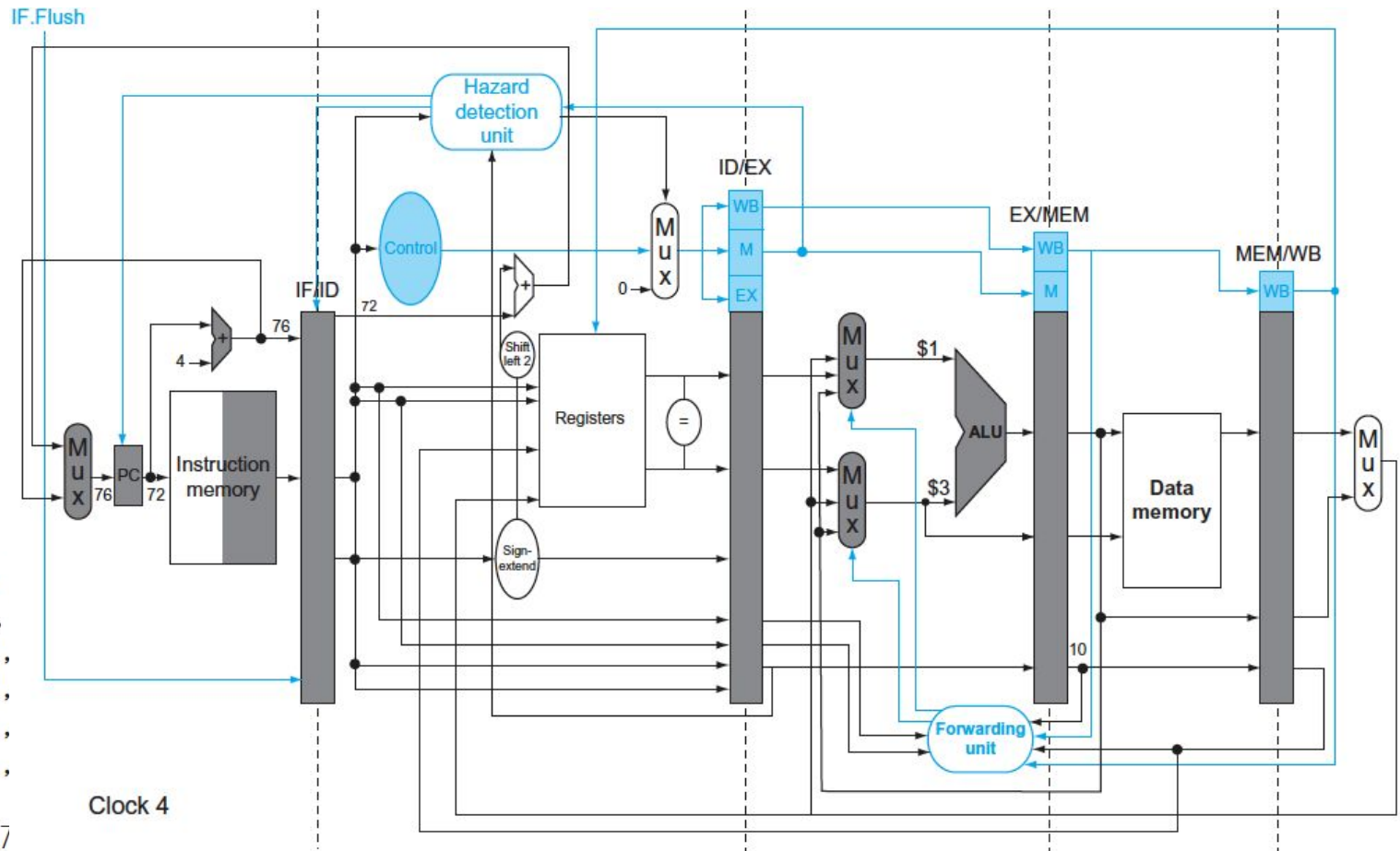
Código com Delayed Branch

```
36 beq $1, $3, 7 #
40 sub $10, $4, $8
44 and $12, $2, $5
48 or  $13, $2, $6
52 add $14, $4, $2
56 slt $15, $6, $7
. . .
72 lw  $4, 50($7)
```

Hazards de Controle

• Ciclo 1

beq \$1, \$3, 7



```

36 beq $1, $3,
40 sub $10, $4,
44 and $12, $2,
48 or $13, $2,
52 add $14, $4,
56 slt $15, $6,
. . .
72 lw $4, 50($7)
    
```

Clock 4

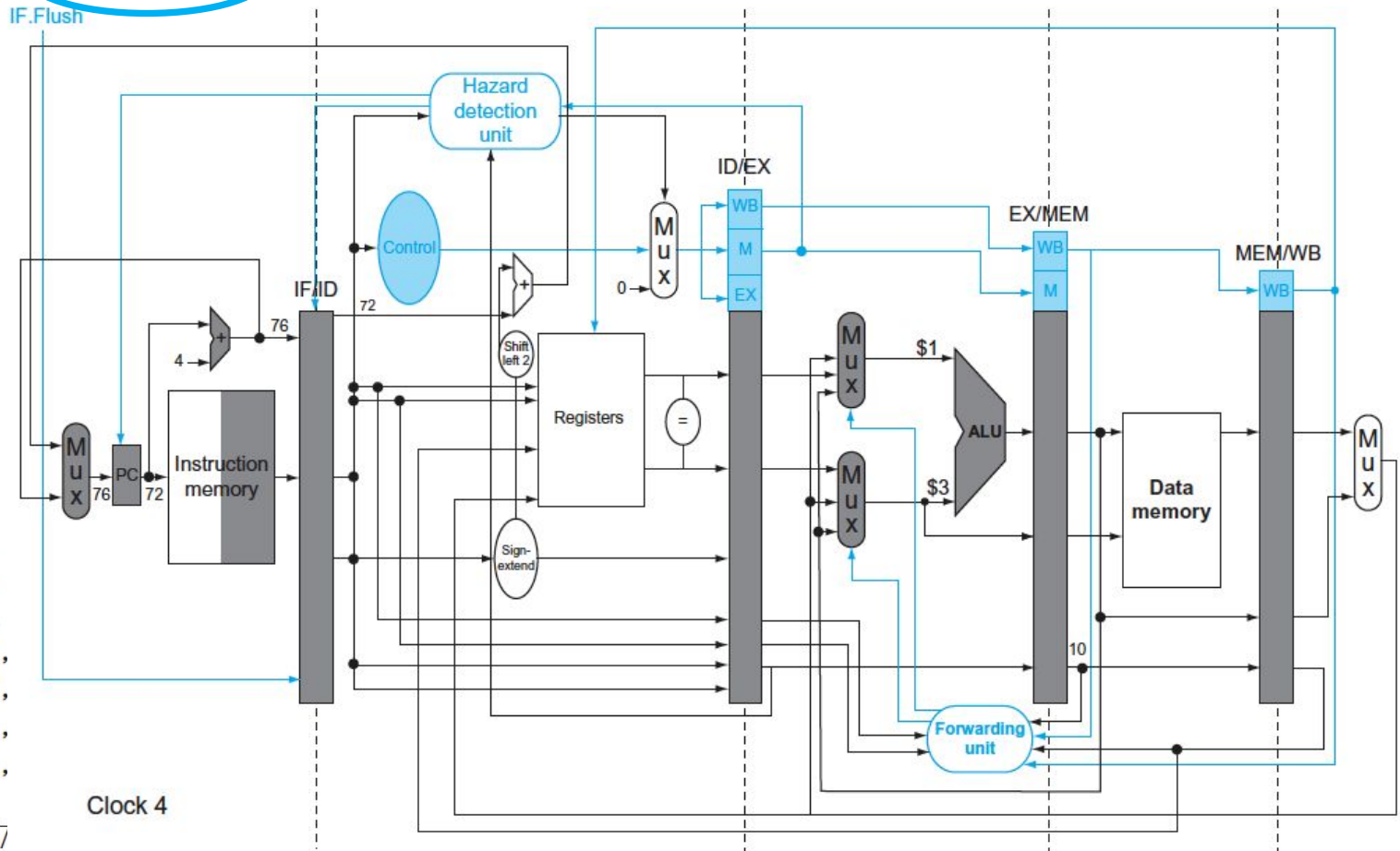
Hazards de Controle

• Ciclo 2

Instrução não condicional

sub \$10, \$4, \$8

beq \$1, \$3, 7



36 beq \$1, \$3,
40 sub \$10, \$4,
44 and \$12, \$2,
48 or \$13, \$2,
52 add \$14, \$4,
56 slt \$15, \$6,
...
72 lw \$4, 50(\$7)

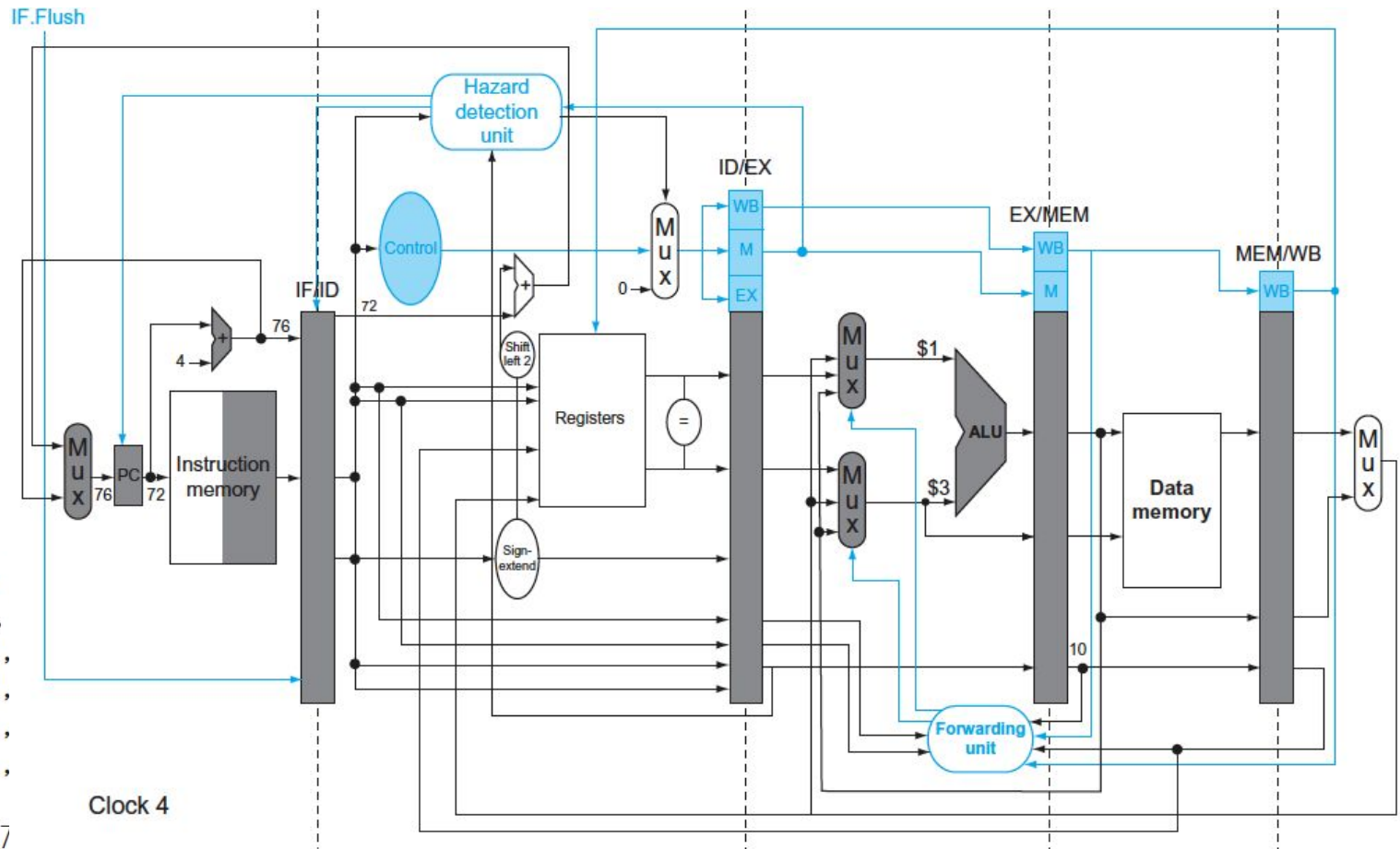
Clock 4

Hazards de Controle

- Ciclo 2 – Decisão será tomada neste ciclo

sub \$10, \$4, \$8

beq \$1, \$3, 7



```

36 beq $1, $3,
40 sub $10, $4,
44 and $12, $2,
48 or $13, $2,
52 add $14, $4,
56 slt $15, $6,
. . .
72 lw $4, 50($7)
    
```

Clock 4

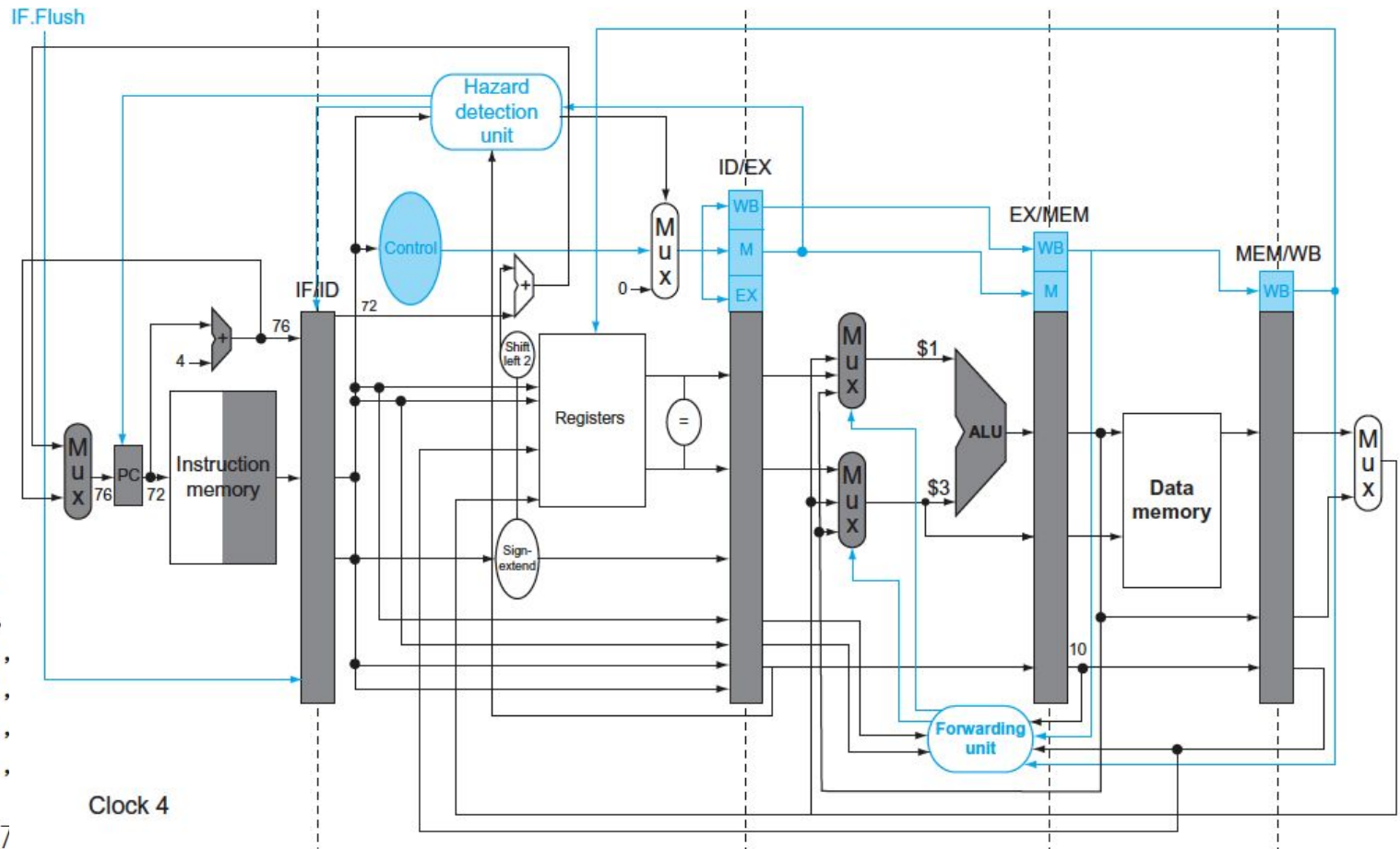
Hazards de Controle

- Ciclo 3 – Mesmo se o branch for taken, não haverá bolha

lw \$4, 50(\$7)

sub \$10, \$4, \$8

beq \$1, \$3, 7



```

36 beq $1, $3,
40 sub $10, $4,
44 and $12, $2,
48 or $13, $2,
52 add $14, $4,
56 slt $15, $6,
. . .
72 lw $4, 50($7)
    
```

Clock 4

Hazards de Controle

- Previsão Dinâmica
 - Decisão se a previsão será *taken* ou *not taken* é realizada em tempo de execução
 - Baseada no comportamento dos desvios no pipeline
 - Comportamento é adquirido em tempo de execução pelas repetidas execuções dos desvios (loops, chamadas de funções, etc)
 - Uma tabela é utilizada para armazenar o comportamento e decidir se a execução atual do desvio será *taken* ou *not taken*
 - Tabela é chamada de *Branch History Table (BHT)* e está situada no estágio de busca da instrução

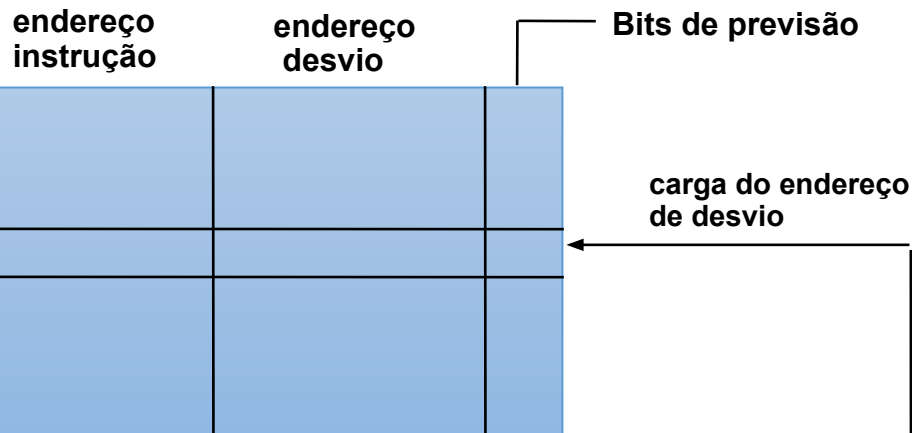
Previsão Dinâmica

Branch History Table

1º Execução do beq

Mem. Instr

```
0x80: beq  r1, r2, 0x90  
0x84: or   r8, r2, r6  
0x88: and  r13, r6, r2  
0x8C: add  r9, r4, r2  
0x90: sub  r7, r8, r9
```



PC

0x80

Memória
de
Instruções

IF

DEC

EX

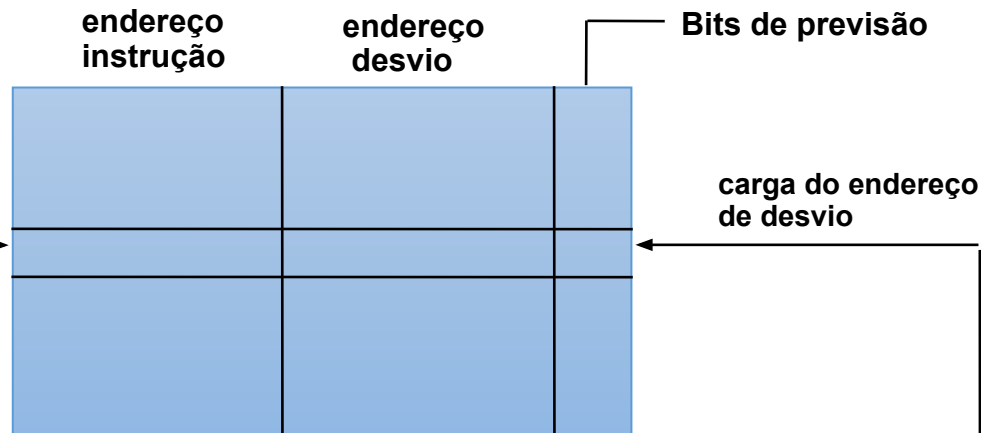
MEM

WB

beq

Previsão Dinâmica

Branch History Table



1º Execução do beq

Mem. Instr

```
0x80: beq  r1, r2, 0x90
0x84: or   r8, r2, r6
0x88: and  r13, r6, r2
0x8C: add  r9, r4, r2
0x90: sub  r7, r8, r9
```

PC

0x84

Memória
de
Instruções

IF

DEC

EX

MEM

WB

or

beq

Previsão Dinâmica

Branch History Table

endereço instrução	endereço desvio	Bits de previsão
0x80	0x90	NT

1º Execução do beq

Mem. Instr

```
0x80: beq  r1, r2, 0x90
0x84: or   r8, r2, r6
0x88: and  r13, r6, r2
0x8C: add  r9, r4, r2
0x90: sub  r7, r8, r9
```

PC

0x88

Memória de Instruções

IF

DEC

EX

MEM

WB

and

or

beq

Previsão Dinâmica

Branch History Table

endereço instrução	endereço desvio	Bits de previsão
0x80	0x90	NT

1º Execução do beq

Mem. Instr

```
0x80: beq  r1, r2, 0x90
0x84: or   r8, r2, r6
0x88: and  r13, r6, r2
0x8C: add  r9, r4, r2
0x90: sub  r7, r8, r9
```

PC

0x8C

Memória de Instruções

IF

DEC

EX

MEM

WB

add

and

or

beq

Previsão Dinâmica

Branch History Table

endereço instrução	endereço desvio	Bits de previsão
0x80	0x90	NT

2ª Execução do beq

Mem. Instr

```
0x80: beq  r1, r2, 0x90
0x84: or   r8, r2, r6
0x88: and  r13, r6, r2
0x8C: add  r9, r4, r2
0x90: sub  r7, r8, r9
```

PC

0x80

Memória de Instruções

IF

DEC

EX

MEM

WB

beq

Previsão Dinâmica

Branch History Table

endereço instrução	endereço desvio	Bits de previsão
0x80	0x90	NT

2º Execução do beq

Mem. Instr

```
0x80: beq  r1, r2, 0x90
0x84: or   r8, r2, r6
0x88: and  r13, r6, r2
0x8C: add  r9, r4, r2
0x90: sub  r7, r8, r9
```

PC

0x80

Memória de Instruções

IF

DEC

EX

MEM

WB

beq

Dependendo do Algoritmo de Previsão – Poderia decidir por Taken

Previsão Dinâmica

Branch History Table

endereço instrução	endereço desvio	Bits de previsão
0x80	0x90	NT

2º Execução do beq

Mem. Instr

```
0x80: beq  r1, r2, 0x90
0x84: or   r8, r2, r6
0x88: and  r13, r6, r2
0x8C: add  r9, r4, r2
0x90: sub  r7, r8, r9
```

PC

0x90

Memória de Instruções

IF

DEC

EX

MEM

WB

sub

beq

Dependendo do Algoritmo de Previsão – Poderia decidir por Taken

- Branch History Table armazena:
 - Endereços de memória das instruções de desvio condicional executadas
 - Endereços de memória de salto destes desvios
 - Bits de previsão, indicando qual deve ser a próxima previsão deste desvio
- Processo de previsão: Em todo ciclo de relógio o PC é comparado a todos os endereços de instruções já armazenados na tabela:
 - se endereço é encontrado, a instrução de branch é identificada, os bits de previsão da linha da BHT são utilizados para realizar a previsão
 - No próximo ciclo o PC será atualizado para PC+4 ou para o endereço de salto que está na BHT dependendo do que os bits de previsão indicam
 - se o endereço não for encontrado, o PC +4 é buscado no próximo ciclo
 - Se, no estágio de decodificação, a instrução seja identificada como branch, o conteúdo do PC - 4 (endereço de memória deste branch) é inserido na BHT; e no estágio de execução, endereço de salto do desvio e bits de previsão são atualizados na tabela

Previsão Dinâmica



- Algoritmos de previsão são utilizados para atualizar os bits de previsão da BHT
 - Exemplos:
 - Predição de 1 bit
 - Predição de 2 bits (Bimodal)
 - Tournament

Exemplos

```
for (i=0;i<4;i++)
{
    a=b+c;
}
```

```
$L3:
    lw    $3,12($fp)
    lw    $2,16($fp)
    addu  $2,$3,$2
    sw    $2,20($fp)
    lw    $2,8($fp)
    addiu $2,$2,1
    sw    $2,8($fp)

$L2:
    lw    $2,8($fp)
    slt   $2,$2,4
    bne $2,$0,$L3 (taken - corpo do loop)
    move  $sp,$fp (not taken - PC + 4)
    lw    $fp,28($sp)
    addiu $sp,$sp,32
    j     $31
    nop
```

```
for (i=0;i<4;i++)
{
    if(i%2)
        a=b+c;
    else
        a=b-c;
}
```

```
$L5:
    lw $2,8($fp)
    andi $2,$2,0x1
    beq $2,$0,$L3 //branch do if
    lw $3,12($fp) //corpo do if (not taken - PC + 4)
    lw $2,16($fp)
    addu $2,$3,$2
    sw $2,20($fp)
    b $L4 // branch saltar corpo do else

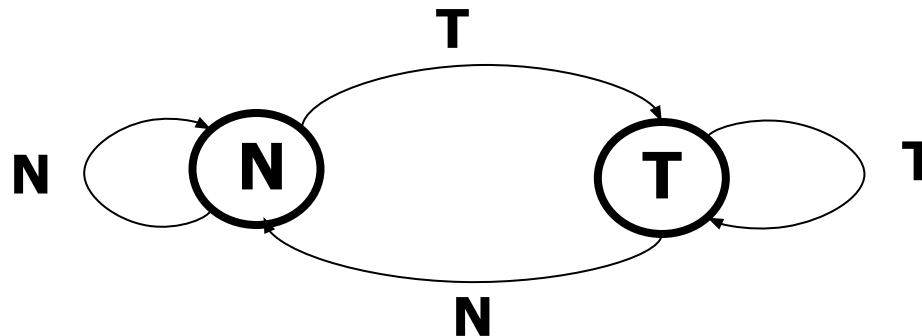
$L3:
    lw $3,12($fp) // corpo do else (taken - corpo do else)
    lw $2,16($fp)
    subu $2,$3,$2
    sw $2,20($fp)

$L4:
    lw $2,8($fp) //controle do loop
    addiu $2,$2,1
    sw $2,8($fp)

$L2:
    lw $2,8($fp)
    slt   $2,$2,4
    bne $2,$0,$L5 // branch do loop
    move  $sp,$fp
    lw $fp,28($sp)
    addiu $sp,$sp,32
    j     $31
```

Predição de 1 bit

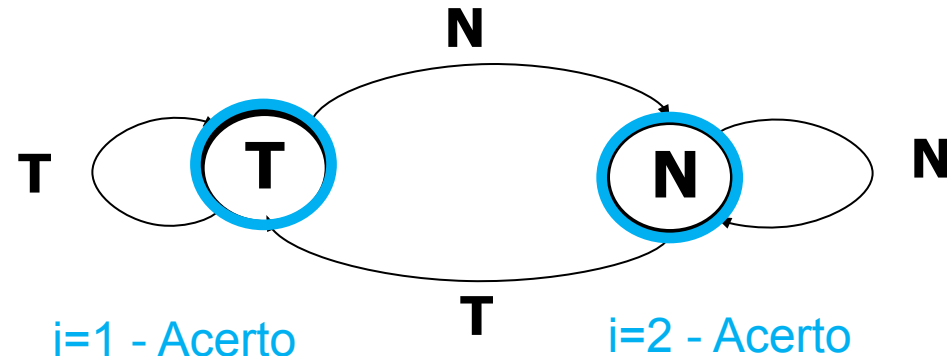
- Somente um bit indica se o branch será taken ou not taken
 - T – taken, N – not taken
- Atualização da tabela segue a máquina de estados abaixo:



- A alteração da previsão ocorre no primeiro erro

Predição de 1 bit

```
for (i=0;i<4;i++) (taken)
{
    a= b+c;
}
(not taken)
```



i=0 - Erro

End I	End S	
0000		Not
	Tak	

i=1 - Acerto

End I	End S	
0000	1000	
	Taken	

i=2 - Acerto

End I	End S	
0000	1000	Taken

i=3 - Acerto

End I	End S	
0000	1000	Taken

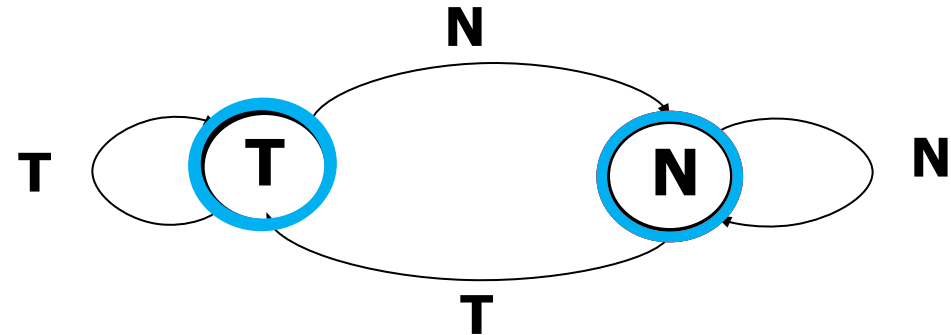
i=4 - Erro

End I	End S	
0000	1000	
	Taken	

End I	End S	
0000	1000	Not Tak

Predição de 1 bit

```
for (i=0;i<4;i++)
{
    if(i%2)
        (not taken)
    }else{
        (taken)
    }
}
```



i=0 - Erro

End I	End S	
1000		Not
	Tak	

i=1 - Erro

End I	End S	
1000	1111	Taken

i=2 - Erro

End I	End S	
1000	1111	Not Tak

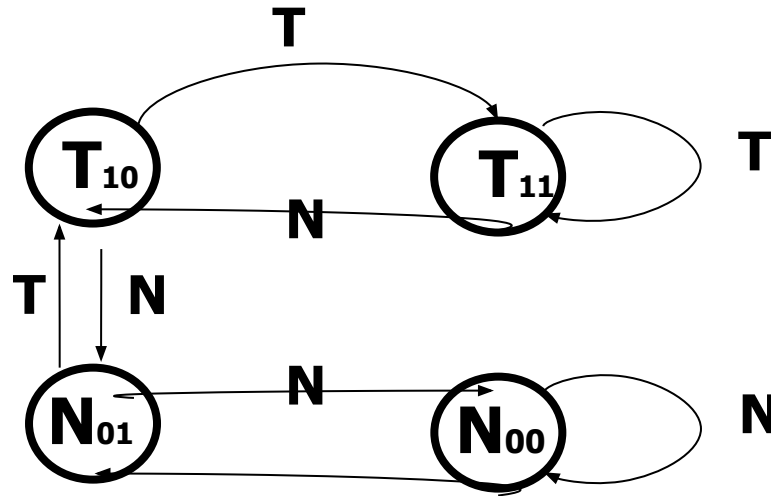
i=3 - Erro

End I	End S	
1000	1111	
	Taken	

End I	End S	
1000	1111	Not
	Tak	

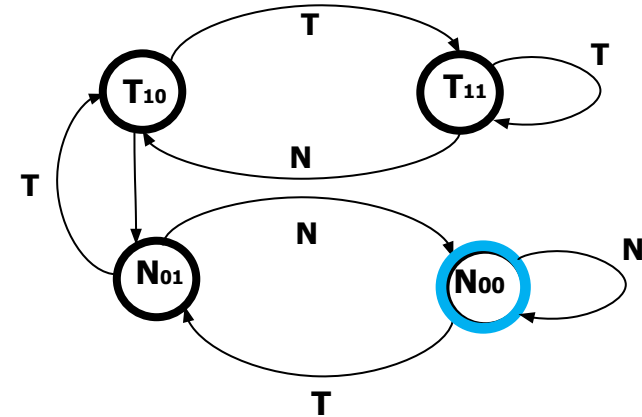
Predição Bimodal

- Dois bits armazenados na BHT informam qual será a previsão
- Utilizada no Pentium I



Predição Bimodal

```
for (i=0;i<4;i++) (taken)
{
    a= b+c;
}
(not taken)
```



i=0 - Erro

End I	End S	
0000		Not
	Tak	

i=1 - Erro

End I	End S	
0000	1000	Not
	Tak	

i=2 - Acerto

End I	End S	
0000	1000	Taken

i=3 - Acerto

End I	End S	
0000	1000	
	Taken	

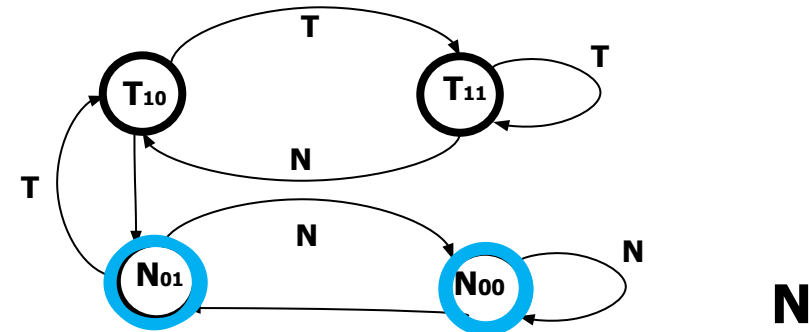
i=4 - Erro

End I	End S	
0000	1000	
	Taken	

End I	End S	
0000	1000	Taken

Predição Bimodal

```
for (i=0;i<4;i++)
{
    if(i%2)
        (not taken)
    }else{
        (taken)
    }
}
```



i=0 - Erro

End I	End S	
1000		Not
	Tak	

i=1 - Acerto

End I	End S	
1000	1111	Not Tak

T i=2 - Erro

End I	End S	
1000	1111	Not Tak

i=3 - Acerto

End I	End S	
1000	1111	Not
	Tak	

End I	End S	
1000	1111	Not
	Tak	

Exercício

Supondo que os processadores descritos abaixo executem a aplicação A. Qual o tempo cada processador leva para executar a aplicação? Quão mais eficiente um processador é em relação ao outro?

Processador A

MIPS Pipeline 5 estágios

333MHz

Cálculo BEQ 3 estágio

Não implementa forwarding

Preditor Desvios Estático Not Taken

Processador B

MIPS Pipeline 5 estágios

100MHz

Cálculo BEQ 2 estágio

Implementa forwarding

Preditor Desvios Estático Not Taken

Aplicação A

1.200.000 instruções

BEQ = 20% (40% taken e 60% not taken)

ADD/SUB = 38%

LW = 22%

SW = 20%

Existem dependências RAW - 25% ADD/SUB - instruções subsequentes

add \$1,\$2,\$3

sub \$6,\$1,\$7

Tex Aplicação A no Processador A

Tciclo = $1/\text{Freq} = 1/333\text{MHz} = 1/333 \times 10^6 \text{ Hz} = 0,003 \times 10^{-6} \text{ seg} = 0,003 \text{ useg} = 3 \text{ nseg}$

BEQ - 20% - 60% - CPI = 1

40% - CPI = 4

LW - 22% - CPI = 1

SW - 20% - CPI = 1

ADD/SUB - 38% - 75% - CPI = 1

25% - CPI = 3

$\text{CPI} = 0.2 \times (0.6 \times 1 + 0.4 \times 4) + 0.22 \times 1 + 0.2 \times 1 + 0.38 \times (0.75 \times 1 + 0.25 \times 3)$

$\text{CPI} = 0.44 + 0.22 + 0.2 + 0.57 = 1.43$

$\text{Tex} = 1.43 \times 1.2 \times 10^6 \text{ instr} \times 3 \times 10^{-9} \text{ seg} = 5.148 \times 10^{-3} \text{ seg}$

Tex Aplicação A no Processador B

Tciclo = $1/\text{Freq} = 1/100\text{MHz} = 1/100 \times 10^6 \text{ Hz} = 0,01 \times 10^{-6} \text{ seg} = 0,01 \text{ useg} \times 10^{-3} = 10 \text{ nseg}$

BEQ - 20% - 60% - CPI = 1

40% - CPI = 2

LW - 22% - CPI = 1

SW - 20% - CPI = 1

ADD/SUB - 38% - 75% - CPI = 1

25% - CPI = 1

$\text{CPI} = 0.2 \times (0.6 \times 1 + 0.4 \times 2) + 0.22 \times 1 + 0.2 \times 1 + 0.38 \times 1$

$\text{CPI} = 0.28 + 0.22 + 0.2 + 0.38 = 1.08$

$\text{Tex} = 1.08 \times 1.2 \times 10^6 \text{ instr} \times 10 \times 10^{-9} \text{ seg} = 12.96 \times 10^{-3} \text{ seg}$

$\text{Speedup} = \text{Proc B} / \text{Proc A} = 12.96 \times 10^{-3} \text{ seg} / 5.148 \times 10^{-3} \text{ seg} = 2,51 \text{ vezes}$

Exceções no Pipeline

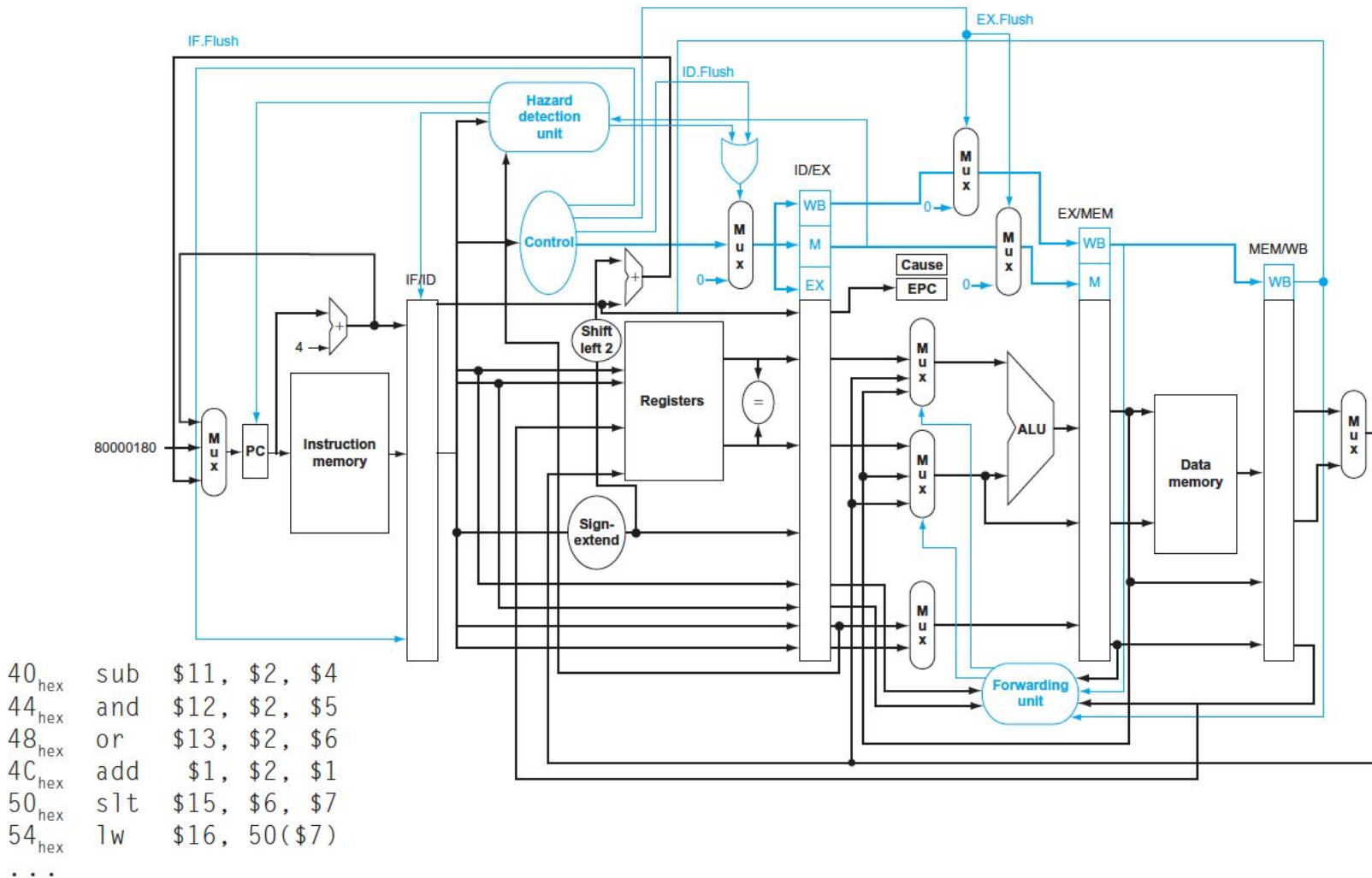
- Baseado na mesma metodologia do Multiciclo
 - Registrador EPC armazena o endereço da instrução causadora da exceção (Na verdade, armazena o PC+4, deve-se subtrair 4)
 - Registrador Status contém o tipo da exceção
 - Overflow ou Instrução indefinida
 - Para implementar a manipulação via registrador status o pipeline deve:
 - ser modificado para saltar para o endereço de decodificação da exceção (0x8000 0180)
 - dar flush das instruções que estão nos estágios anteriores aquela que gerou a interrupção

Exceções no Pipeline

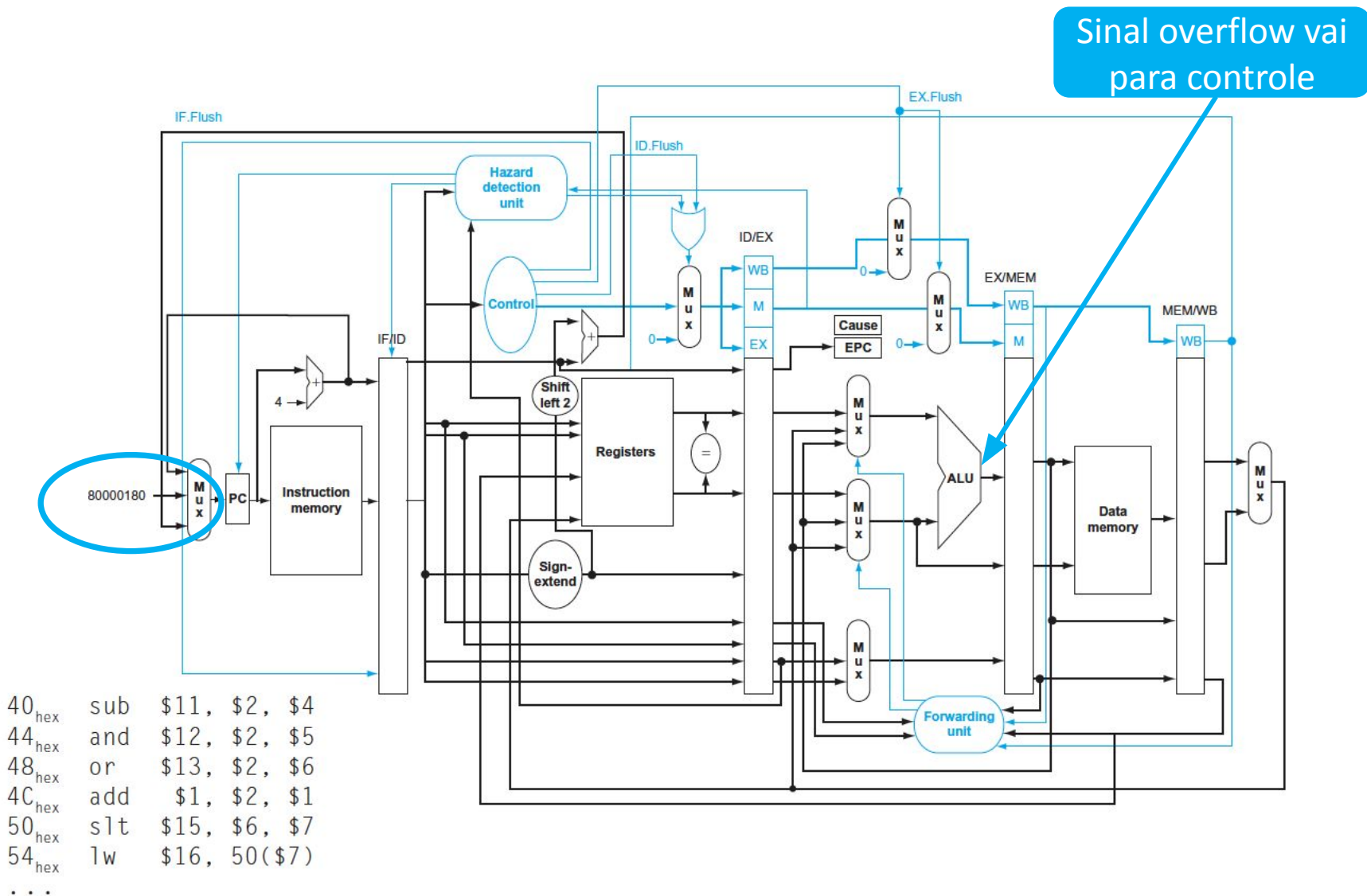
- Dada a sequência de instruções abaixo
 - Supondo que a instrução add cause uma exceção de overflow

```
40hex  sub  $11, $2, $4
44hex  and  $12, $2, $5
48hex  or   $13, $2, $6
4Chex  add   $1, $2, $1
50hex  slt  $15, $6, $7
54hex  lw   $16, 50($7)
... 
```

Exceções no Pipeline



Exceções no Pipeline



Exceções no Pipeline

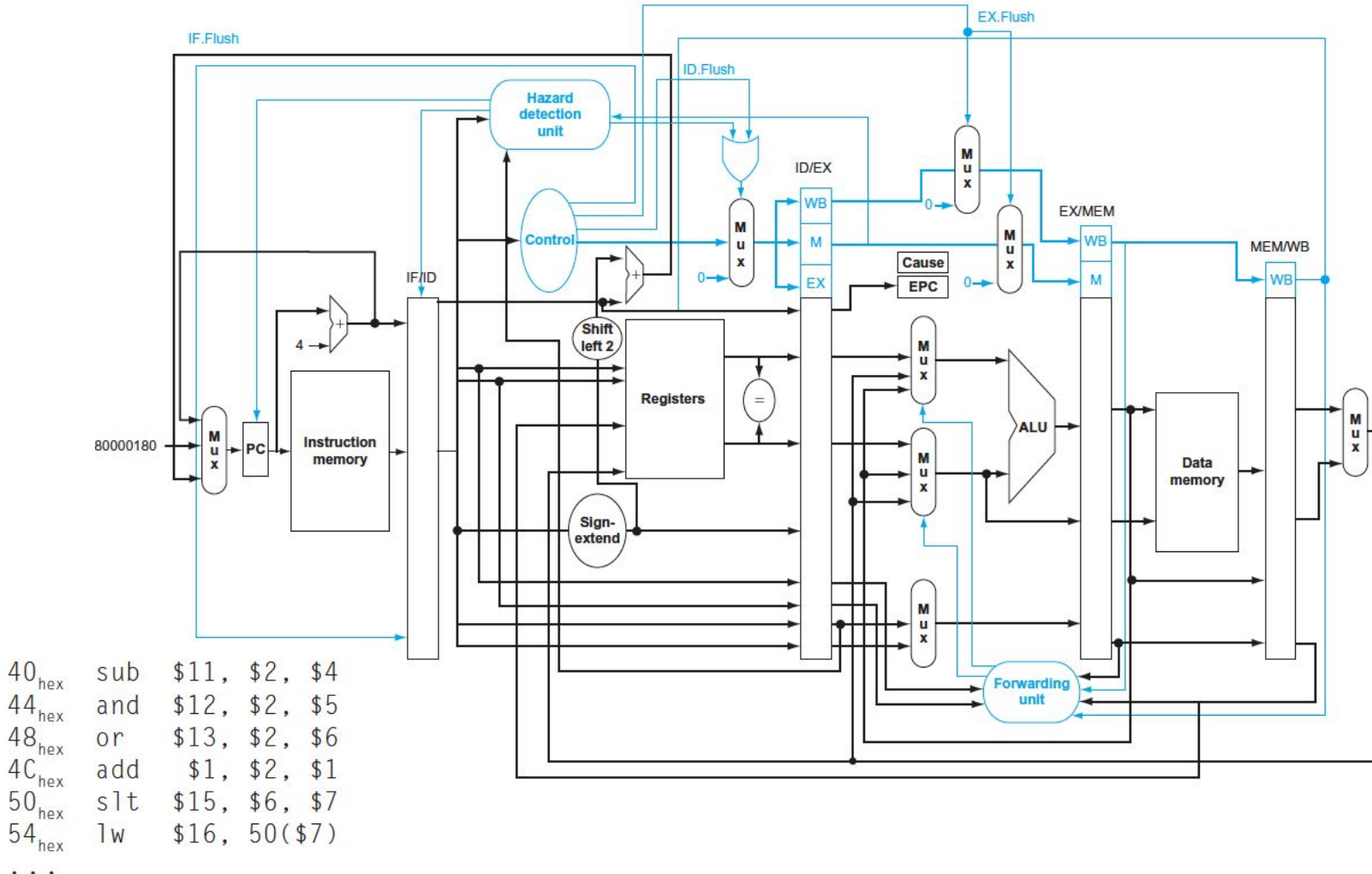
slt \$15, \$6, \$7

add \$1, \$2, \$1

or \$13, \$2, \$6

and \$12, \$2, \$5

sub \$12, \$2, \$4



Exceções no Pipeline

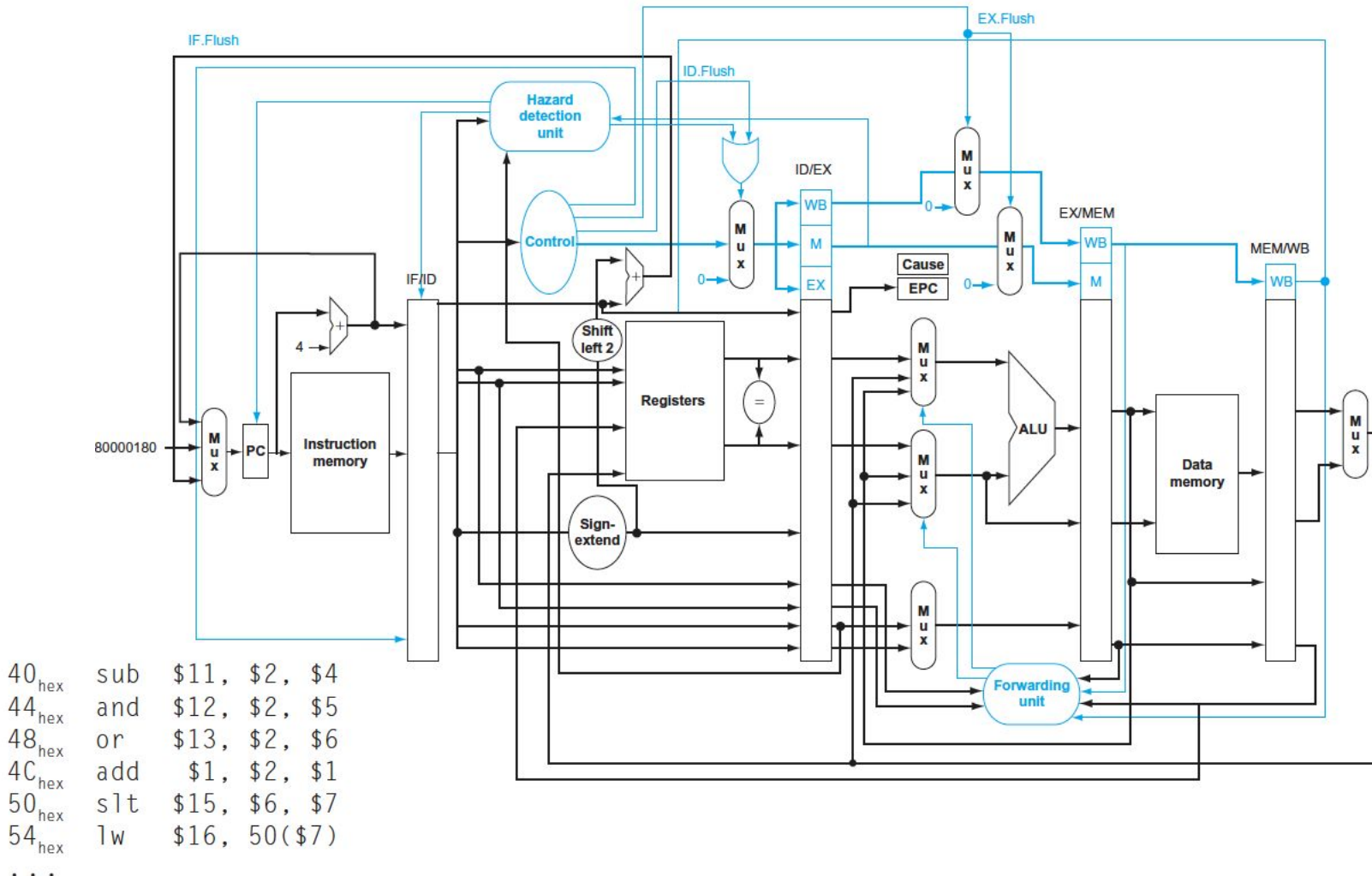
lw \$16, 50(\$7)

slt \$15, \$6, \$7

add \$1, \$2, \$1

or \$13, \$2, \$6

and \$12, \$2, \$5



Exceções no Pipeline

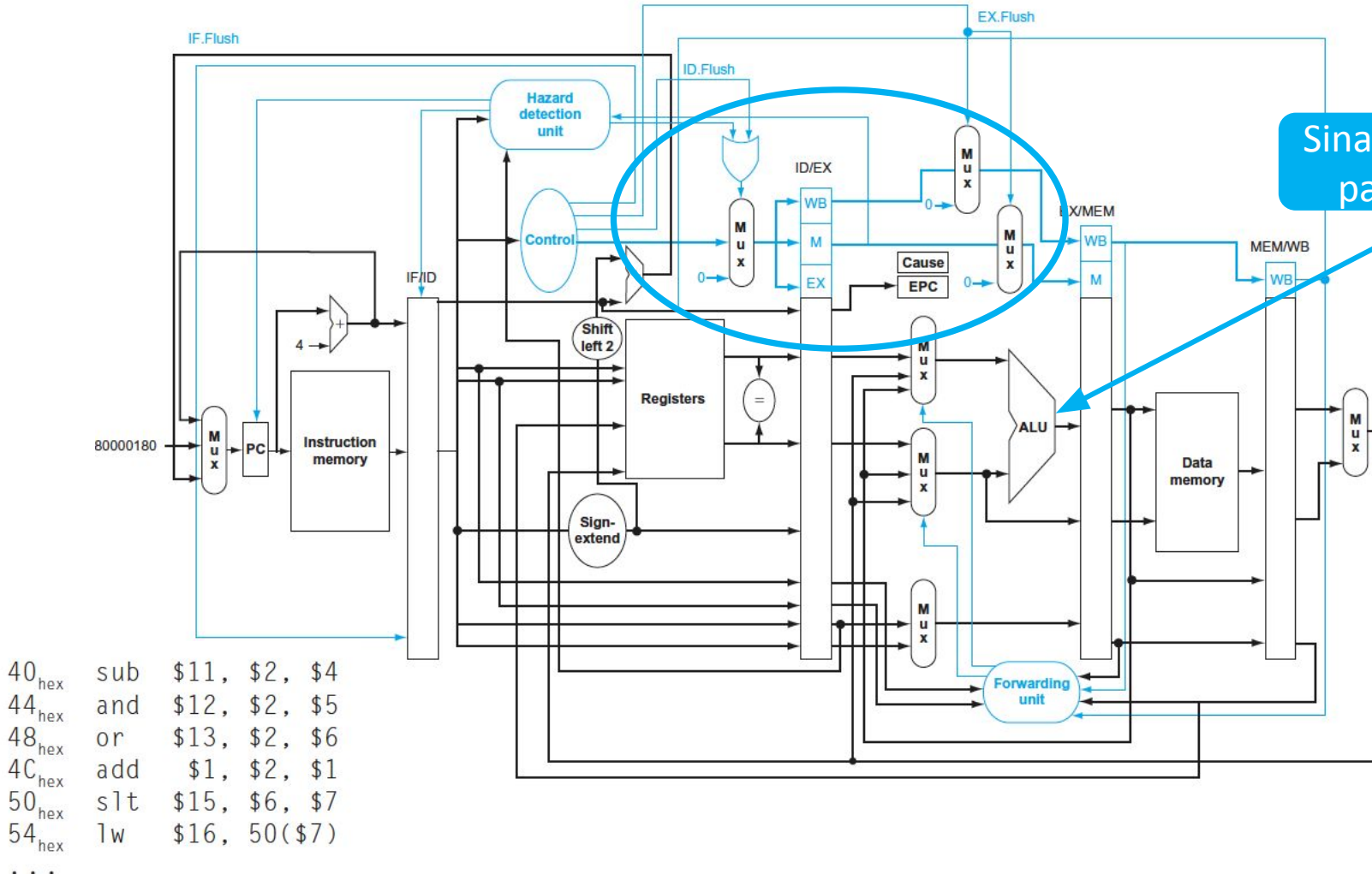
lw \$16, 50 (\$7)

slt \$15, \$6, \$7

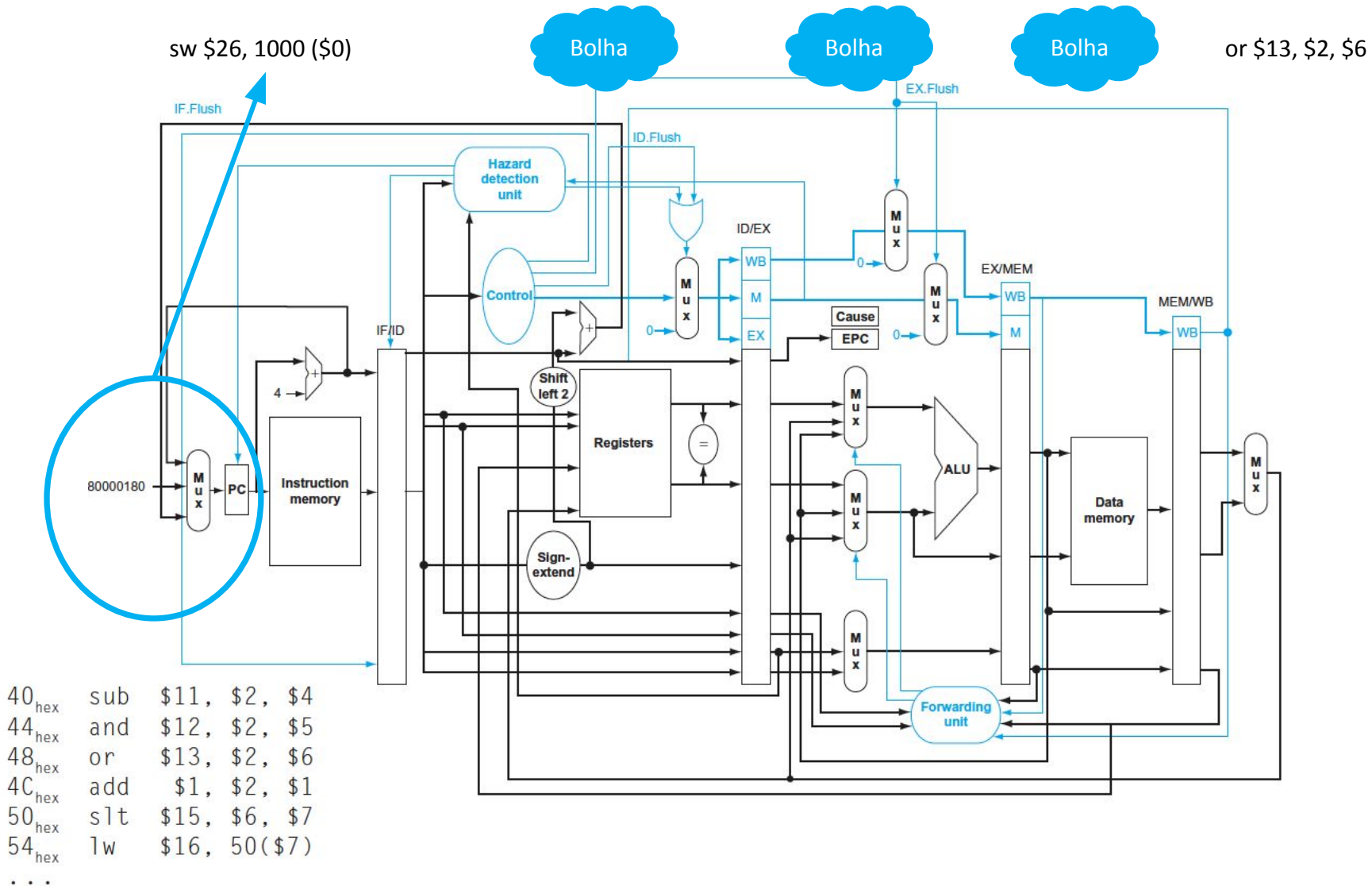
add \$1, \$2, \$1

or \$13, \$2, \$6

and \$12, \$2, \$5



Exceções no Pipeline



Paralelismo via Instruções

- Pipelining explora paralelismo via instruções
 - ILP (Instruction Level Paralellism)
 - Existem 2 métodos para aumentar a exploração do ILP
 - Aumentar o número de estágios de pipeline
 - Mais instruções concomitantes sendo executadas
 - Mais estágios – Menor caminho crítico – Maior frequência de operação
 - Problemas com misprediction
 - Multiple Issue
 - Issue é a capacidade de enviar instruções para serem executadas em unidades funcionais
 - Acoplar mais unidades funcionais para executar mais de uma instrução por ciclo (mais de uma instrução em cada estágio de pipeline)
 - $CPI < 1$ ou $IPC > 1$ (Instruction per Cycle)

Paralelismo via Instruções

- Dois modos de implementar Multiple Issue:
 - Static
 - Paralelismo é extraído em tempo de compilação
 - Instruções podem ser empacotadas pelo compilador para serem executadas em paralelo pelo processador
 - Compilador é responsável por lidar com as dependências de controle e dados
 - Empacotar instruções sem dependências para que possam ser executadas em paralelo
 - Exemplos: Processadores VLIW- Very Long Instruction Word (Intel Itanium)
 - Dynamic
 - Paralelismo é extraído em tempo de execução por HW dedicado
 - Processador é responsável por lidar com as dependências de controle e dados
 - Exemplos: Processadores Superescalares (Intel Pentium - Intel Core i7)

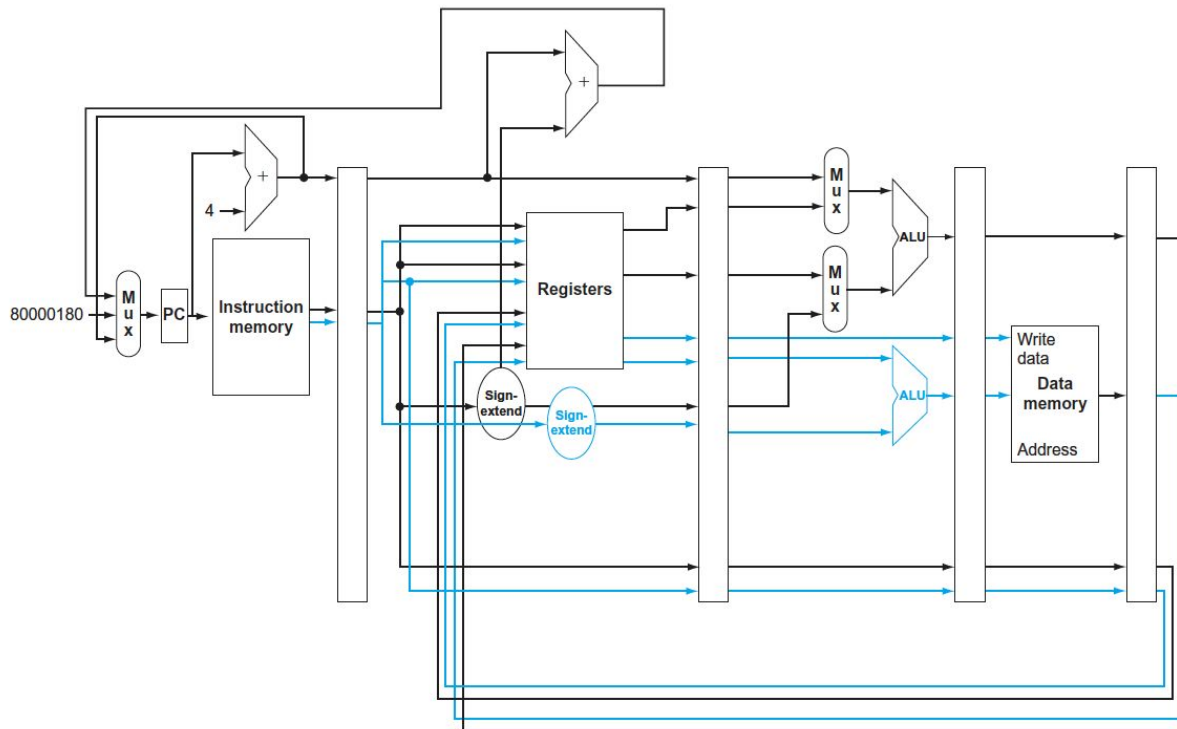
Paralelismo via Instruções

- MIPS Static Multiple Issue (two-Issue):
 - Duas Unidades funcionais
 - Uma para executar instruções lógicas e aritméticas ou branch
 - Uma para cálculo de endereço de load ou store
 - Compilador deve empacotar 2 instruções de 32 bits alinhadas as unidades funcionais, perfazendo uma instrução VLIW de 64-bits
 - Caso compilador não consiga empacotar 2 instruções, um NOP deve ser colocado no slot da instrução

Instruction type	Pipe stages							
ALU or branch instruction	IF	ID	EX	MEM	WB			
Load or store instruction	IF	ID	EX	MEM	WB			
ALU or branch instruction		IF	ID	EX	MEM	WB		
Load or store instruction		IF	ID	EX	MEM	WB		
ALU or branch instruction			IF	ID	EX	MEM	WB	
Load or store instruction			IF	ID	EX	MEM	WB	
ALU or branch instruction				IF	ID	EX	MEM	WB
Load or store instruction				IF	ID	EX	MEM	WB

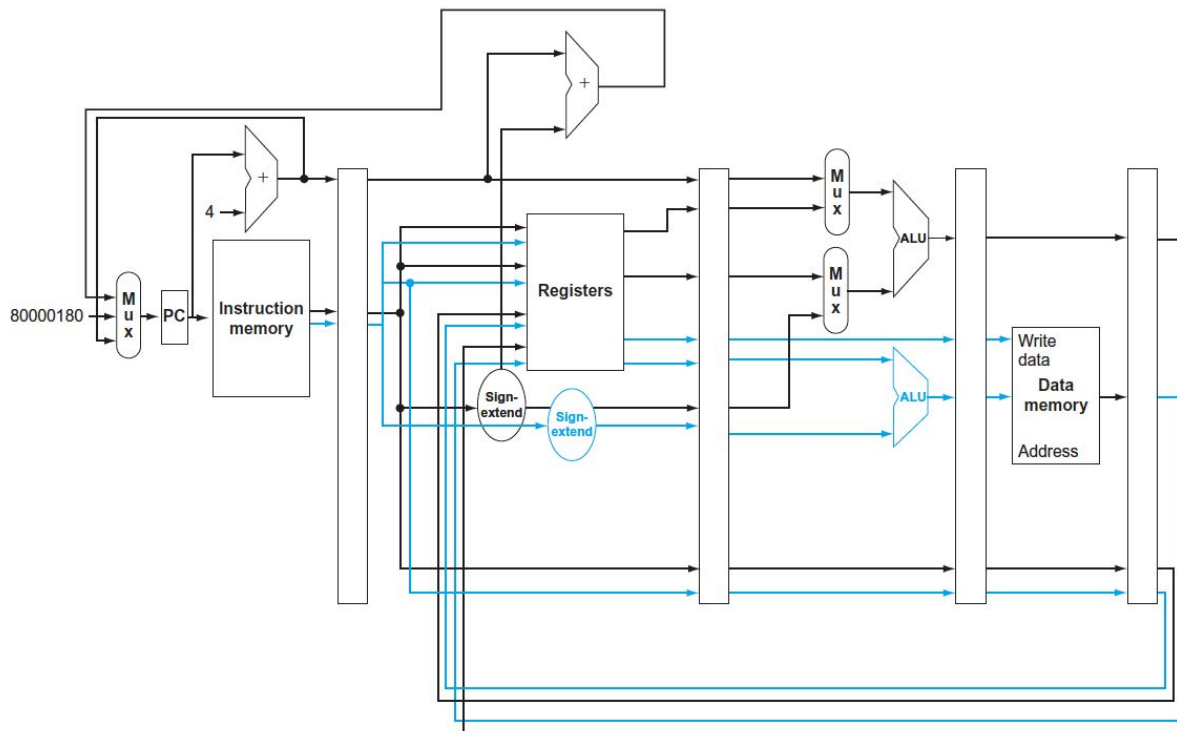
Paralelismo via Instruções

- MIPS Static two-Issue (HW adicionado destacado em azul)
 - 1 porta de leitura na memória de instruções (IF)
 - 2 portas de leitura e 1 porta de escrita no banco de registradores (ID)
 - 1 SigExt para calcular o endereço do load/store (ID)
 - 1 somador para calcular o endereço do load/store (EX)



Paralelismo via Instruções

- MIPS Static two-Issue (HW adicionado destacado em azul)
- LOAD tem um **use latency** de um ciclo:
 - As 2 instruções empacotadas na próxima instrução VLIW não poderão ir para o estágio de execução (EX) (mesmo com forwarding) se necessitarem do dado buscado na memória pelo LOAD. Neste caso, deve ser dado um stall de um ciclo no pipeline.



Paralelismo via Instruções

- Processo de Scheduling do Compilador para gerar Instruções MIPS VILW 64-bits

- Exemplo de um loop

```

Loop: lw    $t0, 0($s1)    # $t0=array element
      addu  $t0,$t0,$s2# add scalar in $s2
      sw    $t0, 0($s1)# store result
      addi  $s1,$s1,-4# decrement pointer
      bne   $s1,$zero,Loop# branch $s1!=0
  
```

- Código do loop escalonado e **reordenado** para VLIW 64 bits

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:		lw \$t0, 0(\$s1)	1
	addi \$s1,\$s1,-4		2
	addu \$t0,\$t0,\$s2		3
	bne \$s1,\$zero,Loop	sw \$t0, 4(\$s1)	4

- Execução no MIPS single-issue com forwarding e reordenamento do addu
 - 5 instruções em 5 ciclos (supondo pipeline cheio) - IPC = 1
- Execução no MIPS VLIW two-issue
 - 5 instruções em 4 ciclos (supondo pipeline cheio) - IPC = 1.25 (best case IPC = 2)

Paralelismo via Instruções

- Loop Unrolling

- Técnica de compiladores para aumentar desempenho na execução de loops
- Passos:
 1. Fazer a cópia do código quatro iterações do loop anterior

```
Loop: lw    $t0, 0($s1)    # $t0=array element
      addu  $t0,$t0,$s2# add scalar in $s2
      sw    $t0, 0($s1)# store result

      lw    $t0, 0($s1)    # $t0=array element
      addu  $t0,$t0,$s2# add scalar in $s2
      sw    $t0, 0($s1)# store result

      lw    $t0, 0($s1)    # $t0=array element
      addu  $t0,$t0,$s2# add scalar in $s2
      sw    $t0, 0($s1)# store result

      lw    $t0, 0($s1)    # $t0=array element
      addu  $t0,$t0,$s2# add scalar in $s2
      sw    $t0, 0($s1)# store result
      addi  $s1,$s1,-16 decrement pointer
      bne   $s1,$zero,Loop# branch $s1!=0
```

Elemento 0 array

Elemento 1 array

Elemento 2 array

Elemento 3 array

Paralelismo via Instruções

- Loop Unrolling

- Técnica de compiladores para aumentar desempenho na execução de loops
- Passos:

1. Fazer a cópia do código quatro iterações do loop anterior
2. Renomear os registradores que geram dependências falsas

```
Loop: lw    $t0, 0($s1)    # $t0=array element
      addu  $t0,$t0,$s2    # add scalar in $s2
      sw    $t0, 0($s1)    # store result

      lw    $t0, 0($s1)    # $t0=array element
      addu  $t0,$t0,$s2    # add scalar in $s2
      sw    $t0, 0($s1)    # store result

      lw    $t0, 0($s1)    # $t0=array element
      addu  $t0,$t0,$s2    # add scalar in $s2
      sw    $t0, 0($s1)    # store result

      lw    $t0, 0($s1)    # $t0=array element
      addu  $t0,$t0,$s2    # add scalar in $s2
      sw    $t0, 0($s1)    # store result
      addi  $s1,$s1,-4     # decrement pointer
      bne   $s1,$zero,Loop # branch $s1!=0
```

Renomear para \$t1

Renomear para \$t2

Renomear para \$t3

Paralelismo via Instruções

- Loop Unrolling

- Técnica de compiladores para aumentar desempenho na execução de loops
- Passos:
 1. Fazer a cópia do código quatro iterações do loop anterior
 2. Renomear os registradores que geram dependências falsas
 3. Realizar o scheduling das instruções

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:	addi \$s1,\$s1,-16	lw \$t0, 0(\$s1)	1
		lw \$t1, 12(\$s1)	2
	addu \$t0,\$t0,\$s2	lw \$t2, 8(\$s1)	3
	addu \$t1,\$t1,\$s2	lw \$t3, 4(\$s1)	4
	addu \$t2,\$t2,\$s2	sw \$t0, 16(\$s1)	5
	addu \$t3,\$t3,\$s2	sw \$t1, 12(\$s1)	6
		sw \$t2, 8(\$s1)	7
	bne \$s1,\$zero,Loop	sw \$t3, 4(\$s1)	8

- Paralelismo entre instruções de diferentes iterações do loop
 - Execução no MIPS VLIW two-issue: $IPC = 14/8$ – $IPC = 1.75$ (best case $IPC = 2$)

Paralelismo via Instruções

- Dynamic Multiple Issue (Superscalar Processors)
 - Instruções chegam no processador superscalar da mesma forma que chegam em organizações mono, multiciclo, pipe, etc.
 - Hardware acoplado ao processador é responsável por explorar o paralelismo em nível de instrução
 - Dependências são resolvidas por hardware
 - Vantagens sobre Static Issue (VLIW) :
 - Mantém a compatibilidade binária de organizações anteriores (mono, multiciclo, pipeline single-issue, etc)
 - Evoluções de organizações superescalares não requerem recompilação do código
 - Desvantagens sobre Static Issue (VLIW):
 - Hardware de detecção de ILP consome potência significativa

Paralelismo via Instruções

- In-order Dynamic Multiple Issue (Superscalar Processors)
 - Instruções são enviadas para as unidades funcionais na ordem que foram compiladas
 - No exemplo abaixo:
 - Existe uma dependência verdadeira (RAW) criada entre as instruções LW e ADDU
 - Esta dependência impossibilita o envio das instruções subsequentes para as unidades funcionais enquanto a instrução ADDU não seja executada

```
lw      $t0, 20($s2)
addu    $t1, $t0, $t2
sub      $s4, $s4, $t3
slti    $t5, $s4, 20
```

- Hardware de exploração de paralelismo muito mais simples
- Limitados ganhos na exploração de paralelismo de instruções
- Exemplo: ARM Cortex A8, Intel Pentium MMX (1993), Intel Atom (2008-hoje)

Paralelismo via Instruções

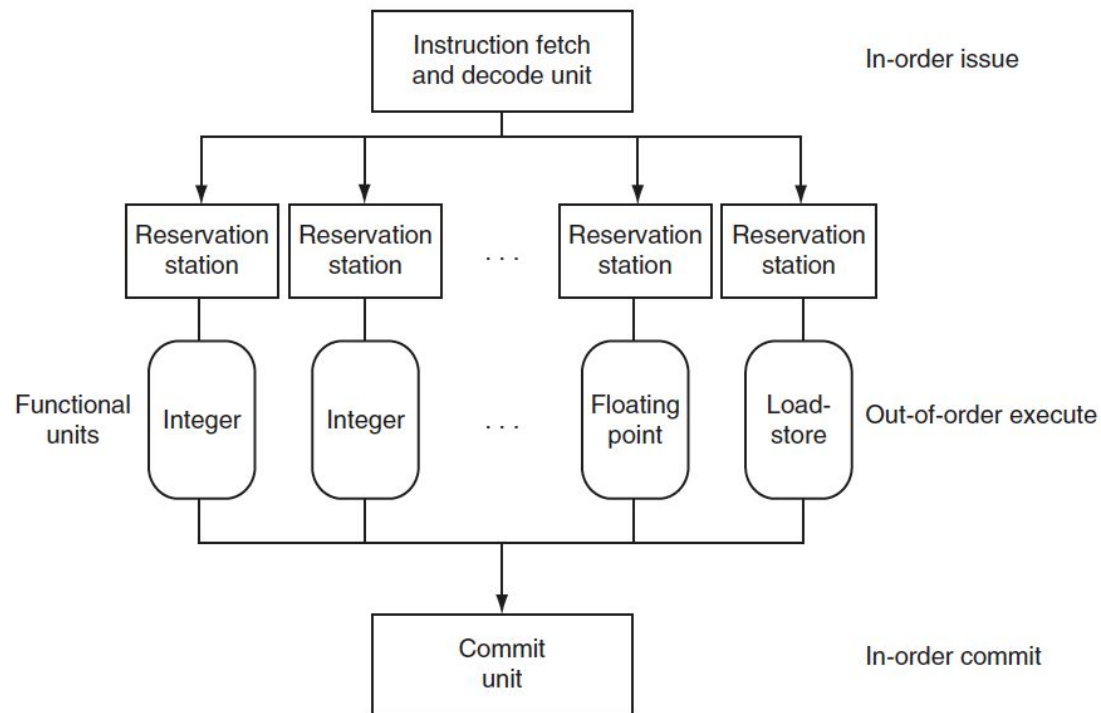
- Out-of-Order Dynamic Multiple Issue (Superscalar Processors)
 - Instruções são reordenadas em tempo de execução e enviadas para as unidades funcionais fora da ordem que foram compiladas
 - No exemplo anterior:
 - A dependência entre LW e ADDU não impossibilita o envio das instruções subsequentes para as unidades funcionais
 - A instrução SUB não possui dependência verdadeira com LW e ADDU, ela pode ser enviada para uma unidade funcional e executar até mesmo antes das instruções LW e ADDU
 - A instrução SLTI possui RAW com SUB e deve ser enviada para uma unidade funcional após SUB

```
lw      $t0, 20($s2)
addu    $t1, $t0, $t2
sub      $s4, $s4, $t3
slti     $t5, $s4, 20
```

- Hardware de exploração de paralelismo muito mais complexo que o processador in-order
- Limitados ganhos na exploração de paralelismo de instruções
- Exemplo: ARM Cortex A73, Intel Pentium PRO (1995) – Intel Core i9 (Hoje)

Paralelismo via Instruções

- Out-of-Order Dynamic Multiple Issue (Superscalar Processors)
 - Pipeline é dividido em 3 grandes partes: Fetch/decode/issue, múltiplas unidades funcionais e commit



Paralelismo via Instruções

- Out-of-Order Dynamic Multiple Issue (Superscalar Processors)
 - Passos para execução:
 - Várias instruções são buscadas da memória e decodificadas por ciclo
 - Instruções decodificadas são enviadas para as Reservation Stations (armazenam os operandos e operações de cada instrução)
 - No momento que uma instrução possui todos os operandos para execução ela é enviada para a unidade funcional (fora da ordem)
 - O resultado da execução da instrução é enviado para todas as unidades de reserva (existem instruções esperando por este resultado-RAW)
 - Ao mesmo tempo, o resultado é enviado para a commit unit (reorder buffer), que escreverá o resultado no banco de registradores na ordem em que o programa foi compilado

Paralelismo via Instruções

- Register Renaming é realizado para possibilitar execução paralela de instruções com dependências falsas (WAW e WAR)
 - Realizado através do uso do Reorder Buffer e Reservation Station:
 - Quando uma instrução é enviada para RS, os operandos que estão disponíveis no RF e no RB são enviados imediatamente para a RS
 - Se o operando não estiver no RF ou RB (ou seja, ele está sendo calculado por uma unidade funcional), a instrução deve esperar para ser enviada para a unidade funcional até que aquele operando esteja no slot da RS
 - Quando a unidade funcional gerar o operando requisitado, ele é enviado diretamente ao slot da RS da instrução
 - Na RS os operandos não são identificados pelos registradores físicos

Paralelismo via Instruções

- In-Order Commit
 - Todos os resultados são escritos no banco de registradores e na memória na ordem em que o programa foi compilado
 - A última instrução executada de fato é a última que escreveu no banco de registradores ou na memória
 - Importante abordagem para evitar inconsistências na presença de interrupções e exceções,
 - A instrução que realizou a última escrita no banco de registradores ou na memória é considerada aquela que programa parou a execução
 - As instruções que geraram resultados no RB e não escreveram no banco de registradores ou na memória são descartadas

Paralelismo via Instruções

- Out-of-Order Dynamic Multiple Issue (Superscalar Processors)
 - Apesar de existir processadores 4- a 6-issue é difícil manter IPC maior do que dois devido às:
 - Dependências de dados verdadeiras (RAW)
 - Dependências estruturais (unidades funcionais)
 - Cache misses na memória de dados e instruções
- Evolução dos Processadores Intel x86

Microprocessor	Year	Clock Rate	Pipeline Stages	Issue Width	Out-of-Order/ Speculation	Cores/ Chip	Power	
Intel 486	1989	25 MHz	5	1	No	1	5	W
Intel Pentium	1993	66 MHz	5	2	No	1	10	W
Intel Pentium Pro	1997	200 MHz	10	3	Yes	1	29	W
Intel Pentium 4 Willamette	2001	2000 MHz	22	3	Yes	1	75	W
Intel Pentium 4 Prescott	2004	3600 MHz	31	3	Yes	1	103	W
Intel Core	2006	2930 MHz	14	4	Yes	2	75	W
Intel Core i5 Nehalem	2010	3300 MHz	14	4	Yes	1	87	W
Intel Core i5 Ivy Bridge	2012	3400 MHz	14	4	Yes	8	77	W

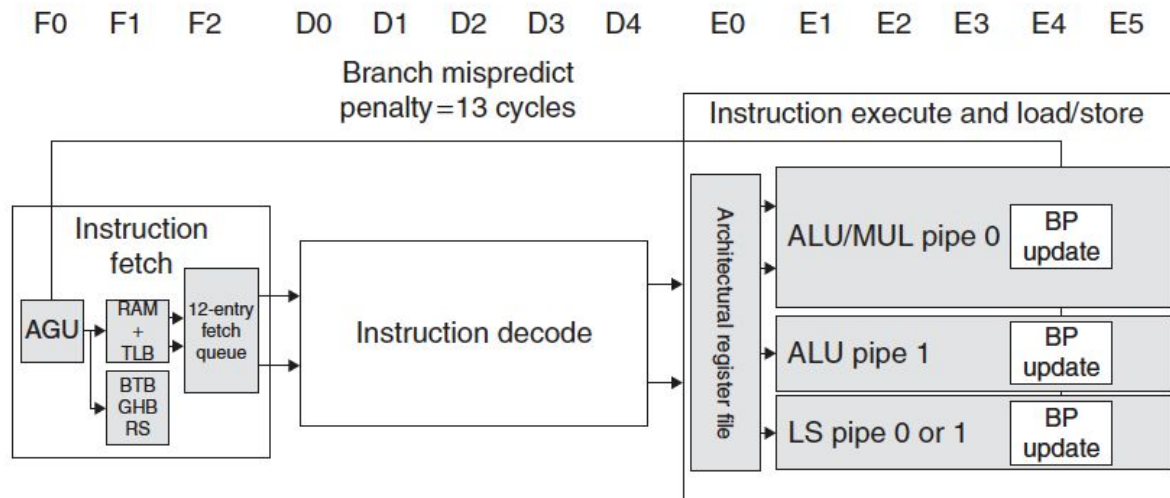
Paralelismo via Instruções

- Comparação entre ARM In-Order e Intel Out-of-Order

Processor	ARM A8	Intel Core i7 920
Market	Personal Mobile Device	Server, Cloud
Thermal design power	2 Watts	130 Watts
Clock rate	1 GHz	2.66 GHz
Cores/Chip	1	4
Floating point?	No	Yes
Multiple Issue?	Dynamic	Dynamic
Peak instructions/clock cycle	2	4
Pipeline Stages	14	14
Pipeline schedule	Static In-order	Dynamic Out-of-order with Speculation
Branch prediction	2-level	2-level
1st level caches / core	32 KiB I, 32 KiB D	32 KiB I, 32 KiB D
2nd level cache / core	128 - 1024 KiB	256 KiB
3rd level cache (shared)	–	2 - 8 MiB

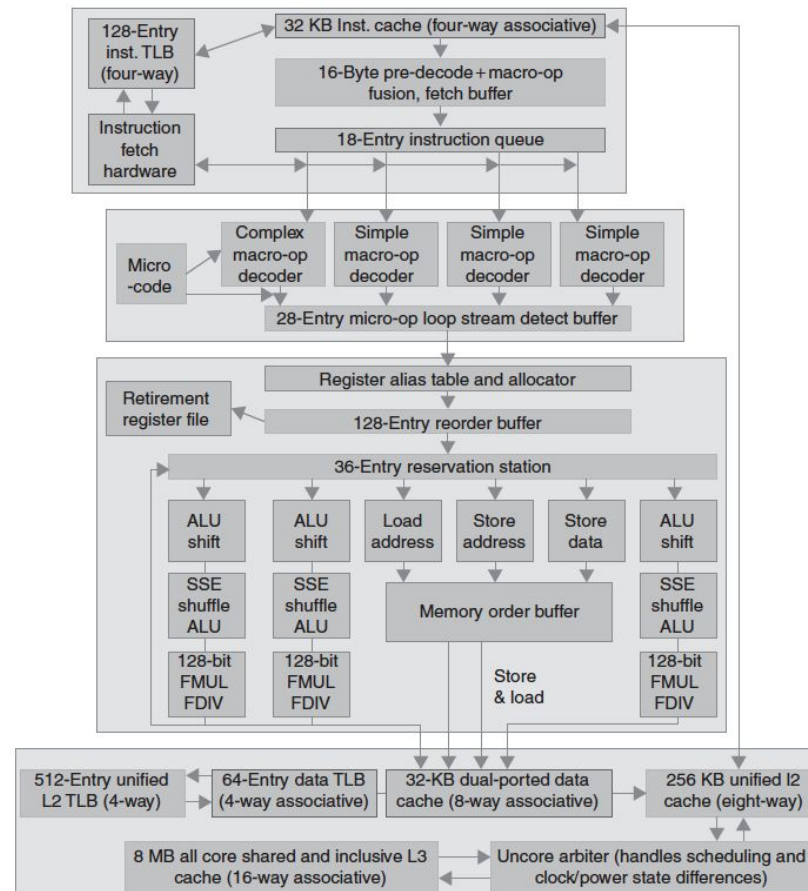
Paralelismo via Instruções

- Comparação entre ARM In-Order e Intel Out-of-Order
 - ARM Cortex A8 Pipeline



Paralelismo via Instruções

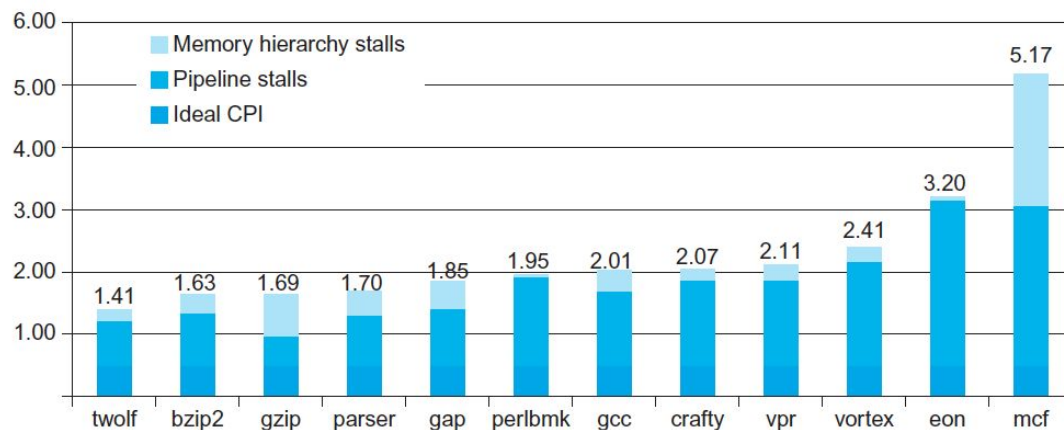
- Comparação entre ARM In-Order e Intel Out-of-Order
 - Intel Core i7 920



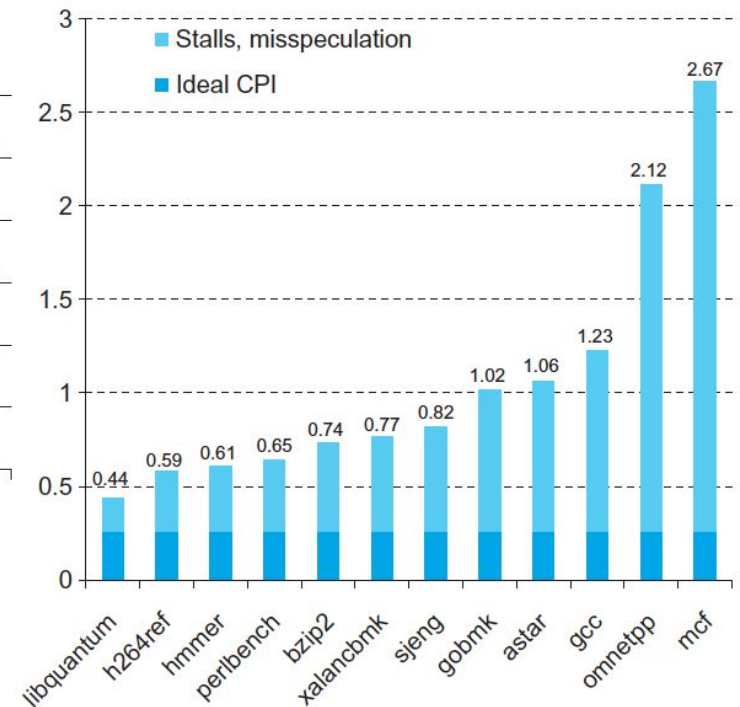
Paralelismo via Instruções

- CPI na Execução de Aplicações do SPEC2000

ARM Cortex A8 In-Order

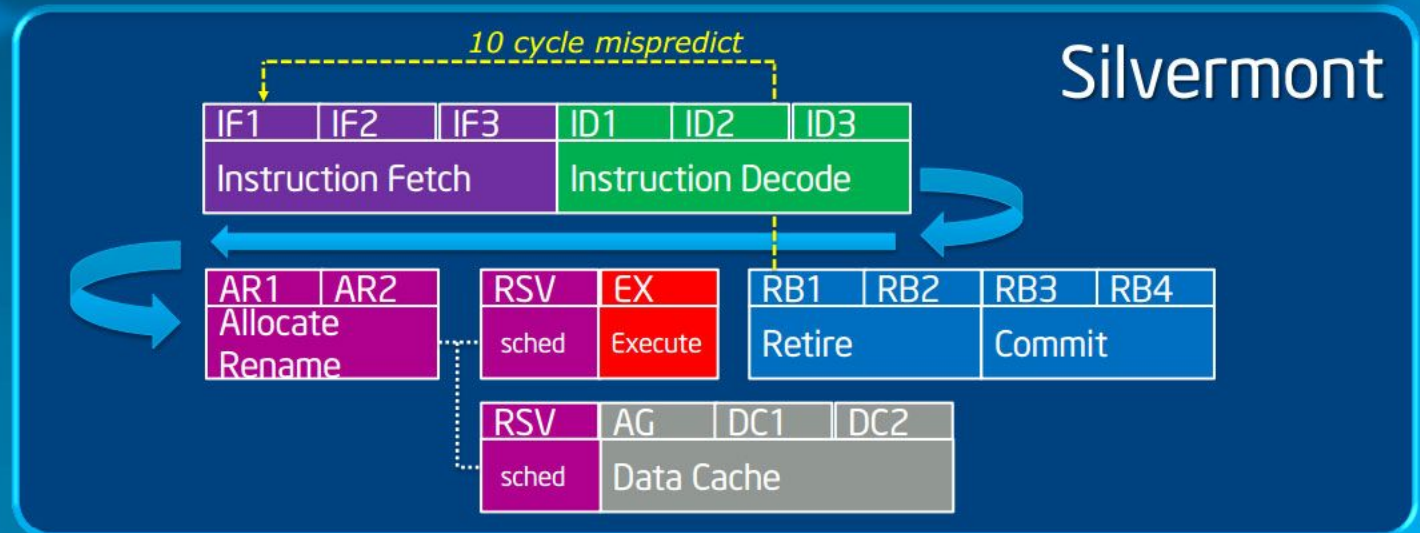
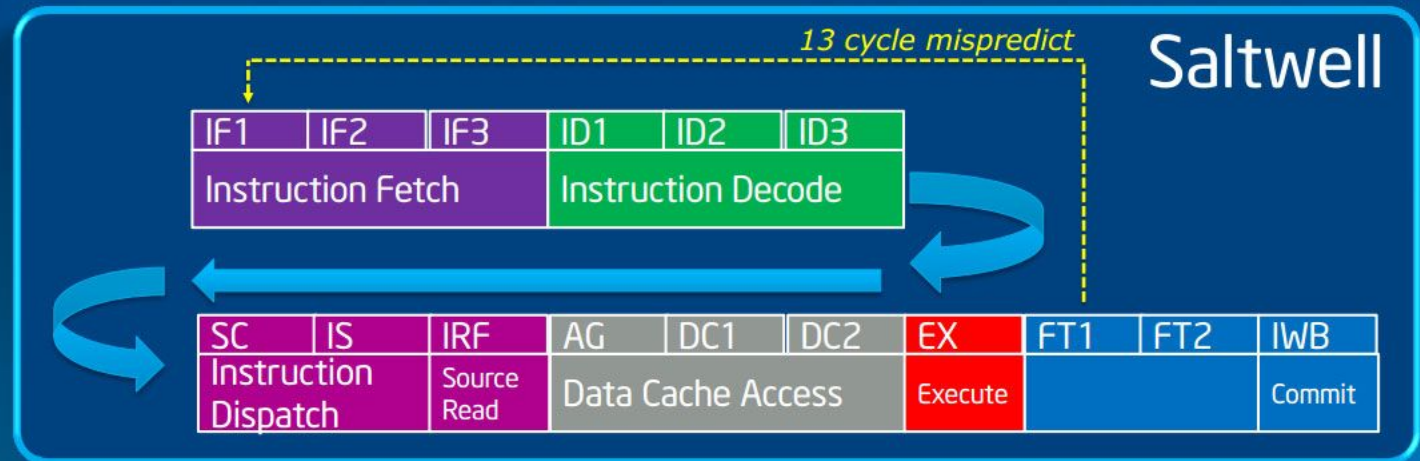


Intel Core i7 920 Out-of-Order



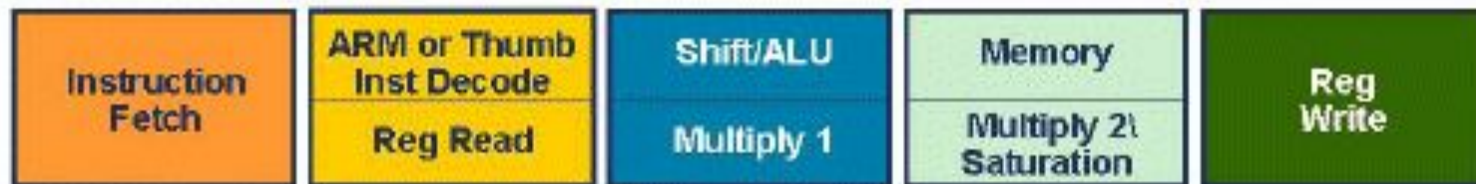
Exemplos – Core i7

Pipelines

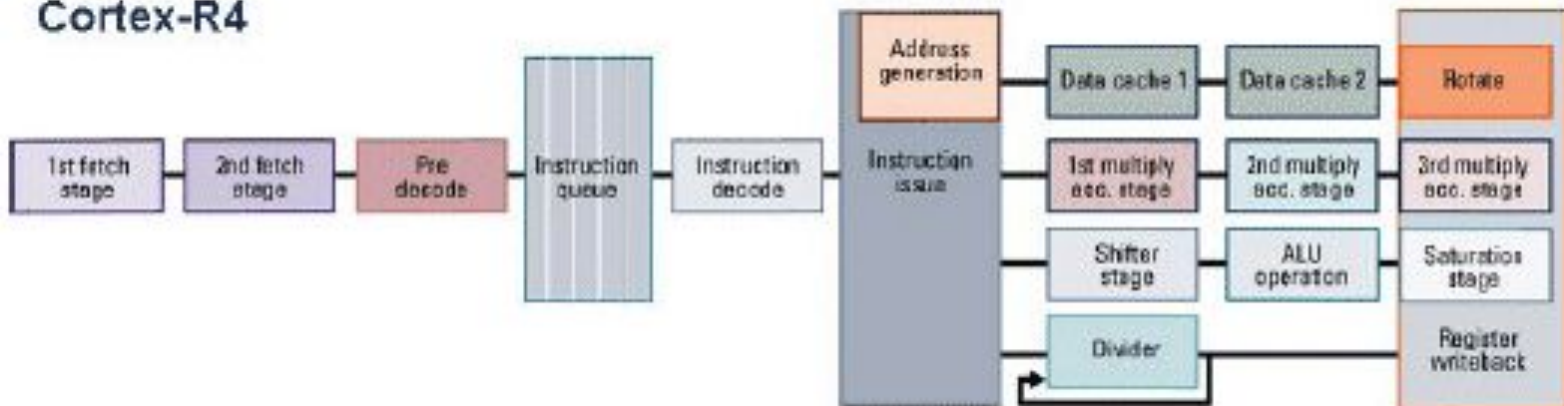


Exemplos - ARM

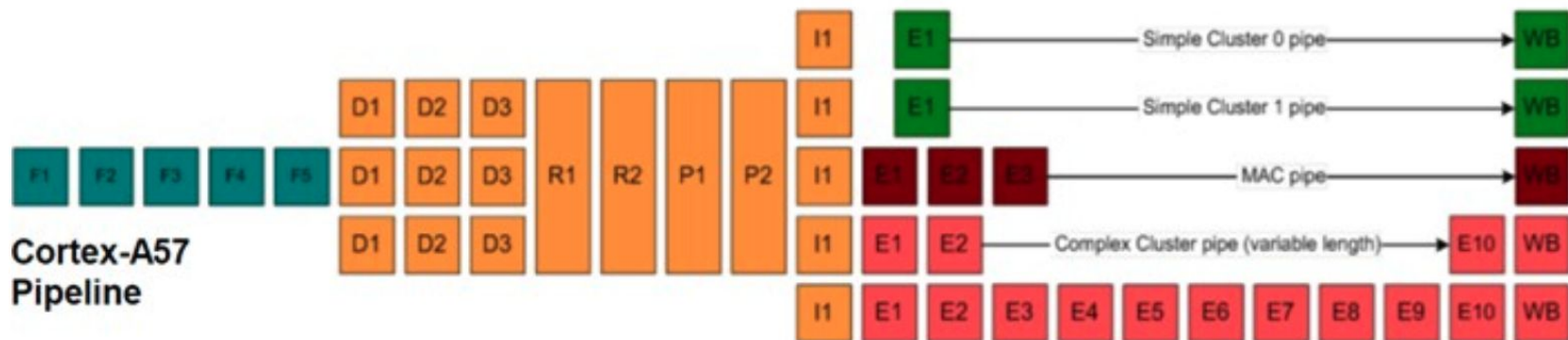
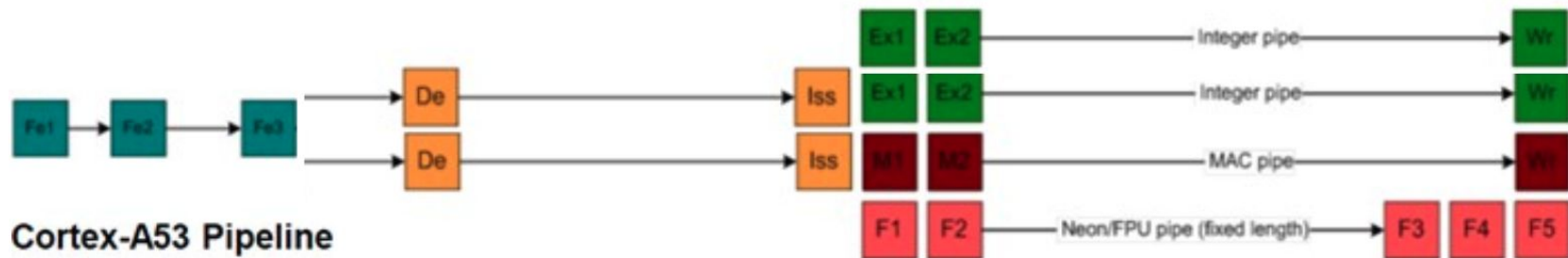
ARM946E-S



Cortex-R4



Exemplos - ARM



D: Decode
R: Rename
P: Dispatch
I: Issue
E: Execute
WB: Write Back

Note: Branch and Load/Store pipelines not shown
(1x Load/Store pipeline for the Cortex A-53 and
2x Load/Store and 1x Branch pipeline for the Cortex-A-57)

Exercícios

4.1 Consider the following instruction:

Instruction: `AND Rd, Rs, Rt`

Interpretation: `Reg[Rd] = Reg[Rs] AND Reg[Rt]`

4.1.1 [5] <\$4.1> What are the values of control signals generated by the control in [Figure 4.2](#) for the above instruction?

4.1.2 [5] <\$4.1> Which resources (blocks) perform a useful function for this instruction?

4.1.3 [10] <\$4.1> Which resources (blocks) produce outputs, but their outputs are not used for this instruction? Which resources produce no outputs for this instruction?

4.2 The basic single-cycle MIPS implementation in [Figure 4.2](#) can only implement some instructions. New instructions can be added to an existing Instruction Set Architecture (ISA), but the decision whether or not to do that depends, among other things, on the cost and complexity the proposed addition introduces into the processor datapath and control. The first three problems in this exercise refer to the new instruction:

Instruction: `LWI Rt, Rd(Rs)`

Interpretation: `Reg[Rt] = Mem[Reg[Rd]+Reg[Rs]]`

4.2.1 [10] <\$4.1> Which existing blocks (if any) can be used for this instruction?

4.2.2 [10] <\$4.1> Which new functional blocks (if any) do we need for this instruction?

4.2.3 [10] <\$4.1> What new signals do we need (if any) from the control unit to support this instruction?

4.3 When processor designers consider a possible improvement to the processor datapath, the decision usually depends on the cost/performance trade-off. In the following three problems, assume that we are starting with a datapath from [Figure 4.2](#), where I-Mem, Add, Mux, ALU, Regs, D-Mem, and Control blocks have latencies of 400 ps, 100 ps, 30 ps, 120 ps, 200 ps, 350 ps, and 100 ps, respectively, and costs of 1000, 30, 10, 100, 200, 2000, and 500, respectively.

Consider the addition of a multiplier to the ALU. This addition will add 300 ps to the latency of the ALU and will add a cost of 600 to the ALU. The result will be 5% fewer instructions executed since we will no longer need to emulate the MUL instruction.

4.3.1 [10] <\$4.1> What is the clock cycle time with and without this improvement?

4.3.2 [10] <\$4.1> What is the speedup achieved by adding this improvement?

4.3.3 [10] <\$4.1> Compare the cost/performance ratio with and without this improvement.

Exercícios

4.4 Problems in this exercise assume that logic blocks needed to implement a processor's datapath have the following latencies:

I-Mem	Add	Mux	ALU	Regs	D-Mem	Sign-Extend	Shift-Left-2
200ps	70ps	20ps	90ps	90ps	250ps	15ps	10ps

4.4.1 [10] <§4.3> If the only thing we need to do in a processor is fetch consecutive instructions (Figure 4.6), what would the cycle time be?

4.4.2 [10] <§4.3> Consider a datapath similar to the one in Figure 4.11, but for a processor that only has one type of instruction: unconditional PC-relative branch. What would the cycle time be for this datapath?

4.4.3 [10] <§4.3> Repeat 4.4.2, but this time we need to support only conditional PC-relative branches.

The remaining three problems in this exercise refer to the datapath element Shift-left-2:

4.4.4 [10] <§4.3> Which kinds of instructions require this resource?

4.4.5 [20] <§4.3> For which kinds of instructions (if any) is this resource on the critical path?

4.4.6 [10] <§4.3> Assuming that we only support beq and add instructions, discuss how changes in the given latency of this resource affect the cycle time of the processor. Assume that the latencies of other resources do not change.

Exercícios

4.5 For the problems in this exercise, assume that there are no pipeline stalls and that the breakdown of executed instructions is as follows:

add	addl	not	beq	lw	sw
20%	20%	0%	25%	25%	10%

4.5.1 [10] <§4.3> In what fraction of all cycles is the data memory used?

4.5.2 [10] <§4.3> In what fraction of all cycles is the input of the sign-extend circuit needed? What is this circuit doing in cycles in which its input is not needed?

4.6 When silicon chips are fabricated, defects in materials (e.g., silicon) and manufacturing errors can result in defective circuits. A very common defect is for one wire to affect the signal in another. This is called a cross-talk fault. A special class of cross-talk faults is when a signal is connected to a wire that has a constant logical value (e.g., a power supply wire). In this case we have a stuck-at-0 or a stuck-at-1 fault, and the affected signal always has a logical value of 0 or 1, respectively. The following problems refer to bit 0 of the Write Register input on the register file in [Figure 4.24](#).

4.6.1 [10] <§§4.3, 4.4> Let us assume that processor testing is done by filling the PC, registers, and data and instruction memories with some values (you can choose which values), letting a single instruction execute, then reading the PC, memories, and registers. These values are then examined to determine if a particular fault is present. Can you design a test (values for PC, memories, and registers) that would determine if there is a stuck-at-0 fault on this signal?

4.6.2 [10] <§§4.3, 4.4> Repeat 4.6.1 for a stuck-at-1 fault. Can you use a single test for both stuck-at-0 and stuck-at-1? If yes, explain how; if no, explain why not.

Exercícios

4.8 In this exercise, we examine how pipelining affects the clock cycle time of the processor. Problems in this exercise assume that individual stages of the datapath have the following latencies:

IF	ID	EX	MEM	WB
250ps	350ps	150ps	300ps	200ps

Also, assume that instructions executed by the processor are broken down as follows:

alu	beq	lw	sw
45%	20%	20%	15%

4.8.1 [5] <\$4.5> What is the clock cycle time in a pipelined and non-pipelined processor?

4.8.2 [10] <\$4.5> What is the total latency of an LW instruction in a pipelined and non-pipelined processor?

4.8.3 [10] <\$4.5> If we can split one stage of the pipelined datapath into two new stages, each with half the latency of the original stage, which stage would you split and what is the new clock cycle time of the processor?

4.8.4 [10] <\$4.5> Assuming there are no stalls or hazards, what is the utilization of the data memory?

4.8.5 [10] <\$4.5> Assuming there are no stalls or hazards, what is the utilization of the write-register port of the “Registers” unit?

Exercícios

4.9 In this exercise, we examine how data dependences affect execution in the basic 5-stage pipeline described in Section 4.5. Problems in this exercise refer to the following sequence of instructions:

```
or r1,r2,r3
or r2,r1,r4
or r1,r1,r2
```

Also, assume the following cycle times for each of the options related to forwarding:

Without Forwarding	With Full Forwarding	With ALU-ALU Forwarding Only
250ps	300ps	290ps

4.9.1 [10] <\$4.5> Indicate dependences and their type.

4.9.2 [10] <\$4.5> Assume there is no forwarding in this pipelined processor. Indicate hazards and add `nop` instructions to eliminate them.

4.9.3 [10] <\$4.5> Assume there is full forwarding. Indicate hazards and add `NOP` instructions to eliminate them.

4.9.4 [10] <\$4.5> What is the total execution time of this instruction sequence without forwarding and with full forwarding? What is the speedup achieved by adding full forwarding to a pipeline that had no forwarding?

4.9.5 [10] <\$4.5> Add `nop` instructions to this code to eliminate hazards if there is ALU-ALU forwarding only (no forwarding from the MEM to the EX stage).

4.9.6 [10] <\$4.5> What is the total execution time of this instruction sequence with only ALU-ALU forwarding? What is the speedup over a no-forwarding pipeline?

Exercícios

4.10 In this exercise, we examine how resource hazards, control hazards, and Instruction Set Architecture (ISA) design can affect pipelined execution. Problems in this exercise refer to the following fragment of MIPS code:

```
sw  r16,12(r6)
lw  r16,8(r6)
beq r5,r4,Label # Assume r5!=r4
add r5,r1,r4
slt r5,r15,r4
```

Assume that individual pipeline stages have the following latencies:

IF	ID	EX	MEM	WB
200ps	120ps	150ps	190ps	100ps

4.10.1 [10] <\$4.5> For this problem, assume that all branches are perfectly predicted (this eliminates all control hazards) and that no delay slots are used. If we only have one memory (for both instructions and data), there is a structural hazard every time we need to fetch an instruction in the same cycle in which another instruction accesses data. To guarantee forward progress, this hazard must always be resolved in favor of the instruction that accesses data. What is the total execution time of this instruction sequence in the 5-stage pipeline that only has one memory? We have seen that data hazards can be eliminated by adding nops to the code. Can you do the same with this structural hazard? Why?

4.10.2 [20] <\$4.5> For this problem, assume that all branches are perfectly predicted (this eliminates all control hazards) and that no delay slots are used. If we change load/store instructions to use a register (without an offset) as the address, these instructions no longer need to use the ALU. As a result, MEM and EX stages can be overlapped and the pipeline has only 4 stages. Change this code to accommodate this changed ISA. Assuming this change does not affect clock cycle time, what speedup is achieved in this instruction sequence?

Exercícios

4.10.3 [10] <\$4.5> Assuming stall-on-branch and no delay slots, what speedup is achieved on this code if branch outcomes are determined in the ID stage, relative to the execution where branch outcomes are determined in the EX stage?

4.10.4 [10] <\$4.5> Given these pipeline stage latencies, repeat the speedup calculation from 4.10.2, but take into account the (possible) change in clock cycle time. When EX and MEM are done in a single stage, most of their work can be done in parallel. As a result, the resulting EX/MEM stage has a latency that is the larger of the original two, plus 20 ps needed for the work that could not be done in parallel.

4.10.5 [10] <\$4.5> Given these pipeline stage latencies, repeat the speedup calculation from 4.10.3, taking into account the (possible) change in clock cycle time. Assume that the latency ID stage increases by 50% and the latency of the EX stage decreases by 10ps when branch outcome resolution is moved from EX to ID.

4.10.6 [10] <\$4.5> Assuming stall-on-branch and no delay slots, what is the new clock cycle time and execution time of this instruction sequence if beq address computation is moved to the MEM stage? What is the speedup from this change? Assume that the latency of the EX stage is reduced by 20 ps and the latency of the MEM stage is unchanged when branch outcome resolution is moved from EX to MEM.

Exercícios

4.11 Consider the following loop.

```
loop:lw  r1,0(r1)
      and r1,r1,r2
      lw  r1,0(r1)
      lw  r1,0(r1)
      beq r1,r0,loop
```

Assume that perfect branch prediction is used (no stalls due to control hazards), that there are no delay slots, and that the pipeline has full forwarding support. Also assume that many iterations of this loop are executed before the loop exits.

4.11.1 [10] <\$4.6> Show a pipeline execution diagram for the third iteration of this loop, from the cycle in which we fetch the first instruction of that iteration up to (but not including) the cycle in which we can fetch the first instruction of the next iteration. Show all instructions that are in the pipeline during these cycles (not just those from the third iteration).

4.11.2 [10] <\$4.6> How often (as a percentage of all cycles) do we have a cycle in which all five pipeline stages are doing useful work?

Exercícios

4.12 This exercise is intended to help you understand the cost/complexity/performance trade-offs of forwarding in a pipelined processor. Problems in this exercise refer to pipelined datapaths from [Figure 4.45](#). These problems assume that, of all the instructions executed in a processor, the following fraction of these instructions have a particular type of RAW data dependence. The type of RAW data dependence is identified by the stage that produces the result (EX or MEM) and the instruction that consumes the result (1st instruction that follows the one that produces the result, 2nd instruction that follows, or both). We assume that the register write is done in the first half of the clock cycle and that register reads are done in the second half of the cycle, so “EX to 3rd” and “MEM to 3rd” dependences are not counted because they cannot result in data hazards. Also, assume that the CPI of the processor is 1 if there are no data hazards.

EX to 1 st Only	MEM to 1 st Only	EX to 2 nd Only	MEM to 2 nd Only	EX to 1 st and MEM to 2 nd	Other RAW Dependences
5%	20%	5%	10%	10%	10%

Assume the following latencies for individual pipeline stages. For the EX stage, latencies are given separately for a processor without forwarding and for a processor with different kinds of forwarding.

IF	ID	EX (no FW)	EX (full FW)	EX (FW from EX/MEM only)	EX (FW from MEM/ WB only)	MEM	WB
150 ps	100 ps	120 ps	150 ps	140 ps	130 ps	120 ps	100 ps

Exercícios

4.12.1 [10] <§4.7> If we use no forwarding, what fraction of cycles are we stalling due to data hazards?

4.12.2 [5] <§4.7> If we use full forwarding (forward all results that can be forwarded), what fraction of cycles are we staling due to data hazards?

4.12.3 [10] <§4.7> Let us assume that we cannot afford to have three-input Muxes that are needed for full forwarding. We have to decide if it is better to forward only from the EX/MEM pipeline register (next-cycle forwarding) or only from the MEM/WB pipeline register (two-cycle forwarding). Which of the two options results in fewer data stall cycles?

4.12.4 [10] <§4.7> For the given hazard probabilities and pipeline stage latencies, what is the speedup achieved by adding full forwarding to a pipeline that had no forwarding?

4.12.5 [10] <§4.7> What would be the additional speedup (relative to a processor with forwarding) if we added time-travel forwarding that eliminates all data hazards? Assume that the yet-to-be-invented time-travel circuitry adds 100 ps to the latency of the full-forwarding EX stage.

4.12.6 [20] <§4.7> Repeat 4.12.3 but this time determine which of the two options results in shorter time per instruction.

Exercícios

4.13 This exercise is intended to help you understand the relationship between forwarding, hazard detection, and ISA design. Problems in this exercise refer to the following sequence of instructions, and assume that it is executed on a 5-stage pipelined datapath:

```
add r5,r2,r1
lw  r3,4(r5)
lw  r2,0(r2)
or  r3,r5,r3
sw  r3,0(r5)
```

4.13.1 [5] <§4.7> If there is no forwarding or hazard detection, insert nops to ensure correct execution.

4.13.2 [10] <§4.7> Repeat 4.13.1 but now use nops only when a hazard cannot be avoided by changing or rearranging these instructions. You can assume register R7 can be used to hold temporary values in your modified code.

4.13.3 [10] <§4.7> If the processor has forwarding, but we forgot to implement the hazard detection unit, what happens when this code executes?

4.13.4 [20] <§4.7> If there is forwarding, for the first five cycles during the execution of this code, specify which signals are asserted in each cycle by hazard detection and forwarding units in [Figure 4.60](#).

4.13.5 [10] <§4.7> If there is no forwarding, what new inputs and output signals do we need for the hazard detection unit in [Figure 4.60](#)? Using this instruction sequence as an example, explain why each signal is needed.

Exercícios

4.14 This exercise is intended to help you understand the relationship between delay slots, control hazards, and branch execution in a pipelined processor. In this exercise, we assume that the following MIPS code is executed on a pipelined processor with a 5-stage pipeline, full forwarding, and a predict-taken branch predictor:

```
        lw r2,0(r1)
label1: beq r2,r0,label2 # not taken once, then taken
        lw r3,0(r2)
        beq r3,r0,label1 # taken
        add r1,r3,r1
label2: sw r1,0(r2)
```

4.14.1 [10] <\$4.8> Draw the pipeline execution diagram for this code, assuming there are no delay slots and that branches execute in the EX stage.

4.14.2 [10] <\$4.8> Repeat 4.14.1, but assume that delay slots are used. In the given code, the instruction that follows the branch is now the delay slot instruction for that branch.

Exercícios

4.15 The importance of having a good branch predictor depends on how often conditional branches are executed. Together with branch predictor accuracy, this will determine how much time is spent stalling due to mispredicted branches. In this exercise, assume that the breakdown of dynamic instructions into various instruction categories is as follows:

R-Type	BEQ	JMP	LW	SW
40%	25%	5%	25%	5%

Also, assume the following branch predictor accuracies:

Always-Taken	Always-Not-Taken	2-Bit
45%	55%	85%

4.15.1 [10] <§4.8> Stall cycles due to mispredicted branches increase the CPI. What is the extra CPI due to mispredicted branches with the always-taken predictor? Assume that branch outcomes are determined in the EX stage, that there are no data hazards, and that no delay slots are used.

4.15.2 [10] <§4.8> Repeat 4.15.1 for the “always-not-taken” predictor.

4.15.3 [10] <§4.8> Repeat 4.15.1 for for the 2-bit predictor.

4.15.4 [10] <§4.8> With the 2-bit predictor, what speedup would be achieved if we could convert half of the branch instructions in a way that replaces a branch instruction with an ALU instruction? Assume that correctly and incorrectly predicted instructions have the same chance of being replaced.

Exercícios

4.15.5 [10] <§4.8> With the 2-bit predictor, what speedup would be achieved if we could convert half of the branch instructions in a way that replaced each branch instruction with two ALU instructions? Assume that correctly and incorrectly predicted instructions have the same chance of being replaced.

4.15.6 [10] <§4.8> Some branch instructions are much more predictable than others. If we know that 80% of all executed branch instructions are easy-to-predict loop-back branches that are always predicted correctly, what is the accuracy of the 2-bit predictor on the remaining 20% of the branch instructions?

4.16 This exercise examines the accuracy of various branch predictors for the following repeating pattern (e.g., in a loop) of branch outcomes: T, NT, T, T, NT

4.16.1 [5] <§4.8> What is the accuracy of always-taken and always-not-taken predictors for this sequence of branch outcomes?

4.16.2 [5] <§4.8> What is the accuracy of the two-bit predictor for the first 4 branches in this pattern, assuming that the predictor starts off in the bottom left state from [Figure 4.63](#) (predict not taken)?

4.16.3 [10] <§4.8> What is the accuracy of the two-bit predictor if this pattern is repeated forever?

4.16.4 [30] <§4.8> Design a predictor that would achieve a perfect accuracy if this pattern is repeated forever. Your predictor should be a sequential circuit with one output that provides a prediction (1 for taken, 0 for not taken) and no inputs other than the clock and the control signal that indicates that the instruction is a conditional branch.

Exercícios

4.16.5 [10] <§4.8> What is the accuracy of your predictor from 4.16.4 if it is given a repeating pattern that is the exact opposite of this one?

4.16.6 [20] <§4.8> Repeat 4.16.4, but now your predictor should be able to eventually (after a warm-up period during which it can make wrong predictions) start perfectly predicting both this pattern and its opposite. Your predictor should have an input that tells it what the real outcome was. Hint: this input lets your predictor determine which of the two repeating patterns it is given.

4.17 This exercise explores how exception handling affects pipeline design. The first three problems in this exercise refer to the following two instructions:

Instruction 1	Instruction 2
BNE R1, R2, Label	LW R1, 0(R1)

4.17.1 [5] <§4.9> Which exceptions can each of these instructions trigger? For each of these exceptions, specify the pipeline stage in which it is detected.

Exercícios

4.18 In this exercise we compare the performance of 1-issue and 2-issue processors, taking into account program transformations that can be made to optimize for 2-issue execution. Problems in this exercise refer to the following loop (written in C):

```
for(i=0; i!=j; i+=2)
    b[i]=a[i]-a[i+1];
```

4.18.4 [10] <\$4.10> Repeat 4.18.2, but this time use your MIPS code from 4.18.3.

4.18.5 [10] <\$4.10> What is the speedup of going from a 1-issue processor to a 2-issue processor from [Figure 4.69](#)? Use your code from 4.18.1 for both 1-issue and 2-issue, and assume that 1,000,000 iterations of the loop are executed. As in 4.18.2, assume that the processor has perfect branch predictions, and that a 2-issue processor can fetch any two instructions in the same cycle.

4.18.6 [10] <\$4.10> Repeat 4.18.5, but this time assume that in the 2-issue processor one of the instructions to be executed in a cycle can be of any kind, and the other must be a non-memory instruction.

4.18.7 [10] <\$4.10> If the loop exits after executing only two iterations, draw a pipeline diagram for your MIPS code from 4.18.1 executed on a 2-issue processor shown in [Figure 4.69](#). Assume the processor has perfect branch prediction and can fetch any two instructions (not just consecutive instructions) in the same cycle.

4.18.8 [10] <\$4.10> Rearrange your code from 4.18.1 to achieve better performance on a 2-issue statically scheduled processor from [Figure 4.69](#).

Exercícios

4.18.4 [10] <\$4.10> Repeat 4.18.2, but this time use your MIPS code from 4.18.3.

4.18.5 [10] <\$4.10> What is the speedup of going from a 1-issue processor to a 2-issue processor from [Figure 4.69](#)? Use your code from 4.18.1 for both 1-issue and 2-issue, and assume that 1,000,000 iterations of the loop are executed. As in 4.18.2, assume that the processor has perfect branch predictions, and that a 2-issue processor can fetch any two instructions in the same cycle.

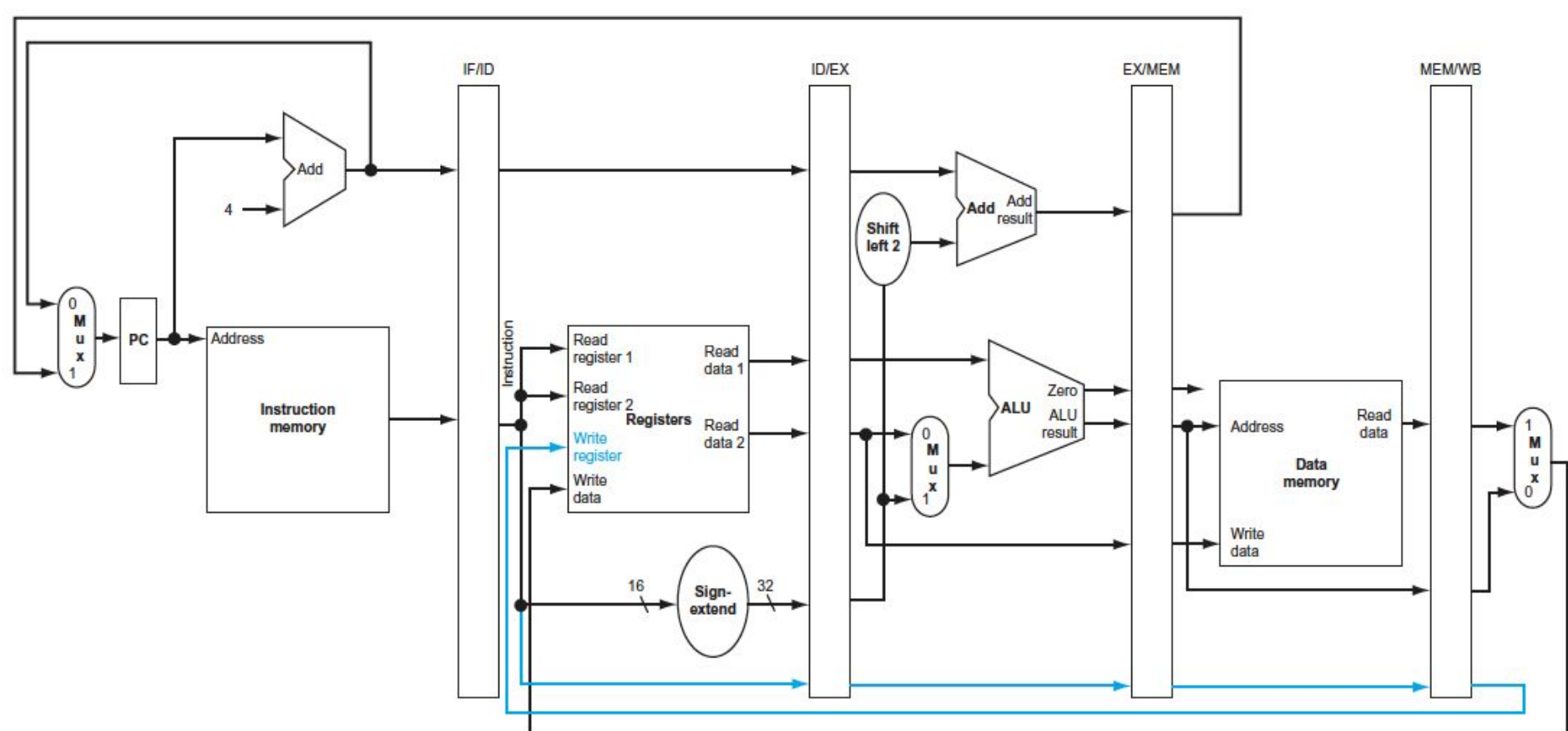
4.18.6 [10] <\$4.10> Repeat 4.18.5, but this time assume that in the 2-issue processor one of the instructions to be executed in a cycle can be of any kind, and the other must be a non-memory instruction.

Exemplo no Pipeline

- Ciclo 2

`lw $t2, 4($t0)`

`lw $t1, 0($t0)`



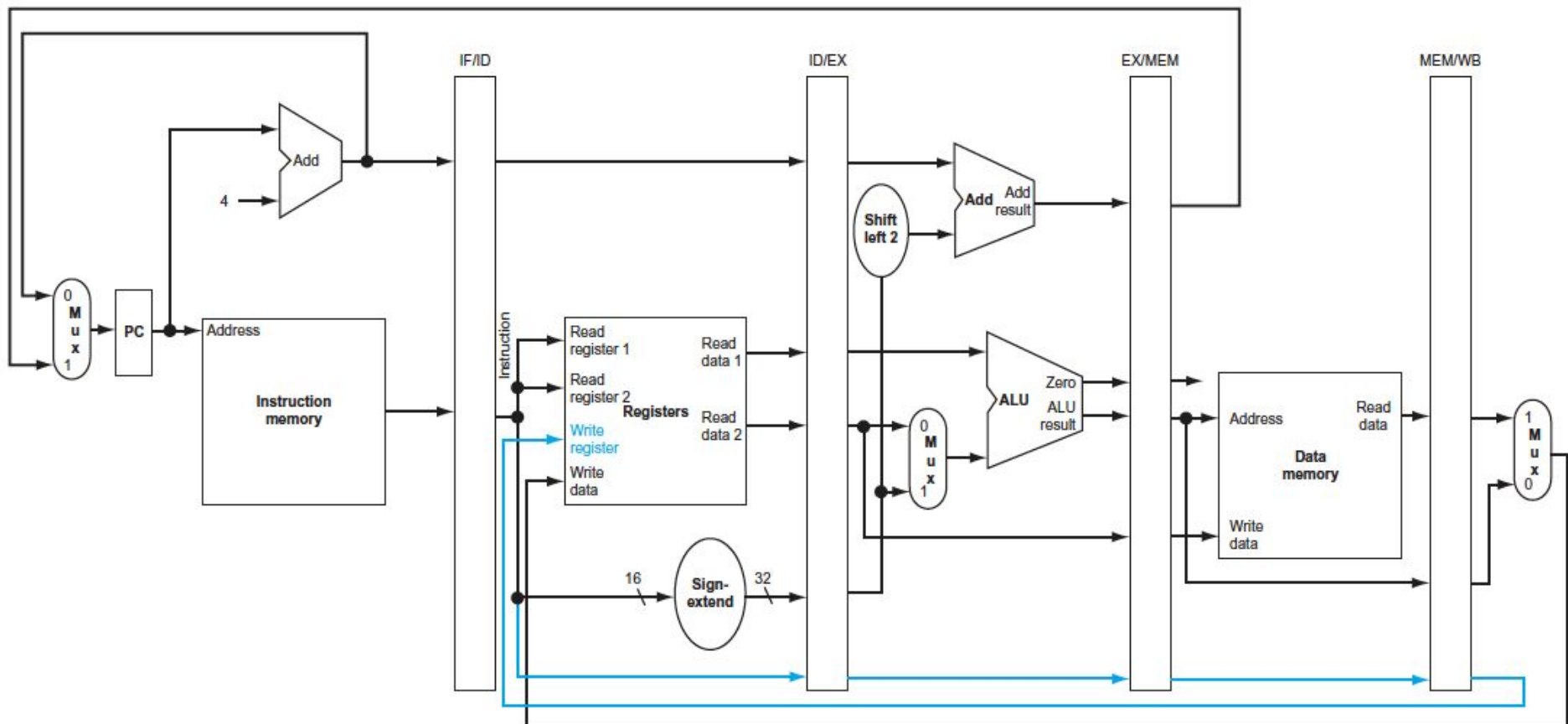
Exemplo no Pipeline

- Ciclo 3

add \$t3, \$t1, \$t2

lw \$t2, 4(\$t0)

lw \$t1, 0(\$t0)



Hazard de Datos

```
lw $t1, 0($t0)
```



```
lw $t1, 0($t0)
```



Exemplo no Pipeline

- Ciclo 6 – Pipeline Stall

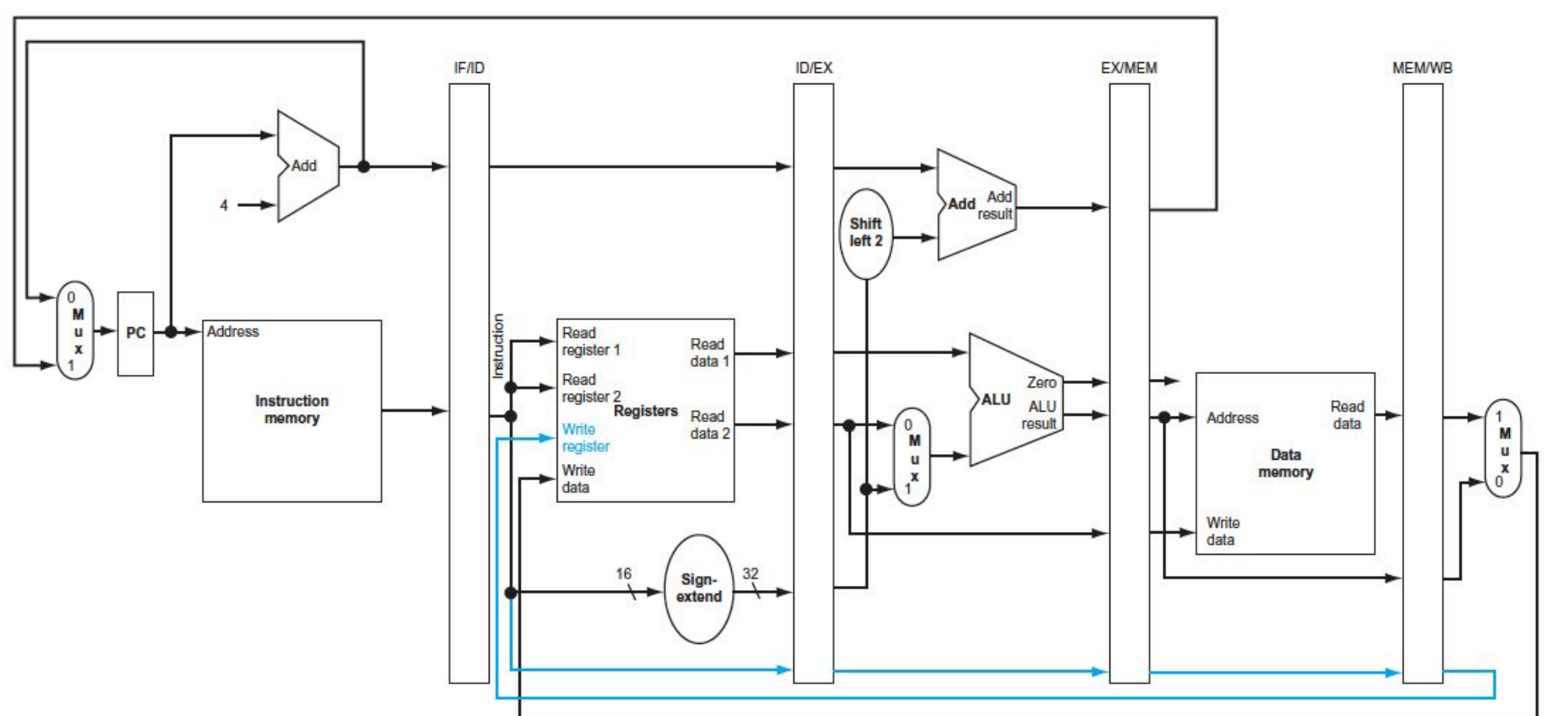
sw \$t3, 12(\$t0)

add \$t3, \$t1, \$t2

Bolha

Bolha

lw \$t2, 4(\$t0)



```
add $t3, $t1, $t2
```

Bolha



Exemplo no Pipeline

- Ciclo 8

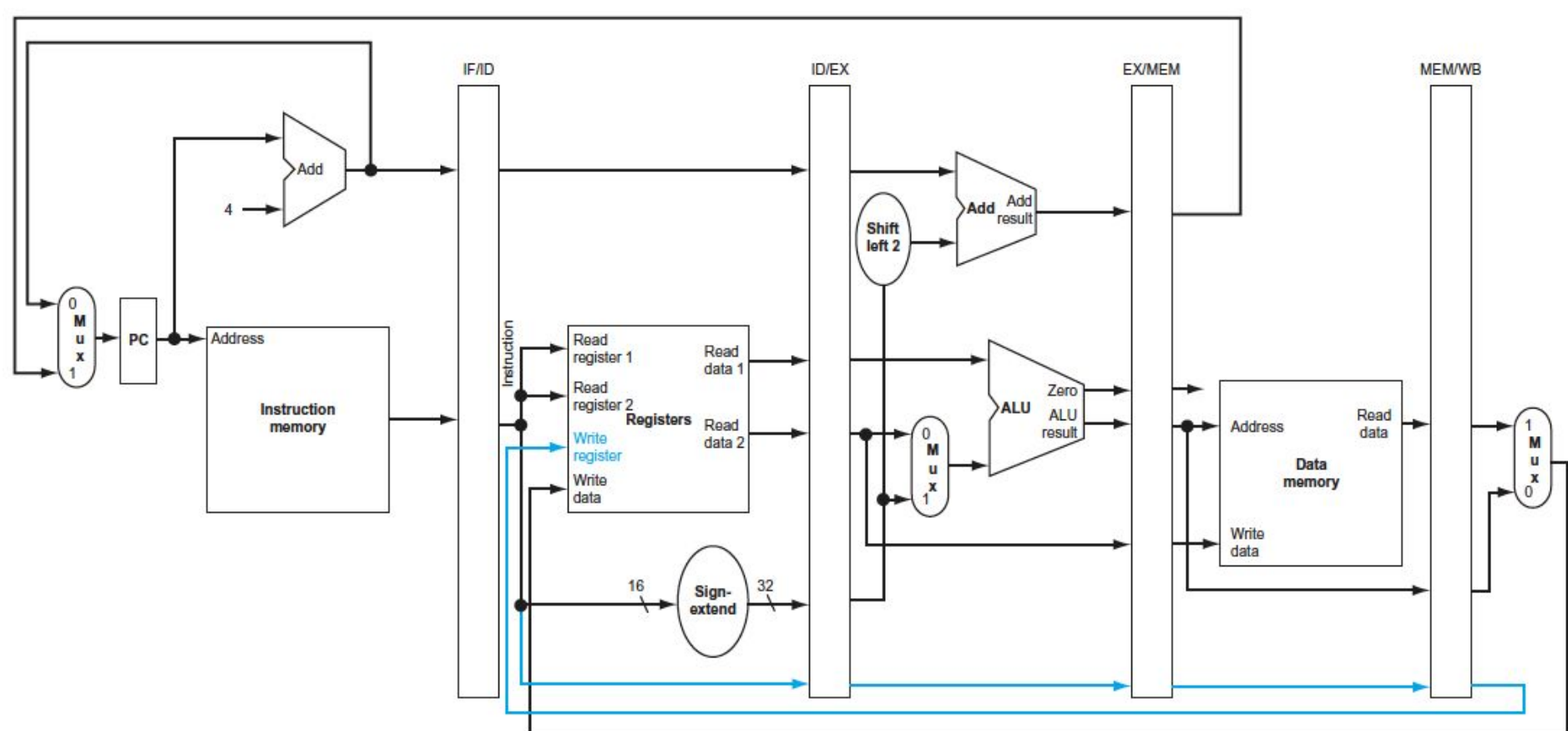
add \$t5, \$t1, \$t4

lw \$t4, 8(\$t0)

sw \$t3, 12(\$t0)

add \$t3, \$t1, \$t2

Bolha



Exemplo no Pipeline

- Ciclo 9

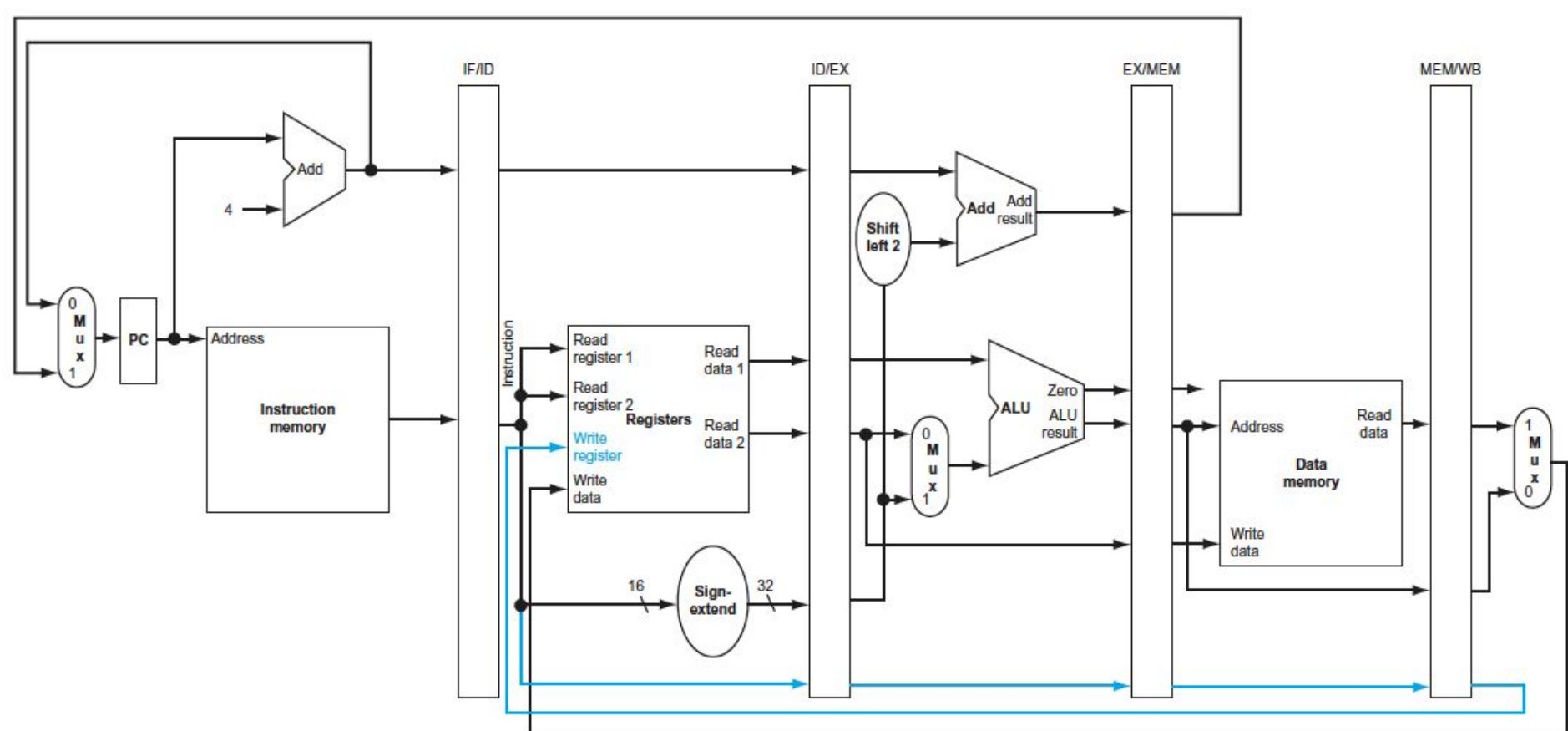
sw \$t5, 16(\$t0)

add \$t5, \$t1, \$t4

lw \$t4, 8(\$t0)

sw \$t3, 12(\$t0)

add \$t3, \$t1, \$t2



Exemplo no Pipeline

- Ciclo 9

Hazard de
Dados

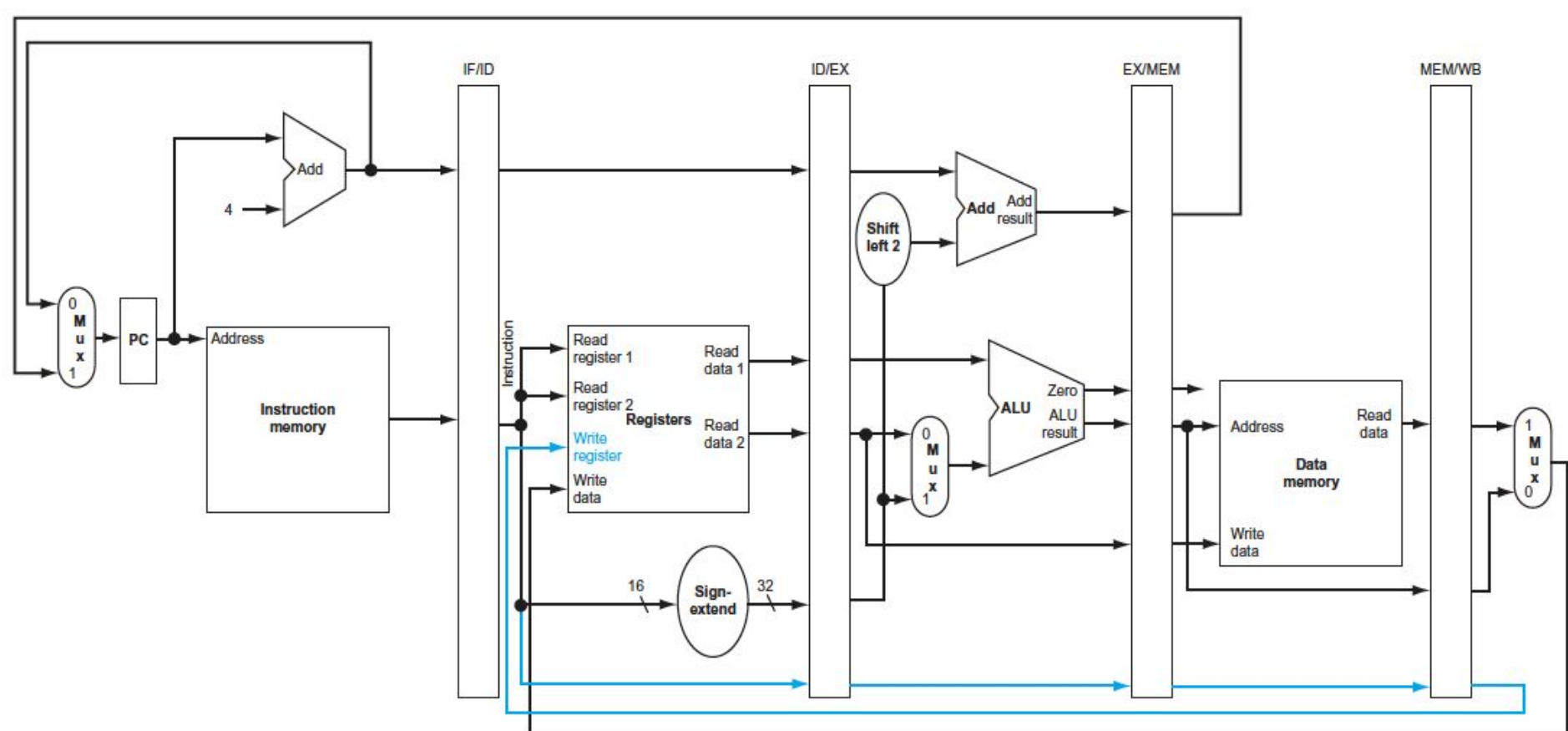
sw \$t5, 16(\$t0)

add \$t5, \$t1, \$t4

lw \$t4, 8(\$t0)

sw \$t3, 12(\$t0)

add \$t3, \$t1, \$t2



Exemplo no Pipeline

- Ciclo 10 – Pipeline Stall

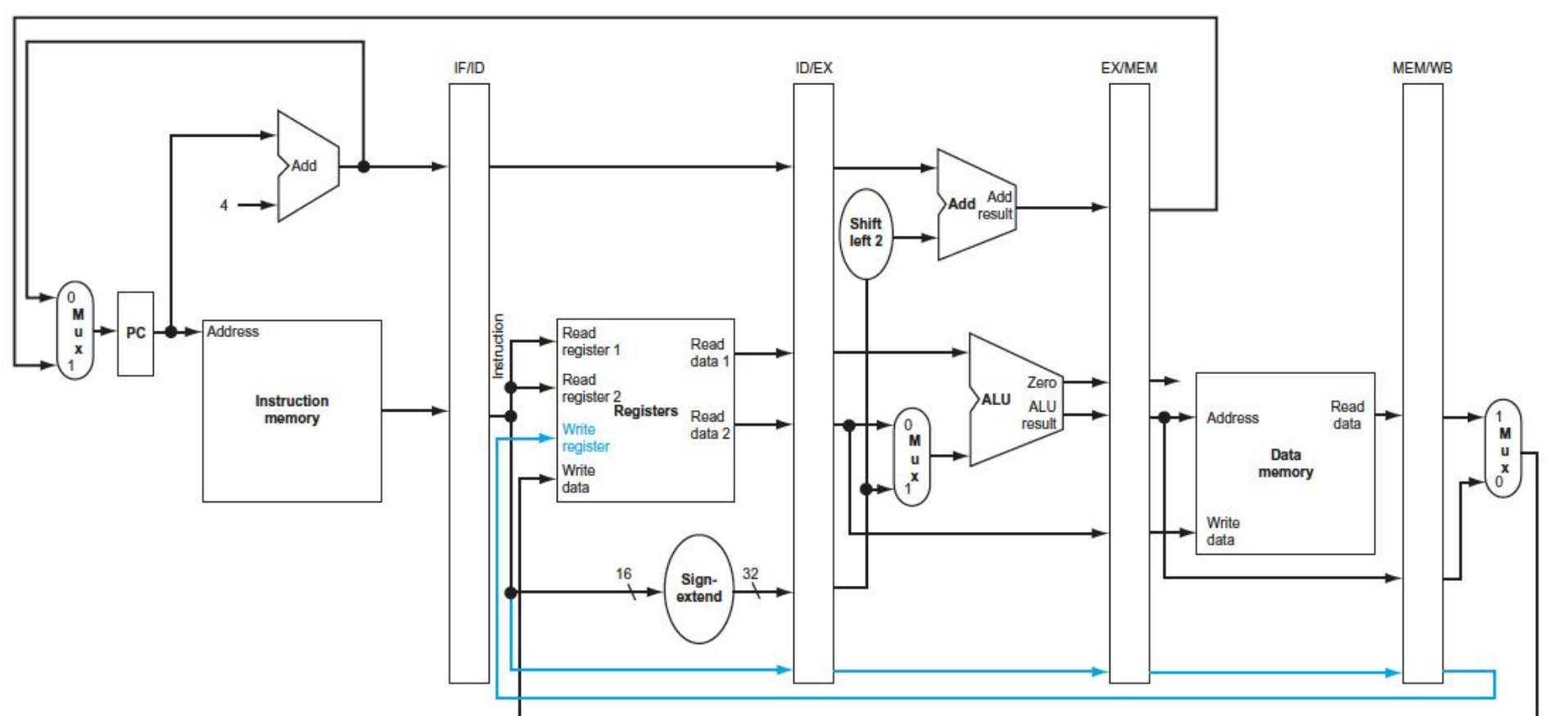
sw \$t5, 16(\$t0)

add \$t5, \$t1, \$t4

Bolha

lw \$t4, 8(\$t0)

sw \$t3, 12(\$t0)



Exemplo no Pipeline

- Ciclo 11 – Pipeline Stall

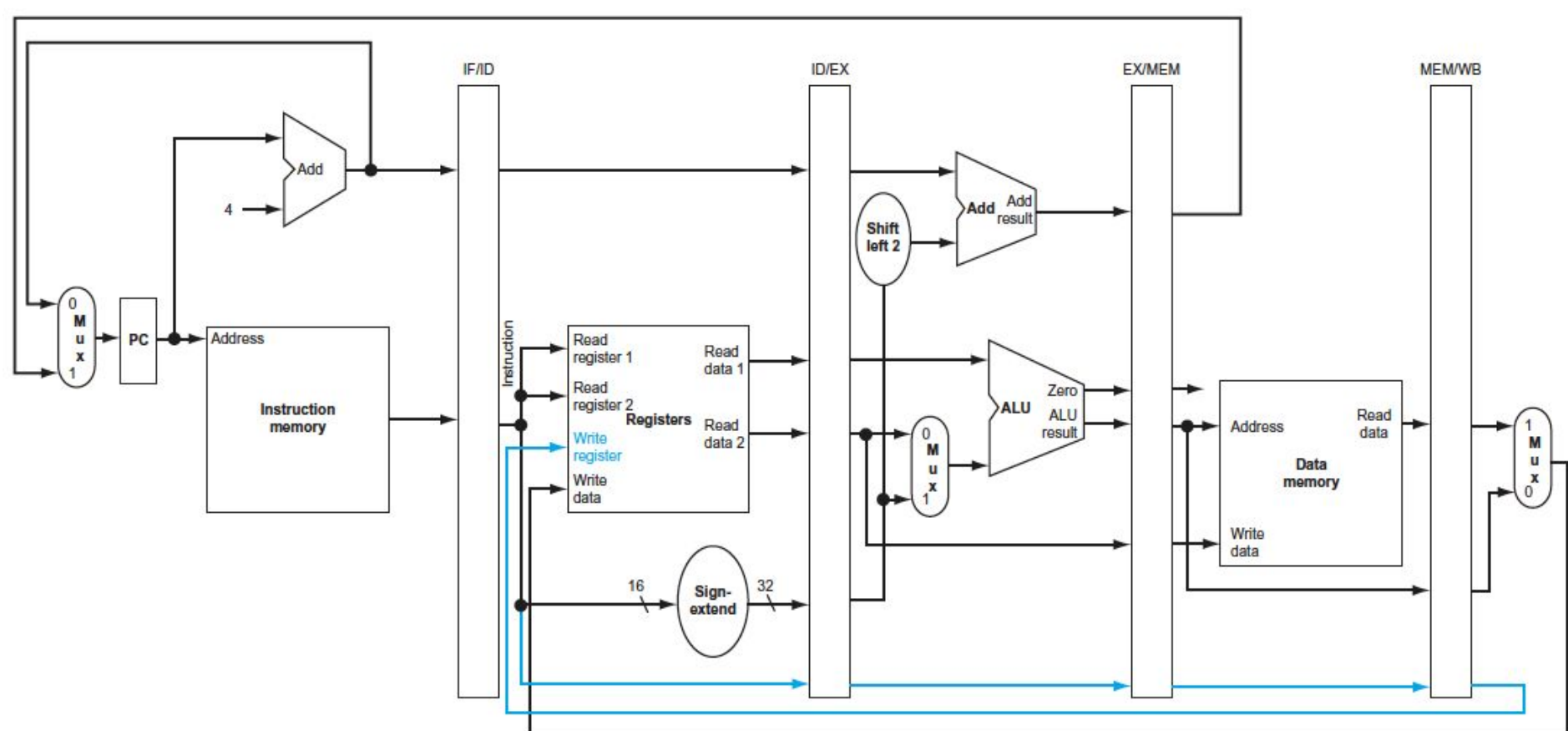
sw \$t5, 16(\$t0)

add \$t5, \$t1, \$t4

Bolha

Bolha

lw \$t4, 8(\$t0)



Exemplo no Pipeline

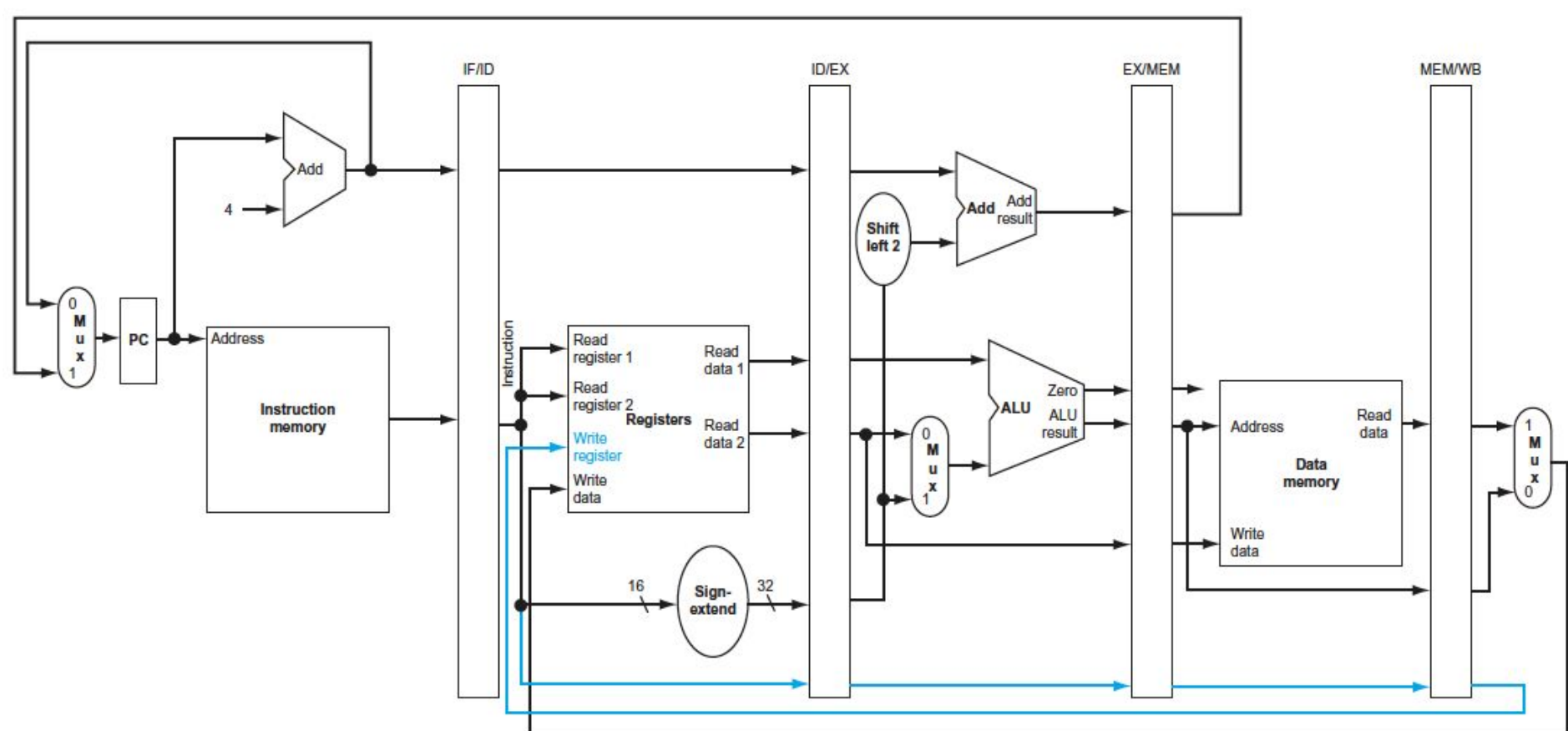
- Ciclo 12

sw \$t5, 16(\$t0)

add \$t5, \$t1, \$t4

Bolha

Bolha



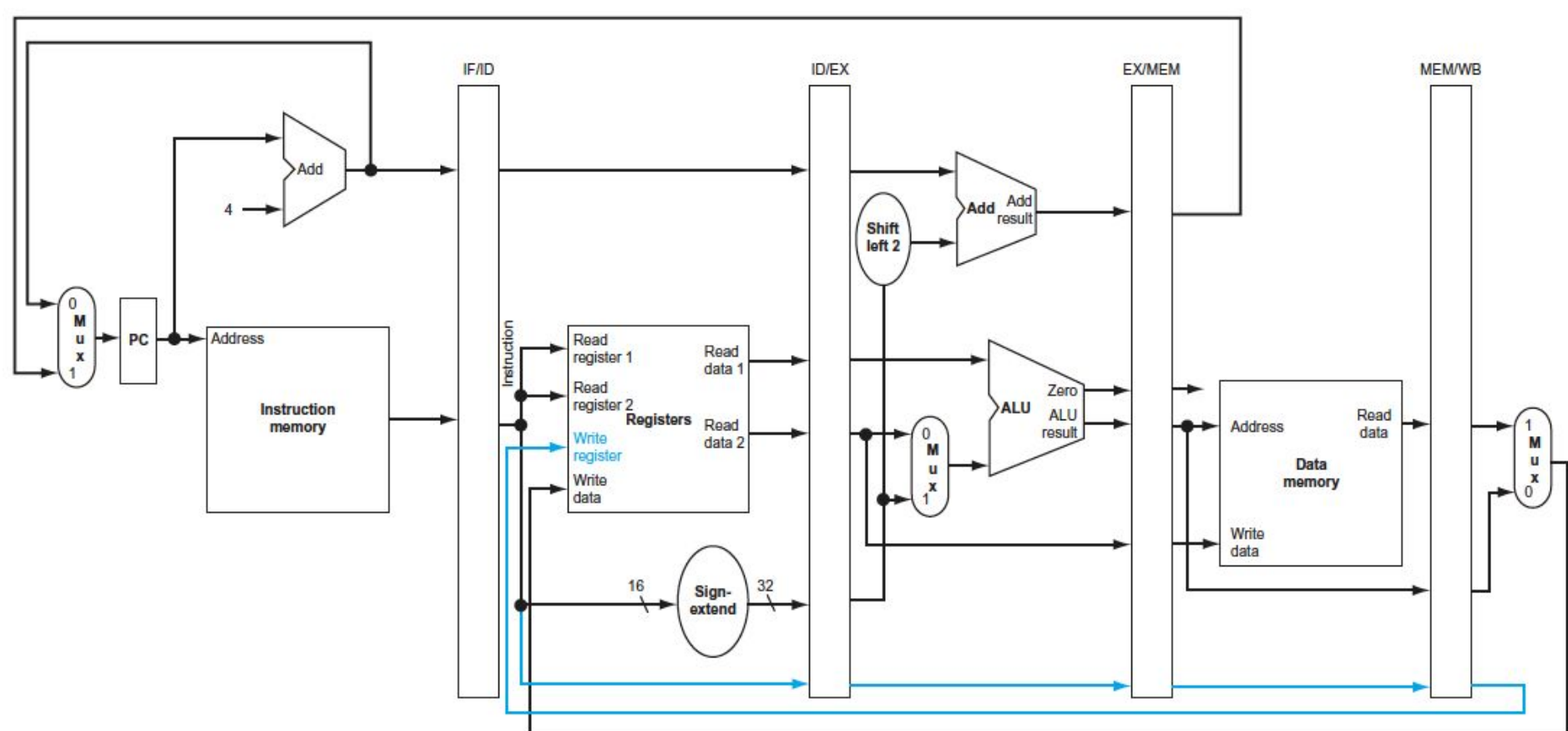
Exemplo no Pipeline

- Ciclo 13

sw \$t5, 16(\$t0)

add \$t5, \$t1, \$t4

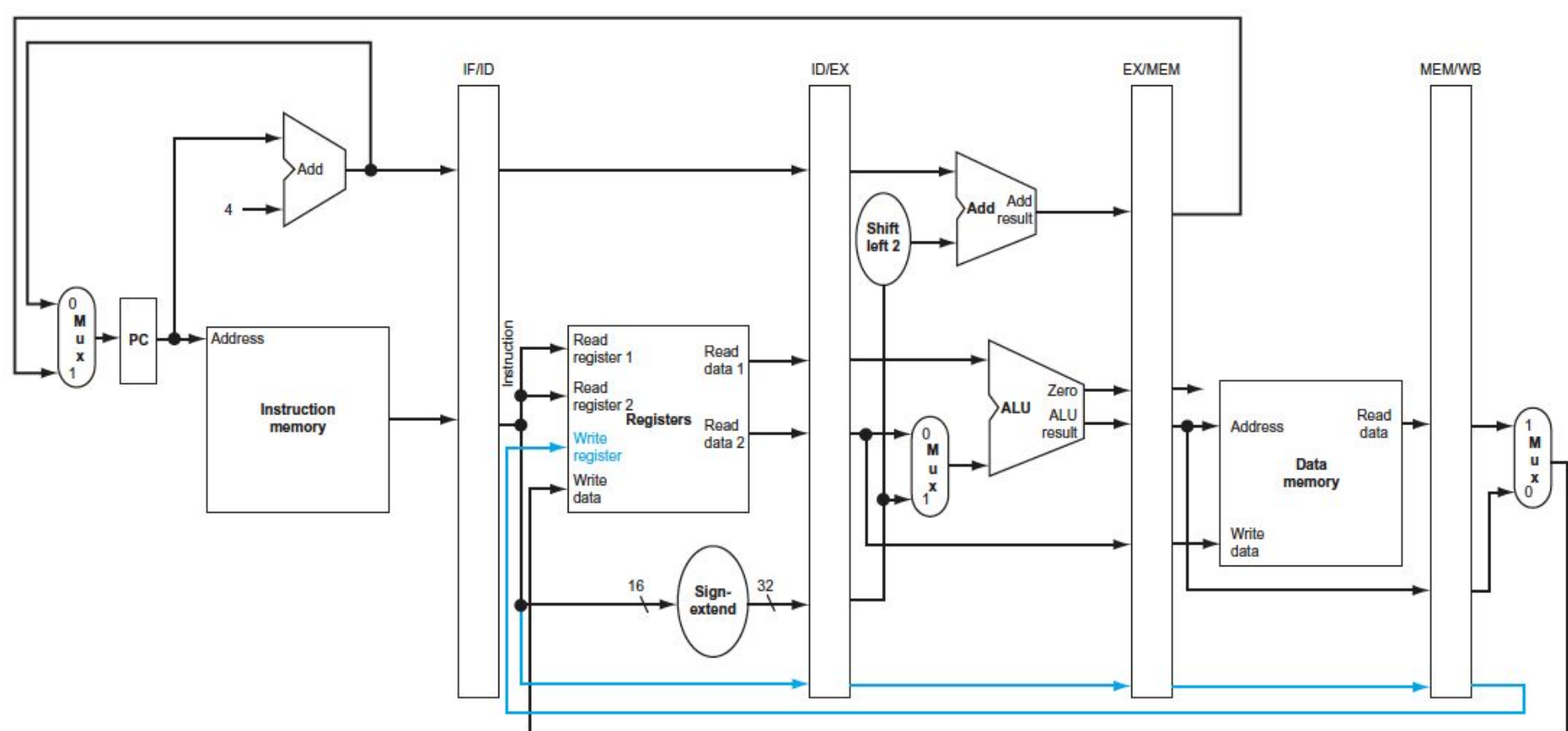
Bolha



Exemplo no Pipeline

- Ciclo 14

sw \$t5, 16(\$t0) add \$t5, \$t1, \$t4



Exemplo no Pipeline

- Ciclo 15

sw \$t5, 16(\$t0)

